SNAPSHOT 을 자동적으로 REFRESH 하는 방법

Bulletin no : 10075

SNAPSHOT 을 REFRESH 하기위해서 SHANPSHOT에 관련된 PARAMETER (?/dbs/initSID.ora) 들을 다음과 같이 SETTING 한다.

```
SNAPSHOT_REFRESH_PROCESSES = 1 (1 이상)
SANPSHOT_REFRESH_INTERVAL = 60
SNAPSHOT_REFRESH_KEEP_CONNECTIONS = FALSE
```

각 PARAMETER 의 기 을 살펴보면 다음과 같다.

. SNAPSHOT_REFRESH_PROCESS (Default : 0)

각 INSTANCE 당 SNAPSHOT REFRESH PROCESS 의 수를 의미한다. 만일 SNAPSHOT 을 자동으로 REFRESH 하기위해서는 1 이상의 수를 SETTING 하고 동시에 REFRESH 하는 SNAPSHOT 수가 대량이 아니 1 이 충분하다. SANPSHOT PROCESS 는 최대 10 까지(SNP0 - SNP9) 가능한데 이 PROCESS 들은 주기적으로 WAKE-UP 하여 자동적으로 REFRESH 되도록 SCHEDULE 되어있는 SNAPSHOT 을 REFRESH 하고 PROCESS 가 여러개일때는 SHARE 를 한다.

. SNAPSHOT_REFRESH_INTERVAL (Default : 60 초)

SNAPSHOT REFRESH PROCESS 가 WAKE-UP 하는 INTERVAL 이다.

. SNAPSHOT_REFRESH_KEEP_CONNECTIONS (Default : FALSE)

SNAPSHOT PROCESS 가 모든 SNAPSHOT 을 REFRESH 후에 REMOTE 데이타 베이스 CONNECTION 을 보존 할 것인지를 결정한다. 만일 FALSE 이면 모든 SNAPSHOT 이 COMPLETE 된후에 SNAPSHOT 을 REFRESH 하기 위해 만들었던 REMOTE DATABASE CONNECTION 을 CLOSE 한다.

SNAPSHOT 를 MANUAL 로 REFRESH 하기위해서는 DBMS_SNAPSHOT.REFRESH_ALL을 실행한다. SNPn 이 실행중일때는 DBMS_SNAPSHOT.REFRESH_ALL 은 필요없다.

SNAPSHOT의 기본 개념과 예제

Bulletin no : 10358

먼저 Oracle7 에서의 snapshot 에 대해 논하고 implementation 하는 방법과 refresh 하는 방법에 대해 알아보자 .

1) 서론

Oracle7 에서는 분산 network 환경에서 서로 다른 노드의 갯수에 제한이 없이 master table 의 replication 을 위한 table snapshot 을 제공한다.

Snapshot 은 update 가 불가능하다. 이것은 read 만 가능한 table 과 같다고 할수 있겠다. Update 는 master table 에서만 가능하다.

각 snapshot (replicate)의 변경된 사항은 master table 에 비동기적으로 반영 되어진다. refresh 는 SQL 명령어를 이용해 수동적으로 수행되어질 수도 있고 , refresh 시간에 따라 자동적으로 수행되어질 수도 있다.

2) Simple 과 Complex

Simple snapshot 은 각 row 가 한개의 remote table1 의 1개의 row 에 기초한 것이다. 즉 Group by, Order by, subqueries, joins, set operations 이 없는 경우이다. 반대로 snapshot 에 정의된 query 가 이러한 구문을 포함하면 complex snapshot 이라 한다.

3) snapshots 의 장점

. remote table 에 대한 snapshot 을 local 에 가짐으로써 network 을 통한 query 를 하지 않아도 되어 시간을 절약할수 있다.

. 여러가지 원인에 의해 master table 을 access 하지 못하더라도 master site 사용자들은 여전히 snapshots을 사용할 수 있다.

4) 사용 방법

```
CREATE SNAPSHOT .snapshot_name
        PCTFREE              integer
        PCTUSED              integer
        INITRANS              integer
        MAXTRANS            integer
        TABLESPACE          tablespace
        STORAGE         storage_clause
        CLUSTER         cluster (column_list)
        REFRESH


        START WITH date
        NEXT date
```

5)Snapshot 에 관련한 오브젝트들

Master table
Snapshot base table
Snapshot log

6) Snapshot 에 관련한 Data dictionary Views:

DBA_SNAPSHOTS
DBA_SNAPSHOT_LOGS
USER_SNAPSHOTS
ALL_SNAPSHOTS
MVIEW$

7) Snapshot 에 관련한 Data dictionary tables:

SNAP$
MLOG$

8) 연관된 Packages

*DBMS_SNAPSHOT

이 것은 다음의 두 스크립터를 순서적으로 수행하면 생성된다.

(1) dbmssnap.sql
(2) catproc.sql (which runs prvtsnap.plb, besides other scripts)

```
-- 이 Package 에서 수행되는 것들
--
-- purge_log          - 필요없는 row 의 log 를 purge
-- refresh            - 주어진 snapshot 의 refresh
-- refresh_all         - 모든 snapshots 을 refresh
-- drop_snapshot     - snapshot 을 drop
-- set_up             - snapshot refresh 를 위해 master쪽을 setup
-- wrap_up            - master site 에서 refresh를 record
-- get_log_age        - log 에서 가장 오래된date 를 check
-- testing             - snapshots을 test (currently null)
-- I_am_a_refresh     - flag used to let triggers identify refreshes
```

이 package 는 snapshot 를 생성하기 전에 반드시 존재 하여야 한다.

9) init.ora 에 적용되는 파라미터들
snapshot 의 refresh process 를 위한 파라미터는 다음과 같다.

- o SNAPSHOT_REFRESH_PROCESSES (number)
- o SNAPSHOT_REFRESH_INTERVAL (seconds)
- o SNAPSHOT_REFRESH_KEEP_CONNECTIONS (true/false)

10) Snapshot Mechanism
Snapshot이 생성되면, Oracle은 이 snapshot에 관련한 여러 다른 오브젝트들도 함께 생성한다. 이들 오브젝트는 변화되어지지 않는다.

snapshot site에서는 snapshot 에서 정의한 query 에 의해 가져온 row 를 저장할 base table 을 생성한다. 또한 이 table 을 기반으로 view 를 생성한다. 이 view 는 read-only 이고 snapshot 을 이용하는 사용자자 이용가능하다 .
또 다른 view 로는 snapshot 을 refresh 하기 위한 view 가 있다.

Snapshot refreshing:

주기적으로 snapshot 은 refresh 하여 최근의 변화된 사항들을 master table 에 반영한다. Snapshot process는 snapshot 에서 정의된 query를 이용하여 변화된 사항들을 가져와 이전의 snapshot data 를 변환한다.
각 snapshot 은 각기 트렌젝션에서 refresh 된다.

Snapshot Log:
snapshot log 는 마스터 테이 과 관련이 있고 마스터 DB 쪽에 존재한다. 여기에는 마스터 테이블에서 update/delete/insert 되는 row 가 저장되며 snapshot refresh 시 이용된다. 임의의 마스터 테이블에 기초한 simple snapshot 이 refrsh 되면 snapshot log의 해당 row 가 반영된다. 이것을 "fast refresh" 라 일컫는다. 한 snapshot log 는 같은 master table 에 설정된 여러개의 snapshot log 에 사용되어 진다. Log 의 특정 row 는 모든 snapshot 에 의해 필요 없게 되면 삭제가 되어 그 크기가 작게 유지된다.
그러나 필요로 하는 snapshot 이 하나라도 있으면 지워지지 않는다.

Snapshot 이 simple snapshot 이 아니고 snapshot log 가 없으면 fast refresh 는 불가능하며 이 경우 "complete refresh" 에 의해 refresh 된다.

======
사용 예 :
======

****
[1] Simple snapshot 과 Snapshot log (on Master):
****

MASTER SITE (REMOTE)

Create a snapshot log on the master table:

```
sqlplus scott/tiger
```

```
create snapshot log on emp
tablespace users
storage (initial 10K pctincrease 50)
pctfree 5;
```

NOTE: init.ora 파라미터는 필요하지 않다.

SNAPSHOT SITE (LOCAL)

---

1. 다음처럼 init.ora 파라미터를 설정한다.

```
snapshot_refresh_interval = 20

snapshot_refresh_processes = 2

snapshot_refresh_keep_connections = true

open_cursors = 250
```

2. sqldba

```
connect internal

shutdown

startup  (init.ora 파라미터 적용)
```

3. sqlplus system/manager

```
create public database link chicago

connect to scott identified by tiger

using 'chicago';
```

NOTE: SQL*Net V2 의 'chicago' 는 tnsnames.ora에 정의되어 있어야 한다.

4. sqlplus scott/tiger

```
create snapshot emp_snap

pctfree 5

pctused 60
```

```
    refresh fast

    start with sysdate

    next sysdate + (1/288)          /* 5 분마다 refresh */

    as select * from emp@chicago;
```

MASTER SITE (REMOTE)
---

1. sqlplus scott/tiger

```
    insert into emp (empno,ename,job,hiredate,deptno)

    values (1234,'SCOTT','DBA',sysdate,10);

    commit;
```

SNAPSHOT SITE (LOCAL)
---

1. sqlplus scott/tiger (or simply switch to client window)

```
    select count(*) from emp_snap;
```

    snapshot 이 자동으로 refresh 될때까지 반복한다.(약 5 분)
    이 방법은 fast refresh 가 작동됨을 보여준다.

    snapshot 은 dbms_snapshot.refresh procedure 를 이용함으로써 수동으로 작동 시킬수 있다.

MASTER SITE (REMOTE)
---

1. sqlplus scott/tiger

```
    insert into emp (empno,ename,job,hiredate,deptno)

    values (2345,'SCOTT','DBA',sysdate,10);

    commit;
```

SNAPSHOT SITE (LOCAL)
---

1. sqlplus system/manager

```
    select count(*) from scott.emp_snap;

    execute dbms_snapshot.refresh('SCOTT.EMP_SNAP','?');

    PL/SQL procedure successfully completed.

    select count(*) from scott.emp_snap;

    NOTE:  Snapshot 이 수동으로 refresh 되었음을 확인할수 있다.
```

snapshot 은 dbms_snapshot.refresh_all 을 수행하여 refresh 할 수 없다.
만일 주어진 snapshot 이 1:00 에 refresh 되도록 되어 있다면, 1:00시 전에 refresh_all 을 수행하도 refresh 는 불가능하다. 이 예제는 아래에 제시되어 있다.

```
MASTER SITE (REMOTE)
--------------------

1.  sqlplus scott/tiger

    insert into emp (empno,ename,job,hiredate,deptno)

    values (3456,'SCOTT','DBA',sysdate,10);

    commit;

SNAPSHOT SITE (LOCAL)
---------------------

1.  sqlplus system/manager

    select count(*) from scott.emp_snap;

    execute dbms_snapshot.refresh_all;

    PL/SQL procedure successfully completed.

    select count(*) from scott.emp_snap;

    NOTE:  refresh_all 프로시져를 이용하여 수동으로 Snapshot 을 refresh 할수 없다.

*****************************************
[2] snapshot log 가 없는 simple Snapshot
*****************************************

(1) FAST REFRESH OPTION 을 이용한 SNAPSHOT .
```

이 예는 마스터 테이블이 snapshot log 를 가질 경우만 fast refresh 가 가능 함을 보여준다. 만일 "refresh snapshot"으로 정의된 snapshot 이 snapshot log 가 없다면 마스터 테이블이 수정될때 자동으로 refresh 되지 않는다.

```
====================
SNAPSHOT SITE (LOCAL)
====================

Create Snapshot
---------------
SQL)connect system/manager
Connected.
SQL) grant resource to saj;

Grant succeeded.

SQL) connect saj/saj
Connected.

Create Database Link
--------------------

SQL) create database link aixlink
  2  connect to system identified by manager
  3  using 't:tcaix:V716';

Database link created.

Check the Database Link created
-------------------------------

SQL) desc dept@aixlink
 Name                           Null?    Type
 ------------------------------          --------                --                DEPTNO
NUMBER(2)
 DNAME                                   VARCHAR2(14)
 LOC                                     VARCHAR2(13)

Create the Snapshot (to be refreshed every 5 minutes)
-----------------------------------------

SQL) create snapshot deptsnap
  2  pctfree 5
  3  pctused 60
  4  refresh fast
  5  start with sysdate
```

```
  6  next sysdate + (1/288)
  7  as select * from system.dept@aixlink;

Snapshot created.

Check the Snapshot created
--------------------------

SQL) select * from deptsnap;

    DEPTNO DNAME          LOC
---------- -------------- -------------
        10 ACCOUNTING     NEW YORK
        20 RESEARCH       DALLAS
        30 SALES          CHICAGO
        40 OPERATIONS     BOSTON
        50 a,b

====================
MASTER SITE (REMOTE)
====================

SQL) connect system/manager
Connected.

Check the Master table
----------------------

SQL) select * from dept;

    DEPTNO DNAME          LOC
---------- -------------- -------------
        10 ACCOUNTING     NEW YORK
        20 RESEARCH       DALLAS
        30 SALES          CHICAGO
        40 OPERATIONS     BOSTON
        50 a,b

SQL) delete from dept where deptno = 50;

1 row deleted.

SQL) commit;

Commit complete.
```

```
====================
SNAPSHOT SITE (LOCAL)
====================

SQL) select * from deptsnap;

    DEPTNO DNAME          LOC
---------- -------------- -------------
        10 ACCOUNTING     NEW YORK
        20 RESEARCH       DALLAS
        30 SALES          CHICAGO
        40 OPERATIONS     BOSTON
        50 a,b

SQL)/

    DEPTNO DNAME          LOC
---------- -------------- -------------
        10 ACCOUNTING     NEW YORK
        20 RESEARCH       DALLAS
        30 SALES          CHICAGO
        40 OPERATIONS     BOSTON
        50 a,b

After 10 minutes:

SQL) /

    DEPTNO DNAME          LOC
---------- -------------- -------------
        10 ACCOUNTING     NEW YORK
        20 RESEARCH       DALLAS
        30 SALES          CHICAGO
        40 OPERATIONS     BOSTON
        50 a,b
```

RESULT: Snapshot 이 refreshed fast 로 만들어져 있고 , snapshot log 가 없기 때문에 이 snapshot 은 refresh 되지 않는다.

dbms_snapshot.refresh 패키지 프로시져를 수행하면 refresh 가 수행된다.

(2) REFRESH COMPLETE OPTION 을 이용한 SNAPSHOT.

```
====================
SNAPSHOT SITE (LOCAL)
====================
```

Snapshot 생성

```
SQL) connect system/manager
Connected.
SQL) grant resource to saj;

Grant succeeded.
```

Database Link 생성

```
SQL) create database link aixlink
  2  connect to system identified by manager
  3  using 't:tcaix:V716';

Database link created.
```

Database Link 생성을 점검

```
SQL) desc dept@aixlink
 Name                           Null?    Type
 ------------------------------ --------
 DEPTNO                                  NUMBER(2)
 DNAME                                   VARCHAR2(14)
 LOC                                     VARCHAR2(13)
```

Create the Snapshot

```
SQL) connect saj/saj
Connected.
SQL) create snapshot deptsnap2
  2  pctfree 5
  3  pctused 60
  4  refresh complete
  5  start with sysdate
6  next sysdate + 1/(288*20)
       /* REFRESH EVERY 15 SECONDS */
  7  as select * from system.dept@aixlink;

Snapshot created.
```

Snapshot 생성 점

```
SQL) select * from deptsnap2;

    DEPTNO DNAME          LOC
---------- -------------- -------------
        10 ACCOUNTING     NEW YORK
        20 RESEARCH       DALLAS
        30 SALES          CHICAGO
        40 OPERATIONS     BOSTON
        50 a,b

====================
MASTER SITE (REMOTE)
====================

SQL) connect system/manager
Connected.

마스터 테이블 점검
----------------------

SQL) select * from dept;

    DEPTNO DNAME          LOC
---------- -------------- -------------
        10 ACCOUNTING     NEW YORK
        20 RESEARCH       DALLAS
        30 SALES          CHICAGO
        40 OPERATIONS     BOSTON
        50 a,b

SQL) delete from dept where deptno = 50;

1 row deleted.

SQL) commit;

Commit complete.

=====================
SNAPSHOT SITE (LOCAL)
=====================

SQL> select * from deptsnap2;

    DEPTNO DNAME          LOC
---------- -------------- -------------
```

```
        10 ACCOUNTING     NEW YORK
        20 RESEARCH       DALLAS
        30 SALES          CHICAGO
        40 OPERATIONS     BOSTON
        50 a,b

After 15 seconds:

SQL> /

    DEPTNO DNAME          LOC
---------- -------------- -------------
        10 ACCOUNTING     NEW YORK
        20 RESEARCH       DALLAS
        30 SALES          CHICAGO
        40 OPERATIONS     BOSTON
```

RESULT: 이 Snapshot 이 refreshed complete 로 생성 되었기 때문에 자동으로 refresh 된다.

```
====================
MASTER SITE (REMOTE)
====================

Check the Master table
----------------------

SQL) select * from dept;

    DEPTNO DNAME          LOC
---------- -------------- -------------
        10 ACCOUNTING     NEW YORK
        20 RESEARCH       DALLAS
        30 SALES          CHICAGO
        40 OPERATIONS     BOSTON

SQL> insert into dept values(50, 'FINANCE', 'SFO');

1 row inserted

SQL> commit;

Commit complete.

SQL) select * from dept;

    DEPTNO DNAME          LOC
---------- -------------- -------------
```

```
        10 ACCOUNTING     NEW YORK
        20 RESEARCH       DALLAS
        30 SALES          CHICAGO
        40 OPERATIONS     BOSTON
        50 FINANCE        SFO

=====================
SNAPSHOT SITE (LOCAL)
=====================

SQL) select * from deptsnap2;

    DEPTNO DNAME          LOC
---------- -------------- -------------
        10 ACCOUNTING     NEW YORK
        20 RESEARCH       DALLAS
        30 SALES          CHICAGO
        40 OPERATIONS     BOSTON

After 15 seconds:

SQL) select * from deptsnap2;

    DEPTNO DNAME          LOC
---------- -------------- -------------
        10 ACCOUNTING     NEW YORK
        20 RESEARCH       DALLAS
        30 SALES          CHICAGO
        40 OPERATIONS     BOSTON
        50 FINANCE        SFO
```

RESULT: Snapshot 이 refreshed complete 로 생성되었기 때문에 자동으로 refresh 된

TABLE SIZE 와 INDEX SIZE 계산

Bulletin no : 10156

---

1. TABLE SIZE 계산 공식(ORACLE BLOCK SIZE : 2K 로 가정)

```
$ sqlplus scott/tiger
  SQL> SELECT   GREATEST(4, ceil(ROW_COUNT /
              ((round(((1958 - (initrans * 23)) *
              ((100 - PCT_FREE) /100)) / ADJ_ROW_SIZE)))) * 2)
              TableSize_Kbytes
       FROM dual;
```

*. 한개의 BLOCK에 Available 한 Bytes - 1958
*. 각 initrans 는 23 Bytes
*. PCT_FREE : Table 의 pctfree 값(default 10)
*. ADJ_ROW_SIZE : 각 row 의 평균 SIZE 추정치
*. ROW_COUNT : table 의 row 의 갯수

2. INDEX SIZE 계산공식

```
SQL> SELECT   GREATEST(4, (1.01) * ((ROW_COUNT /
              ((floor(((2048 - 113 - (initrans * 23)) *
              (1 - (percent_free/100))) /
              ((10 + uniqueness) + number_col_index +
              (total_col_length)))))) *2))
              IndexSize_Kbytes
       FROM dual;
```

*. 한개의 BLOCK에 Available 한 Bytes (1935 or 2048 - 113)
*. 각 initrans 는 23 Bytes
*. ROW_COUNT : table 의 row 의 갯수
*. PCT_FREE : Index 의 pctfree 값(default 10)
*. number_col_index : Index 에서 column 의 수
*. total_col_length : Index 의 길이 추정치
*. uniqueness : 만일 unique index 면 1, non-unique index 면 0.

TABLE 크기 추정 방법

Bulletin no : 10085

TABLE(NON-CLUSTED TABLE)의 초기 사용량을 추정하는 방법을 살펴보자.

여기서는 ORACLE7 SERVER를 기준으로 하여 설명 하기 하며, TABLE의 초기 사용량을 계산하는 단계를 다음과 같이 나눌수 있다.

STEP 1 - 총 블럭 헤드 크기(BLOCK HEADER SIZE)를 계산
STEP 2 - 데이타 블럭 당 사용 가능한 데이타 영역을 계산
STEP 3 - 평균 ROW의 전체 컬럼의 길이(COMBINED COLUMN LENGTH)를 계산
STEP 4 - 총 평균 ROW 크기를 계산
STEP 5 - 데이타 블럭내의 평균 ROW 수를 계산
STEP 6 - 테이블에서 요구되는 블럭과 바이트 수를 계산

1. [STEP 1] : 총 블럭 헤드 크기(TOTAL BLOCK HEADER SIZE)를 계산

1) total block header

```
total block header = block header, part A +
                     block header, part B

block header, part A = (fixed header + variable transaction header)
block header, part B = (table directory + row directory)

** fixed header          : 57 bytes (고정된 블럭 헤드)
   variable transaction  : 23 * I
   header                ( I는 해당 TABLE의 INITRANS의 값)

   table directory       : 4
   row directory         : 2 * R
                         ( R은 블럭의 ROW 수, STEP 5에서 계산)
```

2) 예를 들어, INITRANS = 1일때 경우의 total block header

```
total block header = (57 + (23*1)) + (4 + (2*R))
                   = 80 + (4 + (2*R)) bytes
```

2. [STEP 2] : 데이타 블럭 당 사용 가능한 데이타 영역
(DATA SPACE PER DATA BLOCK)을 계산

1) available data space

available data space = (block size - total block header) -
((block size - block header, part A) *
(PCTFREE/100))

** block size : 데이타베이스의 블럭 크기
(SQL*DBA의 SHOW PARAMETER에서
db_block_size를 확인)

2) 예를 들어, db_block_size=2K, PCTFREE=10 일 경우
available data space = (2048 - (80 + (4 + 2R)) -
((2048 - 80) * (10 / 100))
= (1964 - 2R) - (1968 * 0.1)
= (1964 - 2R - 196)
= (1768 - 2R) bytes

3. [STEP 3] : 평균 ROW의 전체 컬럼의 길이
(COMBINED COLUMN LENGTH)를 계산

ROW 길이를 계산하기 위하여 TABLE 정의에서 컬럼 수, 각 컬럼의 데이타 타입, 가변 길이 컬럼의 평균 크기등을 참조한다.

1) D (data space/average row) : table T 가 다음의 키마로 정의되어
있을 경우의 ROW의 평균 길이
- create table T (A char(10), B date, C number(10,2))

D (data space/average row) = (A + B + C)

** A(컬럼 A의 길이) : 10 bytes -----> CHAR 타입인 경우 fixed length
B(B의 길이) : 7 bytes -----> DATE 타입인 경우
C(C의 길이) : 5 bytes = (P / 2) + 1
-----> NUMBER 타입인 경우에는
PRECISION을 고려해서 계산됨
(단, PRECISION에서 NUMBER의 길
이를 나타내므로 SCALE은 무관함)

4. [STEP 4] : 총 평균 ROW 크기를 계산(TOTAL AVERAGE ROW SIZE)

여기서는 테이블의 ROW에 의해 요구되는 최소 사용 영역을 계산하는 공식을 제공한다.

1) bytes/row

bytes/row = row header + F + V + D

** row header : ROW 당 3 bytes (NON-CLUSTERED TABLE)
F : 250 bytes 이하를 사용하는 컬럼의 TOTAL LENGTH BYTES
(각 럼 당 1 bytes)
V : 250 bytes 이상을 사용하는 컬럼의 TOTAL LENGTH BYTES
(각 컬럼 당 3 bytes)
D : [STEP 3]의 D (data space/average row)

2) 예를 들어, [STEP 3]의 테이블 T 의 TOATL AVERAGE ROW SIZE
- 250 byte 하의 컬럼 수 = 3
250 byte 이상의 컬럼 수 = 0
D (data space/average row) = 22 인 경우
- avg. row size, table T = (3 + (1 * 3) + (3 * 0) + 22)
= 28 bytes

5. [STEP 5] : 데이타 블럭내의 평균 ROW 수(AVERAGE ROWS PER BLOCK)를 계산

1) R (avg. # of rows/block)

R (avg. # of rows/block) = available space / average row size

** available space : [STEP 2]의 available data space
average row size : [STEP 4]의 bytes/row

2) 예를 들어, [STEP 2]의 available space가 (1768 - 2R) bytes이고,
[STEP 4]의 테이블 T의 bytes/row가 28bytes일 경우

- R = (1768 - 2R)bytes / 28bytes
28R = 1768 - 2R
30R = 1768
R = 58 rows/block

6. [STEP 6] : 테이블에서 요구되는 블럭과 바이트 수를 계산
(NUMBER OF BLOCKS AND BYTES)

1) # blocks for table과 # bytes for table

blocks for table = # rows / R

** # rows : TABLE의 ROW 수
R : [STEP 5]의 R rows/block

bytes for table = # blocks for table * 2048 bytes/block

** 2048 bytes/block : 데이타베이스의 블럭 크기 (db_block_size)

2) 예를 들어, 테이블 T가 10000개의 rows을 가질 경우의 테이블 당 블럭 수

- # blocks for table T = 10000 rows / 58 rows/block
  = 173 blocks

- # bytes for table T = 173 blocks * 2048 bytes/block
  = 354304 bytes
  = 346 Kbytes

TABLESPACE 의 내용을 다른 TABLESPACE 로 옮기는 방법

Bulletin no : 10087

현재 USER가 사용중인 TABLESPACE의 데이타를 다른 TABLESPACE의 영역으로 옮기는 방법을 살펴보기로 하자. 우선 EXPORT UTILITY를 이용하여 다른 TABLESPACE로 이전할 데이타에 대한 백업을 수행한다.

(1) EXPORT UTILITY를 이용하여 USER별 데이타 백업

- USER DATA의 EXPORT

```
$ exp user_name1/user_password

  Export: Release 7.0.15.4.0 - Production
       .
  Export file : expdat.dat > EXP100.dmp
  (2) U(sers), or (3) T(ables) : (2) U > U
  Export grants (yes/no) : yes > YES
  Export table data (yes/no) : yes > YES
  Compress extents (yes/no) : yes > NO

  About to export USER_NAME1's objects .....
   .exporting snapshots
   .exporting snapshot logs
   .
   .
```

다음으로 이전하고자 하는 TABLESPACE를 생성하고

(2) USER 용 TABLESPACE 를 생성

- SQLDBA> CONNECT INTERNAL;
  SQLDBA> CREATE TABLESPACE tablespace_name
  DATAFILE '/user1/oracle_data/aaa.dbf' SIZE 100M;

3) 해당 USER를 아래와 같은 방법으로 다시 생성하여야함

- SQLDBA> DROP USER user_name1 CASCADE;

- SQLDBA> CREATE USER user_name2 IDENTIFIED BY user_password
  DEFAULT TABLESPACE tablespace_name

```
        QUOTA 100M ON tablespace_name;
SQLDBA> GRANT CONNECT TO user_name2;
```

4) 생성된 user_name 과 tablespace_name 확인

- $ sqlplus user_name2/user_password
  SQL> SELECT * FROM USER_TS_QUOTAS;

5) IMPORT UTILITY를 이용하여 변경예정인 TABLESPACE에 데이타 RESTORE

- $ imp system/manager file=exp100.dmp fromuser=user_name1
  touser=user_name2 commit=y

6) IMPORT 후 각 TABLE이 해당 TABLESPACE로 변경되었는지 확인

- $ sqlplus user_name2/user_password
  SQL> SELECT * FROM USER_TABLES;

TABLESPACE 크기를 확장하는 방법

Bulletin no : 10165

오라클 7.1 이하에서는 테이블 스페이스를 확장하려면 해당 테이블 스페이스에 데이타 화일을 추가하는 방법을 사용한다. 이 때 추가하는 데이타 화일의 이름은 기존 화일과 동일한 이름이 아니기만 하면 되지만 편의상 기존의 화일에 일련번호를 붙여서 사용하는 것이 일반적이다.

예를 들어 테이블 스페이스 TOOLS 를 확장한다고 가정하면

```
$sqlplus system/manager

SQL>select file_name, bytes
    from dba_data_files
    where tablespace_name = 'TOOLS';
```

이와같이 하면 현재 TOOLS 테이블 스페이스를 구성하고 있는 화일이름과 크기 (bytes)라 출력된다. 여기서 출력된 file_name 이 /oracle/dbs/tools0RA.dbf 라고 한다면 다음과 같이 하여 테이블 스페이스를 확장한다.

```
SQL>alter tablespace tools
    add datafile '/oracle/dbs/tools20RA.dbf' size 50M;
```

여기서는 화일의 크기를 50M 로 주었는데 이것은 디스크의 FREE SPACE 와 앞으로의 기존의 데이타 화일의 크기 및 앞으로 들어갈 데이타의 크기 등을 고려하여 적절한 값으로 결정하도록 한다.

오라클 7.2 에서는 위의 방법 외에도 기존의 데이타 화일의 크기를 변경시켜서 확장시킬 수 있다.

예를 들어 TOOLS 테이블 스페이스가 현재 50M 크기의 /oracle/dbs/tools0RA.dbf 화일로 구성되어 있다면 다음과 같이 해서 이 화일의 크기를 100M 로 늘릴 수 있다.

```
SQL>alter database datafile
    '/oracle/dbs/tools0RA.dbf' resize 100M;
```

RESIZE 옵션은 V7.2 에서 추가된 것으로 기존의 데이타 화일을 확장 또는 축소할 수 있다. 축소하는 경우는 데이타가 들어 있는 경우 하한선 이하로 내려가지는 않는다.

한편, 다음과 같은 명령을 이용하여 데이타가 계속 들어가서 테이블 스페이스를 꽉 채우게 되면 자동적으로 테이블 스페이스를 확장할 수도 있다.

```
SQL>alter database datafile
    '/oracle/dbs/tools0RA.dbf'
    autoextend on
    next 10M
    maxsize 200M;
```

이렇게 하면 데이타가 늘어 면서 자동적으로 10M 씩 데이타화일의 크기가 늘어나게 된다. 여기서는 최대 200M 까지 늘어 날 수 있도록 설정하였다.

TAPE 으로 ARCHIVE 를 직접 받는 방법

Bulletin no : 10155

ARCHMON: REDO LOG ARCHIVING TO TAPE

OVERVIEW

This paper describes how to archive redo logs to tape and how to monitor it through ARCHMON.

UNDERSTANDING LOG ARCHIVING

Any changes made to the database are recorded both in database and redo log buffers. Both buffers exist in the SGA. The redo log buffer is written to an online redo log file whenever a transaction is commited or the redo buffers are full. Every database needs at least 2 online redo log files. The point where the system stops writing to one file and begins writing to another is called a log switch.

A log switch triggers a checkpoint and checkpoints are done at every log_checkpoint_interval. A checkpoint is when all the modified database buffers are written to the datafiles on disk.

When the database is running in ARCHIVELOG mode, the archive daemon ARCH archives filled online redo log files off line or to the archive destination. These archived files can then be used to recover the database from a consistent backup in case of media failure. In versions 6.0.27 and later, ARCH can archive redo log files directly to tape for some Unix platforms.

REQUIREMENTS FOR ARCHIVING TO TAPE
----

1. One terminal session must be dedicated to the Archive Monitor Program, ARCHMON, which communicates with ARCH. It prompts when a new tape must be mounted or when a full tape must be dismounted, confirming when it is successful, and displays error messages. An operator must check it from time to time for prompts, error messages, etc.

2. Tape device: any type of supported tape device.

3. Network hardware and software: BSD-based systems require SQl*Net TCP/IP to be installed on top of TCP/IP networking hardware and software to permit communications between ARCH and ARCHMON.

FACTORS TO CONSIDER FOR ARCHIVING TO TAPE

Database operations will halt if the system cannot write to the current online redo log file. This happens when it has filled the active redo log file but for some reason cannot write to the next one. This may be caused by file corruptions or instance failure (e.g. LGWR died).

If the database is running in archivelog mode, this can happen when the current redo log file cannot be written to because ARCH is still archiving the log file that LGWR is trying to log switch to. Factors that can effect this include the size and number of log files and writing rate. Writing a large file to tape may have a significant impact on database performance.
More log files of smaller size are recommended for archiving to tape.

Tape archiving includes ARCHMON which communicates with ARCH. When the end of tape is reached, ARCH stops and signals ARCHMON to prompt the user for a new tape. When ARCHMON knows the tape is mounted, it tells ARCH to resume the archiving operation. It needs regular operator monitoring.

SETTING UP REDO LOG ARCHIVING

The init.ora parameters that must be specified for automatic archiving of redo log files are:

log_archive_dest: It indicates the destination for the archiving of the redo log file.

syntax: log_archive_dest=device[:volumeM¦K¦B]
where,

device: specifies the location (device file or regular file) where
the redo file is to be archived.

volume: This is applicable only for tape archiving. It indicates the size of the tape volume. The tape must be able to hold at least the size of one log file, plus an overhead of 5K bytes.

M, K, B: indicates whether volume value refers to megabytes(M),
kilobytes(K) or bytes(B). The default is bytes.

To archive the online redo log files to tape device /dev/rmt0 to
tape which can hold 100M, specify:

log_archive_dest = /dev/rmt0:100M

log_archive_start: It indicates whether archiving is to be automatic or manual when the instance is started. It can have value 'false' or 'true'. The default is 'false'. This parameter is only meaningful if the database is running in archivelog mode.

Perform following steps to set up automatic archiving to tape:

1. Set init.ora parameters log_archive_start and log_archive_dest.

2. Starting ARCHMON

ARCHMON for System V based systems: On System V-based operating systems, ARCHMON can only monitor tape archival of database instances local to its machine.

To start ARCHMON, type:

$ archmon hoststring

For example, to start ARCHMON for database identified by ORACLE_SID value "TEST" enter:

$ archmon @P:TEST

ARCHMON for BSD-based systems: On BSD-based system, the ARCHMON requires SQL*Net TCP/IP to work. It communicates with ARCH through orasrv. To start orasrv enter:

$ tcpctl start

To start archmon enter:
$ archmon hoststring

where hoststring is a valid SQL*Net TCP/IP hoststring. For example, to start ARCHMON on host named "landshark" with ORACLE_SID of "TEST" enter:

$ archmon @T:landshark:TEST

3. Put the database in archivelog mode and set automatic archiving. Enter the following SQL*DBA statements:

```
SQLDBA> startup mount
SQLDBA> connect internal
SQLDBA> alter database archivelog;
SQLDBA> archive log start 'destination';
SQLDBA> alter database open;
```

The destination specifies the destination for archive files. It is same as specification for log_archive_dest init.ora parameter.

You get the message: 'connected.' from the ARCHMON session, which is confirmation that it has connected to the archiving process.

When the tape volume is full, ARCH suspends operation, tells ARCHMON to prompt for a new tape, waits for user's response, and then resume log archiving.

STOPPING ARCHMON AND ARCHIVING

ARCHMON will stop when the database is shutdown or log archiving has been stopped. When the database is shutdown with the normal or immediate option, ARCHMON signs off with following message, after archiving all remaining redo log files:

```
Message from ora_arch_TEST@landshark at 2:35....
Normal, successful termination.
```

If the database is shutdown with the abort option, the archiving of some logfiles will be incomplete. This will not ensure complete recovery up to the last log file written to.

ARCHMON can be interrupted, killed and reinvoked. However, if ARCHMON is not reinvoked, the database will hang when all redo logs are full until it is restarted.

Archiving can be turned off when the database is up or down. When the database is up, enter following SQL*DBA statements to turn off archivlog mode:

```
SQLDBA> connect internal
SQLDBA> alter database close;
SQLDBA> alter database noarchivelog;
SQLDBA> disconnect;
SQLDBA> shutdown
SQLDBA> startup
```

If the database is down, enter the following SQL*DBA statements to turn off archivelog mode:

```
SQLDBA> startup mount
SQLDBA> connect internal
SQLDBA> alter database noarchivelog;
SQLDBA> alter database open;
```

To check the status of database, whether it is in archivelog mode or not, enter following SQL*DBA statement:

```
SQLDBA> archive log list;
Database log mode              NOARCHIVELOG
Automatic archival             DISABLED
Archive destination            /usupport/6033p/dbs/arch.dbf
Oldest online log sequence     47
Current log sequence           48
```

The online redo log is archived on tape as filename 'arch_.dbf'.

For example, online redo log with sequence# 28 will be stored on tape as file 'arch_28.dbf'.

If offline redo log files are required to recover the database, you may retrieve archived log files using the 'cpio' command. For example, to check the contents of the tape for drive /dev/rmt0, enter:

```
$ cpio -itBc < /dev/rmt0
```

It will display the contents of the tape:

```
arch_28.dbf
arch_29.dbf
arch_30.dbf
...... etc....
```

Use the cpio command to read necessary files from the tape. For example, to get arch_28.dbf enter:

```
$ cpio -icBv arch_28.dbf < /dev/rmt0
```

TKPROF 사용시 CORE DUMP 조치 사항.

Bulletin no : 10310

TKPROF 사용시 Core Dump 조치 사항

Oracle V7.3.2.1 에서 tkprof를 이용하여 Explain을 떠 보려면 Core Dump가 떨어 집니다. 이것은 Oracle Bug 376851 때문이며 V7.3.3에서 Fix 될 예정 이며, 이 현상을 피해 가는 방법 2가지 방법은 아래와 같습니다.

1. $ORACLE_HOME/rdbms/admin/utltkprf.sql 을 sys로 수행

2. utltkprf.sql 에서 생성된tk profer 라는 role을 tkprof 사용 할 user에게 권한 부여

```
SQL> grant tkprofer to scott;
```

3.1 SQL*PLUS 에서 생성된 trace file을 아래와 같이 재 작성 후 tkprof 수행

```
$ grep -v "APPNAME" file.trc > newfile.trc
$ tkprof newfile.trc a explain=scott/tiger
```

3.2 또는, SQL*PLUS 대신 Server Manager에서 수행하면 grep을 사용 안해도 정상 작동됨

```
SVRMGR> connect scott/tiger
SVRMGR> alter session set sql_trace=true;
SVRMGR> select * from tab;
```

* 간단히 PLAN을 떠 보기 위해서는 SQL*PLUS 에서 set autotrace on 을 이용해도 됩니다.(V7.3에서)

TPC Q&A

Bulletin no : 10190

- TPC란 무엇입니까 ?

TPC는 Transaction Processing Council의 약자로서 여러 하드웨어 및 소프트웨어 회사 및 소수의 사용자 기관들을 멤버로 하는 비영리 단체입니다. TPC의 구성 목적은 "트랜잭션 프로세싱 벤치마크를 정의 하고 그 적과 판명가능한 성능을 널리 알리는 데" 두고 있습니다.

따라서 멤버들은 TPC의 벤치마크 테스팅을 통해 자사의 제품이 타사의 제품(예, H/W+S/W)과 얼마나 잘 결합될 수 있는지에 대한 증빙자료를 마련하게 됩니다.

- TPC-C란 무엇입니까 ?

TPC-C는 9개의 테이블로 구성된 데이타베이스에 주문입력 시스템과 주문처리 시스템을 모의, 읽기전용 및 갱신위주의 혼합 업무부담율을 처리해내는 OLTP 벤치마킹 입니다.

- TPC에서 정의하는 다른 종류의 벤치마킹에는 어떤 것이 있습니까 ?

우선 TPC-C보다 단순한 형태의 OLTP를 벤치마킹하는 A와 B가 있었습니다. A, B는 1995년 6월부로 폐지되었으며, 간단히 설명하면 A는 일종의 Banking System Simulation으로서 3 Updates와 1 Insert를 포함하는 한가지 트랜잭션 타입만을 벤치마킹하 것이었고, B는 이와 유사하나 "Tink Time"요구사항 및 터미널, 네트워크 에뮬레이션이 빠진 것이었습니다.

TPC-D라고 하는 새로운 DSS 벤치마크 테스트가 95년 4월부로 공식화되어 (TPC-C와 유사하며 대용량 DB에 17개의 Queries로 테스) 최적화 및 질의 프로세싱의 효율성을 측정하기로 되었으나 아직 그 누구도 실시 하지 않았으며 앞으로 TPC-S라고 하는 서버 벤치마크, TPC-C/S라고 하는 클/서 벤치마크 및 TPC-E라고 하는 엔터프라이즈 벤치마크 등이 금년 하반기 중 공식화 될 전망입니다.

- TPC-C는 실제상황을 그대로 대변합니까 ?

TPC-C는 단순비교를 위해서는 유용할 수 있으나 실제상황을 그대로 반영한다고 하긴 어렵습니다. 대부분의 벤치마크가 하나의 트랜잭션으로 수행된다는 점을 감안하면 실제상황과는 많이 다를 수 있고 따라서 이와 관련한 데이타의 충돌 등이 테스트되지 않는다고 볼 수 있으며, 실제로 중요한 Data Integrity나 Backup/Recovery는 아예 테스트되질 않습니다.

- 이제까지 오라클이 TPC-C에 참여하지 않은 이유는 무엇입니까 ?

가 큰 이유는 TPC-C의 규정에 맞추지 못하였기 때문입니다. 이 부분은 TPC-C의 규정 중 "Transaction Isolation"에 국한되는 것으로 이제 까지의 오라클의 Concurrency Control기법과는 다른 Spec을 요구하였던것입니다. 따라서 Oracle7이 TPC-C를 수행할 수 없었다기 보다는 결과가 만족스럽지 못할 것으로 생각되어 7.3까지 기다린 것입니다.

- TPC-C 테스트를 위해 7.3에 추가된 것은 무엇입니까 ?

한마디로 "Serializable Transaction"이라는 새로운 Concurrency Control 기능을 추가한 것입니다.(자세한 내용은 원본 P6 참조)

- Oracle7 Release 7.3은 아직 베타인데 결과치를 발표할 수 있습니까 ?

그렇습니다. TPC의 "Fair Use Rule"에 따르면, 테스트 된 제품의 "배급시기" 가 명시만 되면 Press Release하는 데 문제가 되진않습니다. 그러나 테스팅에 관한 모든 것을 밝혀주는 Full Disclosure Report는 96년 2월경에 배포가 가능하니 이점 양지하시기 바랍니다.

- 만약 가망 고객이 다른 플랫폼에 Oracle7의 TPC-C 결과치를 원하면 어떡하나요 ?

가장 예상되는 문제 중의 하나이지만 현재 HP/Oracle 발표이후 다른 하드웨어 업체들도 매우 적극적으로 오라클과 협력하여 그들 플랫폼 에서의 결과치를 원하고 있습니다. 이는 그간 TPC-C에 오라클이 불참 함으로써 하드웨어 플랫폼의 비교 그 자체가 무의미 했기 때문입니다. 대부 분의 TPC 테스트가 사실 그간 DBMS 보다는 H/W 선정에 많은 영향력을 보였던 것을 고려해도 앞으로 다른 플랫폼에 Oracle을 TPC-C 벤치마킹하는 것은 시간 문제라고 생각됩니다. 오라클 서버 기술부 및 각 제품부는 각 하드웨어 벤더와 긴밀히 협조 조만간 새로운 결과들을 속속 발표할 것입니다. 따라서 고객들에게 이 점을 주지하셔야 할 것 입니다.

- Informix가 HP9000/T500 8CPU 에 테스팅을 한 것과 이번의 오라클 결과치와는 CPU 갯수의 차이 때문에 어렵지 않은가요 ? (오라클 12CPU를 이용)

얼핏 보면 오라클이 결과를 생각해서 인포믹스보다 더 많은 CPU를 이용한 것처럼 보입니다. 결과는 인포믹스가 3118 tpms, 오라클이 5369 tpms로서 CPU 갯수 차이에 의한 당연한 차이로 인식되기 쉽습니다. 그러나 정말 중요한 사실은 Informix가 8개 CPU를 넘는 하드웨어로는 벤치마킹을 하지 않는다는 것 입니다. 따라서 Scalability면에서 오라클을 따라오지 못한다는 사실이 담겨있는 것 입니다. 이 점을 강조 하시기 바랍니다.

TRANSACTION ENTRY란 무엇인가?

Bulletin no : 10293

---

* Transaction Entry는 Table Create시 정의되는 Initrans와 Maxtrans에 연관되어 있다.

- Transaction Entry는 사용자 Process가 Row를 변경하는 모든 Data Block에 있는 Transaction의 제어를 위해 사용된다. 이것은 하나의 Transaction이 각각의 변경된 Data Block을 위해서 Transaction Table내에서 하나씩의 Slot만을 가짐을 의미한다.

예를 들어, Process A가 5개의 Data Block에 걸쳐 있는 100개의 Row에 대해 갱신 연산을 하고자 한다고 하자. Oracle은 Transaction 내에서 변경되어지는 각 Block에 대해 하나씩의 슬롯만을 취하면 된다. 즉 여기에서는 5개이다.

- Process A가 해야 할 첫번째 일은 Transaction Entry를 할당하는 것이다. 그리고 이 Transaction Entry에 Lock을 건 후 Transaction에 대한 동적(Dynamic)인 정보를 기록한다. 만약 Transaction이 갱신하고자 하는 Row가 Lock이 걸려 있음을 Process가 발견하면 Transaction 공간 내의 Lock을 풀것이다. 일단, Transaction이 Commit을 하거나 Rollback을 한 후에 Entry에는 새로운 Transaction이나, 입력 혹은 삭제가 자유롭게 이루어질 수 있다.

- 낮은 INITRANS Parameter의 제점은 큰 Transactional Table에서 Process 가 Transaction Entry를 검색하고 할당하기 위해 공간을 찾는 중 찾지 못했을 때 발생하는 경쟁(Contention)이다. Process는 다른 Entry가 Free 될 때까지 기다려야 한다.

- Default INITRANS는 Table이 1이고 인덱스는 2이다. 모든 Transaction Entry는 23 Bytes 정도를 사용한다. 이것은 OS에 따라 다르며, DBA는 V$Type_Size View를 참조하여 확인할 수 있다.

TRANSACTION을 ROLLBACK SEGMENT에 할당하는 방법

Bulletin no : 10383

Oracle은 다음과 같은 규칙에 의해 각 Transaction이 사용하는 Rollback segment를 결정한다.

1. Active Transaction의 수가 가장 작은 Rollback Segment에 할당한다.
2. 만약 1의 조건에 만족하는 Rollback segment가 하나 이상이면, 가장 마지막에 할당된 rbs 다음의 rbs를 할당한다. 이것은 undo 기능을 더 오래 지속시키도록 해 준다.

그러나 Application이 큰 rollback segment가 필요하다면 위와 같이 자동적인 rollback segment의 할당이 아닌 Manual하게 특정 rollback segment를 할당 가능하다. 다음의 방법을 사용한다.

a. 큰 rollback segment 를 만든다. 큰 rollback segment 를 만들기 위해서는 rbs tablespace 에 영역이 충분히 커야하므로 필요할경우 tablespace 확장하는 작업을 한다.

```
$ sqlplus system/manager
sql> alter tablespace rbs add datafile '?/rbs1SID.dbf' size 100m;

sql> create rollback segment big_rbs storage(initial 10m next 10m)
     tablespace rbs;
sql> alter rollback segment big_rbs online;
```

b. Transaction 를 assign 한다.

```
SQL> set transaction use rollback segment big_rbs;



....
exec sql commit work;
exec sql set transaction use rollback segment big_rbs;
```

이와 같이 지정하면 이후에 commit 또는 rollback이 일어날 때 까지 한 transaction에만 해당된다.

TRIGGER 에서 ERROR 확인
(SET SERVEROUTPUT ON 사용)

Bulletin no : 10382

---

1. Trigger 생성 (trg.sql)

```
create trigger dept_trg
after insert on dept
for each row
begin
 insert into emp(empno,ename,deptno)
        values(7000,'TEST',:new.deptno);
 exception
   when others then
    DBMS_OUTPUR.PUT_LINE('sqlcode : '||to_char(sqlcode));
end;
/
```

2. SQL*PLUS 에서 SET SERVEROUTPUT ON setting

```
$ sqlplus scott/tiger
SQL> set serveroutput on
```

3. DEPT Table 에 Data 입력

```
SQL> insert into dept values(10,'Sales','Seoul');
```

-- Error 가 발생하는 경우 sqlcode 가 Display 됨

TWO-PHASE COMMIT

Bulletin no : 10294

---

1. Two-Phase Commit 이란?

(1) Two-Phase Commit은 Application Programmer가 별도의 Program없이 Global Database의 Integrity를 보장하는 기법이다.
(2) Two-Phase Commit 은 분산 환경 Database(DDB)를 사용하는 경우에 사용된다. 이는 Oracle V7.0이후 Version에서 제공된다.

2. Two-Phase Commit의 구성

- Two-Phase Commit은 Prepare Phase와 Commit Phase로 나누어 진다.

(1) Prepare Phase
Global Coordinator(분산 Transaction을 일으킨 Node)가 분산 Transaction에 참석한 Node들에 대해 Commit이나 Rollback을 수행해도 되는가를 확인하는단계이다.

(2) Commit Phase
분산 Transaction에 참여한 Node가 Global Coordinator에게 분산 Commit해도좋다는 Response(즉 Prepare 되었음)를 받고 Transaction을 Commit한다. 어떠한 Node라도 Prepare가 안되었다는 Response가 있으면 Transaction을
Rollback한다.

3. Two-Phase Commit의 수행

(1) Prepare Phase에 대한 Response는 Prepared, Read-Only, Abort가 있다.
A. Prepared는 Data가 정상적으로 수정되었으며 Prepare되었음을 의미한다.
B. Read-Only 는 해당Node에 Data의 수정이 없었다는 의미이다.
C. Abort는 Prepare되지 않았음을 의미한다.

(2) 모든 Node에서 Prepared가 되면 Commit Phase가 수행된다.
그러나 한 Node라도 Abort되면 Commit Phase에서 전 Node에 Rollback을
수행한다.

Using Oracle Database Manager 7.2 & 7.3

Document ID: 11126316.61

Title: Using Oracle Database Manager 7.2 & 7.3

Product: Database Manager

Product Version: 7.2/7.3

Platform: Windows NT, Windows 95

Information Type: Advisory

Impact: Medium

Abstract: This bulletin explains how to perform a few common
Database Manager tasks.

Keywords: database;manager;ORACLE.INI,VS10.INI;ALIAS;configuration

---

Using Oracle Database Manager 7.2 & 7.3

Oracle Database Manager is a remote database administration
tool that allows a database administrator to perform three
key functions:

1.) starting up a database

2.) shutting down a database

3.) checking the status (running or not) of a database.

This bulletin is intended to clarify the steps to perform a few common Database Manager tasks that are not easily understood from the provided documentation.

The following topics are covered in this bulletin:

- Creating a new database alias

- How Database Manager uses the ORACLE.INI file

- Creating a new database configuration

- Finding a configuration file

- Shutting down a database

1. Creating a new database alias
---

1. Click on the Aliases button.
   A dialog box titled Database Connection appears.

2. Fill in the Alias and Database fields.
The Database field must contain a valid Oracle database connect string. A connect string could be a local one such as "2:" or "2:SID", a SQL*Net V1 connect string such as "t:hostname:SID", or a SQL*Net V2 alias.

3. Click on the Add button to save the Alias/Database pair.

2. How Database Manager uses the ORACLE.INI file

The above process is straightforward, however the use of the ORACLE.INI file by the 32-bit Database Manager (the Windows NT and Windows 95 versions) needs clarification. When you click on the Add button in the Database Connection dialog box the following line is created in the [Oracle] section of the ORACLE.INI file:

```
SQLNET DBNAME MyAlias=My_DB_Name
```

where MyAlias and My_DB_Name are the values you typed in the Alias and Database fields. This is an abnormal behavior since all 32-bit Oracle tools must use the Windows NT or the Windows 95 registry to store initialization information. None of the 32-bit tools are affected by this abnormality,but the 16-bit tools may be affected since these tools normally use the ORACLE.INI file for storing initialization information.

If you experience connection problems with your 16-bit tools on a 32-bit platform after a database alias is created via Database Manager, try removing the above line. NOTE: The ORACLE.INI file is normally located in the Windows home directory, (typically C:\WINDOWS.)

3. Creating a new database configuration

Database Manager includes many preset configurations, which are available from the Configuration drop-down list. Clicking on the Configure button brings up the Configure Initialization Parameters dialog box. This is where you can edit a few parameters of the selected configuration. unfortunately, Database Manager does not allow you to create a new configuration and edit all of its parameters in one step. You must save an existing configuration with a different name and edit the .ORA file of the new configuration.

Although this may be awkward, creating a new configuration for your database instance is a very important task because the preset configurations are designed to use the default instance, whose SID is ORCL. A Windows NT configuration, for instance, will try to use the CTL1ORCL.ORA control file, but if an instance tries to use this control file while the default ORCL instance is already running on the same computer, you will get an ORA-600 [2141] error.

1. Choose one of the preset configurations from the Database Manager main window and click on the Configure button to open the Configure Initialization Parameters dialog box .

2. In the dialog box, click on the Save As button.

3. Another dialog box titled "Configuration: Save As" appears.

4. Enter the name of your new configuration and the name of the configuration file which will store your new initialization parameters information. By default, this file will be saved in the

C:\ORANT\VS10\EDITED directory,
where C:\ORANT is the local Oracle home directory
(C:\ORAWIN95 for Windows 95
(C:\ORAWIN for Windows 3.1x, or 16-bit Database Manager
running on Windows NT or Windows 95)

You can click on the Browse button to choose another location for the file.

5. After a new configuration is created, exit Database Manager and restart it. You should also do this after making any change to a configuration file. This step is necessary because Database Manager will use the cached version rather than reading the configuration file each time.

Once the new configuration is created, you can modify the initialization parameters by editing the configuration file using any ASCII text editor, such as Notepad. The initialization parameter normally used is the INIT.ORA file on the computer hosting your Oracle7 database. To use the same parameters in Database Manager, simply copy the INIT.ORA and rename the copy with a new configuration file name, i.e. ALIAS1.ORA.

You may wonder why Database Manager does not automatically use the INIT.ORA file. It is because Database Manager is designed as a remote administration tool; your Oracle7 server's file system is not always accessible to Database Manager running remotely.

4. Finding the configuration file of the selected configuration

---

Database Manager finds a configuration name by looking up the config_name=file_name pair in the [CONFIGS_EDITED] section of the C:\ORANT\VS10\VS10.INI file for Windows NT, C:\ORAWIN95\VS10\VS10.INI for Windows 95, or :\ORAWIN\VS10\VS10.INI for Windows 3.1x or 16-bit

Database Manager running on Windows NT or Windows 95.

5. Shutting down a database

Database Manager always issues a SHUTDOWN IMMEDIATE command rather than SHUTDOWN NORMAL to stop a running database. Database Manager also checks for current user sessions before initiating a shutdown. When there are open sessions, Database Manager displays a warning dialog and waits for your approval to proceed with the shutdown.

WINDOWS NT와 ORACLE7 TUNING GUIDE

Bulletin no : 10315

ORACLE7 RDBMS FOR WINDOWS NT의 구조

NT에서 동작하는 ORACLE7은 Microsoft 32 API를 사용하는 완전한 32bit 프로그램으로서 하드웨어와 완전하게 결합되어 동작한다. NT상의 ORACLE7은 NT의 메모리 모델에 맞도록 SINGLE PROCESS/MULTI THREADED 구조를 갖도록 설계되었다.

UNIX에서 동작하는 ORACLE7과는 달리 SINGLE PROCESS상에서 MULTI THREAD 를 갖기 때문에 단일 주소 공간하에서 메모리를 공유하게 된다. ORACLE7은 NT가 제공하는 선점형 스케쥴링과 여러 개의 CPU를 최대한 활용하는 LOAD BALANCING 기능을 활용하고 있다.

NT상의 ORACLE INSTANCE는 SGA(System Global Area)와 여러개의 BACKGROUND THREADS로 구성된다. SGA는 하나의 ORACLE INSTANCE 를 위한 다양한 정보를 갖고 있다. 최고의 퍼포먼스를 내기 위해서는 SGA는 주어진 메모리 한도 내에서 충분한 크기를 갖어야 한다. 만약 SGA를 제외한 나머지 메모리가 너무 작게 면 PAGING과 SWAPPIG이 계속 일어나게 되므로 퍼포먼스를 크게 저하시키게 되므로 SGA 영역이 과도하게 커서도 안된다. SGA는 다음과 같은 부분들로 이루어져 있다.

DATABAES BUFFER CACHE

데이타 화일에서 읽어들인 데이타 블럭이 저장되는 곳으로 일종의 CACHE 역할을 한다. 디스크에 기록되기 이전의 데이타가 저장되는 버퍼의 기능도 하는데 적절한 크기로 세팅함으로써 DISK I/O를 줄여서 퍼포먼스를 증가시킬 수 있다.

LOG BUFFER AREA

데이타 블럭의 이전 정보와 변경되고 난 후의 정보가 기록되는 곳이다. LOG_BUFFER 파라미터가 이 영역의 크기를 결정한다(BYTE 단위). 이 영역의 크기가 너무 작으면 LOG WRITER는 디스크에 WRITE 작업을 자주 해야 하기 때문에 퍼포먼스가 저하하게 된다. MULTIPROCESSOR MACHINE에서는 LOG_SIMLUTANEOUS_COPIES 파라미터를 CPU 갯수의 2배로 설정함으로써 REDO COPY LATCH에 대한 경합을 줄일 수 있다.

SHARED POOL AREA

SHARED_POOL_SIZE(BYTE 단위) 파라미터로 그 크기가 결정되는데 SHARED SQL/PLSQL 명령을 저장하고 DICTIONARY CACHE 역할을 한다. SQL문의 PARSING 정보가 기록이 되므로 같은 명령은 REPARSING 없이 바로 실행될 수 있다.

CURSORS

커서는 SQL 명령과 관련된 메모리 영역을 지시하는 HANDLE이다. 대부분의 경우 커서는 내부적으로 생

성되는데 직접 사용자가 커서를 생성하여 명령을 수행할 수도 있다.

BACKGROUND THREADS는 다음과 같이 이루어져 있다.

DBWR(DATABASE WRITER) THREAD : DBWR는 BUFFER CACHE의 변경된 데이타 블럭을 디스크로 쓰는 작업을 한다.

LGWR(LOG WRITER) THREAD: LGWR는 REDO LOG ENTRY를 디스크에 기록하는 작업을 한다.

CKPT(CHECKPOINT) THREAD: LOG SWITCH가 일어나 DBWR는 버퍼내의 모든 변경된 블럭을 디스크에 WRITE하며 CKPT는 CONTROL FILE과 DATA FILE을 UPDATE한다. 만약 CKPT가 ENABLE되어 있지 않으면 LGWR가 CKPT의 기능까지 함께 수행한다. 퍼포먼스 증대를 위해서는 CKPT 를 ENABLE 시켜 두는 것이 좋다.

SMON(SYSTEM MONITOR) THREAD: INSTANCE STARTUP시에 INSTANCE RECOVERY를 수행한다.

PMON(PROCESS MONITOR) THREAD: USER PROCESS가 비정상적으로 끊어지는 등 FAILURE가 발생했을 경우 PROCESS RECOVERY를 수행하여 점유한 RESOURCE 를 RELEASE한다.

RECO(RECOVERY) THREAD: NETWORK/SYSTEM FAILURE 등으로 인해 발생하는 분산 트랜잭션의 PENDING 문제를 해결한다.

ARCH(ARCHIVER) THREAD: ONLINE REDO LOG FILE이 재사용 되기 전에 다른 곳으로 복사하는 역할을 한다.

==========

PERFORMANCE BOTTLENECKD을 찾는 방법과 SYSTEM RESOURSE

시스템의 다음과 같은 부분에서 퍼포먼스 저하가 많이 발생하게 된다.

SYSTEM MEMORY,CPU, DISK I/O SUBSYSTEM, NETWORK

위의 사항들을 최적상태로 튜닝함으로써 ORACLE의 퍼포먼스를 증대시킬 수 있다. 메모리는 그 크기가 한정되어 있으므로 PROCESS들은 한정된 메모리를 서로 이용하려고 하기 때문에 PAGING과 SWAPPING이 일어나는 경우가 생긴다. 여러명의 사용자가 동시에 디스크를 읽거나 쓰고자 할 경우 DISK CONTENTION이 발생하게 된다. 마찬가지로 여 개의 프로그램이 CPU를 동시에 사용하고자 하면CPU CONTENTION이 발생한다. PHYSICAL MEMORY가 부족하게되면 CPU TIME의 많은 부분이 PAGING 과 SWAPPING에 할당되게 되므로 퍼포먼스가 저하하게 된다. 한정된 BANDWIDTH에 다다르도록NETWORK TRAFFIC이 증가하게 되면 NETWORK BOTTLENECK이 발생한다.

PERFORMANCE MONITOR를 통한 CPU BOTTLENECK 검사

PERFORMANCE MONITORING은 응용프로그램이 CPU TIME과 MEMORY와 같은 시스템 리소스를 얼마나 사용하는 가를 측정하는 것을 말한다. 따라서 CPU/MEMORY BOTTLENECK을 알아 는데 있어서 매우 유용하게 사용되는 방법이다.

CPU는 프로세스들간에 CONTENTION이 일어나는 가장 중요한 시스템 요소의 하나로 만약 시스템안에 CPU가 여러개가 존재한다면 다른 레벨의 CONTENTION이 일어날 것이다. PERCENTAGE PROCESSOR TIME 카운터는 하나의 PROCESS 내의 모든 THREADS가 명령을 수행하기 위해서 소요된 시간 백분율을 알려준다. 만약 이 값이 85%를초과하게 된다면 CPU에서 BOTTLENECK이 발생하고 있음을 알 수 있다.

PERCENTAGE PRIVILEGED TIME와 PERCENTAGE USER TIME의 합은 PROCESSOR TIME 전체를 알려준다. PERCENTAGE PRIVILEGED TIME은 CPU가 응용프로그램을 위해서 사용된 시간이 아닌 OS 를 위해서 사용된 시간을 나타낸다. 이 시간의 대부분은 디스크에서 데이터를 가져오고 저장하는데 사용된다.

WINDOWS NT는 각각의 SUBSYSTEM을 분리하기 위하여 PROCESS BOUNDARY를 사용한다. 모든 응용프로그램 코드는 사용자의 SUBSYSTEM에 존재하며 % USER TIME COUNTER를 통하여 알아볼 수 있다. 만약 시스템이 효율적으로 작동한다면 대부분의 시간이 USER PROCESS에 할당되어 있을 것이므로 % USER TIME이 매우 높은 을 나타낼 것이다. 하지만 CPU를 많이 사용하는 작업을 수행하고 있고, 이 COUNTER의 값이 높다면 응용프로그램의 튜닝이 요구된다. 이상의 사항들은 각각의 PROCESS와 THREADS 별로 CPU사용현황을 알려주는 WINDOWS PERFORMANCE MONITOR를 통하여 알수 있다.

==========

ORACLE7 SERVER의 퍼포먼스에 영향을 끼치는 사항들

RAW PARTITION

ORACLE7은 I/O 작업이 많은 경우 퍼포먼스를 증가시키기 위하여 RAW PARTITION을 사용할 수 있다. RAW PARTITION은 FILE SYSTEM OVERHEAD가 없으므로 I/O 성능이 우수하다. 그러나 RAW PARTITION은 FILE SYSTEM에 비해서 유연성이 떨어지기 때문에 관리하는데 있어 많은 주의와 노력이 요구된다. 적절한 크기로 설정된 충분한 갯수의 RAW PARTITION을 확보하는 것이 쉬운 작업이 아니기 때문에 RAW PARTITION을 이용하여 DATABASE CONFIGURATION을 잡는 것은 까다로운 문제에 속한다. 하나의 데이타 화일은 하나의 RAW PARTITION에 해당된다. 만약 RAW PARTITION을 사용한다면 RAW PARTITION의 크기를 모두 같게 유지함으로써 I/O BALANCING과 관리상의 유연성을 가질 수 있다.

ORACLE7 SERVER BACKGROUND THREAD PRIORITY 높이기

ORACLE7 SERVER와 BACKGROUND THREAD는 NORMAL PRIORITY CLASS상태에서 수행되도록 세팅되어 있다. 이 CLASS에서는 SCHEDULLER는 PROIRITY를 1에서 15 사이에서 동적으로 변화를 시킨다. 하지만 16~31 사이에 해당되는 REAL-TIME PRIORITY CLASS로는 올라가지 못한다.
ORACLE은 NORMAL PRIORITY CLASS 상태에서 가장 좋은 PERFORMNACE를 보인다는 사실이 알려져 있다. 이 경우는 주로 OLTP 와 같은 성격의 트랜잭션일 때에 대한 것으로 WINDOWS NT의 REGISTRY에 저장된 ORACLE_PRIORITY 파라미터를 변경함으로써 ORACLE7 PROCESS와 관련 THREADS 의 PRIORITY를 변경할 수 있다. 한편 각 BVACKGROUND THREAD의 PRIORITY를 하나씩 별도로 지정할 수도 있다. 하지만 모든 BACKGROUND THREADS의 PRIORITY를 동일한 레벨에 설정하는 것이 가장 바람직하다. 만약 보다 높은 PRIORITY를 갖는 THREAD가 CPU를 요구할 경우 그보다 낮은 PRIORITY를 갖는 THERAD는 CPU를 할당 받지 못하게 된다. ORACLE_PRIORITY 값을 조정하여 안정성에 문제가 발생하는 경우에는 이 값을 다시 원래대로 돌려놓으면 된다. 오라클의 CONFIGURATION PARAMETER 는 REGISTRY 상의 \HKEY_LOCAL_MACHINE\SOFTWARE\ORACLE 에 저장되어 있는데 REGIOSTRY EDITOR인 REGEDT32.EXE 이용하여 변경이 가능하다. REGISTRY 변경 후에는 ORACLE7 의 SERVICE와 DATABASE를 다시 기동하여야만 새로운 값이 적용된다.

OPTIMIZATION

WINDOWS NT SERVER의 최적화는 기본적으로 MAXIMIZE THROUGHPUT FOR FILE SHARING으로 세팅되어 있다. 만약 시스템이 데이타베이스 작업을 위해서 주로 사용된다면 CONTROL PANEL/NETWORK/SERVER에서 MAXIMIZE THROUGHT FOR NETWORK APPLICATIONS 로 변경하면 된다.

WINDOWS NT SYSTEM CACHE는 DISK I/O를 위한 BUFFER를 가지고 있다. 32MB RAM을 가진 시스템에서는 NT는 보통 4MB 를 SYSTEM CACHE로 할당한다. ORACLE7 DATABASE는 독자적인 BUFFERING MECHANISM을 갖고 있기 때문에 DISK에 기록 SYSTEM CACHE를 거치지 않게 된다. REGISTRY의 \HKEY_LOCAL_MACHINE\Control\Session Manager\Memory Management 에서 LargeSystemCache 값을 0으로 변경함으로써 WINDOWS NT OS의 SYSTEM CACHE가 0이 되어 오라클에 맞게 시스템을 최적화 할 수 있다.

SERVICES

ORACLE7 DATABSAE와 관련된 NETWORK LISTENER 들은 WINDOWS NT의 SERVICE로 이루어져 있다. 사용되지 않는 SERVICE 들도 START 되어 있으면 시스템에 부하를 주게 되므로 이들을 STOP 시킴으로써 퍼포먼스를 증가시킬 수 있다.

TUNING DISK I/O

SEQUENTIAL I/O

REDO LOG FILE은 순차적으로 사용된다. 퍼포먼스를 증가시키려면 LOG FILE 들은 I/O가 별로 일어나지 않는 디스크에 두는 것이 좋다. 그리고 ARHIVING THREAD가 기동되면 각각의 LOG FILE이 서로 다른 디스크 상에 있어야지만 LGWR가 LOG FILE을 복사하거나 ARCHIVER THREAD가 CLOSED REDO LOG FILE을 읽을 때 DISK CONTENTION이 일어나지 않는다.

RANDOM I/O

OLTP 또는 DECISION SUPPORT 환경하에서는 I/O는 디스크와 디스크 콘트롤러 상에 걸쳐서 골고루 일어나야 최고의 퍼포먼스를 얻을 수 있다. RAID 0 DISK ARRAY를 사용하는 것이 관리도 편하고 I/O LOAD BALANCING도 잘 이루어진다고 알려져 있다. DISK ARRAY CONTROLLER는 I/O의 LOAD BALANCE를 유지함으로써 디스크에 대한 병렬접근이 가능하게 한다. SQL*DBA의 MONITOR FILE I/O는 ORACLE에 의해서 사용되는 모든 화일과 I/O 활동상황에 관한 통계정보를 제공한다. 한 디스크에 대한 전체 I/O는 READ/S와 WRITE/S의 합으로 계산한다. 각 디스크에 대한 이들 값을 계산한 다음 이를 디스크의 I/O CAPACITY와 비교하여 DISK CONTENTION 상태를 알 수 있다.

과도한 DISK FRAGMENTATION의 점검

화일 크기와 디스크 사용계획을 세울 때 앞으로 확장될 상황을 예상하여 결정하여야 한다. 그렇지 않으면 데이타화일이 불연속적인 EXTENT 들로 구성됨에 따라서 I/O 에 많은 시간이 걸리게 된다. DATABASE FRAGMENTATION은 퍼포먼스를 저하시키므로 EXPORT/IMPORT를 수행함으로써 FRAGMENTATION을 제거하여야 한다. 그리고 SORT_AREA_SIZE 파라미터가 작게 설정된 가운데 이 크기에 넘치는 소팅작업을 실행하게 되면 I/O가 많이 발생하여 퍼포먼스가 저하된다.

메모리 튜닝

SGA 영역의 튜닝

DISK에서 데이타를 읽는 것 보다 메모리에서 읽는 것이 수천, 수만 배 빠르기 때문에 메모리에 대한 튜닝은 매우 중요하다.

SGA 영역은 반드시 PHYSICAL MEMORY 내에 존재해야 한다. 그렇지 않으면 PAGING에 의해서 퍼포먼스가 크게 저하된다. SQLDBA에서 SHOW SGA 명령을 통하여 SGA에 당된 메모리를 확인할 수 있다.
INIT.ORA 에 PRE_PAGE_SGA=TRUE 라고 세팅함으로써 SGA 영역을 항상 PHYSICAL MEMORY 상에 고정시킬 수 있다. 이는 DATABASE의 STARTUP에 더 많은 시간을 요구하지만 DATABASE 사용중에 발생하는 CACHE MISS를 줄여 준다.

BUFFER CACHE의 튜닝

DB_BLOCK_BUFFER 파라미터는 시스템의 퍼포먼스에 큰 영향을 미친다. 여기에는 TABLE,INDEX,ROLLBACK SEGMENTS 의 블럭내용을 보관하고 있으며 HIT RATIO는 몇 퍼센트의 데이타가 이곳에서 사용되었는 지를 알려주는 값이다. 계산방법은 다음과 같다.

HIT RATIO = (LOGICAL READS - PHYSICAL READS) / (LOGICAL READS)

BSTAT/ESTAT 스크립트를 수행함으로써 HIT RATIO을 알아낼 수 있다. 만약 이 값이 0.95 이하라면 DB_BLOCK_BUFFER 값을 늘려주도록 한다.

WINDOWS NT 상의 ORACLE7의 DB_BLOCK_SIZE 값은 기존적으로 2K BYTES이다. 이 값은 WINDOWS NT의 PAGE SIZE와 같은 값이다. 가능한 OS의 PAGE SIZE의 정수배로 DB_BLOCK_SIZE를 결정하는 것이 바람직하다. 만약 응용프로그램이 OLTP 와 같은 작업 위주라면 DB_BLOCK_SIZE를 2K BYTES 로 하도록 하고 응용프로 램이 FULL TABLE SCAN과 같은 작업을 많이 한다면 DB_BLOCK_SIZE를 8K BYTES 정도로 키워주는 것이 퍼포먼스를 크게 증대시키는 요인이 될 수 있다.

사용자 메모리

사용자 프로그램에서 요구하는 메모리는 WINDOWS NT의 PERFORMANCE MONITOR나 PVIEW.EXE(WINDOWS NT RESOURCE KIT에 있음)를 통하여 알아낼 수 있다.
응용프로그램이 실행되기 전과 후의 남아있는 메모리 양을 검사함으로써 응용프로그램에서 사용하는 메모리양을 알아 낼 수 있다. ORACLE의 SHADOW THREADS는 ORACLE WORKING SET에 할당된 메모리를 사용한다. WINDOWS NT의 특성상 WORKING SET의 크기는 최대 256MB를 넘을 수 없도록 제한되어 있다.

WINDOWS NT PAGING

WINDOWS NT상에서 오라클을 운영하기 위해서는 최소 32MB 이상의 메모리를 확보하는 것이 좋다. 만약 SGA 영역에 나무 많은 메모리 가 할당되어서 남 메모리가 너무 적게 되면 PAGING이 발생하게 되며 과도한 PAGING/SWAPPING은 시스템의 퍼포먼스를 크게 저하시키게 된다. 이 문제를 해결하려면 보다 많은 PHYSICAL MEMROY를 갖도록 시스템을 확장하거나 아니면 SGA영역을 줄이거나, 동시 사용자수를 줄여야 한다.

PAGE FAULTS/SEC은 PROCESS의 PAGE FAULT 횟수를 보여준다. PAGE FAULT는 한 PROCESS가 자신의 PHYSICAL MEMORY내의 WORKING SET에 들어있지 않는 PAGE(VIRTUAL MEMORY PAGE)를 요구하는 경우에 발생한다. PAGE FAULT가 발생하더라도 해당 PAGE가 STANDBY LIST상에 있어서 MAIN MEMORY 상에 남아 있는 경우라면 실제의 PAGING은 일어나지 않는다. 잘 튜닝된 시스템에서는 PAGING이 거의 일어나지 않아야 한다.

지원되는 SQL*NET PROTOCOLS

WINDOWS NT 상의 ORACLE에서 지원되는 NETWORK PROTOCOL로는 TCP/IP, NAMED PIPES, SPX/IPX 등이 있다. CLIENT/SERVER 환경하에서는 TCP/IP 를 NETWORK TRANSPORT로 사용하는 경우 가장 좋은 퍼포먼스를 얻을 수 있다고 알려져 있다.

BENCHMARKING TIPS

LOADING DATA

SQL*LOADER를 사용할 때에는 DIRECT PATH OPTION을 사용하도록 한다.

PROGRAMMATIC INTERFACE

OCI(ORACLE CALL INTERFACE)는 PRO*C 에 비해서 ORACLE에 대한 보다 섬세한 제어가 가능하다. 만약 OCI를 사용한다면 DEFERRED LINKING과 PARSING을 사용하면 좋은데, 이 OPTION은 CLIENT/SERVER 환경에서 NETWORK TRIP을 줄여줌으로써 퍼포먼스를 높여주게 된다.

CLIENT APPLICATION의 튜닝

INDEX 생성을 빠르게 하려면 SORT_AREA_SIZE 값을 키워주면 된다.
게다가 SORT 작업과 SORT/MERGE 작업 또한 PHYSICAL MEMORY상에서 일어나게 되므로 DISK I/O를 줄여주는 장점도 있다. DISCRETE_TRANSACTIONS_ENABLED=TRUE로 세팅하면 길이가 짧고 분산 트랜잭션외의 다른 트랜잭션의 퍼포먼스를 증대시켜 준다. 이 OPTION을 사용하면 트랜잭션이 COMMMIT 명령을 만나기 전까지 모든 변경된 데이타가 저장되지 않고 기다리게 된다. 그리고 UNDO INFORMATION이 발생하지 않는다. STORED PROCEDURE, DATABASE TRIGGER, ARRAY PROCESSING 또한 NETWORK TRAFFIC을 줄여주는 등 퍼포먼스 증가에 큰 영향을 줄 수 있다.

DIAGNOSTIC TOOLS

EXPLAIN PLAN과 TKPROF 유틸리티를 사용하여 OPTIMIZER의 EXECUTION PLAN을 점검할 수 있다. COST-BASED OPTIMIZER가 디폴트 이므로 이를 사용하는 것이 좋다. OPTIMIZER는 해당 OBJECT에 관련된 통계정보를 이용하여 최적의 EXCUTION PATH를 선정하여 준다.

ROLLBACK SEGMENTS

OLTP TRANSACTION의 특징은 처리건수는 많으나 TRANSACTION 하나 하나의 크기는 다는 데 있다. 이때에는 각 TRANSACTION에 10K BYTES 정도의 매우 작은 ROLLBACK SEGMENT 를 할당하는 것이 매우 효과적이다. 한편으로는 BENCHMARKING APPLICATION과 같이 큰 데이타를 다루는 경우라면 역시 ROLLBACK SEGMENT도 큰 것을 사용하는 것이 좋다.

새로운 DATABASE 생성

Bulletin no : 10092

---

1. 환경 설정

새로운 데이타베이스를 생성하기 위해서는 기존의 ORACLE HOME 디렉토리를 이용하는 경우 새로운 ORACLE_SID를 사용하여 데이타베이스를 생성한다.

1) ORACLE_SID 결정

새로운 OS ACCOUNT 를 생성하고 기존 ORACLE HOME 디렉토리에 있는 .profile(.cshrc) 화일을 새로운 ACCOUNT의 HOME 디렉토리에 복사하여 ORACLE_SID 부분을 다음과 같이 변경 한다.

예) 기존 환경 : ORACLE_SID=ORACLE7;export ORACLE_SID

새로운 환경 : ORACLE_SID=ORATEST;export ORACLE_SID

2) Parameter 화일 및 config 화일 생성

- 기존의 $ORACLE_HOME/dbs/initORACLE_SID.ora 화일을 이용하여 새로운 ORACLE_SID를 갖는 Parameter 화일을 만든다.

```
$ cd $ORACLE_HOME/dbs
$ cp initORACLE7.ora  initORATEST.ora
$ cp configORACLE7.ora configORATEST.ora
```

- initORATEST.ora 및 configORATEST.ora의 내용중에 ORACLE7 부분을 ORATEST로 수정한다.

2. CREATING DATABASE

1) 기존의 DATABASE를 생성하면서 사용된 SHELL SCRIPT를 복사하여 수정하고 새로운 환경의 DATABASE를 생성

```
$ cd  $ORACLE_HOME/dbs
$ vi  crdb0RATEST.sql       <--   ORACLE7으로 되어있는 부분을
$ vi  crdb20RATEST.sql            ORATEST로 변경한다.
```

2) 수정된 SCRIPT FILE을 수행을 통해서 새로운 데이타베이스 생성.

```
$ cd $ORACLE_HOME/dbs
$ sqldba lmode=y
SQLDBA> connect internal
SQLDBA> startup nomount
   Statements processed.
SQLDBA> @crdb0RATEST
             :
SQLDBA> @crdb20RATEST
             :
```

3) CATALOG VIEW 생성 및 PROCEDURAL OPTION INSTALL

```
$ cd $ORACLE_HOME/rdbms/admin
$ sqldba lmode=y
SQLDBA> connect internal;
```

```
SQLDBA> @catalog
SQLDBA> @catproc
SQLDBA> @catexp
SQLDBA> create user scott identified by tiger
        default tablespace tools
        temporary tablespace temp
        quota unlimited on tools
        quota unlimited on temp;
SQLDBA> grant connect to scott;
```

3. 각 TOOL에 관련된 TABLE 생성

1) SQL*Plus install

```
$ cd  $ORACLE_HOME/sqlplus/admin/help
$ SYSTEM_PASS=system/manager;export SYSTEM_PASS
$ helpins
$ cd  $ORACLE_HOME/sqlplus/admin
$ sqlplus system/manager@pupbld
$ demobld scott/tiger
```

2) SQL*Forms30 install

```
$ cd  $ORACLE_HOME/forms30/admin
$ sqlplus system/manager
  SQL> set compatibility v6
  SQL> @iadtable
  SQL> @iadindex
  SQL> @iadview
  SQL> @iadgrant
```

3) SQL*Menu50 install

```
$ cd  $ORACLE_HOME/menu5/admin
$ sqlplus system/manager
  SQL> set compatibility v6
  SQL> @menutabs
  SQL> @menuvws
  SQL> @menuidxs
  SQL> @menugrts
```

4) SQL*Reportwriter install

```
$ cd $ORACLE_HOME /sqlreport/admin
$ sqlplus system/manager
  SQL> @srw_updpro
  SQL> @srw_prac
  SQL> @srw_icen
  SQL> @srw_newid
  SQL> @srw_cmdn
  SQL> @srw_grnt
```

인포믹스 STATIC PARTITIONING KNOCKOFFS

Bulletin no : 10177

인포믹스는 "데이타 파티셔닝(Data Partitioning)" 혹은 "테이블 파티셔닝(Table Partitioning)"을 최강점으로 내세우며 자사의 소위 "Shared Nothing"기반 아키텍쳐를 오라클에 비해 월등한 것으로 주장합니다. 이와 관련하여 오라클은 인포믹스의 주장대로 "Shared Disk"에 특화되어 있는 것이 아니며 하드웨어의 어떠한 형태의 구성에도 최적화 될 수 있는 "Hybrid Architecture"를 지닌다고 할 수있습니다. 이는 하드웨어가 "Shared Nothing"인 경우는 물론 "Shared Disk"를 포함 "Shared Anything", "Shared Everything"등
모든 형태 configuration 이나 architecture를 오라클은 지원할 수 있다는 뜻입니다. 반대로 인포믹스는 Shared Disk 구조의 지원을 하지 못하며(예, 오라클의 OPS와 같은 기능이 없음), 이를 교묘히 위장하고 있습니다.
게다가 인포믹스의 파티셔닝은 "Static Partitioning"으로서 다음과 같은 한계를 지닙니다.

1. Informix partitioning is done manually.
인포믹스의 병렬질의는 전적으로 파티셔닝에 의존합니다. 또한 이같은 파티셔닝은 전적으로 DB 관리자가 책임지도록 되어있습니다. 따라서 성공적인 병렬질의 수행을 위해서는 DBA가 반드시 데이타베이스내의 데이타 분산 상태를 알아야 함은 물론 사용자의 데이타 액세스 패턴도 알고 있어야만 합니다. 이들 파라미터는 항상 변하는 것이며, 기본 디자인이 잘못되면 성능은 최저가 됩니다.

2. Static partitioning is inflexible.
인포믹스의 병렬처리는 파티션내에서는 불가능합니다. 만약 하나의 파티션내에서 데이타를 많이 가져오는 경우는 파티션내에서 병렬처리가 되는 경우와 비교하여 상당한 성능의 차이를 가져올 수 있습니다.

3. No partitioning Tools
인포믹스 사용환경에서는 위에서 설명한 대로 DBA가 파티셔닝의 디자인을 최적화해야만 하는데도 데이타 distribution 이나 data access path에 관한 정보를 주고 모니터링하는 툴을 제공하고 있질 않습니다.

4. Data distribution changes require repartitioning.
OLTP환경에서는 데이타가 항상 변합니다. 즉, data는 dynamic 반면 인포믹스의 파티셔닝은 static입니다. 즉, 파티션내의 데이타를 어떻게 배치하는가에 따라 즉, 파티션을 어떻게 나누는가에 따라 성능이 영향을 받게 되므로 데이타의 변동에 따라 최적의 성능을 내기 위해서는 그때그때 파티션을 새로 해 주어야 합니다.

5. Schema changes require repartitioning.
데이타베이스를 사용하다 보면 데이타의 structure를 바꾸어 주어야 할 때도 생깁니다. 이때도 역시 인포믹스는 파티셔닝을 다시 해주어야 합니다.

6. Hardware changes require repartitioning.
인포믹스의 파티셔닝 처리는 파티션별 프로세서의 할당으로 이루어집니다. 이는 하드웨어의 spec변동시에도 새로이 파티셔닝을 해주어야만 한다는 것을 의미합니다.

7. Parallelism or Administration ?
인포믹스의 static partitioning은 결국 성능과 관리의 두가지 측면에서 사용자들을 어려움에 빠리게 됩니다.

8. Repartitioning data means downtime.
파티셔닝을 새로 한다는 것은 결국 데이타베이스를 off line해야 된다는 것을 감안하면 dynamic data partitioning과 static(manual) partitioning과의 차이가 분명히 availability로 드러납니다.

(V3.0) BASE TABLES FOR MENU 5.0 BASE TABLES

Bulletin no : 10283

The Sql*Menu base tables store information for all menu applications installed in the database. These are needed to make application library files and check user security. If you do not have these tables, you will not be able to create or save an application.

In sqlplus as SYSTEM, run the following sql statement to see if these base table objects already exist:

```
1  select owner||'    '||rpad(object_name,20,' ')||'    '||object_type
2  from sys.dba_objects
3  where object_name like 'MENU_%'
4  and (object_type='VIEW' or object_type='TABLE')
5* order by owner, object_type, object_name
```

```
SYSTEM    MENU_B_APPL             TABLE
SYSTEM    MENU_B_APPL_GRP         TABLE
SYSTEM    MENU_B_CIRCLE           TABLE
SYSTEM    MENU_B_GROUP            TABLE
SYSTEM    MENU_B_GRP_PRIV         TABLE
SYSTEM    MENU_B_INFO             TABLE
SYSTEM    MENU_B_OBJ_TEXT         TABLE
SYSTEM    MENU_B_OPTION           TABLE
SYSTEM    MENU_B_PARAM            TABLE
SYSTEM    MENU_B_PARM_XREF        TABLE
SYSTEM    MENU_B_PRIV             TABLE
SYSTEM    MENU_B_PROCEDURE        TABLE
SYSTEM    MENU_B_REF              TABLE
SYSTEM    MENU_B_USER             TABLE


SYSTEM    MENU_V_APPL             VIEW
SYSTEM    MENU_V_APPL_GRP         VIEW
SYSTEM    MENU_V_CIRCLE           VIEW
SYSTEM    MENU_V_GROUP            VIEW
SYSTEM    MENU_V_GRP_PRIV         VIEW
SYSTEM    MENU_V_INFO             VIEW
SYSTEM    MENU_V_OBJ_TEXT         VIEW
SYSTEM    MENU_V_OPTION           VIEW
SYSTEM    MENU_V_PARAM            VIEW
SYSTEM    MENU_V_PARM_XREF        VIEW
```

```
SYSTEM     MENU_V_PRIV              VIEW
SYSTEM     MENU_V_PROCEDURE         VIEW
SYSTEM     MENU_V_REF               VIEW
SYSTEM     MENU_V_USER              VIEW

28 rows selected.
```

You need the run the base table scripts if you get no rows or any of the records missing for above query. In addition, if other base objects like synonyms,sequences, index, and grants were not created properly, it is likely that you will encouter problems and will need to reinstall.
In the Oracle RDBMS V7, if the tables have data type of CHAR instead of VARCHAR2 for the base table columns, then you need to reinstall the base tables with VARCHAR2 data type.

These are the scripts you need to run to re-install the base tables.
You should log into sqlplus as user system, and start these scripts in the order specified.
Note that if you have V7 database, then you would need to do

```
SQL> SET COMPATIBILITY V6
```

before you run them to ensure char vs varchar2 difference is resolved.
Also, before running the scripts, you should unload all the applications in the existing tables or have a back up of the database. Note that the scripts will drop all the related existing base objects first if there are any before creating them.

```
menutabs.sql   menuidxs.sql  menuvws.sql  menugrts.sql
```

After the base table objects have been reinstalled correctly, you can put the menu applications back from the unloaded sql files. For locations of these files, you should look into your installation guides or maybe you can do some kind of global file search (like find in unix) to look for these files. In unix, for example, these are usually under $ORACLE_HOME/menu50/admin directory.

(V3.0) FIELD를 이동할 때 자동으로 한글/영문 MODE를 변경하려면?

Bulletin no : 10289

---

1. emulator나 터미널에서 사용하는 한글/영문 toggle escape code값을 알아둔다.
2. oraterm 수정
   o. device attribute : 사용하지 않는 attribute를 선택하여 한글, 영문 toggle하는
                         escape code값을 입력한다.
   o. example : underline bold : 한글변환 code
                 underline inverse : 영문변환 code
   o. mapping attribute : product name, device video name을 확인한다.
   o. example : BOLD-UNDERLINE : underline bold
                 INVERSE-UNDERLINE : underline inverse
   o. save한다.

3. forms30에서 trigger생성 :
                : emp.ename에 field level trigger를 생성한다.
   o. POST-FIELD : display_field(*emp.ename*,*BOLD-UNDERLINE*);
   o. PRE-FIELD : display_field(*emp.ename*,*INVERSE-UNDERLINE*);

      display_field에서 주는 attribute name은
      oraterm의 mapping attribute의 product attribute name을 주어야 한다.

(V3.0) FORMS3.0에서 USER EXIT이란?

Bulletin no : 10246

USER EXIT 이란?

User Exit이란 사용자가 직접 만든 C나 Cobol등의 Subroutine을 Oracle Forms를 사용하면서 특정 목적을 위해 실행되어 질 수 있도록 하는 기능이다.

▶ 이용 형태

1) oracle database이외의 정보를 처리
2) SQL*Forms로부터의 정보를 파일에 기록
3) array data 처리
4) .frm file의 크기 축소등

▶ 작성 방법

1) 현재 디렉토리를 $ORACLE_HOME/forms30/lib 로 위치한다.

2) genxtb scott/tiger
   : scott user에 iapxtb 테이블을 생성한다.

3) iag30 -to genxtb scott/tiger
   : genxtb.inp 로 genxtb.frm file을 생성

4) runform30 -c vt220 genxtb scott/tiger
   : genxtb를 띄워 "User Exit"부분에 새로 만든 userexit 프로그램의 function name을 등록하고 "Type"부분에는 C혹은 COB(대문자로)를 등 한다.

5) genxtb scott/tiger iapxtb.c
   : compile과 link작업이 될 IAPXTB definition을 담고 있는 iapxtb.c 를 생성

6) sqlforms30.mk를 editor로 열어서 IAPXIT부분에 userexit 프로그램의 object file명을 등록
   예) IAPXIT=iapxtb.o
   ---> IAPXIT=iapxtb.o sample5.o
   (여기서 등록한 userexit program은 sample5.o이며 실제 OS상에는 sample5.pc가 존재)

7) make -f sqlforms30.mk sqlforms30x
   : sqlforms30x와 runform30x를 생성

8) 수행시에는 sqlforms30x -c vt220 scott/tiger로 들어가서
USER_EXIT('userexit_name'); 을 기술한다.
예) KEY-NXTFLD trigger에서
user_exit('test'); ---> test는 4)에서 등록한 function name

참고) COBOL user exit 사용자는
$ORACLE_HOME/forms30/lib/cobol 디렉토리에서 수행하며 1 - 5 의
단계는 같고

6) procobxt.mk 화일내의 IAPXIT 부분에 user exit 프로그램의 object
화일명을 등록

7) make -f procobxt.mk sqlforms30x

(V3.0) FORMS30에서의 LOCKING

Bulletin no : 10113

Forms내에서 Locking 사용

* FOR UPDATE Option은 Record가 Select되는 순간 Lock을 걸어 다른 User들이 Change를 못하게 한다.

예) 두 User가 같은 Record를 Query한 상태일때 한 User에 의해

```
SELECT {all database fields in the block}
FROM {table which the block is based on}
WHERE rowid={current row}
FOR UPDATE OF {all database fields in the block}
```

가 수행되었다면, Select되는 Record들은 Lock이 걸려 다른 모든 User들은 Update를 할 수 없게 된다.

* 한 User가 Record를 Update 할때, SQL*Forms는 현재 그 Record가 Lock이 걸려있는지를 확인하기 위해 SELECT...FOR UPDATE문을 발생시키며, 만약 LOCK상태가 아니라면, 그 다음단계로 현재 Database내의 값과 Forms내의 값이 같은지를 비교하여, 같다면 Data를 Change하고, 값이 다르다면 다른 User에 의해 Change된 후 Commit된 것이므로 다시 Query하기를 요구한다.

(FRM-40654:Record changed by another user.Re-query to see change)

* 관련 trigger와 procedure

- ON-LOCK Trigger : Row가 Lock될 때마다 Fire.
- LOCK_RECORD Procedure : 현재 Record에 해당하는 Database의 Row를 Lock.
- ENTER_QUERY(FOR_UPDATE), EXECUTE_QYER(FOR_UPDATE)
  : 실제로 fetch되는 record들만 lock을 건다.
- LAST_RECORD를 만나거나, ENTER_QUERY(ALL_RECORDS,FOR_UPDATE)는 Query에 만족하는 모든 Record를 Lock시킨다.

* PL/SQL을 사 한 Locking 제어

ON-ERROR/MESSAGE trigger로는 Record가 Lock이 걸려 있는지 확인할 수 없기 때문에, PL/SQL PRAGMA EXCEPTION을 사용한다.

```
DECLARE
  dummy CHAR(1);
  could_not_obtain_lock  EXCEPTION;
  PRAGMA EXCEPTION_INIT(could_not_obtain_lock,-54);
BEGIN
   SELECT 'X' INTO dummy
   FROM emp
   WHERE rowid=:emp.rowid
   FOR UPDATE OF empno NOWAIT;
```

```
  MESSAGE('Locked the Current Row Successfully');
EXCEPTION
  WHEN could_not_obtain_lock THEN
    MESSAGE('Record is Already Locked. Please Try Later...');
    RAISE form_trigger_failure;
END;
```

Locking으로 인한 예기치 않은 결과

* 만약 한 Form내에 두개의 Block이 같은 Record를 Query하면서 동시에 Update 하였다면, Block Definition에서 더 높은 Sequence Number를 가진 Block의 Change가 Commit된다.
* 한 Form에 두개의 Database Block A와 B가 있고 Block A에는 PRE-UPDATE Trigger(:A.field:='X')가 있고 Block B에는 PRE-INSERT Trigger(:A.field:='Y') 가 있다. Block A는 Update작업을 Block B는 Insert 작업을 수행 일때 Commit을 하면 Block A는 PRE-UPDATE가 fire가 되어

```
UPDATE A
SET column=:A.field
WHERE rowid={current rowid}  ='X' (the actual value)
```

를 수행하며, SQL*Forms는 다음 Block B로 옮겨가며 PRE-INSERT Trigger를 Fire한다. 따라서 block A에 :A.field를 다시 'Y'로 Update하려고 하면

FRM-40654: Record changed by another user ...

가 나온다.

* NUMBER(5,2)인 Column을 가진 Table X를 Forms에서 그 Field를 Access 해서 10.243을 입력하고 Commit을 한 후 다시 Edit하면 FRM-40654 Error가 생한다. 왜냐하면 Database에 저장된 10.24라는 값과 Forms상의 10.243이 서로 일치하지 않기 때문이다.

* 두 User A와 B가 동시에 같은 Record를 Query하고 있을때, User A가 Modify를 하고 Commit을 하면, User B가 Modify하려면 FRM-40654 Error 가 발생한다. 이때 User A가 다시 그 Record를 Modify하려고 하면 "Attempting To Reserve Record For Update/Delete"가 나타난다. 이는 User B가 Modify를 시도했을 때 이미 SELECT...FOR UPDATE문이 실행되었기 때문이다. B는 A가 Commit을 수행했으므로 Lock을 걸 수 있었으며 Database에 저장된 값과 Forms의 값이 불일치하여 Error를 발생시켰다.

* calling form FORM_A와 called form FORM_B가 있을때, user A가 한 record를 수정한 후 CALL FORM_B를 한 상태에서 FORM_A에서 수정된 그 record를 다시 수정이 가능하다. 이때 FORM_B가 commit을 수행하면 FORM_A가 걸어놓은 lock이 clear됨을 주의해야 한다. 따라서 그 이후에 다른 user가 그 record를 update할 수 있게 된다.

(V4.5,3.0) FRM-40735 ERROR에 관하여

Bulletin no : 10277

I. Introduction

FRM-40735 Error는 ORACLE Forms가 실행중에Unexpected Error를 만나면 발생하는 Error이다. 이 Error는 대부분 다음과 같은 형태로 표시된다.

FRM-40735: Raised An Unhandled Exception

은 Unexpected Event 가 발생하였을 때 수행되고 있던 Trigger명을 나타낸다. Trigger에 기술한 내용내에서 발생한 Unexpected Event를 처리 할 수 없을 때 ORACLE Forms는 사 자에게 FRM-40735 Error를 표시해 주는 것이다.
은 Trigger수행중에 발생한 Unexpected Event 의 이름을 나타내준다. 이것은 PL/SQL에 미리 선언되어있는 Exception일 수도 있고, User가 Define한 Exception이거나, Server나 Application에서 나오는 것일 수도 있다.

FRM-40735 Error를 만나게되면, [Display Error] Key를 눌러 자세한 Error Message를 확인하여야 한다. 여기에서는 이 Error를 발생하는 몇가지의 조건과 해결방법에 관해 살펴본다.

II. FRM-40735와 함께 나타나는 일반적인 FRM-Xxxxx Error

FRM-40831: Truncation Occurred: Value Too Long For Field .

원 인
Item이나 Parameter에 값을 지정할 때, 가능한 길이보다 긴 값을 준 경우에 발생.
조치사항
해당 값을 지정받을 Item이나 Parameter의 Maximum Length를 늘려준다.
Item에 값을 지정하는 경우에는그 Item의 Query Length도 늘려준다.
더 자세한 정보는 PL/SQL에서의 Value_Error Exception부분을 참조한다.

III. FRM-40735와 함께 나타나는 일반적인 PL/SQL Exception들

o. Unhandled Exception이란?

Trigger가 수행중에 PL/SQL에 정의되어있는 Exception 사항을 만나면 Program은 이미 정의되어있는 PL/SQL Exception을 처리하는 Routine으로 떨어지게 된다. 만약 해당 Exception을 처리하는 Rountine이 없다면, 사용자는 FRM-40735 Unhandled Exception Error를 만나게 된다.

o. Value_Error
계산시,변환시, 절단시,제약사항 등에 관한 Error가 발생한다.

원 인

1. Char Type의 Out Parameter를 가진 Stored Procedure를 호출할 때
2. Subprogram을 호 할 때 Parameter간의 Type이 맞지 않을 때
3. Item이나 Parameter에서 갖을 수 있는 값보다 큰 값을 지정하려 할 때
4. ORACLE Forms 4.5.6.X 이전 Version에서 Runtime시 Get_Menu_Item_Property나 Set_Menu_Item_Property 에서 Label Property를 바꾸려 할때

조치사항

1. Out Parameter의 Data Type을 Varchar2로 변경한다.
   만약 단지 한개의 값을 Return하는 Procedure라면, Procedure가 아닌 Function으로 바꾸어 생성하라. Function은 Char Type을 Return할 수 있기 때문이다.
   사용되는 Item이나 Parameter의 Datatype이 Subprogram의 것과 같거나 호환되는 것이어야 한다.
3. 해당 Item이나 Parameter의 Maximum Length를 증가시켜라.
   만약 Item에 값을 지정하는 것이라면, 해당 Item의 Query Length도 증가시켜준다.
   PL/SQL Variable에 값을 지정하는 경우는 Variable의 Size를 증가시켜준다.

Variable Character나 Fixed Character를 정의하는 Format은 다음과 같다.

```
Char[ Maximum Length ]
Varchar2[ Maximum Length ]
```

Maximum Length를 지정하지 않으면 Default값은 1이 지정된다.

Examples

A. PL/SQL Program Unit에서 Variable에 값을 지정하는 경우

```
Declare
    My_String   Varchar2;
Begin
    My_String := 'Abcdefg';
End;
```

위의 경우는 Runtime시에 Value_Error가 발생한다.
다음과 같이 변경해준다.

```
Declare
   My_String   Varchar2(7);
Begin
    My_String := 'Abcdefg';
End;
```

B. PL/SQL Program Unit에서 Item에 값을 지정하는 경우

```
Item Name:       Empno
Maximum Length:  4
Query Length:      4

:Empno := 12345678;
```

위의 경우는 Runtime시에 Value_Error 발생한다.

다음과 같이 변경해준다.

```
Item Name:       Empno
Maximum Length:  8
Query Length:     8
:Empno := 12345678;
```

4. Label Property 를 바꾸기위해 Get_Menu_Item_Property 나 Set_Menu_Item_Property Built-Ins를 꼭 사용해야 한다면 4.5.6.X로 Upgrade해야 한다.

Cursor_Already_Open

원 인

이미 Open되어있는 Cursor를 Explicit하게 다시 Open하려할 때 발생한다.
일반적으로 대부분 Cursor For Loop문에서 Open 문을 사용한 경우 발생한다.

조치사항

Cursor를 Open하는 문장을 기술하기 전에 Cursor가 이미 Open되어 있는지 확인하기 위하여 %ISopen을 사용한다.

Example

```
If  C1%ISopen Then
    ...
Else
   Open C1;
End If;
```

Invalid_Cursor

원 인

Open되어 있지 않은 Cursor를 Close하려고 한 경우 발생한다.

조치사항

1. Close하기전에 Open되어 있는지 확인한다.
2. Cursor를 Close하기전에 이미 Open되어 있는지 확인하기 위하여 %ISopen을 사용한다.

Example

```
If  C1%ISopen Then
    Close C1;
Else
    ...
End If;
```

No_Data_Found

원 인

SELECT... INTO ... 문장에 대한 결과가 없을 때 발생한다.

조치사항

1. SQL*Plus에서 같은 Select문장을 실행하여, 실제 결과가 나오는지 확인한다.
2. 이 Exception을 잡아낼수 있는 Exception 처리문장을 삽입한다.

Not_Logged_On

원 인

DATABASE에 대한 Call을 수행하고자 할때, RDBMS에 현재 Login되어 있지 않은경우에 발생 DATABASE에 Session이 맺어지지 않았거나, Session이 Time Out에 걸렸거나, Connection이 끊긴 경우에도 발생한다.

조치사항

Server에 다시 Logon한다.

Too_Many_Rows

원 인

SELECT... INTO ... 문장에서 Return되는 결과가 한건 이상일때 발생한다.

조치사항

1. 한건의 Record가 Return되기를 바라는 경우라면 SQL문장을 수정한다.
2. 한건 이상의 자료가 Return되기를 바라는 경우라면 Explicit한 Cursor를 사용한다.

IV. FRM-40735와 함께 나타나는 일반적인 ORA-Xxxxx Error

ORA-604: Error Occurred At Recursive SQL Level %S

원 인

ORACLE Forms Application에서 Server에 대하여Cursor를 Open하는 SQL문을 실행하고자 하는데, Server에서는 Cursor를 Open할 수 있는 Maximum Number에 도달한 경우에 발생한다.

조치사항

1. ORA-00604 Error는 DATABASE Parameter중 Open_Cursors의 값을 증가시켜주면 해결된다.

   * Cursor란 무엇인가?

   Cursor는 SQL문장을 실행하기위해 DATABASE이 사용하는 Memory의 영역을 말한다.
   DATABASE에서 갖는 Open_Cursor의 Default값은 50이다.
   Maximum값은 User가 사용하는 System 에 따라 결정된다.
   User의 환경에 따라 Open_Cursor의 적정값을 설정할 필요 있다.

주의 : Open_Cursors는 ORACLE7 Server DATABASE의 Parameter이지 ORACLE Forms에만 해당하는 Parameter가 아니다.

2. Open_Cursors의 수를 증가시키는 것이 용이하지 않을 때에는 ORACLE Forms Application에서 사용하는 Cursor의 수를 줄인다.

   * ORACLE Forms Application에서 사용하는 Cursor수의 절감

가능하다면 Implicit Cursor대신에 Explicit Cursor를 사용한다. 이것은 ORACLE Forms가 종료하면서 Cursor가 Close될 때 Memeory Resource가 Release되므로 개발자가 좀더 많이 관리할 수 있게 해준다.

Example
다음의 문장을 Implicit한 Cursor를 Open한다.

```
SELECT  Ename
  FROM  Emp
WHERE  Empno = :Empno;
```

만약 Form에서 Cursor Mode Property가 Open으로 되어있다면,이 Cursor는 이 SQL문을 사용한 Form이 Close될때까지 남아있게 된다.

User는 위의 SQL문을 Explicit한 Cursor를 사용하도록 문장을 수정할 수 있다.

```
Declare
   Cursor My_Cursor Is
      SELECT Ename
       FROM Emp
      WHERE Empno = :Empno;
Begin
  Open My_Cursor;
  Fetch My_Cursor INTO :Ename;
  Close My_Cursor;
End;
```

이 SQL문에서는 Close문을 수행하면 Cursor는 Close된다.

기본적으로 ORACLE Forms는 DATABASE에 각각의 SQL문마다 Implicit한 Cursor를 지정한다. 이것은 Processing을 향상시키기는 하지만, Runforms Session시에 ORACLE Forms가 더 많은 Cursor를 사용하게 한다. Runform Parameter 중Optimizetp 를 "No"로 설정하면, ORACLE Forms는 INSERT, UPDATE, DELETE, SELECT FOR UPDATE Statements에 사용한 Cursor를 재사용한다. 이렇게 사용하면 Application에서 Cursor의 수를 적게 사용하게 되지만, Performance는 저하된다.

ORA-942: Table Or View Does Not Exist

원 인

해당 Object가 없거나, 잘못 Reference했거나,.해당 Object에 대한 권한이 없을 때 발생

조치사항

1. SQL*Plus로 Login하여 해당 Table이나 View의 Description을 확인한다.
만약 ORA-942이 다시 나온다면, 해당 Object에 대한 권한을 부여 받았는지 확인한다.
2. Object명을 다음과 같이 기술한다.

```
Schema_Name.Object_Name.
```

3. Object에 대한 Synonym을 생성하고, 해당 User에 적절한 권한을 준다.

ORA-1000: Maximum Number Of Open Cursors Exceeded

원 인

ORA-604를 참조

조치사항

ORA-604 의 조치사항 참조

ORA-2289: Sequence Does Not Exist

원 인

해당 Sequence가 없거나, 작업할 수 있는 권한이 없는 경우에 발생한다.

조치사항

1. 정확한Sequence명을 기술한다. 다음의 Data Dictionary Views 에서 가능한 Sequences를 확인한다.

   User_Sequences

   Dba_Sequences

   All_Sequences

2. 해당 Sequence에 대한 Select권한이 있는지 확인한다. 해당 Sequence로 부터 다음 번호를 받아오려면 Select권한이 반드시 있어야 하기 때문이다. 이 사항을 확인하기 위해서 SQL*Plus에서 다음 문장을 실행한다.

```
SQL> SELECT Sequence_Name, Sequence_Owner  FROM  All_Sequences
```

3. 해당 Sequence에 대해 Public Synonym을 생성한다. 만약 해당 Sequence가 다른 User가 생성한 것이라면 반드시 다음과 같이 기술하여 사용하여야 한다

```
SELECT .Nextval  INTO :Block.Field_Name FROM Dual;
```

Sequence의 Owner는 다른 User들이 Schema_Name을 기술하지 않고 해당 Sequence를 사용할 수 있도록 다음과 같이 해당 Sequence에 대해 Synonym을 생성할 수 있다.

```
SQL> Create Public Synonym   FOR  Schema_Name.Sequence_Name>
```

다른 User들은 위에서 생성한 Synonym을 다음과 같이 기술하여 사용할 수 있다.

```
SELECT.Nextval INTO :Block.Field_Name FROM Dual;
```

4. 만약 Sequence가 Remote DATABASE에 있는 것이라면, 다음과 같은 방법으로 사용할 수 있다.

1> Remote Sequence에 대해 DATABASE Link명을 감출 수 있는 Synonym을 생성한다.

```
SQL> CONNECT System/Manager
SQL> CREATE Public Synonym Remote_Seq_Syn
  2  FOR  Remote_Seq@Db_Link;
ORACLE Forms Application에서는 DATABASE Link명 없이 Synonym을 사용한다.
SELECT.Nextval FROM Dual;
```

2>ORACLE Forms에서 Sequence와 Link명을 주어 사용한다.

```
SELECT .Nextval FROM Dual@Db_Link
```

ORA-106552: DDE Exception Dmlerr_Dataacktimeout

원 인

DDE Data Request가 Timed Out에 걸린 경우에 발생한다.

조치사항

1. DDE.Execute Built-In에서 사용하는 Timeout Parameter의 값을 증가시켜준다.
2. DDE.Dmlerr_Dataacktimeout Exception에 대한 처리문장을 기술해 준다.

ORA-106556: DDE Exception Dmlerr_No_Conv_Established

원 인

DDE Conversion을 하려는 시도가 실패한 경우 발생한다

조치사항

1. DDE.Initiate 에서의 Service와 Topic Name 을 Checking한다.
   Excel의 경우
     Service Name : *Excel*
     Topic Name : Spreadsheet Name 예를 들어 'C:\Test.Xls'
   Word의 경우
     Service Name : 'Winword'
     Topic Name : Document Name 예를 들어 'C:\Test.Doc'

2. DDE.App_Begin 에서 Open하고자 하는 File명을 Checking한다.
   Excel의 경우

```
Appid := DDE.App_Begin('C:\MsoFFIce\Excel\Excel.Exe C:\Abc.Xls',
                                                    DDE.App_Mode_Normal)
```

   Word의 경우

```
Appid := DDE.App_Begin('C:\MsoFFIce\Winword\Winword.Exe
                                        C:\Test.Doc', DDE.App_Mode_Normal)
```

ORA-304500: Failed ORA_FFI Call

원 인

지정한 DLL File의 Fuction Call을 할 수 없을 경우에 발생한다.

조치사항

1. 해당 DLL File이 ORACLE_HOME 이나 DOS Path에 포함되어 있는지 확인한다.
2. 32-Bit Application을 사용할 경우 32-Bit DLL인지 확인한다. 마찬가지로, 16-Bit Applicaiotn에서는 16-Bit DLL인지 확인한다.

V. General Information

==========

FRM-40735 Error가 발생하면, User는 때로는 해당 Exception을 잡아서 처리 할 수 있는 문장을 기술하여야 한다. 이 Exception Handler는 Error가 언제 발생하든지 같은 처리를 수행할 수 있다. ORA_FFI 와DDE에 관한 Package를 사용할 때는 Package안에 해당 Exception에 관한 Handler를 기술해 두는 것이 좋다.

(V4.5,5.X) SUPPORT FOR VBX AND OCX CONTROLS

Bulletin no : 10404

** Summary of Current and Planned Support : 더 자세한 사항은 Release Notes를 참조바람.

1. Release 1.3 Win95/NT

OCX - Limited OCX support for 32 bit OCX controls (see below)
VBX - No VBX support

2. Release 1.3 Win 3.1

OCX - Limited OCX support for 16 bit OCX controls (see below)
VBX - Continued VBX support with bug fixes and increased stability

3. Release 2.0 Win95/NT

OCX - Full support including events for 32-bit OCX controls. Automatic generation of PL/SQL "wrapper" packages for methods and events. Ability to set
properties at design time.
VBX - Open, edit and save a form containing a VBX. No support for modifying an existing VBX item or creating a new VBX item. NOTE: You will not be
able to run a form containing a vbx under a 32 bit runtime. This support is provided only to allow you to edit an existing form to compile and run it under a 16
bit runtime.

4. Release 2.0 Win 3.1

NOTE: current plans for Win 3.1 are for a runtime only (no designer) in 2.0. The mechanism for creating a 16 bit OCX in the 32 bit designer is still under
review.
OCX - Support including methods for 16-bit OCX controls. May exclude ability to set properties at design time (as in 1.3).
VBX - Run an upgraded form containing a VBX. No support for modifying an existing VBX item or creating a new one.

5. Corrections to the Release Notes

The release notes for 1.3 contain some errors:

4.16.1. OCX Support

We have limited OCX support in Developer/2000 Release 1.3. This will be expanded to full support in Developer/2000 Release 2.0. In Forms 4.5.7 you
will be able to:

o Insert an OCX into an OLE container

o Bring up the OCX's property sheet to set properties

by invoking the Edit action from the right-mouse-menu

>>>This is incorrect; although can attempt to set properties in the designer, they are not retained at runtime! See the White Paper for details on
setting properties.

o Calling the OLE-style automation interfaces at runtime

Until Release 2.0, you will not be able to :

o Show and set the OCX properties in the property palette

>>>In 2.0 you will (probably) set the properties through the OCX's own dialog, not the property palette

o Programmatically get and set properties at runtime

>>>This is incorrect; in fact you MUST get and set properties at runtime in 1.3. See the White Paper and the sample.

o Receive events at runtime

>>>See the White Paper and the sample for a workaround for this restriction.

o Call OCX-style methods.

>>>In many cases this is not a problem because the OCX exposes an OLE automation interface

(V4.5.7 LATER) SQL*NET V2 SUPPORT

Bulletin no : 10405

From Release 1.3.2 of Developer/2000, we are only including SQL*Net V2 on the product CD as a coporate policy.
Note that Release 1.3.2 supports RDBMS 7.3 which doesn't support SQL*Net V1.

(V4.5) CDE2 AND RDBMS

Bulletin no : 10008

Does anyone out there know that what is the latest rdbms version certified with the cde2 tools? possible hardware will be hp/ux or sun solaris 2.4...

There's a lot of confusion about this issue. Let me try to shed some light on the subject. There are actually two separate issues involved in this question.
They are:

(a) Can the CDE tools connect to a particular version of the database?

(b) Can the CDE tools co-exist in the same $ORACLE_HOME as a particular version of the database?

Note that these are *not* the same question.

The answer to question (a) is almost always "yes". The CDE tools can connect to *any* Oracle 7.X database. If the proper tables have been installed in the database, the CDE tools can save forms/reports/documents in any 7.X database without any problems.

The answer to question (b) is "it depends".

On UNIX, the most current version of the CDE tools is the CDE1 Production 2 bundle, with Forms 4.0.12 and Reports 2.0.13. On most ports, this bundle was built using the Toolkit and Core libraries from the 7.0.16 RDBMS version. This means that the only RDBMS version with which the CDE tools are *guaranteed* to co-exist in the same $ORACLE_HOME is the 7.0.16 RDBMS version.

Now, some, *BUT NOT ALL*, of the product lines have certified their particular release of CDE with later versions of the RDBMS. For example, the Sun SBU has said that they support installing CDE1P2 in the same $ORACLE_HOME as the 7.1.3 RDBMS, but the HP SBU has specifically said that this combination is *not* supported. I know of no SBU who has said that you can install CDE1P2 and the 7.1.4 RDBMS in the same $ORACLE_HOME.

My personal recommendation is that you *never* install the CDE tools and the RDBMS in the same $ORACLE_HOME. Instead, install them in separate $ORACLE_HOME's, and set the $TWO_TASK environment variable to control which database you are connecting to. This strategy has the following benefits:

(1) There is no performance penalty, since UNIX always uses a two-task architecture anyway.

(2) It is possible to upgrade the CDE tools version without being forced to upgrade the RDBMS at the same time.
(3) It is possible to upgrade the RDBMS version beyond the latest version supported by the CDE tools. (For example, using this strategy it's no problem to have CDE1P2 and RDBMS V7.1.4 on the same machine.)
(4) It is possible to upgrade the RDBMS version without being forced to upgrade the CDE tools at the same time.
(5) This strategy completely avoids problems in which applying a RDBMS patch will cause problems in the CDE tools, and vice versa.

Anyway, I hope this clears things up.

-p

(V4.5) FORMS4.5 NEW FEATURE

Bulletin no : 10013

New Features

The following tables summarize new features in Oracle Forms 4.5 in six areas: Designer interface, object and code reusability, GUI control, PL/SQL interface, database integration, and MS Windows extensibility.

Designer Interface

Object Navigator The Object Navigator presents a hierarchical view of an entire Oracle Forms design session, showing all form, menu, and library modules in an easy-to-manipulate list For more information, refer to the Oracle Forms Developer's Guide, Ch. 2, "Working with the Design Tools."

Property Palette The Property Palette displays an object's properties in a list format, and replaces the Release 4.0 property sheets. For more information, refer to the Oracle Forms Developer's Guide, Ch. 2, "Working with the Design Tools."

Graphical Layout Editor The graphical Layout Editor lets you size, position, and edit stacked canvas-views in the same Layout Editor window with the content view. For more information on editing windows and canvas-views, refer to the Oracle Forms Developer's Guide, Ch. 11, "Working with Windows and Canvas-views."

Graphical Menu Editor The graphical Menu Editor allows you to create pull -down menus quickly, working from a visual display of your menu. For information, refer to the Oracle Forms Developer's Guide, Ch. 22, "Basic Menu Design."

PL/SQL Editor The PL/SQL Editor provides a common editing environment for both triggers and PL/SQL subprograms. The PL/SQL Editor lets you enter and compile PL/SQL code and view and resolve any errors. A global search and replace function allows you to find all occurrences of any string in trigger or program unit code. For more information, refer to the Oracle Forms Developer's Guide, Ch. 2, "Working with the Design Tools."

Object and Code Reusability

Copy/Paste Property The Copy Property command lets you copy a list of property properties from one object or a group of selected objects. To apply these properties to another object or group of objects, use the Paste Property command.
For more information, refer to the Oracle Forms Developer's Guide, Ch. 4, "Setting Object Properties."

Property You can save a collection of current property settings Classes as a named property class, then base objects on the class, making local overrides as necessary. For more information, refer to the Oracle Forms Developer's Guide, Ch. 5, "Reusing Objects and Code."

Object Groups Object groups allow you to share common objects among many modules. You define an object group by naming a group of functionally related objects. For example, a calendar function might include a window, a content canvas, several blocks and their items, as well as related triggers. By creating an object group, you can include this calendar window in every form in your application with a single drag-and -drop operation. For more information, refer to the Oracle Forms Developer's Guide, Ch. 5, "Reusing Objects and Code."

Copy/The drag-and-drop Copy/Reference function now Reference allows referencing modules in the file system as well as Modules in the database. For more information, refer to the the File Oracle Forms Developer's Guide, Ch. 5, "Reusing Objects and Code."

Source You can use Oracle Forms 4.5 with PVCS (Polytron Control Version Control System) for source control. For more information, refer to online Help for the File->Administration command.

GUI Control

Responding You may want to write code that responds to mouse to Mouse events, such as a single or double click. Now you can Events determine where a mouse event occurs, whether it is a single or double click, and whether the mouse has entered or left an item, canvas, or window. For information, refer to Oracle Forms Advanced Techniques, Ch. 6, "Responding to Mouse Events."

Dynamic List Dynamic list items allow you to programmatically Items change list item elements, including deriving values and labels from the database at runtime. For more information, refer to the Oracle Forms Developer's Guide, Ch. 9, "Defining Items."

Combo Box Combo box list items, which combine the functionality Style List of a text item and a poplist, are supported in Items Version 4.5. For more information, refer to the Oracle Forms Developer's Guide, Ch. 9, "Defining Items."

Toolbar You can create a horizontal, vertical, or MDI window Support toolbar as a canvas and assign it to a window. For more information on editing stacked canvas-views, refer to the Oracle Forms Developer's Guide, Ch. 11, "Working with Windows and Canvas-views." Set Cursor In Oracle Forms 4.5, you can programmatically control Style the cursor shape using the new Cursor_Style property. For more information, refer to the Oracle Forms Developer's Guide, Ch. 6, "Basic Form Design."

Dynamic In Oracle Forms 4.5, you can dynamically change Format Mask format masks using the Format_Mask property Support with the SET_ITEM_PROPERTY and GET_ITEM_PROPERTY built-ins. For more information, refer to the Oracle Forms Reference Manual, Vol.2, Ch. 5, "Properties."

Maximize You can now programmatically maximize or minimize Window a window by calling the SET_WINDOW_PROPERTY built-in, using the new Window_State property, with "state" settings of MINIMIZED, MAXIMIZED, or NORMAL.
For more information, refer to the Oracle Forms Reference Manual, Vol. 2, Ch. 5, "Properties."

Display You can specify for each form which window should Console display the console (message line and status line), using the form-level property Console Window.
For more information, refer to the Oracle Forms Reference Manual, Vol. 2, Ch. 5, "Properties."

No Root A form is not required to have a root window in Oracle Window Forms 4.5.
Required For more information on editing windows, refer to the Oracle Forms Developer's Guide, Ch. 11, "Working with Windows and Canvas-views."

Display Single You can now independently control the number of and Multiple instances of each item displayed in a multi-record Item Instances block, using the new Items Displayed property. in a Block For more information, refer to the Oracle Forms Reference Manual, Vol. 2, Ch. 5, "Properties."

Base Multiple You can now have more than one item corresponding Items on the to the same base table column, using the new item Same Column property, Mirror Item. For more information, refer to the Oracle Forms Reference Manual, Vol. 2, Ch. 5, "Properties."

Alter Default You can alter the default navigation sequence at the Navigation item level using the new Next Navigation Item and Previous Navigation Item properties. For more information, refer to the Oracle Forms Reference Manual, Vol. 2, Ch. 5, "Properties."

Display You can create a named visual attribute and apply it to Current Record the current row using the Current Record Attribute in Color property.
For more information, refer to the Oracle Forms Reference Manual, Vol. 2, Ch. 5, "Properties."

PL/SQL Interface

Source-Level The source-level debugger allows you to debug Debugger PL/SQL code at runtime, including setting breakpoints, single stepping, examining and depositing values, and examining the call stack. For more information, refer to the Oracle Forms Developer's Guide, Ch. 21, "Debugging Applications."

Editing Through Oracle Forms 4.5, you have access to a new Environment editing facility for creating and modifying stored for Stored procedures and database triggers.
Procedures For information, refer to Oracle Forms Advanced Techniques, Ch. 2, "Stored Procedures and Database Triggers."

Incremental Compiling In Oracle Forms 4.5, the Generate utility will recompile only code which has been changed. You can use a new COMPILE_ALL argument to override this default functionality when you make a change you know will affect many of your program units. For more information, refer to the Oracle Forms Reference Manual, Vol. 1, Ch. 1, "Components and Options."

Trigger Execution Style Using a new Execution Style trigger property, you will be able to specify whether you want the lowest-level trigger to override any like-named triggers at higher levels (Oracle Forms 4.0 default behavior), or if you want the lower-level trigger to fire before or after higher-level triggers fire.
For more information, refer to the Oracle Forms Reference Manual, Vol. 2, Ch. 5, "Properties."

Calling Foreign Functions In Oracle Forms 4.5, you can call foreign functions (functions written in any 3GL programming language, such as C) from a PL/SQL interface.
For more information, refer to Oracle Forms Advanced Techniques, Ch. 13, "PL/SQL Interface to Foreign Functions."

PECS The Performance Event Collection Services (PECS) is a tool you can use to locate performance problems, and to measure resource usage and test coverage.
For more information, refer to Oracle Forms Advanced Techniques, App. C, "Analyzing Performance with PECS."

Database Integration

Multiple Oracle Forms applications can now open multiple Open Forms forms simultaneously. Complex applications can be developed as a collection of modular forms, which may cooperate at runtime. Any number of these forms may be active at once on the desktop.
For information, refer to Oracle Forms Advanced Techniques, Ch. 5, "Multiple-Form Applications."

Multiple Sessions In Oracle Forms 4.5, you can establish a separate transaction environment for each form in a Connect multiple-form application, so an independent form can issue commits even if other forms have pending transactions.
For information, refer to Oracle Forms Advanced Techniques, Ch. 5, "Multiple-Form Applications."

Application Security Oracle Forms 4.5 allows you to implement menu security using the Oracle7 Server application security Using Oracle7 roles stored in the Oracle7 data dictionary.
Roles For information, refer to the Oracle Forms Developer's Guide, Ch. 23, "Customizing Menus."

Insert/Update Changed Columns Only You can use the new Update Changed Columns Only property to save on network traffic by specifying that only changed columns should be included in UPDATE statements.

For more information, refer to the Oracle Forms Reference Manual, Vol. 2, Ch. 5, "Properties."

MS Windows Extensibility

VBX Controls VBX controls, such as knobs and sliders, are available from several third-party sources and can now be included in Oracle Forms applications as custom items. For more information, refer to Oracle Forms Advanced Techniques, Ch. 11, "VBX Controls."

OLE2 Object Linking and Embedding (OLE) allows MS Container Windows applications to create compound documents, Support documents that include objects from other Windows applications. Oracle Forms 4.5 provides OLE container support, allowing you to, for instance, display an Excel spreadsheet in an Oracle Forms application. For more information, refer to Oracle Forms Advanced Techniques, Ch. 10, "Object Linking and Embedding (OLE)."

OLE2 PL/SQL built-ins can interrogate and call the methods Automation of any registered OLE2 server that supports automation. For more information, refer to Oracle Forms Advanced Techniques, Ch. 10, "Object Linking and Embedding (OLE)."

SDK Support If you use the MS Windows Software Developer's Kit (SDK), you can use the new Window_Handle property to programmatically obtain the Windows handle for any Oracle Forms object so you can refer to it in user exit code. For more information, refer to the Oracle Forms Reference Manual, Vol. 2, Ch. 5, "Properties."

64K Limit In Oracle Forms 4.5, the 64K limit on record size has Lifted been removed, which particularly affected applications that displayed image files. There is now essentially no limit on record size for MS Windows.

(V4.5) CDE2 의 ON-LOCK 기능

Bulletin no : 10048

Subject: CDE2의 ON-LOCK기능

현재 PowerBuilder와 FORMS간의 OLTP비교설명시 다음 사항을 추가 합니다.

1. 동시에 2명의 사용자가 같은 record를 update할때.

forms45 - 사용자A의 commit여부에 관계없이 수정하였을때,
"이미 타 사용자에 의해 수정되었다"거나,
"타 사용자에의해 수정lock이 걸려 있다"는 message제공.

PB40 - 타 사용자의 작업여부에 관계없이 최종 사용자의 update된 값이
무조건 입력되어 자료의 오류가 발생한다.

PowerBuiler PB사용자들의 이견.

- forms45처럼 lock이 발생하면, lock이 필요없는 업무에서는 오히려 문제가 많다.(화면에서만 수정하고 commit하지 않은 상태에서 아무런 작업을 하지 않고 있으면, 같은 레코드를 사용하는 타 모든 사용자는 lock이 발생하여 업을 할수 없다).

해결책.

- forms45의 form-level trigger에 on-lock을 이용하여 내용을 NULL;만 입력하여 작업하면, PowerBuiler와 같은 lock이 발생하지 않는 작업처리를 할 수 있다.

- 그러므로 , 기본적으로 동시사용자가 많은 경우 forms45처 lock 기능을 제공하며, 필요시 lock을 제거하여 programming할수 있다고 유도.

(V4.5) DEVELOPER/2000 REGENERATE FORM

Bulletin no : 10057

알고 계시겠지만, 이전 forms version에서 developer/2000 - Forms로 upgrade하여 program를 수행시킬경우 -- old version이므로 re-gen하여 다시 run하시요--라는 message를 종종 받게 됩니다.
또한 수많은(수백copy 이상 program)을 하나하나씩 다시 gen 하기 해서는 보통 많은 시간이 소비되는것이 아닙니다.

이럴때 방법은 다음과 같습니다.

1. 모든 program을 하나의 directory로 모은다.
2. 모든 프로그램의 이름을 임의의 table을 생성하여 db내에 저장한다.
   c:\dir *.fmb > filename.txt
   이 filename.txt 의 내용을 sqlloader를 이용하여 임의의 table에 load.

3. forms프로그램에서 이 program name이 저장되어 있는 table를 query한다.
4. 이때 query시 post-query 트리거에....

```
declare
 form_name varchar2(100);

begin
 first_record;
 loop

  host('f40gen c:\...\form_name un/pw .....');
  if :system.last_record='TRUE' then
    exit;
  end if;
  next_record;
 end loop;
end;
---------------
```

(V4.5) FORMS4.5간의 PARAMETER PASSING

Bulletin no : 10122

CALLED FORM에서의 SETTING

1. PARAMETER를 생성한다.
   (1) Navigator에서 Parameters를 선택
   (2) Navigator * Create
   (3) Parameter의 Property
      * Data Type : CHAR, NUMBER, DATE 만 가능
      * Default : 해당 Parameter의 Default Value
      * Length
        - CHAR 만 입력가능 : 64K까지 가능
        - NUMBER : Default 23 Bytes
        - DATE : Default 7 Bytes
      * Name : Called Form에서 사용될 이름

2. CALLED 된 상태에서의 PARAMETER값 REFERENCE
   (1) 직접적인 Reference
      * :block.item := :PARAMETER.parameter_name;
        - PARAMETER로 기술된 것은 Reserved Word이다
   (2) 간접적인 Reference
      * NAME_IN,COPY등의 Built-in Procedure를 사용한다.
        - Name_In('PARAMETER.parameter_name');
        - Copy('SMITH','PARAMETER.parameter_name');
   (3) 다음의 Object들에 대한 Property 정의시에 해당 Parameter를 사용할 수 있다.
      * Base Table Block의 WHERE나 ORDER BY절
        - Property화면이나 SET_BLOCK_PROPERTY에서 가능
      * LOV에서의 Return Item Property
      * Item의 Default Value Property
      * Text Item의 Range Low Value혹은 Range High Value Property

3. Initial Values Of Parameters in a Called Form
   (1) Calling Form 에서 Parameter List 없이 부르면
      * Called Form 에서 Design시에 지정된 Default값이 정의된다.
        지정된 Defult값이 없으면 NULL이 된다.
   (2) Calling Form에서 Default Parameter List를 Passing하면
      * Calling Form 의 Parameter들이 갖고있는 현재값을 받는다.
   (3) Calling Form에서 Program에서 생성한 Parameter List를 Passing하면
      * ADD_PARAMETER Built-in에서 지정한 값을 받는다.

4. Called Form의 Parameter들은 Design시에 정의가 되어 있어야 한다.
(1) 정의되지 않은 Parameter를 Calling Form에서 지정하면
* FRM-47023 : No such parameter named exists in
Error가 발생한다.
(2) Called Form에 지정된 Parameter들은 Pass되는 Parameter들과 Data Type, Length가 Match되어야 한다. 그렇지않으면
* FRM-47024:Parameter type does not match definition in
Error가 발생한다.

* CALLING FORM에서의 SETTING

Called Form으로의 Parameter Passing은 Parameter List의 Passing으로 가능하다. Call_Form,Open_Form,New_Form등의 Built-In Procedure를 이용하는 Called Form으로의 Parameter Passing이 가능하다. 다른 Tool로는 Run_Product Built-in을 사용한다.
a) Parameter 값은 여러개의 Forms간에 호환이되지 않는다.
b) 만약 Calling Form과 Called Form에서 같은 이름의 Parameter는 생성하여도 서로에게 영향을 미치지 않는다.
c) 만약 Called Form에서도 영향을 주는 가시적인방법을 원한다면 Global Variable을 사용하여야 한다.
d) Global Variable은 ERASE Built-in을 사용하여 Explicit 하게 삭제하거나, Session이 끝나기전에는 Called Form에 Active상태로 남아있게 된다.

1. Creating Parameter List
(1) Default Parameter List
* 각각의 Form은 Default라는 Built-in Parameter List를 포함하고 있다.
- Default Parameter List는 Form Design시에 생성한 모든 Parameter를 포함한다.
- 예를 들어 Form A에서 Design시에 Patameters를 p1,p2, p3 생성하였을 때, 그것들은 Form A의 Default Parameter List에 자동으로 등록된다.
* 다른 어떤 Parameter List와 마찬가지로 Default Parameter List도 OPEN_FORM,CALL_FORM,NEW_FORM Built-in을 통하여 Passing가능하다.

예)
```
Declare
   the_list Paramlist;
Begin
   the_list := Get_Parameter_List('default');
   Open_Form('Form_B',ACTIVE,NO_SESSION,'default');
End;
```

(2) Programmatic Parameter List
* Calling Form에 Parameters를 갖지않은 경우에는 Trigger나 User-named Routine에서 다음사항들을 리하여 Parameter를 Passing한다.
* Parameter Creation 및 Passing
- Create_Parameter_List
a) 이 Function은 생성된 Parameter List의 ID를 Return한다.
b) ID의 Type은 ParamList로 지정한다.
c) 처음 생성된 Parameter List는 Parameter이름과 Value를 가질 Construct에 불과하다.
d) 이 빈 Construct에 Parameter를 지정하기 위해서는 ADD_PARAMETER Built-in을 사용한다.

- Add_Parameter

a) Parameter List에 Parameter를 Add한다.

b) 지정하는 Parameter명을 Called Form의 Parameters에 정의된 이름을 사용하여야 한다.

c) 지정할 Parameter의 갯수만큼 Call한다.

* Parameter의 Type

- Text Parameters

a) CHAR String

b) CALL_FORM,OPEN_FORM,NEW_FORM Built-in에 의해 Invoke된다.

- Data Parameters

a) Called된 Product로 Pass될 Data Parameter의 값은 현재사용중인 Form에 정의된 Record Group의 이름이어야 한다. (Record Group은 Query에 의해 생성된 Record들이나 Program상에 Assign된 Record들의 Data Structure를 말한다.)

* Reference Manual참조

b) Data Parameter들은 RUN_PRODUCT Built-in을 통하여 다른 Product (Reports나 Graphics)에 자료를 Pass한다.Forms로도 Pass가 가능하다.

* Parameter List의 Manipulate Built-in Subprogram들

(Oracle Forms Reference Manual 참조)

- ADD_PARAMETER
- CREATE_PARAMETER_LIST
- DELETE_PARAMETER
- DESTROY_PARAMETER_LIST
- GET_PARAMETER_ATTR
- GET_PARAMETER_LIST
- SET_PARAMETER_ATTR

(V4.5) FORMS4.5에서 REPORT2.5로의 DATA PARAMETER PASSING

Bulletin no : 10124

RECORD GROUP 이란?

: Record Group은 Database Table의 Row/Column구조와 유사한 Oracle Forms의 내부적 구조라 할수 있다. 즉, 프로그램적으로 그 기능이 Two-Dimensional Array와 같은 역할을 수행하게끔 해 준다. 또한 이는 Column의 총 갯수가 64K를 초과하지 않 한도내에서 Char, Long, Number, Date Type의 Column들을 제공한다.

Record Group의 3가지 Type

1. Query Record Group
: SELECT문과 관련된 Record Group으로 Default로 Database Column의 Type과 길이를 갖는 Column들이 생성.
- Design Time에 Create
  (1) Navigator * Record Group에서 Create
  (2) Based on the Query Below를 선택한 후 SELECT 문 기술
  (3) Populate_Group Built-In Subprogram으로 Query를 실행
- Runtime에 Create
  (1) Trigger에서 Query문을 직접 기술

```
group_id := Create_Group_From_Query('RUNQUERY',
            'select deptno,dname from dept');
```

  (2) Populate_Group Built-in Subprogram으로 Query 실행
- Populate_Group이 성공적으로 실행된 경우 Return Value는 0.
  Populate_Group_With_Query는Group의 현재Query를 Replace할때 사용

2. Non_query Record Group
- Runtime에 프로그램적으로 그 구조와 값을 변경
  (1) Create_Group으로 Group 생성
  (2) Add_Group_Column, Add_Group_Row로 Column과 Row를 추가
  (3) Set_Group_Char_Cell등으로 Group 의 각 Cell에 값을 Assign

3. Static Record Group
- Design Time때에 그 구조와 값을 정의해 놓고 Runtime에 값이 고정되어 사용.
  (1) Navigator * Record Group에서 Create
  (2) Static Values를 선택한 후 Column Name, Type, Length, Value를 Assign
  (3) Runtime에 바로 사용(get_group_char_cell등으로)

Record Group을 이용하여 Report2.5로의 Data Parameter Passing (Non_query Record Group을 이용)

(1) Record Group을 Create한다. (Create_Group)
(2) Column들을 Add한다. (Add_Group_Column)
(3) Row를 Add한다. (Add_Group_Row)
(4) 각 Cell에 값을 Assign 한다. (Set_Group_Char_Cell)
(5) Report로 Record Group을 전송한다.

<예제1> Record Group을 Create하고 값을 Assign한다. Cursor를 사용하여 Field에 Display 하고 그 값들을 Record Group에 입력

```
DECLARE
    Rg_Name   Varchar2(15) := 'My_Group';
    Rg_Id     Recordgroup;
    Gc_Id     Groupcolumn;
    Cursor Temp Is Select Deptno,Empno,Ename,Job  From Emp;
    Cnt       Number(5) := 0;
BEGIN
    Rg_Id := Find_Group(Rg_Name);
     If Not Id_Null(Rg_Id) Then  Delete_Group(Rg_Id);
     End If;
     If Id_Null(Rg_Id) Then
        Rg_Id := Create_Group(Rg_Name);
        Gc_Id := Add_Group_Column(Rg_Id,'Deptno',Number_Column);
        Gc_Id := Add_Group_Column(Rg_Id,'Empno',Number_Column);
        Gc_Id := Add_Group_Column(Rg_Id,'Ename',Char_Column,10);
        Gc_Id := Add_Group_Column(Rg_Id,'Job',Char_Column,9);
     End If;
     If Not Id_Null(Rg_Id) Then
        Delete_Group_Row(Rg_Id,All_Rows);
       Open Temp;
     Loop
        Cnt := Cnt+1;
        Fetch Temp Into :Emp.Deptno,:Emp.Empno,:Emp.Ename,:Emp.Job;
        Exit When Temp%Notfound;
        Add_Group_Row(Rg_Id,End_of_Group);
      Set_Group_Number_Cell('My_Group.Deptno',Cnt,Name_in('Emp.Deptno'));
      Set_Group_Number_Cell('My_Group.Empno',Cnt,Name_in('Emp.Empno'));
      Set_Group_Char_Cell('My_Group.Ename',Cnt,Name_in('Emp.Ename'));
      Set_Group_Char_Cell('My_Group.Job',Cnt,Name_in('Emp.Job'));
      Down;
     End Loop;
    Close Temp;
  End If;
 END;
```

<예제2> Report로 전송

이때 Report에는 'Emp_Query'라는 Query가 존재하고 보내고자 하는 Record Group의 Column명과 Type이 일치하는 Column들이 있어야 한다.

- Forms에서

```
Declare
     Record_Pa   Paramlist;
      Begin
        Record_Pa := Get_Parameter_List('a');
     if Not Id_Null(Record_Pa) Then
     Destroy_Parameter_List(Record_Pa);
  End If;
  Record_Pa := Create_Parameter_List('a');
  Add_Parameter(Record_Pa,'Emp_Query',Data_Parameter,'My_Group');
  Run_Product(Reports,'C:\Orawin\Report25\Data_Pa',Synchronous,Runtime,
        Filesystem,Report_Pa,Null);
      End;
```

- Report 의 Query 'Emp_Query'

```
  Select Eptno,Empno,Ename,Job From Emp
```

<예제3> 현재 들어있는 Record Group의 Row들을 Delete

```
Declare
        Rg_Id   Recordgroup;
Begin
     Rg_Id := Find_Group('My_Group');
     If Not Id_Null(Rg_Id) Then
           Delete_Group_Row(Rg_Id,All_Rows);
     End If;
End;
```

<예제4> WHEN-MOUSE-CLICK Trigger에서 Mouse로 Click 는 Record만 Record Group으로 담는 Logic

```
DECLARE
     Rg_Name   Varchar2(15) := 'My_Group';
     Rg_Id     Recordgroup;
BEGIN
    :Global.Count := to_Number(:Global.Count)+1;
    Rg_Id := Find_Group(Rg_Name);
    Add_Group_Row(Rg_Id,End_of_Group);
   Set_Group_Number_Cell('My_Group.Deptno',to_Number(:Global.Count),
      Name_in('Emp.Deptno'));
   Set_Group_Number_Cell('My_Group.Empno',to_Number(:Global.Count),
      Name_in('Emp.Empno'));
   Set_Group_Char_Cell('My_Group.Ename',to_Number(:Global.Count),
      Name_in('Emp.Ename'));
   Set_Group_Char_Cell('My_Group.Job',to_Number(:Global.Count),
      Name_in('Emp.Job'));
END;
```

(V4.5) DDE(DYNAMIC DATA EXCHANGE)

Bulletin no : 10125

---

1. About DDE?

(1) MS Windows와 Communicate하거나 Data교환을 할 수 있는 기법을 말한다.
(2) Oracle Forms 4.5에서는 DDE를 가능케하는 PL/SQL Package를 제공한다.
(3) Oracle Forms 4.5에서의 DDE Function들은 MS Windows의 다른 DDE Server들과 다음과 같은 방법으로 Data를 교환할 수 있다.
* Data를 Import하는 형식
* Data를 Export하는 형식
* DDE Server에 대한 Commnad를 실행하는 형식

2. Forms 4.5에서의 DDE 제한사항

(1) Data Linking : Data Item이 변경된 경우 자동으로 변경된 내용을 인식하거나,Update가 되지 않는다. 다시 한번 조회하여 처리하여야 한다.
(2) Server Support : DDE Client로 부터의 Command나 Data요구에 응답 할 수 없다.

3. Function Overview

Forms에서 제공하는 DDE Package를 기능에 따라 분류하면 다음과 같다.
(1) Support Functions : DDE Server Application을 시작,종료시키는 기능을 한다.

- DDE.App_Begin : DDE Server를 시작한다.

Syntax: DDE.APP_BEGIN(Command,Mode)
Return Value
Application Identifier (PLS_INTEGER Type)

DDE.App_Begin에서 Return된 Application Identifier는
DDE.APP_END,DDE.APP_FOCUS Fucntion Call시에 사용된다.
Command
Type : Varchar2

Program을 실행하기위한 Command
Path가 지정되지 않은 경우 Forms는 다음 Directory를 찾는다.
Current Directory
Windows Directory
Windows System Directory
Current Task를 위한 Executable File을 포함한 Directory
PATH환경에 설정된 Directory
Network에 설정된 Directory

```
    Command String내에 Application에 필요한 Argument가  필요한 경우
    Application Program Name과 Space로 분리하여 표기하면 된다.
 Mod
   Type : PLS_INTEGER

    실행할  Application의  Window의 Mode를 말한다.
    DDE.APP_MODE_NORMAL    : Normal Size 로 Start한다.
    DDE.APP_MODE_MINIMIZED : Minimized Size로 Start한다.
    DDE.APP_MODE_MAXIMIZED : Maximized Size로 Start한다.
+-------------------------------------------------------------------------+
 [ Example ]

 DECLARE
    Appid PLS_INTEGER;
 BEGIN
   /* MS Excel을 Start한다 */
    Appid := DDE.App_Begin('EXCEL.EXE',DDE.APP_MODE_MINIMIZED);
 EXCEPTION
   WHEN DDE.DDE_APP_FAILURE
        Message('Excel 실행에 실패하였읍니다');
 END;
+-------------------------------------------------------------------------+

 - DDE.App_End  : DDE Server를 종료한다.

  Syntax : DDE.APP_END(Appid)
  Return Value : 없음.
  Appid
           Type : PLS_INTEGER
  DDE.APP_BEGIN에 의해 얻어진 Application Identifier를 지정한다.
  Application은 Application자체의 Close에 의하거나,  Application자체의
  Control Menu를 Double Click함으로써 종료될 수도 있다.
  DDE.App_End를 실행하기전에  반드시 DDE.App_Begin이 실행되어 있어야 한다.
+-------------------------------------------------------------------------+
 [ Example ]
 DECLARE
   Appid      PLS_INTEGER;
 BEGIN
   /* MS Excel을 Start한다 */
   Appid := DDE.App_Begin('EXCEL.EXE',DDE.APP_MODE_MINIMIZED);
   ...
   .../* Excel Program 을 종료한다. */
    DDE.App_End(Appid);
 EXCEPTION
    WHEN DDE.DDE_APP_FAILURE THEN
            Message('Excel 실행에 실패하였읍니다');
```

```
    WHEN DDE.DDE_APP_NOTFOUND THEN
            Message('Excel을 먼저 실행하십시오.');
  END;
+-------------------------------------------------------------------------+
```

- DDE.App_Focus : Window의 Focus를 DDE Server로 옮긴다.

```
  Syntax : DDE.APP_FOCUS(Appid)
  Return Value : 없음.
  Appid
         Type : PLS_INTEGER
  DDE.APP_BEGIN에 의해 얻어진 Application Identifier를 지정한다.
  DDE.App_End를 실행하기전에  반드시 DDE.App_Begin이 실행되어 있어야 한다.
+-------------------------------------------------------------------------+
 [ Example ]
 DECLARE
    Appid     PLS_INTEGER;
 BEGIN
    /* MS Excel을 Start한다 */
    Appid := DDE.App_Begin('EXCEL.EXE',DDE.APP_MODE_MINIMIZED);
    ...
    ...
    /* Application Window를 Activate시킨다.*/
    DDE.App_Focus(Appid);
 EXCEPTION
    WHEN DDE.DDE_APP_FAILURE THEN
            Message('Excel 실행에 실패하였읍니다');
    WHEN DDE.DDE_APP_NOTFOUND THEN
            Message('Excel을 먼저 실행하십시오.');
  END;
+-------------------------------------------------------------------------+
```

(2) Connect/Disconnect Functions : DDE Server에 접 ,분리하는 기능을 한다.

- DDE.Initiate : 다른 Application과 DDE작업을 시작한다.

```
  Syntax: DDE.INITIATE(Service,Topic)
  Return Value
          Conversation Identifier( PLS_INTEGER Type)
           DDE.App_Initiate Function에서 Return되는 Convid는
           Execute,Poke,Request,Terminate Function Call시에 사용된다.
  Service
          Type : Varchar2
          Server Program의  DDE Service Code
          Service의 값은 일반적으로 Application Program명이 지정된다.
  Topic
```

Type : Varchar2
Conversation의 Topic Name
File-Based로 실행되는 Operate되는 Application의 경우 Topic의 값은
일반적으로 Document의 File명이 지정된다.
Service와 Topic의 값은 DDE Server Application에 의해 달라진다.
DDE.Initiate로 시작된 Conversation은 DDE.Terminate를 만나야 끝이난다.

+--+

[Example]

```
DECLARE
   Convid     PLS_INTEGER;
BEGIN
    /* MS Excel과 DDE Conversation을 Open한다 */
   Convid := DDE.Initiate('EXCEL','SHEET1');
EXCEPTION
    WHEN DDE.DDE_INIT_FAILED THEN
         Message('Excel SHEET1 Init실행에 실패하였읍니다');
END;
```

+--+

- DDE.Terminate : 지정된 Application과의 DDE작업을 종료한다.

Syntax: DDE.Terminate(Convid)
Return Value
없음.
Conv
Type : PLS_INTEGER
DDE.INITIATE에서 Return된 Conversion Identifier
DDE.Terminate를 실행하기전에 반드시 DDE.Initiate가 실행되어 있어야 한다.
DDD.Terminate가 실행된 이후에 모든 Execute,Poke,Request,Terminate등의
Command는 Error이다.

+--+

[Example]

```
DECLARE
    Convid    PLS_INTEGER;
BEGIN
    /* MS Excel과 DDE Conversation을 Open한다 */
    Convid := DDE.Initiate('EXCEL','SHEET1');
     ....
     ....
    /* MS Excel과의 DDE Conversation을 끝낸다 */
     DDE.Terminate(Convid);
END;
```

+--+

(3) Transaction Functions : DDE Server와 Data를 교환하는데 사용한다.

- DDE.Execute : Application에 의해 인식된 Command를 실행한다.

Syntax: DDE.Execute(Convid,Command,Timeout)
Return Value 없음.
Conv
Type : PLS_INTEGER
DDE.INITIATE에서 Return된 Conversion Identifier
Command
Type : VARCHAR2
Server에서 실행에 필요한 명령어
Server Application에 서 제공하는 Command를 사용한다.
Timeout
Type : VARCHAR2
Millisecond단위
Default : 1000

+--+

[Example]

```
DECLARE
   Convid     PLS_INTEGER;
BEGIN
   /* MS Excel과 DDE Conversation을 Open한다 */
   Convid := DDE.Initiate('EXCEL','SHEET1');
    ....
    ....
    DDE.Execute(Convid,'[Calcaulate.Now()]',1000);
   /* Word에서 File Close */
   DDE.Execute(Convid,'[Filecloseall(0)]',1000);
 END;
```

+--+

- DDE.Poke : Application에 자료를 제공한다.

Syntax: DDE.Poke(Convid,Item,Data,Dataformat,Timeout)
Return Value 없음.
Conv
Type : PLS_INTEGER
DDE.INITIATE에서 Return된 Conversion Identifier
Item
Type : VARCHAR2
Application으로 보내지는 Data Item명
Data
Type : VARCHAR2
전송할 Data Buffer
Dataformat

```
    Type : PLS_INTEGER
     MS Window에 기 지정된 Data Format명을    다.
    (Oracle Forms 4.5 Advanced Techniques Manual참조)

Timeout
    Type : VARCHAR2
     Millisecond단위
     Default : 1000
+-------------------------------------------------------------------------+
[ Example ]

DECLARE
    Convid    PLS_INTEGER;
BEGIN
    /* MS Excel과 DDE Conversation을 Open한다 */
    Convid := DDE.Initiate('EXCEL','SHEET1');

    ....
    ....
   /*  EXCEL의  Row 2, Column 2 Cell에 Ename  Item의 값을 보낸다 */
   DDE.Poke(Convid,'R2C2',:Ename,DDE.CF_TEXT,1000);
END;
+-------------------------------------------------------------------------+
- DDE.Request :  Application으로부터 자료를 요청한다.

Syntax: DDE.request(Convid,Item,buffer,Dataformat,Timeout)
Return Value          없음.
Conv
    Type : PLS_INTEGER
    DDE.INITIATE에서 Return된 Conversion Identifier
Item
    Type : VARCHAR2
    Application으로  부터 보내지는 Data Item명
Buffer
   Type : VARCHAR2
    전송받을  Data Buffer
Dataformat
    Type : PLS_INTEGER
    MS Window에 기 지정된 Data Format명을 준다.
    (oracle forms 4.5 advanced techniques manual참조)
Timeout
    Type : VARCHAR2
    Millisecond단위
    Default : 1000
+-------------------------------------------------------------------------+
[ Example ]
```

```
DECLARE
   Convid     PLS_INTEGER;
BEGIN
    /* MS Excel과 DDE Conversation을 Open한다 */
   Convid := DDE.Initiate('EXCEL','SHEET1');
    ....
    ....
   /*  EXCEL의  Row 2, Column 2 Cell에 해당된 값을 Ename  Item으   받는다*/
   DDE.Request(Convid,'R2C2',:Ename,DDE.CF_TEXT,1000);
END;
```

(4) Datatype Translation Functions

- DDE.Getformatnum : Data Format String을 Number로 Convert/Register한다.
- DDE.Getformatstr : Data Format Number를 String으로 Convert한다

4. Sample Program - MS Windows의 EXCEL과의 Data Exchange

(1) Package Spec

```
PACKAGE DDE_EXE IS
    APPID PLS_INTEGER;
       CONVID PLS_INTEGER;
       ISRUN BOOLEAN := FALSE;
    PROCEDURE EXCEL_BEGIN;
    PROCEDURE EXCEL_FOCUS;
    PROCEDURE EXCEL_POKE;
    PROCEDURE EXCEL_REQ;
    PROCEDURE EXCEL_END;
END;
```

(2) Package Body

```
PACKAGE BODY DDE_EXE IS

PROCEDURE EXCEL_BEGIN IS
BEGIN
   IF DDE_EXE.ISRUN = TRUE THEN
      MESSAGE('ALREADY STARTED');
   ELSE
      DDE_EXE.Appid :=
      DDE.App_Begin('C:\Msoffice\Hexcel\Excel.Exe',Dde.App_Mode_Normal);
      DDE_EXE.ISRUN := TRUE;
   END IF;
```

```
EXCEPTION
   WHEN DDE.DDE_APP_FAILURE THEN
      MESSAGE('BEGIN ERROR');
END;

/* tips :  Excel이 시작되는 시간이 필요합니다.  When-New-Form-Instance나
   When-Button-Pressed에서 처리하되 DDE.Initiate를 실행하기전에 Excel이
   Start되기위한 충분한 시간을 가져야 합니다.  만약 Excel이 정상 Start되기
   전에 DDE.Initiate를 실행하면 FRM-106556 Error가 발생합니다 */

PROCEDURE EXCEL_END IS
BEGIN
  IF DDE_EXE.ISRUN = FALSE THEN
       MESSAGE('먼저 START시키십시오');
  ELSE
       DDE.APP_END(DDE_EXE.APPID);
       DDE_EXE.ISRUN := FALSE;
  END IF;
EXCEPTION
  WHEN DDE.DDE_APP_NOT_FOUND THEN
       MESSAGE('APPID FOUND ERROR');
END;

PROCEDURE EXCEL_FOCUS IS
BEGIN
  IF DDE_EXE.ISRUN = FALSE THEN
       MESSAGE('먼저 BEGIN을 실행하십시오!');
  ELSE
       DDE.APP_FOCUS(DDE_EXE.APPID);
  END IF;
EXCEPTION
  WHEN DDE.DDE_INIT_FAILED THEN
       MESSAGE('INITIATE 실패');
  WHEN DDE.DDE_APP_NOT_FOUND THEN
       MESSAGE('해당 APPID가 없음');
END;

PROCEDURE EXCEL_POKE IS
   Y     Number := 1;
   AXIL  VARCHAR2(6);
BEGIN
    IF DDE_EXE.ISRUN = FALSE THEN
         MESSAGE('먼저 BEGIN을 실행하십시오!');
   ELSE
         DDE_EXE.CONVID := DDE.INITIATE('EXCEL',:CNT.SHEET);
   END IF;
   Go_Block('Emp');
```

```
   execute_query;
    LOOP
    IF :SYSTEM.LAST_RECORD = 'TRUE' THEN
          EXIT;
    ELSE
     AXIL := 'R'||TO_CHAR(Y)||'C'||TO_CHAR(1);
            DDE.POKE(DDE_EXE.CONVID,AXIL,TO_CHAR(:EMPNO),DDE.CF_TEXT,100000);
      AXIL := 'R'||TO_CHAR(Y)||'C'||TO_CHAR(2);
        DDE.POKE(DDE_EXE.CONVID,AXIL,:ENAME,DDE.CF_TEXT,100000);
      AXIL := 'R'||TO_CHAR(Y)||'C'||TO_CHAR(3);
        DDE.POKE(DDE_EXE.CONVID,AXIL,:JOB,DDE.CF_TEXT,100000);
      AXIL := 'R'||TO_CHAR(Y)||'C'||TO_CHAR(4);
        DDE.POKE(DDE_EXE.CONVID,AXIL,TO_CHAR(:HIREDATE,'YY/MM/DD'),
                 DDE.CF_TEXT,100000);
    END IF;
      Down;
      Y := Y+1;
   END LOOP;
   DDE.TERMINATE(DDE_EXE.CONVID);
   FIRST_RECORD;
EXCEPTION
  WHEN DDE.DDE_INIT_FAILED THEN
       MESSAGE('INITIATE 실패');
END;
/* Tips :  Emp Block에 Query되어 있는 Data를 Excel Sheet에 전송합니다.
     Numeric이나 Date Type의 Item은 Char Type으로 변형하여 전송하여야 합니다. */

  DDE_EXE.CONVID := DDE.INITIATE('EXCEL',:CNT.SHEET);
   FOR y IN 1..5 LOOP
      AXIL := 'R'||TO_CHAR(y)||'C'||TO_CHAR(1);
        DDE.REQUEST(DDE_EXE.CONVID,AXIL,DUMMY,DDE.CF_TEXT,100000);
        :empno := to_number(substr(dummy,1,length(:dummy)-2));
      AXIL := 'R'||TO_CHAR(y)||'C'||TO_CHAR(2);
         DDE.REQUEST(DDE_EXE.CONVID,AXIL,:ENAME,DDE.CF_TEXT,100000);
        :ename := substr(:ename,1,length(:ename)-2);
 next_record;
   END LOOP;
   DDE.TERMINATE(DDE_EXE.CONVID);
   FIRST_RECORD;
END;
/* Tips1 : Excel에서 가져올 수  있는 Data는 Char Type이어야 합니다.
 Numeric이나 Date  Item으로 Data를 가져오려면 먼저 Char Type인 Variable에
 받아서 Numeric Item으로 Assign하여야 합니다. */
/* Tips2 : Excel에서 가져오는 Data뒤에 Space가  2 Byte가 붙어서
 들어옵니다.  DB Item에  Assign하기전에 뒤에서 2 Byte를 잘라내야 합니다 */
END;
```

(V4.5) FORMS45와 TEXT FILE간의 INTERFACE

Bulletin no : 10160

Forms에서 text file 로 write하거나 혹은 text file의 data를 forms의 block으로 read하는 기능은 Forms가 제공하는 TEXT_IO라는 built-in package에 의해 가능하다.

```
TEXT_IO.FCLOSE(file file_type)
 : close할 file을 지정

TEXT_IO.FILE_TYPE
 : file의 datatype을 지정

TEXT_IO.FOPEN(spec varchar2, filemode varchar2)
  spec : filename
  filemode :  R   읽기전용의 file을 open
              W   file에 있는 내용을 지우고 write를 할때
              A   file에 있는 내용에 덧붙여서 write를 할때

TEXT_IO.IS_OPEN(file file_type)
  지정한 file이 현재 open되어 있는지 check

TEXT_IO.GET_LINE(file file_type, item OUT varchar2);
  한 line에서 newline character(즉 carriage return)이나 end-of-
  file(EOF)를 만날때까지 character를 읽어서 item에 입력

TEXT_IO.NEW_LINE
  새로운 line을 지정

TEXT_IO.PUT(file file_type, item varchar2)
  open된 file의 현재line에 제공되는 data를 덧붙인다.

TEXT_IO.PUTF(file file_type¦format varchar2¦arg varchar2)
  open된 file에 format대로 arg   쓴다.
  예) TEXT_IO.PUTF(out_file,'Today is %s\n',
                   TO_CHAR(SYSDATE));

TEXT_IO.PUT_LINE(file file_type, item varchar2)
  open된 file의 현재line에 제공된 data를 덧붙인 후 newline
  character(즉 carriage return)을 string   마지막에 추가한다.

PROCEDURE text_read IS
```

```
  in_file  TEXT_IO.FILE_TYPE;
  linebuf   varchar2(80);

BEGIN
  in_file := TEXT_IO.FOPEN('c:\temp\read.txt','r');
  go_block('emp');
  first_record;
  loop
   TEXT_IO.GET_LINE(in_file,linebuf);
   :emp.empno := substr(linebuf,1,10);
   :emp.ename := substr(linebuf,13,10);
   TEXT_IO.NEW_LINE;
   next_record;
  end loop;
EXCEPTION
  when no_data_found then
    TEXT_IO.FCLOSE(in_file);
    message('읽기가 종료되었습니다.');
  when others then
    TEXT_IO.FCLOSE(in_file);
    message('읽기에 실패하였습니다.');
end;

PROCEDURE text_write IS

  out_file  TEXT_IO.FILE_TYPE;
  linebuf   varchar2(80);

BEGIN
  out_file := TEXT_IO.FOPEN('c:\temp\echo.txt','a');
  go_block('emp');
  first_record;
  loop
   TEXT_IO.PUTF(out_file,' %s %s %s \n',
     rpad(Name_in('emp.empno'),10),
     rpad(Name_in('emp.ename'),20),
     rpad(Name_in('emp.job'),20));

    if :system.last_record = 'TRUE' then
      message('File Write가 종료되었습니다.');
      exit;
   end if;
   next_record;
  end loop;
  TEXT_IO.FCLOSE(out_file);
END;
```

(V4.5) DEVELOPER2000으로 작성한 MODULE을 DATABASE에 저장하려면?

Bulletin no : 10216

User가 작성한 Forms4.5나 Report2.5 Module을 DB에 save하기 위해서는 그 Module을 수용하는 table들이 먼저 create되어 있어야 한다. 이 table의 생성 script들은 Install당시에 자동적으로 만들어지며 SQLPLUS 의 SYSTEM account에서 돌려주어야 한다.

1) Oracle Forms를 위한 Table Create

```
SQL> @C:\ORAWIN\CDE2\SQL\TOOLBILD.SQL
SQL> @C:\ORAWIN\CDE2\SQL\ROSBILD.SQL
SQL> @C:\ORAWIN\CDE2\SQL\VGBILD.SQL
SQL> @C:\ORAWIN\CDE2\SQL\DEBILD.SQL
SQL> @C:\ORAWIN\FORMS45\SQL\FRM45BLD.SQL
```

2) Oracle Report를 위한 Table Create

```
SQL> @C:\ORAWIN\CDE2\SQL\TOOLBILD.SQL
SQL> @C:\ORAWIN\CDE2\SQL\ROSBILD.SQL
SQL> @C:\ORAWIN\CDE2\SQL\VGBILD.SQL
SQL> @C:\ORAWIN\REPORT25\SQL\SRW2BILD.SQL
```

3) Oracle Graphics를 위한 Table Create

```
SQL> @C:\ORAWIN\CDE2\SQL\ROSBILD.SQL
SQL> @C:\ORAWIN\CDE2\SQL\TOOLBILD.SQL
SQL> @C:\ORAWIN\CDE2\SQL\VGBILD.SQL
SQL> @C:\ORAWIN\CDE2\SQL\DEBILD.SQL
SQL> @C:\ORAWIN\CDE2\SQL\GCBILD.SQL
SQL> @C:\ORAWIN\GRAPH25\SQL\GOBILD.SQL
```

상호간에 중복된 Script를 다시 실행할 필요는 없다.

(V4.5) FORMS4.5 RUNTIME INSTALL 방법

Bulletin no : 10217

Forms Application이 각 client machine에서 작동되기 위해서는 반드시 Forms Runtime이 install되어 있어야만 하는데, 기본적으로 install되어야 할 것들은

GUI Common Files
System Support Files
Tools Utilities
Forms Runtime

이고, 만약 SQL*Net V2를 사용하려면

Required Support Files 7.1
SQL*Net 2.1
Oracle [protocol] Adapter 2.1

을 추가로 install을 해야하고, SQL*Net v1을 사용하려면 위 Product 대신에

Required Support Files 7.0
Required Support Files 7.1
SQL*Net [protocol] 1.x

이 필요하다.

Runtime Module을 install한 후에 PC에 변경된 사항들은 다음과 같다.

1) C:\ORAWIN directory 및 sub directory 생성
2) C:\WINDOWS\ORACLE.INI file 생성
3) C:\WINDOWS\WIN.INI file에
   [Oracle]
   ORA_CONFIG=C:WINDOWSORACLE.INI
   가 insert 됨
4) AUTOEXEC.BAT가 변경
5) program group과 icon의 생성
6) C:\WINDOWS\SYSTEM directory밑에 여러 DLL file의 생성

CTL3D.DLL	CTL3DV2.DLL	COMPOBJ.DLL
OLE2.DLL	OLE2CONV.DLL	OLE2DISP.DLL
OLE2NLS.DLL	OLE2PROX.DLL	TYPELIB.DLL
STORAGE.DLL	VSHARE.386	STDOLE.TLB
MFC250.DLL	MFCD250.DLL	MFCO250.DLL

MFCOLEUI.DLL COMMDLG.DLL MSOLEVBX.DLL
VBOA300.DLL

Install이 끝난 후 RUMFORM 수행시에 FRM-40034 (Cannot attach the library file) Error가 발생한다면?

library를 attach 할 수 없는 이유는 지정한 directory에 attach하려는 PLL file이 존재하지 않기 때문이다.
Form을 작성할때 Attach library를 면 path를 remove할 것인지를 묻는 Alert Box가 뜨게 된다. 이때, remove path를 'NO'로 하게 되면 Form은 그 PLL file의 directory를 기억하게 되어 후에 rumtime module을 install하는 PC에도 같은 directory에 반드시 attach한 PLL file이 존재해야 한다. 편 remove path를 'YES'로 하면 path를 기억하지 않으며 항상 그 form을 실행할때 ORACLE.INI file의 FORMS45_PATH에 setting되어 있는 path밑으로 가서 그 PLL file을 찾게 됨을 유의해야 한다.

(V4.5) FORMS4.5에서 FILE DIALOG 처리 방법

Bulletin no : 10218

File명을 직접 입력하지 않고 Dialog Box를 이용하여 원하는 File을 선택하려면 해당 DLL을 호출하거나 cmdialog.vbx를 이용하는 방법이 있는데 여기서는 'FILEDLL.DLL'을 사용하여 작성된 예이다.
(보통 FILEDLL.DLL은 C:\ORAWIN\PB15\DEMO\FFI\FILEDLL directory에 존재하며 size는 3616Bytes)

PACKAGE SPEC 정의내역

```
---------------------------

PACKAGE SELFILE IS
  FUNCTION selfile return varchar2;
END;
```

PACKAGE BODY 정의내역

```
---------------------------

PACKAGE BODY selfile IS
  dll_handle   ORA_FFI.LIBHANDLETYPE;
  fct_handle   ORA_FFI.FUNCHANDLETYPE;

  FUNCTION selfileicd(handle IN ORA_FFI.FUNCHANDLETYPE,
                      arg1 IN OUT VARCHAR2)
    RETURN PLS_INTEGER;
    pragma  interface(c, selfileicd,11265);
  FUNCTION selfile
    RETURN VARCHAR2
  IS
    tmp VARCHAR2(512) := rpad(' ',512);
  BEGIN
    IF (selfileicd(fct_handle,tmp)) = 1 THEN
      RETURN NULL;
    ELSE
      RETURN tmp;
    END IF;
  END;
BEGIN
  dll_handle := ORA_FFI.REGISTER_LIBRARY(NULL,
                'C:\ORAWIN\PB15\DEMO\FFI\FILEDLL\FILEDLL.DLL');
  fct_handle := ORA_FFI.REGISTER_FUNCTION(dll_handle,
                'ChooseFile',ORA_FFI.PASCAL_STD);
  ORA_FFI.REGISTER_RETURN(fct_handle,  ORA_FFI.C_INT);
  ORA_FFI.REGISTER_PARAMETER(fct_handle, ORA_FFI.C_CHAR_PTR);
END;
```

(V4.5) HOW TO IMPLEMENT MENU SECUTIRY IN FORMS4.5?

Bulletin no : 10243

Oracle Forms 4.5에서의 보안은 ORACLE7 SERVER의 ROLE의 개념에 기초를 둔다. 모든 user들이 database에 저장된 모든 정보를 사용할 수 없기때문에, role은 user들을 grouping하는데 유용하게 사용한다. (role에 대한 자세한 개념은 manual을 참조바람.)
oracle server에서 role이 정의되면, forms개발자는 menu module과 menu item 에 role을 지정하여, menu program에 누가 그것을 사용할 수 있는지를 조정할 수 있게된다.
*MENU APPLICATION을 사용하려면 EXP_FULL_DATABASE PRIVILEGE가 있어야한다.

I. ROLE생성방법

1.ORACLE FORMS4.5 DESIGNER에서 SYSTEM ACCOUNT로 CONNECT
2.FILE ADMINISTRATION DATABASE ROLES
만약, ADMIN PRIVILEGE가 없는 USER로 CONNECT를 하게되면 이 OPTION이 DISABLE상태로 나타난다.
3.새로운 ROLE을 생성하기 위해 NEW를 선택
새로운 ROLE의 이름을 등록할 수 있는 WINDOW가 나타난다.
새로운 ROLE의 이름을 입력하고, 그 ROLE 갖게될 USER명을 입력한다. MENU를 실행시에 사용할 수 있는 OPTION(DEBUG MODE,O/S COMMAND,BACKGROUND MENU)을 원하는 데로 선택한다. MENU에서 위의 OPTION을 사용하지 않으려면 BLANK로 남겨두면된다.

4.OK BUTTON을 선택
ROLE이 생성된다.

II. MENU MODULE에서의 SECURITY SETTING

1.MENU APPLICATION OPEN한 후 MENU APPLICATION의 PROPERTY화면을 띄운다.
2.SECURITY MENU SECTION에서
1)MENU MODULE SECURITY MORE BUTTON 선택 혹은 DOUBLE CLICK
선택한 MENU APPLICATION의 사용을 허락할 ROLE명을 등록한다.
OK
이 작업은 각각의 MENU ITEM을 사용할 수 있는 권한을 부여한 것은 아니다.
단지, 각 ITEM마다 선택할 수 있는 ROLE을 지정한 것에 불과하다.
2)USE SECURITY을 TRUE로 SETTING
만약, TRUE로 SETTING하는 것을 잊어버린다면
모든 USER들이 모든 MENU OPTION을 사용할 수 있게된다.

III. MENU ITEM에 ROLE을 지정하는 방법

1.MENU ITEM의 PROPERTY화면을 띄운다.
2.SECURITY SECTION에서
  1)MENU ITEM ROLES에서 MORE BUTTON선택 혹은 DOUBLE CLICK
    앞의 과정에서 SETTING해 놓은 ROLE들이 나타난다.
    이 ITEM을 사용할 수 있게 하고자 하는 ROLE들을 선택한다.
    만약, 이 LIST에서 아무것도 선택하지 않으면,
    이 MENU는 모든 USER들이 사용할 수 없게된다.
  2)DISPLAY W/O PRIVILEGE는 이 MENU ITEM을 사용할 수 없은 USER에게도 MENU ITEM을 보여 줄 것인가를 의미한다. 만약, TRUE인 경우 이 ITEM 을 사용 수 없는 USER에게는 GRAYED OUT된상태 즉 DISABLE된 상태로 나타난다.
  만약, FALSE인 경우 이 IETM을 사용할 수 없는 USER에게는 이 MENU ITEM이 나타나지 않게된다.

IV. MENU 를 RUN시키는 방법

MENU을 RUN시키려면, 먼저 FORM에 MENU를 ATTCH하는 작업이 필요하다.

1.FORM MODULE을 OPEN한다.
2.FORM MODULE의 PROPERTY화면을 띄운다. (TOOLS PROPERTIES)
3.MENU SECTION에서
  MENU MODULE에 생성해 놓은 MENU MODULE명을 등록한다.
  MENU STYLE 선택한다.
  STARTING MENU를 입력한다.
  이것은 생성해 놓은 MENU MODULE내에 있는 MENU TREE구조중
  어느 POINT에서 부터 사용할 것인가를 지정하는 것이다.
4.FORM을 SAVE한 후 RUN시킨다.

V.나타날 수 있는 ERROR들

FRM-10256 : "User Is Not Authorized To Run Oracle Forms Menu"

이것은 USER가 FORMS TABLE들을 ACCESS할 수 없다는걸 의미한다.
FORMS에서의 SECURITY를 구현하기위해서는, FORMS45에 관련 TABLE과 VIEW들이 생성되어 있어야 하고, 해당 USER는 이 TABLE과 VIEW를 사용할 수 있는 권한이 있어야 한다. FORMS45에 관련된 TABLE과 VIEW를 생성하 기 위해서는 관련된 SQL이SYSTEM ACCOUNT 하에서 실행되었어야 한다.
(INSTALLATION GUIDE참조)

FRM-10249 : "No Authorization To Run Application"

이 ERROR는 MENU APPLICATION을 사용할 수 있도록 지정된 ROLE에 해당 USER가 ASSIGN되어 있지 않다는 것을 의미한다.

(V4.5) DEV/2000에서 DOCUMENTATION 생성방법

Bulletin no : 10252

Forms Doc/Report Doc

1. Running Forms Doc.

Developer/2000의 Forms4.5이전의 Version에서는 Forms Doc를 실행하려면 해당 Form이 Database에 Save가 되어있어야 하고, Doc를 실행하면 내부에서 실제로 F40doc.Rdf가 호출되었다. 그러나 Developer/2000의 Forms4.5에서 Doc을 실행하면 File System에 저장되어있는 Module에서 직접 정보를 읽어 들이도록 되었다.

Forms에서 Doc Utility를 사용하려면 File * Administration * Forms Doc.를 선택하면 된다. 이 Utility는 Object Navigator에 나타나있는 순서에 따라 Text-Formatted File을 생성하게 된다.

Document File에 나타나는 Object의 순서를 바꾸려면, Object Navigator의 Option을 Ownership View에서 Visual View로 바꾸어 선택하면 된다.

1) Navigator*Ownership View(Default) : Object Navigator는 Ownership에의한 Object의 Hierachy에 따라 나타난다.

2) Navigator*Visual View : Object Navigator 는 Display될 수 있는 것들 예를들어 Windows,Canvas, Item등만을 나타낸다. 이러한 Object들은 그것들이 Display되는 Canvas/Window들과 연관된 것들이다.

3) Navigator*Only Object With PL/SQL Option은 On/Off로 Toggle이 된다. 이 Option을 On으로 Setting하면 Trigger,Program Unit등이 있는 Object를 제외한 모든 Object들이 나타나질 않게 된다. 이것은 Document의 양을 줄일 수 있게 해준다. 그러나 이것 Trigger나 Program만이 Forms Documentation에 나타난다는 것을 의미하는 것은 아니다.

2. Output To Expect

Forms4.0에서의 Doc과는 달리 Forms4.5에서 Documentation에서는 PL/SQL Code나 Trigger만을 Print할 수 없다. 이점에 관해서는 아래의 Section에서 처리한 .

Documentation을 선택하면 현재 작업중인 Directory내에 .Txt File이 생성된다. 만약 Form의 이름을 아직 생성하지 않았으면 .Txt가 생성된다.

3. Generating PL/SQL Documentation

Program에서 단지 Trigger나 Program Unit만 Document로 출력하고자 한다면 Forms Doc Utiliti를 사용해서는 할 수가 없다. 대신, Forms Debugger Utility를 사용하여야만 한다. Forms Designer에서 원하는 Program을 Open한 다음 Debug Option을 선택한다.

Forms을 Run시켜 Debugger가 나타나면 PlSQL> Prompt에 다음과 같이 Type한다.

```
.Export  Prog  *  File  C:\.Pls
```

이 작업을 수행하면 현재의 Form에 포함되어있는 PL/SQL Code들이 포함되게 된다. 이뿐아니라

Attach되어 있는 Library의 Code도 포함하게 된다.

주의 : 이 작업을 통하여 생성되는 File내에는 Procedure나 Trigger의 이름들은 출력되지 않는 제약 이 있으므로 각각의 Trigger나 Procedure에 Comment처리를 하여야 한다.

이 작업은 Graphics나 Reports와 같은 다른 Tool에서도 처리가 가능하다.

4. Notes On Forms Doc

Document는 Release에 따라 다르게 성되므로, 이 작업을 Source Control에 사용하거나 Application간의 차이점을 Check하는데 사용하지 않는 것이 좋다.

5. Running Reports Doc

Oracle Reports2.5에서의 Report Doc은 Reports 2.0과 흡사하다. Reports에서 Documentation을 받아내려면 해당 Report Database에 저장되어 있어야 하고, 해당 Table을 Access할 수 있도록 Logon이 되어 있어야 한다.

Report Doc을 선택하면 Landscape나 Portrait방식을 선택하게 된다. 이것은 결국 내부에서 C:\Orawin\Report25밑의 Srwdoclb와 Srwdocpb Report를 선택하여 출력 게 된다.

(V4.5) EXITING MULTIPLE FORM APPLICATIONS

Bulletin no : 10254

---

1. General Overview

가끔 User들은 Menu에서 선택하거나, Button을 선택하여 Open된 든 Form들을 동시에 Close하기를 원한기도 한다. 이렇게 동시에 Open되어 있는 Form들을 Close하기 위해서는 각 Form에 Focus가 갈때 Global값이 Check되어야 한다. 이 Global값은 스스로 Close되어야 하는지 아닌지를 알려주게된다. 이 방법은 Call_Form, Open_Form과 New_Form으로 새로운 Form을 Open할 때 모두 사용된다.

2. Triggers

다음의 3가지 Trigger가 필요하다.

```
+-------------------------------------------------------------------+
        When-New-Form-Instance
              :Global.Quit_All := 'N';

       주의 : 처음시작하는 Form에 걸어야 한다.
            Global변수를 처음 생성한다.
+-------------------------------------------------------------------+
+-------------------------------------------------------------------+
       When-Window-Activated
            If :Global.Quit_All = 'Y' Then
              Exit_Form;
            End If;

   주의 :  이 Trigger는 처음 시작하는 Form을 포함한 모든 Form에 필요하다.
            이 Trigger는  Form에 다시 Forcus가 돌아올때 실행된다.
+-------------------------------------------------------------------+

+-------------------------------------------------------------------+
       When-Button-Pressed
            :Global.Quit_All := 'Y';
            Exit_Form;

   주의 :  이 Trigger는 Close를 시작하게 한다.
      만약 Form들이  같은 Menu에 연결된 것이라면 Menu Item으로 Setting
            하고,  각각의 Form에 걸려있는 이 Trigger를 없애도 된다.
+-------------------------------------------------------------------+
```

단, Open_Form과 Call_Form을 섞어서 사용하지 말아야 한다.

(V3.0,4.5) PRE-QUERY TRIGGER 사용방법

Bulletin no : 10271

Advanced SQL*FORMS Techniques Version 3.0 manual 중 'Performing Dynamic Sorting' 참조.

Enter query mode에서 field에 값을 입력하거나,원하는 조건을 입력함으로써 query를 제한적으로 처리할 수 있다. field에 입력한 문자열은 field에 연결되어있는 database column명과 결합되고, SQL*FORMS에서 사용하는 SQL문으로 변환,추가되어 Database로 부터 자료를 가져오게된다. 예를 들어 NUBER type인 field에 대해 제한적으로 자료를 조회하려면 [enter query] key를 누른 후 다음과 같이 처리된다.

Typed into Field	Effect on Query
12	column의 값이 12인 모든 자료
<12	column 값이 12보다 작은 모든자료
#in (12,13,14)	column의 값이 12,13,14인 모든 자료
#between 12 and 15	column의 값이 12에서 15인 모든 자료
#$	systax error발생

위의 예에서 보여준 것처럼 #$를 입력하여 systax error를 발생시키면 SQL*FORMS에서 생성한 SQL문장을 볼수가 있다. query를 실행하면 다음과 같은 error가 발생한다.

FRM-40505: ORACLE error -- unable to perform query.

이때 [Display Error] key 를 눌러보면 SQL*Forms가 database로 부터 data를 가져오기위해 생성한 error가 포함된 SQL문장의 list를 볼 수 있다.

Enter-query mode를 사용하는 이와같은 제한된 query의 수행은 forms designer에서 PRE-QUERY trigger를 사용하여 처리할 수 있다. PRE-QUERY trigger는 EXECUTE_QUERY event가 처리되기전에 수행(fire)된다. PRE-QUERY trigger는 [Enter Query] key를 누르지 않고도 같은 결과를 갖게한다. PRE-QUERY trigger에 적당한 SQL문을 입력함으로써 [Enter Query] key를 눌렀을 때와 같은 제한적인 query를 수행할 수 있는 것이다.

NUMBER type의 DB-field인 :BLOCK.FIELD를 가정해보자.
위에서 본 enter query와 같은 결과를 갖는 PRE-QUERY trigger의 예를 보자.

Typed into Field	PRE-QUERY Trigger Statement
12	:BLOCK.FIELD := 12;
<12	COPY ('<12', 'BLOCK.FIELD');
#in (12,13,14)	COPY ('#in (12,13,14)', 'BLOCK.FIELD');
#between 12 and 15	COPY ('#between 12 and 15', 'BLOCK.FIELD');

=========
SAMPLE
=========

Special

Field Name	Type	Attributes	Description
CONTROL.N	CHAR		control field for the name
CONTROL.Z	NUMBER	fixed length = 2	control field for two zip code BLOCK.NAME
CHAR		DB-field for the name	
BLOCK.ZIP	NUMBER		DB-field for zip code

PRE-QUERY Trigger Code on BLOCK

```
+============================================================================+
if (:CONTROL.N is null) and (:CONTROL.Z is null) then
  message('You must enter a name and/or a zip to search.');
  raise form_trigger_failure;
end if;
if :CONTROL.N is null then
  copy('#between ' || to_char(:CONTROL.Z*1000) || ' and ' ||
              to_char((:CONTROL.Z+1)*1000-1), 'BLOCK.ZIP');
elsif :CONTROL.Z is null then
  :BLOCK.NAME := :CONTROL.N || '%';
else
  copy('#between ' || to_char(:CONTROL.Z*1000) || ' and ' ||
              to_char((:CONTROL.Z+1)*1000-1), 'BLOCK.ZIP');
  :BLOCK.NAME := :CONTROL.N || '%';
end if;
+============================================================================+
```

========
주의사항
========

1. field query length가 query문장의 길이가 충분하도록 지정하여야 한다.
2. number field의 값을 지정할 때는 assign문(:=)을 사용하여야 한다.
3. num-numeric string을 지정할 때는 COPY packaged procedure를 사용한다.
4. query string에서 지정된 값이 null이면 query가 실행될 때 systax error가 발생하기 때문에 이 경우에 대한 처리를 해주어야 한다.

(V4.5) ALERT을 이용하여 MESSAGE를 처리하는 방법 (ADVANCED)

Bulletin no : 10274

Forms에서 Message Line에 Message를 표시하기보다는 Alert Box를 사용하여 Message를 표시하는 방법에 대해 살펴본다.

Mesage Library를 생성하기위해 다음과 같은 Logic을 사용한다. 만약 User가 Message의 내용을 바꾸고 싶다면, User는 단지 Library만 변경하면 된다.

1. 필요한 Database Object를 생성한다.

```
-----------------------------------------------------------------
Create Table Message_Library
     (Message_Num              Number                 Not Null,
      Message_Text             Varchar2(60)           Not Null,
      Message_Help_Text        Varchar2(2000)         Not Null,
      Message_Type             Varchar2(1)            Not Null,
            Constraint Pk_Message_Num
                       Primary Key (Message_Num),
            Constraint Check_Message_Type
                       Check (Message_Type In ('E','W','I','Y','O'))
     );
-----------------------------------------------------------------
```

2. Message_Library Table이 생성되고나면 필요한 Message들을 등록한다.

```
-----------------------------------------------------------------
    Insert Into Message_Library
          Values(-1,
                 'Unknown Message Type For Specified Message',
                 'Please Contact Your Dba',
                 'E');

    Insert Into Message_Library
          Values(-2,
                 'Requested Message Number Not Found In Message Library',
                 'Please Contact Your Dba',
                 'E');
-----------------------------------------------------------------
```

3. Form에 Message를 표시 할 음과 같은 Aler Box를 생성한다.

```
-----------------------------------------------------------------
```

'Error_Alert'　　:치명적인 Error를 표시한다.
　　Ok와 Help Button을 갖는다.
Warning_Alert　:주의사항을 표시한다.
　　Ok와 Help Button을　갖는다.
'Info_Alert'　　:안내사항을 표시한다.
　　Ok와 Help Button을 갖는다.
'Yes_No_Alert'　: Yes나no의 확인에 사용한다.
　　Yes,No,Help Button을 갖는다.
'Ok_Cancel_Alert': Ok나cance의 확인에 사용한다.
　　Ok, Cancel, Help Button을 갖는다.

4.　해당하는 Message를 표시하는 Procedure를 생　한다.

해당Message의 번호와 5개의 Input Parameter를 갖는다. 만약 Message에 대치될 값이 없다면, Procedure는 Null Value인 5개의 Parameter를 가지고 호출되게 된다.
Procedure에서 Message_Library Table로 부터 해당 Message의 Type이 무엇인지 확인하여 Alert Box의 Type을 결정한다. Message의 Type이 *T*라면 Information Type으로 하고, *W*,*Y*,*O*라면 Warning Type으로 하고, *E*라면 Stop Type을 표시한다. 모등 Alert Box에 나타낼 Message는 Message_Library Table에서 조회하여 표시한다.　Procedure는 입력받은 Button에 대한　을 Out변수에 담아 Return하게된다. Retrun된 값에 따라 User가 어떤 Button을 눌렀는지를 알게되는 것이다.

```
Procedure Display_Error_Prc(
                    P_Message_Number      In     Number,
                    P_Place_Holder1       In     Varchar2,
                    P_Place_Holder2       In     Varchar2,
                    P_Place_Holder3       In     Varchar2,
                    P_Place_Holder4       In     Varchar2,
                    P_Place_Holder5       In     Varchar2,
                    P_Button_Value        Out    Number,
                    P_Error_Help_Text     Out    Varchar2) Is

  Alert_Is        Alert;
  Alert_Button    Number;
  Err_Message     Varchar2(200);
  Err_Help        Varchar2(3000);
  Err_Type        Varchar2(1);

  Cursor   Bmsg_Cur Is
    Select  Message_Text, Message_Help_Text, Message_Type
    From    Message_Library
    Where   Message_Num = P_Message_Number;
Begin
   Open   Bmsg_Cur;
   Fetch  Bmsg_Cur Into Err_Message, Err_Help, Err_Type;
   If Bmsg_Cur%Notfound Then
```

```
      Raise No_Data_Found;
    End If;
    Close Bmsg_Cur;

    Err_Message := Replace(Err_Message,'{{1}}',P_Place_Holder1);
    Err_Message := Replace(Err_Message,'{{2}}',P_Place_Holder2);
    Err_Message := Replace(Err_Message,'{{3}}',P_Place_Holder3);
    Err_Message := Replace(Err_Message,'{{4}}',P_Place_Holder4);
    Err_Message := Replace(Err_Message,'{{5}}',P_Place_Holder5);
    Err_Message := To_Char(P_Message_Number)||': '||Err_Message;

    Err_Help := Replace(Err_Help,'{{1}}',P_Place_Holder1);
    Err_Help := Replace(Err_Help,'{{2}}',P_Place_Holder2);
    Err_Help := Replace(Err_Help,'{{3}}',P_Place_Holder3);
    Err_Help := Replace(Err_Help,'{{4}}',P_Place_Holder4);
    Err_Help := Replace(Err_Help,'{{5}}',P_Place_Holder5);
    P_Error_Help_Text := Err_Help;

    If (Err_Type = 'E') Then
        Alert_Is := Find_Alert('Error_Alert');
    Elsif(Err_Type = 'W') Then
        Alert_Is := Find_Alert('Warning_Alert');
      Elsif (Err_Type = 'I') Then
         Alert_Is := Find_Alert('Info_Alert');
            Elsif (Err_Type = 'Y') Then
               Alert_Is := Find_Alert('Yes_No_Alert');
                 Elsif (Err_Type = 'O')
                   Then Alert_Is := Find_Alert('Ok_Cancel_Alert');
                     Else
                        /* Unknown Value For Error Message Type */
                           Display_Error_Prc(-1,Null,Null,Null, Null,Null,
                                             P_Button_Value,P_Error_Help_Text);
      If P_Button_Value = Alert_Button3 Then
         Display_Error_Help_Prc(P_Error_Help_Text);
      End If;
    End If;

    If (Err_Type In ('E','W','I','Y','O'))
    Then
       Set_Alert_Property(Alert_Is, Alert_Message_Text, Err_Message);
       Alert_Button := Show_Alert(Alert_Is);
       P_Button_Value := Alert_Button;
    End If;
Exception
  When No_Data_Found Then
    /* Error Message Number Provided To This Procedure Is Not Found In The Message_Library
```

```
Table */
    Display_Error_Prc(-2,To_Char(P_Message_Number),Null,Null,Null,Null,
                      P_Button_Value,P_Error_Help_Text);
    If P_Button_Value = Alert_Button3 Then
        Display_Error_Help_Prc(P_Error_Help_Text);
    End If;
    Raise Form_Trigger_Failure;
  When Others Then
     Begin
        If (Sqlcode = 100) Then
           Raise No_Data_Found;
        Elsif (Sqlcode = -100501) Then
           Raise Form_Trigger_Failure;
        Else
           Message(Sqlerrm);
           Raise Form_Trigger_Failure;
        End If;
     End;
End;
```

---

5. Message표시를 요청하는 Trigger를 생성한다.

Error번호 '20710'에 해당하는 Message를 표시해야 하는 경우를 생각해보자.

```
V_Button_Value    Number;
V_Err_Help        Message_Library.Message_Help_Text%Type;

Begin
  Display_Error_Prc(20710,Null,Null,Null,Null,Null,V_Button_Value,V_Err_Help);
  If V_Button_Value = Alert_Button3 Then
    Display_Error_Help_Prc(V_Err_Help);
  End If;
End;
```

'{{1}} Is Not An Active Contract Number'라는 Message Text에서 {{1}}부분을 실제의 Contract Number로 대체되어 표시되게 하려면 다음과 같이 처리한다.

```
V_Button_Value    Number;
V_Err_Help        Message_Library.Message_Help_Text%Type;

Begin
  Display_Error_Prc(20710,To_Char(:Block.Contract_Number),Null,Null,Null,Null,
                    V_Button_Value,V_Err_Help);
  If V_Button_Value = Alert_Button3 Then
```

```
    Display_Error_Help_Prc(V_Err_Help);
  End If;
End;
```

---

5. 조합된 Error Message를 표시하는 Procedure를 생성한다.

```
Procedure Display_Error_Help_Prc(Err_Help In Varchar2) Is
Begin
  :Control_Block_Name.Help_Text := Err_Help;
  Show_Window('Help_Window');
End;
```

(V4.5) ALERT을 이용하여 MESSAGE를 처리하는 방법 (BASIC)

Bulletin no : 10275

Introduction
==========
기본적으로 User가 생성하거나, Oracle Forms4.5에서 내주는 모든Message와 Error는 Message Line에 Display된다. Oracle Forms 4.5에서는 Alert을 이용하여 Error를 처리 할 수 있다. 이것은 필요한 시점에 User에게 좀더 Message를 강조하여 확인하게 하는 효과를 갖는다..

Alert생성방법
==========
Alert은 두 지 Type으로 나누어 볼 수 있다. 하나는 Message를 처리하기 위한 것이고, 하나는 Error가 발생한 경우를 처리하기 위한 것이다.

1. Error용 Alert생성
- Alert Node를 Click한 후에 Navigator --> Create선택
- Properties Window에서
- Alert의 이름설정 : Error_Alert
- Alert Style Property 설정 : Stop
- Alert에서 나타내고 싶은 Button을 선택하거나 Default를 사용

2. Message용 Alert생성

위와 같은 방법을 이용하여 Message용 Alert을 하나 더 생성한다.
- Alert의 이름 설정 : Message_Alert
- Alert Style Property 설정 : Caution

Alert을 사용하기 위한 Triggers 생성
==========
Form Level의 두개의 Trigger를 생성한다. : On-Message And On-Error.
On-Message Trigger는 Forms에 Message가 표시될 때마다 Fire될 것이고, On-Error Trigger는 Error가 발생 할 때마다 Fire될 것이다.

On-Error

```
Declare
     Alert_Button  Number;

Begin
   Set_Alert_Property('Error_Alert', Alert_Message_Text, Error_Text);
   Alert_Button := Show_Alert('Error_Alert');
End;
```

```
On-Message
Declare
   Alert_Button  Number;
Begin
   Set_Alert_Property('Message_Alert', Alert_Message_Text, Message_Text);
   Alert_Button := Show_Alert('Message_Alert');
End;
```

Alert을 이용하여 User가 생성한 Message를 처리하는 방법

==========

위의 두 Trigger들은 Oracle Forms4.5에서 내주는 모든 Error와 Message를 처리해 주지만, 만약 User가 생성한 Message가 다면 그것은 Message Line에 나타나게 된다. 이와 같은 경우의 Message를 Alert을 이용하여 표시하는 방법은 다음과 같다.

1. Alert을 생성한다.
- 위에서와 같은 방법으로 또하나의 Alert을 생성한다.
- 이름을 설정한다 : User_Alert
- Alert Style Property을 설정한 : Note
- 표시하고 싶은 Button을 결정하여 설정한다.

2. Alert을 사용할 Procedure를 생성한다.

```
Procedure Alerts (Mesg Varchar2) Is
   Alert_Button  Number;
Begin
   Set_Alert_Property('User_Alert', Alert_Message_Text, Mesg);
   Alert_Button := Show_Alert('User_Alert');
End;
```

3.필요한 Trigger에서 위에서 생성한 Procedure를 호출한다.

```
Trigger: When-Validate-Item
Item : Deptno
Declare
     Mesg  Varchar2(80);
Begin
   If :Deptno > 50 Then
      Mesg := ('Invalid Number.  Please Try Again.');
      Alerts(Mesg);
      Raise Form_Trigger_Failure;
   End If;
End;
```

(V3.0,4.5) USER가 아무 ACTION이 없이 일정 시간이 지나면 자동으로 LOGOUT시키는 방법

Bulletin no : 10278

---

* SQL*Plus에서
  CREATE PROFILE Profile_name LIMIT IDLE_TIME N; 실행
  : idle time은 분단위로 setting한다.
  SQL Language Reference manual참조.

* 기존에 생성되어 있는 USER에 대하여
  ALTER USER PROFILE Profile_name; 실행
  : session에서 아무 action이 없이 위에서 지정한 시간이 지나면
  error message를 내주고 강제로 rollback이 된 후 session이 끊어지게된다.
  SQL Language Reference manual참조.

* $ORACLE_HOME/dbs/initSID.ora file에서
  RESOURSE_LIMIT parameter의 값을 TRUE로 SETTING한 후
  ORACLE을 RESTART하여야만 위에서 setting한 profile에 대한 정의가 적용된다.
  Admin Manual참조.

(V4.5) CUSTOMIZING LOGON BEHAVIOR

Bulletin no : 10279

Introduction
============
By default, Oracle Forms will attempt to establish a connection to a database when starting up a Runform session. If the connect information (username, password, connect string) is not specified as parameters to the Runform executable, Oracle Forms will pop up a logon window.

This bulletin details methods of commonly-asked-for customization of this logon behavior, including running a form without logging onto a database, defaulting the connect string, and creating a custom logon window.

Objective
=========
This bulletin will primarily focus on the On-Logon trigger and the LOGON built-in. The On-Logon trigger, if defined, replaces the default logon behavior of Oracle Forms. Within that trigger, the LOGON built-in actually establishes the connection. If called from outside the On-Logon trigger, the LOGON built-in calls the On-Logon trigger.

Running Without a Connection
============================
Forms that do not access any data do not require connections to the database. By telling Forms not to establish a connection, startup time is reduced and the operator will not be prompted for connect information. The following On-Logon trigger eliminates logon entirely:

```
On-Logon (Form Level)
  NULL;
```

Small Changes to Default Behavior
=================================
Here is an On-Logon trigger which mimics the default behavior of Oracle Forms when it performs a connect.

```
On-Logon (Form Level)
   DECLARE
      un          VARCHAR2(30);
      pw          VARCHAR2(30);
      cs          VARCHAR2(30);
```

```
    upc         VARCHAR2(200);
    connected   BOOLEAN := FALSE;
    tries       NUMBER := 3;
    PROCEDURE get_connect_info IS
    BEGIN
       un := GET_APPLICATION_PROPERTY(USERNAME);
       pw := GET_APPLICATION_PROPERTY(PASSWORD);
       cs := GET_APPLICATION_PROPERTY(CONNECT_STRING);
    END;
BEGIN
    get_connect_info;
    IF un IS NOT NULL THEN
       LOGON(un, pw||'@'||cs, FALSE);
       IF FORM_SUCCESS THEN            /* Successful logon */
          connected := TRUE;
       END IF;
    END IF;
    WHILE connected = FALSE and tries > 0 LOOP
       upc := un||pw||cs;
       LOGON_SCREEN;
       get_connect_info;
       IF upc = un||pw||cs THEN
          EXIT;
       END IF;
       LOGON(un, pw||'@'||cs, FALSE);
       IF FORM_SUCCESS THEN            /* Successful logon */
          connected := TRUE;
       END IF;
       tries := tries - 1;
    END LOOP;
    IF NOT connected THEN
       RAISE FORM_TRIGGER_FAILURE;
    END IF;
END;
```

Note that the GET_APPLICATION_PROPERTY(CONNECT_STRING) will not work in versions prior to 4.0.13 of Oracle Forms. In prior versions, the connect string is part of the password.

Small deviations from the default connect behavior can be accomplished by modifying this code:

* To allow more or fewer logon attempts, increase the initial value of the counter variable, tries.

* To default the connect string, change the calls to LOGON from:

```
LOGON(un, pw||'@'||cs, FALSE);
```

to use a hard-coded connect string:

```
LOGON(un, pw||'@', FALSE);
```

* These lines can also be modified to hardcode username or password.

Hiding the Logon Information
==========

For security reasons, many customers do not want their users to be able to access the database except through the forms. One way to do this is to have the form handle connecting to the database, which keeps passwords away from the users. In V7, the best way to do this is to have the form set role to a privileged role. However, a quick and dirty way to do it is to have the form connect to a privileged, secret ORACLE account.

The following On-Logon trigger establishes a connection using hard-coded values for username and password. Users will be able to connect to the database without knowing the account or password:

```
On-Logon (Form Level)
   LOGON('SCOTT','TIGER@T:MYHOST:V716');
```

Deferring Logon
==========

Connection can be deferred until later in the Forms processing.

```
On-Logon (Form level)
   DECLARE
      un CHAR(30);
      pw CHAR(30);
      cs CHAR(30);
   BEGIN
      DEFAULT_VALUE('N','global.logon_now');
      IF :global.logon_now = 'Y' THEN
         LOGON_SCREEN;
         un := GET_APPLICATION_PROPERTY(USERNAME);
         pw := GET_APPLICATION_PROPERTY(PASSWORD);
         cs := GET_APPLICATION_PROPERTY(CONNECT_STRING);
         LOGON(un, pw||'@'||cs);
      END IF;
   END;
```

When the form starts up, it will not try to log on since the flag logon_now will be defaulted to N. When you do wish to log onto the database, use the syntax:

```
:global.logon_now := 'Y';
LOGON(NULL, NULL);
```

This sets the flag to Y and causes the On-Logon trigger to fire, so the form connects to the database.

Customizing the Logon Screen
============================
The built-in LOGON_SCREEN calls up the default Forms logon screen to capture the username, password, and connect string. However, you can also use a customized logon screen. The following tutorial creates a form which can be used as a replacement logon screen from any form.

NOTE: Coordinate System is assumed to be pixel.

1. Create a new forms module and change only the following properties:
   Name: LOGON
   Title: LOGON (v4.5)

2. Modify only the following properties for WINDOW0
   (v4.0, Create this window):
   Name: LOGON_WINDOW
   Width: 126
   Height: 108
   Title: Logon
   Style: Dialog
   Modal: TRUE
   Fixed Size: TRUE
   Iconifiable: FALSE
   Inherit Menu: FALSE
   Zoomable: FALSE

3. Create a new canvas (v4.0, Modify CANVAS0) and modify only the following properties:
   Name: LOGON_CANVAS
   Window: LOGON_WINDOW
   Width: 126
   Height: 108

4. Modify only the following properties for LOGON_WINDOW:
   View: LOGON_CANVAS

5. Create a new block and modify only the following properties:
   Block Name: LOGON_BLOCK

6. Create three text items on canvas LOGON_CANVAS for block LOGON_BLOCK with properties as follows:
   Name: USERNAME

X Position: 12
Y Position: 8
Width: 102
Height: 18

Name: PASSWORD
X Position: 12
Y Position: 31
Width: 102
Height: 18
Secure: TRUE

Name: CONNECT
X Position: 12
Y Position: 54
Width: 102
Height: 18

7. Create two push buttons on canvas LOGON_CANVAS for block LOGON_BLOCK with properties as follows:

Name: LOGON
X Position: 12
Y Position: 84
Width: 52
Height: 18
Label: Logon
Mouse Navigate: FALSE
Default Button: TRUE

Name: CANCEL
X Position: 72
Y Position: 84
Width: 52
Height: 18
Label: Cancel
Mouse Navigate: FALSE

8. Code the following program units:

```
PROCEDURE leave(status BOOLEAN DEFAULT TRUE) IS
BEGIN
   IF status THEN
      :global.logged_on := 'TRUE';
   ELSE
      :global.logged_on := 'FALSE';
   END IF;
```

```
   EXIT_FORM;
END;

PROCEDURE initialize_form IS
   un VARCHAR2(30) := GET_APPLICATION_PROPERTY(USERNAME);
   pw VARCHAR2(30) := GET_APPLICATION_PROPERTY(PASSWORD);
   cs VARCHAR2(30) := GET_APPLICATION_PROPERTY(CONNECT_STRING);
BEGIN
   DEFAULT_VALUE('3', 'global.logon_tries');
   DEFAULT_VALUE(un, 'global.username');
   DEFAULT_VALUE(pw, 'global.password');
   DEFAULT_VALUE(cs, 'global.connect_string');
   :logon_block.username := :global.username;
   :logon_block.password := :global.password;
   :logon_block.connect  := :global.connect_string;
END;

FUNCTION connect_to RETURN BOOLEAN IS
BEGIN
   IF :logon_block.username IS NULL THEN
      BELL;
   END IF;
   LOGON(:logon_block.username,
         :logon_block.password||'@'||:logon_block.connect, FALSE);
   IF FORM_SUCCESS THEN
      RETURN TRUE;
   ELSE
      RETURN FALSE;
   END IF;
END;
```

9. Code the following triggers:

```
On-Logon (Form Level)
   LOGON(:logon_block.username,
         :logon_block.password||'@'||:logon_block.connect,
         FALSE);

When-New-Form-Instance (Form Level)
   initialize_form;

When-Button-Pressed (Item Level, on LOGON_BLOCK.CANCEL)
   leave(FALSE);

When-Button-Pressed (Item Level, on LOGON_BLOCK.LOGON)
   BEGIN
```

```
      IF connect_to THEN
         leave(TRUE);
      END IF;
      :global.logon_tries := TO_NUMBER(:global.logon_tries) - 1;
      IF TO_NUMBER(:global.logon_tries) = '0' THEN
         leave(FALSE);
      END IF;
   END;

When-Window-Activated (Form Level)
   DECLARE
     screen_height  NUMBER;
     screen_width   NUMBER;
     window_height  NUMBER;
     window_width   NUMBER;
   BEGIN
     IF :SYSTEM.EVENT_WINDOW = 'LOGON_WINDOW' THEN
        screen_height := GET_APPLICATION_PROPERTY(DISPLAY_HEIGHT);
        screen_width := GET_APPLICATION_PROPERTY(DISPLAY_WIDTH);
        window_height := GET_WINDOW_PROPERTY(:SYSTEM.EVENT_WINDOW, HEIGHT);
        window_width := GET_WINDOW_PROPERTY(:SYSTEM.EVENT_WINDOW, WIDTH);
        SET_WINDOW_PROPERTY('LOGON_WINDOW', X_POS,
                            (screen_width - window_width)/2);
        SET_WINDOW_PROPERTY('LOGON_WINDOW', Y_POS,
                            screen_height/2 - window_height);
     END IF;
   END;
```

10. Generate and save this form.

11. Create a new form. This form is an example of how to use the logon form that has just been created.

12. Create a default block, giving this form some items.

13. Code the On-Logon trigger as follows:

```
On-Logon (Form Level)
   CALL_FORM('logon', NO_HIDE);
   IF :global.logged_on != 'TRUE' THEN
      RAISE FORM_TRIGGER_FAILURE;
   END IF;
```

This will complete the coding for the main form. When the form now requires a connection to the database, it will use this customized logon screen instead of the Forms default one.

(V4.5) BASE TABLES FOR DEVELOPER 2000 TOOLS

Bulletin no : 10282

This document explains how to install base tables for Developer 2000 Tools (i.e. Oracle Forms V4.5, Oracle Reports V2.5 and Oracle Graphics V2.5). Developer 2000 Tools allow saving modules to file system or database. To be able to save modules to the database, all bases tables must exist. If any of the base tables are missing or corrupt, usually ORA-942 : "Table or view does not exist" or other errors will occur.

Types of Base Tables

There are a number of tables, views and synonyms which are shared by Oracle Forms, Oracle Reports and Oracle Graphics. They have prefixes TOOLS, ROS, VG, GC or DE. Base tables for Oracle Forms 4.5, Reports 2.5, and Graphics 2.5 have prefixes FRM45, SRW2, and GO respectively.

Check Common Base Tables, Views and Synonyms

Before you install the base tables, you need to find out if they exist already and thus skip the installation of existing objects. This process can be started
from checking objects shared by all CDE products. Put the following code into a SQL*Plus script called checkobjects.sql.

```
set space 0
set escape off
set pagesize 100
column object_name format a30
column owner format a30
column object_type format a20
select object_name, owner, object_type from sys.dba_objects
where object_name like 'TOOL\_%' escape '\' or
      object_name like 'ROS%'  or
      object_name like 'VG\_%' escape '\' or
      object_name like 'GC\_%' escape '\' or
      object_name like 'DE\_%' escape '\'
order by object_name, object_type;
```

Now log into SQL*Plus as SYSTEM (default password is MANAGER) and type:

```
SQL>start checkobjects.sql
```

You should see the following list of objects returned :

OBJECT_NAME	OWNER	OBJECT_TYPE
DE_ATTACHED_LIBS	PUBLIC	SYNONYM
DE_ATTACHED_LIBS	SYSTEM	VIEW
DE_ATTACHED_LIBS_IDX	SYSTEM	INDEX
DE__ATTACHED__LIBS	SYSTEM	TABLE
GC_CHARTING	PUBLIC	SYNONYM
GC_CHARTING	SYSTEM	VIEW
GC_FIELD	PUBLIC	SYNONYM
GC_FIELD	SYSTEM	VIEW
GC_REFERENCE	PUBLIC	SYNONYM
GC_REFERENCE	SYSTEM	VIEW
GC__CHARTING	SYSTEM	TABLE
GC__FIELD	SYSTEM	TABLE
GC__REFERENCE	SYSTEM	TABLE
ROSATTRIBS	PUBLIC	SYNONYM
ROSATTRIBS	SYSTEM	VIEW
ROSLFDESC	PUBLIC	SYNONYM
ROSLFDESC	SYSTEM	TABLE
ROSLFDESC_ID	SYSTEM	SEQUENCE
ROSLFDESC_ID	PUBLIC	SYNONYM
ROSOBJMAP	PUBLIC	SYNONYM
ROSOBJMAP	SYSTEM	TABLE
ROSSEQUENCES	PUBLIC	SYNONYM
ROSSEQUENCES	SYSTEM	TABLE
ROSSTRINGS	PUBLIC	SYNONYM
ROSSTRINGS	SYSTEM	TABLE
ROSSTRUCTS	PUBLIC	SYNONYM
ROSSTRUCTS	SYSTEM	VIEW
ROSTFDESC	PUBLIC	SYNONYM
ROSTFDESC	SYSTEM	TABLE
ROSTFDESC_ID	SYSTEM	SEQUENCE
ROSTFDESC_ID	PUBLIC	SYNONYM
TOOL_ACCESS	PUBLIC	SYNONYM
TOOL_ACCESS	SYSTEM	VIEW
TOOL_ACCESSIBLE	PUBLIC	SYNONYM
TOOL_ACCESSIBLE	SYSTEM	VIEW
TOOL_ACCESSIBLE_IDX	SYSTEM	INDEX
TOOL_ACCESS_IDX	SYSTEM	INDEX
TOOL_COMMENT	PUBLIC	SYNONYM
TOOL_COMMENT	SYSTEM	VIEW
TOOL_COMMENT_IDX	SYSTEM	INDEX
TOOL_DEPEND	PUBLIC	SYNONYM
TOOL_DEPEND	SYSTEM	VIEW

TOOL_DEPENDENT	PUBLIC	SYNONYM
TOOL_DEPENDENT	SYSTEM	VIEW
TOOL_LIBRARY	PUBLIC	SYNONYM
TOOL_LIBRARY	SYSTEM	VIEW
TOOL_LIBRARY_IDX	SYSTEM	INDEX
TOOL_LRAW	PUBLIC	SYNONYM
TOOL_LRAW	SYSTEM	VIEW
TOOL_LRAW_ID	SYSTEM	SEQUENCE
TOOL_LRAW_ID	PUBLIC	SYNONYM
TOOL_LRAW_IDX1	SYSTEM	INDEX
TOOL_LRAW_IDX2	SYSTEM	INDEX
TOOL_LTEXT	PUBLIC	SYNONYM
TOOL_LTEXT	SYSTEM	VIEW
TOOL_LTEXT_ID	SYSTEM	SEQUENCE
TOOL_LTEXT_ID	PUBLIC	SYNONYM
TOOL_LTEXT_IDX1	SYSTEM	INDEX
TOOL_LTEXT_IDX2	SYSTEM	INDEX
TOOL_MODID	SYSTEM	SEQUENCE
TOOL_MODID	PUBLIC	SYNONYM
TOOL_MODTYPE_IDX	SYSTEM	INDEX
TOOL_MODULE	PUBLIC	SYNONYM
TOOL_MODULE	SYSTEM	VIEW
TOOL_MODULE_IDX	SYSTEM	INDEX
TOOL_PLSQL	PUBLIC	SYNONYM
TOOL_PLSQL	SYSTEM	VIEW
TOOL_PLSQL_IDX	SYSTEM	INDEX
TOOL__ACCESS	SYSTEM	TABLE
TOOL__COMMENT	SYSTEM	TABLE
TOOL__DEPEND	SYSTEM	TABLE
TOOL__LIBRARY	SYSTEM	TABLE
TOOL__LRAW	SYSTEM	TABLE
TOOL__LTEXT	SYSTEM	TABLE
TOOL__MODULE	SYSTEM	TABLE
TOOL__PLSQL	SYSTEM	TABLE
VG_COLOR	PUBLIC	SYNONYM
VG_COLOR	SYSTEM	VIEW
VG_DRAWING	PUBLIC	SYNONYM
VG_DRAWING	SYSTEM	VIEW
VG_FONT	PUBLIC	SYNONYM
VG_FONT	SYSTEM	VIEW
VG_IMAGE	PUBLIC	SYNONYM
VG_IMAGE	SYSTEM	VIEW
VG__COLOR	SYSTEM	TABLE
VG__DRAWING	SYSTEM	TABLE
VG__FONT	SYSTEM	TABLE

VG__IMAGE SYSTEM TABLE

88 rows selected.

Make sure all the objects listed above exist. The owner and type of the objects also have to be the same as above. If there is any discrepancy, you need to re-install those objects. Use SQL*Plus command DESCRIBE to check the structure of some of the tables. All the string data type must be VARCHAR2, otherwise re-installation of base objects is also needed.

Install / Re-install Common Base Tables

BEFORE YOU RUN ANY SCRIPTS, MAKE SURE YOU HAVE A BACKUP OF THE DATABASE AND SOURCE FILES FOR ALL APPLICATIONS (FMB AND MMB FILES FOR ORACLE FORMS; PLL FILES FOR PL/SQL LIBRARY; RDF AND SQL FILES FOR ORACLE REPORTS; OGD FILES FOR ORACLE GRAPHICS). Since the common base tables are shared by different CDE Tools, dropping them may cause loss of data from other tools.

The following scripts need to be run in order to build the base tables. Log into SQL*Plus as SYSTEM to execute them and make sure the order in which they are specified is followed. Note that when you run the "grant" scripts (the xxxgrnt.sql files), you will be prompted for a username. Specify a username if you want to grant access to one user at a time, or you can specify PUBLIC and every user will have access to the base tables.

On Microsoft Windows

The installation process creates a number of program groups which appear on your desktop. The install and upgrade scripts can be run by double clicking the icons in the Developer 2000 Administration program group. The actual scripts to install the common base tables for the Developer 2000 tools are in C:\ORAWIN\Developer 2000\SQL. The product specific script are located in C:\ORAWIN\PRODUCT\SQL, where PRODUCT refers to product name eg. for forms45 the scripts are in C:\ORAWIN\FORMS45\SQL directory.

The followings are scripts needed to be run to rebuild the base tables.

1.	TOOLDROP.SQL	TOOLBILD.SQL	TOOLGRNT.SQL
2.	ROSDROP.SQL	ROSBILD.SQL	ROSGRNT.SQL
3.	VGDROP.SQL	VGBILD.SQL	VGGRNT.SQL
4.	DEDROP.SQL	DEBILD.SQL	DEGRNT.SQL

Additionaly, you need (only for Oracle Graphics 2.5)

5.	GCDROP.SQL	GCBILD.SQL	GCGRNT.SQL

Check Product Specific Base Tables

You are not required to install product-specific base tables for all CDE Tools. Any one of them can be installed independently. Log into SQL*Plus as SYSTEM and use the following SELECT statements to check the existence of the base tables first.

Oracle Forms

```
select object_name, owner, object_type from sys.dba_objects
where object_name like 'FRM45%'
order by object_name;
```

you should get the following list:

```
FRM45_BINDVAR                PUBLIC                       SYNONYM
FRM45_BUFFER                 PUBLIC                       SYNONYM
FRM45_ENABLED_ROLES          PUBLIC                       SYNONYM
FRM45_GRP                    PUBLIC                       SYNONYM
FRM45_OBJECT                 PUBLIC                       SYNONYM
FRM45__BINDVAR               SYSTEM                       TABLE
FRM45__BUFFER                SYSTEM                       TABLE
FRM45__GRP                   SYSTEM                       TABLE
FRM45__OBJECT                SYSTEM                       TABLE
FRM45_ENABLED_ROLES          SYSTEM                       VIEW
10 rows selected.
```

Important: Please note that unlike Oracle Forms 4.0 which utilizes 62 base table objects, Oracle Forms 4.5 utilizes only the above 10 base table objects. As a result, each of the above base table objects tend to be very large in size.

Oracle Reports

```
select object_name, owner, object_type from sys.dba_objects
where object_name like 'SRW2%'
order by object_type;
```

you should get the following list:

```
SRW2_ANCHOR_IDX              SYSTEM                       INDEX
SRW2_BODY_LOCATION_IDX       SYSTEM                       INDEX
SRW2_BOILERPLATE_IDX         SYSTEM                       INDEX
SRW2_COLUMN_IDX              SYSTEM                       INDEX
SRW2_ELEMENT_IDX             SYSTEM                       INDEX
SRW2_FRAME_IDX               SYSTEM                       INDEX
SRW2_LAYOUT_GROUP_IDX        SYSTEM                       INDEX
SRW2_LINK_IDX                SYSTEM                       INDEX
SRW2_MATRIX_IDX              SYSTEM                       INDEX
SRW2_LISTS_IDX               SYSTEM                       INDEX
SRW2_TEXT_SEGMENT_IDX        SYSTEM                       INDEX
SRW2_QUERY_IDX               SYSTEM                       INDEX
SRW2_PARAM_FORM_IDX          SYSTEM                       INDEX
SRW2_OG_DOCUMENT_IDX         SYSTEM                       INDEX
SRW2_OGD_COLUMN_MAP_IDX      SYSTEM                       INDEX
```

SRW2_LAYOUT_IDX	SYSTEM	INDEX
SRW2_GROUP_IDX	SYSTEM	INDEX
SRW2_DISPLAY_TAG_IDX	SYSTEM	INDEX
SRW2_DATA_MODEL_IDX	SYSTEM	INDEX
SRW2_BUTTON_IDX	SYSTEM	INDEX
SRW2_FIELD_IDX	SYSTEM	INDEX
SRW2_ANCHOR	PUBLIC	SYNONYM
SRW2_BODY_LOCATION	PUBLIC	SYNONYM
SRW2_BOILERPLATE	PUBLIC	SYNONYM
SRW2_BUTTON	PUBLIC	SYNONYM
SRW2_COLUMN	PUBLIC	SYNONYM
SRW2_DATA_MODEL	PUBLIC	SYNONYM
SRW2_DISPLAY_TAG	PUBLIC	SYNONYM
SRW2_ELEMENT	PUBLIC	SYNONYM
SRW2_FIELD	PUBLIC	SYNONYM
SRW2_FRAME	PUBLIC	SYNONYM
SRW2_GROUP	PUBLIC	SYNONYM
SRW2_LAYOUT	PUBLIC	SYNONYM
SRW2_LAYOUT_GROUP	PUBLIC	SYNONYM
SRW2_LINK	PUBLIC	SYNONYM
SRW2_LISTS	PUBLIC	SYNONYM
SRW2_MATRIX	PUBLIC	SYNONYM
SRW2_MODULE	PUBLIC	SYNONYM
SRW2_OGD_COLUMN_MAP	PUBLIC	SYNONYM
SRW2_OG_DOCUMENT	PUBLIC	SYNONYM
SRW2_PARAM_FORM	PUBLIC	SYNONYM
SRW2_QUERY	PUBLIC	SYNONYM
SRW2_TEXT_SEGMENT	PUBLIC	SYNONYM
SRW2__ANCHOR	SYSTEM	TABLE
SRW2__DISPLAY_TAG	SYSTEM	TABLE
SRW2__DATA_MODEL	SYSTEM	TABLE
SRW2__TEXT_SEGMENT	SYSTEM	TABLE
SRW2__QUERY	SYSTEM	TABLE
SRW2__PARAM_FORM	SYSTEM	TABLE
SRW2__OG_DOCUMENT	SYSTEM	TABLE
SRW2__OGD_COLUMN_MAP	SYSTEM	TABLE
SRW2__MATRIX	SYSTEM	TABLE
SRW2__LISTS	SYSTEM	TABLE
SRW2__LINK	SYSTEM	TABLE
SRW2__LAYOUT_GROUP	SYSTEM	TABLE
SRW2__LAYOUT	SYSTEM	TABLE
SRW2__GROUP	SYSTEM	TABLE
SRW2__FRAME	SYSTEM	TABLE
SRW2__FIELD	SYSTEM	TABLE
SRW2__ELEMENT	SYSTEM	TABLE
SRW2__BODY_LOCATION	SYSTEM	TABLE

```
SRW2__BOILERPLATE          SYSTEM                    TABLE
SRW2__BUTTON               SYSTEM                    TABLE
SRW2__COLUMN               SYSTEM                    TABLE
SRW2_ANCHOR                SYSTEM                    VIEW
SRW2_FIELD                 SYSTEM                    VIEW
SRW2_ELEMENT               SYSTEM                    VIEW
SRW2_OGD_COLUMN_MAP        SYSTEM                    VIEW
SRW2_MODULE                SYSTEM                    VIEW
SRW2_MATRIX                SYSTEM                    VIEW
SRW2_LISTS                 SYSTEM                    VIEW
SRW2_LINK                  SYSTEM                    VIEW
SRW2_LAYOUT_GROUP          SYSTEM                    VIEW
SRW2_LAYOUT                SYSTEM                    VIEW
SRW2_GROUP                 SYSTEM                    VIEW
SRW2_TEXT_SEGMENT          SYSTEM                    VIEW
SRW2_QUERY                 SYSTEM                    VIEW
SRW2_PARAM_FORM            SYSTEM                    VIEW
SRW2_OG_DOCUMENT           SYSTEM                    VIEW
SRW2_FRAME                 SYSTEM                    VIEW
SRW2_BOILERPLATE           SYSTEM                    VIEW
SRW2_COLUMN                SYSTEM                    VIEW
SRW2_DATA_MODEL            SYSTEM                    VIEW
SRW2_DISPLAY_TAG           SYSTEM                    VIEW
SRW2_BUTTON                SYSTEM                    VIEW
SRW2_BODY_LOCATION         SYSTEM                    VIEW
86 rows selected.
```

Important: Please note that base table objects for Oracle Reports 2.5 and Oracle Reports 2.0 are one and the same.

Oracle Graphics

```
select object_name, owner, object_type from sys.dba_objects
where object_name like 'GO\_%' escape '\'
order by object_type;
```

you should get the following list:

```
GO_COLUMN                  PUBLIC                    SYNONYM
GO_DOCUMENT                PUBLIC                    SYNONYM
GO_PARAMETER               PUBLIC                    SYNONYM
GO_QUERY                   PUBLIC                    SYNONYM
GO_SOUND                   PUBLIC                    SYNONYM
GO_TIMER                   PUBLIC                    SYNONYM
GO_VISLAYER                PUBLIC                    SYNONYM
GO__COLUMN                 SYSTEM                    TABLE
```

```
GO__PARAMETER                  SYSTEM                         TABLE
GO__SOUND                      SYSTEM                         TABLE
GO__TIMER                      SYSTEM                         TABLE
GO__VISLAYER                   SYSTEM                         TABLE
GO__QUERY                      SYSTEM                         TABLE
GO__DOCUMENT                   SYSTEM                         TABLE
GO_COLUMN                      SYSTEM                         VIEW
GO_DOCUMENT                    SYSTEM                         VIEW
GO_PARAMETER                   SYSTEM                         VIEW
GO_QUERY                       SYSTEM                         VIEW
GO_SOUND                       SYSTEM                         VIEW
GO_TIMER                       SYSTEM                         VIEW
GO_VISLAYER                    SYSTEM                         VIEW
21 rows selected.
```

Important: Please note that the base table objects for Oracle Graphics 2.5 and Oracle Graphics 2.0 are one and the same.

Re-install Product Specific Base Tables

BEFORE YOU RUN ANY SCRIPTS, MAKE SURE YOU HAVE A BACKUP OF THE DATABASE AND SOURCE FILES FOR ALL APPLICATIONS (FMB AND MMB FILES FOR ORACLE FORMS; PLL FILES FOR PL/SQL LIBRARY; RDF AND SQL FILES FOR ORACLE REPORTS; OGD FILES FOR ORACLE GRAPHICS).

If any object of a CDE Tools application is missing or the string data type of any table is CHAR instead of VARCHAR2, you need to re-install the base objects. Again, log into SQL*Plus as SYSTEM and execute the scripts. You can grant access to individual user or PUBLIC when you run the "grant" scripts.

Oracle Forms 4.5:
On Windows: In C:\ORAWIN\FORMS45\SQL\

```
FRM45DRP.SQL            FRM45BLD.SQL            FRM45GRT.SQL
```

Oracle Reports 2.5:
On Windows: In C:\ORAWIN\REPORT25\SQL\

```
SRW2DROP.SQL            SRW2BILD.SQL            SRW2GRNT.SQL
```

Oracle Graphics 2.5:
On Windows: In C:\ORAWIN\GRAPH25\SQL

```
GODROP.SQL              GOBILD.SQL              GOGRNT.SQL
```

Upgrading from Earlier Versions:

If you are upgrading from an earlier version of Developer 2000 products, you should run one or more upgrade scripts instead of creating the tables from scratch. Doing so will update the information in your tables (preserve your existing information) rather than rebuild the tables.

Note: During installation, program items are created in the Developer 2000 Administration program group for the SQL scripts needed to upgrade the Developer 2000 tables and the individual product tables. Double-click on the program items to upgrade the tables.

Oracle Forms:
From previous versions of 4.0 to 4.5:

```
Run         C:\ORAWIN\FORMS45\DEBILD.SQL
                              DEGRNT.SQL username
                              FRM45BLD.SQL
                              FRM45GRT.SQL username
```

From previous versions of 4.5:

```
Run         C:\ORAWIN\Developer 2000\SQL\DEALTR.SQL
                                        FRM45UP1.SQL
```

Oracle Reports:
Using the following table, run the upgrade SQL scripts that correspond to the version of Oracle Reports you want to upgrade. The SQL scripts listed are located in:

C:\ORAWIN\REPORT25\SQL

Version to Version	Script
2.0.13.0 to 2.0.14.0	SRW20140.SQL
2.0.14.0 to 2.5	SRW2500.SQL
2.5 to 2.5.3.1.16	SRW2530.SQL

Oracle Graphics:
To upgrade Oracle Graphics tables, you must run the following scripts:

C:\ORAWIN\Developer 2000\GC25UP.SQL
C:\ORAWIN\GRAPH25\SQL\GO25UP.SQL

(V4.5) APPLICATION을 DB에 저장하는 방법

Bulletin no : 10303

- User가 작성한 Forms4.5나 Report2.5 Module을 DB에 Save하기 위해서는 그 Module을 수용하는 Table들이 먼저 Create되어 있어야 한다. 이 Table의 생성 Script들은 Install당시에 자동적으로 만들어지며 SQLPLUS 의 SYSTEM Account
에서 돌려주어야 한다.

1. Oracle Forms를 위한 Table Create

```
SQL> @C:\ORAWIN\CDE2\SQL\TOOLBILD.SQL
SQL> @C:\ORAWIN\CDE2\SQL\ROSBILD.SQL
SQL> @C:\ORAWIN\CDE2\SQL\VGBILD.SQL
SQL> @C:\ORAWIN\CDE2\SQL\DEBILD.SQL
SQL> @C:\ORAWIN\FORMS45\SQL\FRM45BLD.SQL
```

2. Oracle Report를 위한 Table Create

```
SQL> @C:\ORAWIN\CDE2\SQL\TOOLBILD.SQL
SQL> @C:\ORAWIN\CDE2\SQL\ROSBILD.SQL
SQL> @C:\ORAWIN\CDE2\SQL\VGBILD.SQL
SQL> @C:\ORAWIN\REPORT25\SQL\SRW2BILD.SQL
```

3. Oracle Graphics를 위한 Table Create

```
SQL> @C:\ORAWIN\CDE2\SQL\ROSBILD.SQL
SQL> @C:\ORAWIN\CDE2\SQL\TOOLBILD.SQL
SQL> @C:\ORAWIN\CDE2\SQL\VGBILD.SQL
SQL> @C:\ORAWIN\CDE2\SQL\DEBILD.SQL
SQL> @C:\ORAWIN\CDE2\SQL\GCBILD.SQL
SQL> @C:\ORAWIN\GRAPH25\SQL\GOBILD.SQL
```

* 상호간에 중복된 Script는 다시 실행할 요가 없다.

(V4.5) F45에서 특정 FIELD에서 한/영키 자동 변환방법

Bulletin no : 10304

---

1. 한글 Bios를 응용하여 한/영키에서 발생될 Event를 Program에서 발생케 하여 특정 Field에서 자동으로 원하는 Mode로 전환케 한다.

2. Program은 DLL로 만들어졌으며, Ora_Ffi Function으로 Call해야한다.

3. 다음은 Forms4.5의 Package 내용이다.

```
PACKAGE han IS
  PROCEDURE hanon (flag IN NUMBER);
  PROCEDURE hanoff(flag IN NUMBER);
END;

PACKAGE BODY han IS
  dll_handle    ORA_FFI.LIBHANDLETYPE;
  hanon_handle  ORA_FFI.FUNCHANDLETYPE;
  hanoff_handle ORA_FFI.FUNCHANDLETYPE;

  PROCEDURE hanonicd(handle IN ORA_FFI.FUNCHANDLETYPE);
  pragma interface (C, hanonicd, 11265);

  PROCEDURE hanon(flag IN NUMBER) IS
  BEGIN
      hanonicd(hanon_handle);
  END;

  PROCEDURE hanofficd(handle IN ORA_FFI.FUNCHANDLETYPE);
  pragma interface (C, hanofficd, 11265);

  PROCEDURE hanoff(flag IN NUMBER) IS
  BEGIN
      hanofficd(hanoff_handle);
  END;

BEGIN

  dll_handle := ORA_FFI.REGISTER_LIBRARY(NULL,'C:ORA_HAN.DLL');
```

```
  hanon_handle  := ORA_FFI.REGISTER_FUNCTION(dll_handle,
                                        'cvtToHAN',ORA_FFI.PASCAL_STD);
  hanoff_handle := ORA_FFI.REGISTER_FUNCTION(dll_handle,'cvtToENG',
                                         ORA_FFI.PASCAL_STD);

  ORA_FFI.REGISTER_PARAMETER(hanon_handle,  ORA_FFI.C_INT);
  ORA_FFI.REGISTER_PARAMETER(hanoff_handle, ORA_FFI.C_INT);
END;

han.hanon (1);
han.hanoff(1);
```

4. 특정 Field의 pre_text_item의 내용

```
han.hanon (1);    ---> Hangul  Input Mode
han.hanoff(1);    ---> English Input Mode
```

5. "ora_han.dll" file은 천리안에서 go oracle하여, 정보 보물 창고에 가면 Down받을 수 있다.

6. 수정할 부분은 ora_han.dll의 Path만 변경한다.

(V4.5) FORMS4.5에서 GRAPHIC2.5로의 DATA PARAMETER PASSING

Bulletin no : 10307

---

o forms4.5에서 graphic을 data parameter를 이용하여 사용하는 방법.

1) forms에서의 작업

. attached library : OG
. form level에서 사용하는 trigger
- when-new-form-instance의 내용

```
og.open('c:\orawin\graph25\text_f2g.ogd',
        'dirrep_bar.chart_item',TRUE,true);
go_block('managers');
execute_query;
```

- post-form의 내용

```
og.close('c:\orawin\graph25\text_f2g.ogd',
        'dirrep_bar.chart_item');
```

. 사용하는 block

- managers (where clause : where job in ('manager', ' president')
  order by clause : ename)

  emp table을 사용하여 생성

  when-new-record-instance trigger를 사용하여
  data_parameter를 graphic에 넘겨준다.

  when-new-record-instance의 내용은 아래와 같습니다.

```
declare
val  char(20) := 'chart_data';
pl   paramlist;
rg   recordgroup;
qry  varchar(2000);
stat number;
begin
/* prepare a query in a string */
   qry := 'select initcap(ename) name, sal from emp
        where mgr = '||to_char(:managers.empno);
```

```
        /* try to get the ID of the 'chart_data' record group */
           rg := find_group('chart_data');

        /* if it doesn't exist, create the group based on the
        query in the 'qry' string */
           if id_null(rg) then
             rg := create_group_from_query('chart_data',qry);
                   end if;

        /* make sure the 'chart_data' record group is empty */
           delete_group_row(rg, all_rows);

        /* populate the 'chart_data' record group with the query
                in the 'qry' string */
           stat := populate_group_with_query(rg,qry);

           pl := create_parameter_list('foo');
                       add_parameter(pl,'query0',data_parameter,val);

        /*************************************************/
        /*  RUN_PRODUCT를 사용하여 parameter passing */
        /*************************************************/

/*   run_product(graphics,'c:\orawin\graph25\data_f2g.ogd',
                       synchronous,batch,filesystem,pl,'DIRREP_BAR.CHART_ITEM');     */

        /*************************************************/
        /*  OG.INTERPRET를 사용하여 parameter passing  */
        /*************************************************/

/*   og.interpret('c:\orawin\graph25\data_f2g.ogd',
                        'dirrep_bar.chart_item','EXECUTE_ME;',true,pl);  */

                /*************************************************************************/
/*  OG.REFRESH를 사용하여 parameter passing  (OG.CLOSE와 함께
        사용)  */
/*************************************************************************/
                    og.refresh('c:\orawin\graph25\data_f2g.ogd','dirrep_bar.
        chart_item',pl);
                    og.close('c:\orawin\graph25\data_f2g.ogd',
        'DIRREP_BAR.CHART_ITEM');

           destroy_parameter_list(pl);
end;
```

- control block : dirrep_bar (chart_item을 포함)

2) graphic에서의 작업

. attached library : OG

. query0

```
select dname name, deptno sal
from dept
```

. program unit : execute_me <= run_product에서는 필요하지 않습니다.

```
PROCEDURE execute_me IS
BEGIN
  og_execute_query(og_get_query('query0'));
END;
```

. chart

name : ftog_bar

type : bar chart

. categories : name

. value : sal(bar)

(V4.5) FORMS4.5에서 SERVER쪽 APPLICATION을 실행하는 방법

Bulletin no : 10320

방법 :
Server쪽에서 우선 pro*c로 만든 daemon을 띄워놓고 Client의 Forms4.5에서는 DBMS_PIPE package로 만든 Stored Procedure를 call함으로써 message를 Server로 보낸다. daemon은 message를 계속해서 기다리다가 messaage가 receive되는 즉시 실행한다. 이때 'STOP'이라는 message를 만나면 deamon은 실행을 마친다.

준비사항 :
dbms_pipe에 관한 실행권한이 해당 user에 주 져야 한다. dbms_pipe package는 oracle install시에 생성되어 있으나 일반 user들에게는 사용권한이 없도록 되어있기 때문이다. sqldba에서 다음의 문장을 이용하여 해당 user에게 권한을 준다.

```
grant execute on dbms_pipe to ;
```

1) daemon.sql

```
create or replace procedure execute_system
    (command varchar2, timeout number default 10)
is
  status  NUMBER;
begin
  DBMS_PIPE.PACK_MESSAGE(command);
  status := DBMS_PIPE.SEND_MESSAGE('daemon', timeout);
  IF status <> 0 THEN
     RAISE_APPLICATION_ERROR(-20010,'Error while sending .
         status = ' || status);
  END IF;
end;
/
```

2) daemon.pc

```
#include
#include
#include

EXEC SQL BEGIN DECLARE SECTION;
  VARCHAR username[20];
  VARCHAR password[20];
```

```
  int status;
  VARCHAR command[100];
  VARCHAR value[2000];
EXEC SQL END DECLARE SECTION;

main()
{
  EXEC SQL WHENEVER SQLERROR DO sqlerror();
  connect_sql();
  execute_message();
}

connect_sql()
{
  strcpy(username.arr, "scott");
  strcpy(password.arr, "tiger");
  username.len =strlen(username.arr);
  password.len =strlen(password.arr);
  EXEC SQL CONNECT :username IDENTIFIED BY :password;
  printf(" Connect to oracle %s / %s\n",
                        username.arr,password.arr);
}

execute_message()
{
  while (1) {
    printf("Daemon wating ...\n\n");
    EXEC SQL EXECUTE
      BEGIN
        :status := DBMS_PIPE.RECEIVE_MESSAGE('daemon');
        IF :status = 0 THEN
             DBMS_PIPE.UNPACK_MESSAGE(:command);
        END IF;
      END;
    END-EXEC;
    if (status == 0)
    {
      command.arr[command.len] = '\0';
      if (!strcmp((char *) command.arr, "STOP"))
      {
        printf("Daemon exiting.\n");
        break;
      }

      else
      {
```

```
        printf("Will execute system command '%s'\n", command.arr);
        status = system(command.arr);
        if (status)
           printf("Daemon error while responding to system
                     command.");
        else
           printf("\n\nCompleted Successfully!!!\n\n");
      }
    }
  }
}

sqlerror()
{
  EXEC SQL WHENEVER SQLERROR CONTINUE;
  printf("\n% .70s \n",sqlca.sqlerrm.sqlerrmc);

  EXEC SQL ROLLBACK WORK RELEASE;
}
```

3) Forms4.5의 when-button-pressed trigger에서

```
declare
  temp varchar2(100);
begin
  temp := 'runrep emp.rep scott/tiger paramform=no destype=file';
  execute_system(temp,5);
end;
```

(V3.0, MENU V5.0) FORM과 MENU의 연결

Bulletin no : 10387

FORMS3.0과 MENU5.0을 연결하여 사용하는 방법은 크게 2가지로 나누어 볼 수 있다.
첫째는 MENU가 먼저 실행되고, MENU에서 FORM을 호출하는 형식이다.
둘째는 FORM에 MENU가 ATTACH되어 FORM이 먼저 실행되는 형식이다.

다음에서 2가지의 방법에 대해 좀더 자세히 살펴본다.

방법1) MENU가 먼저 실행되고, MENU에서 FORM을 호출
1. sqlforms30에서 사용할 FORM들을 생성
2. sqlmenu50에서 MENU생성
3. runmenu50으로 MENU실행 -> FORM 호출

방법2) FORM에 MENU ATTACH
준비 항) $ORACLE_HOME/bin/sqlforms30에 menu에 관한 library가 link되어 있어야 한다.

```
$ cd $ORACLE_HOME/forms30/lib
$ make -f sqlforms30.mk sqlforms30menux
```

1. sqlmenu50에서 MENU생성
2. sqlforms30에서 FORM생성
3. FORM PROPERTY화면에서 MENU부분을 DEFAULT가 아닌 USER가 생성한 MENU명으로 지정
4. KEY-STARTUP TRIGGER에 SETTING사항

```
SHOW_MENU;
SET_INPUT_FOCUS(MENU);
```

5. FORM을 실행하면 MENU가 나타남

(V3.0) BUFFER AND ARRAY SIZE WITHIN SQL*FORMS 3.0

Bulletin no : 10250

SQL*Forms 3.0 Block Definition에서의 Buffer and Array Size parameters는 tuning parameters이다.
SQL*Forms Designer's Reference에서 보면:
ARRAY SIZE : database에서 한번에 fetch할 수 있는 최대record수를 말한다.
Default는 block의 한번에 display하는 숫자로 되어있다.
RECORDS BUFFERED : block에서 query를 행할 때 memory내에
SQL*Froms가 가져가는 record의 최대수를 말한다.
SQL*Forms는 어떤 추가적인 record들에 대해서는disk위에
temporary file을 이용하여 buffering을 한다.
이 값은 display되는 record의 수보다 커야 한다.

<예>
record 5개씩을 multi로 display하는 block에서 1000건의 record를 query해오는 경우.Array Size를 10으로 setting하면, execute_query를 수행하면 10건의 record가 database로부터 SQL*Forms로 들어온다. 만약 사 자가 11번째의 record로 down키를 누르면, SQL*Forms는 database로부터 11에서 20까지의 record들을 가져오게 된다.
적절한 Array Size를 setting함으로써, 한번에 모든 record를 query해 올 때 시간이 오래 경과되는 것을 눈에 띄지 않도록 적절하게 시간 분배를 할 수 있 한다.

위의 예에서, 10개의 record를 query해오는 것은 1000건의 record를 query하는 것보다 무척 많이 빠르기 때문에 11번째,21번째,31번째의 record를 access할 때 fetch하는 것을 느끼지 못한다.

Unfortunately, due to a bug, versions 3.0.16.7 to 3.0.16.10 of SQL*Forms
only performs the Array processing on the first batch of records. Please see bugs 97173 and 79733. (Abstract: "Arraysize is only used for first arraysize
number of records"). Version 3.0.16.10 and above have this bug fixed.

Records buffered 는 disk의 temporary file에 write하기 전에 memory내에 buffering되는 최대 record수를 나타낸다. disk에서 read/write하는 것보다, memory내에서 read/write하는 이 훨씬 빠르다. 이 값은 Array Size와는 전혀 관계가 없는 value이다.
Records Buffered parameter와 같은 수의 record가 SQL*Forms에 load되고나면, 다음에 query되는 record들은 temporary disk file에 기록된다.

이parameter들은 application을 tunning하기위해 query하는 동안에 setting될 수
있다. 만약 그 값들이 지정되어 있지않다면, SQL*Forms는 block당 query되는 수와 관계없이 모든 block에 대해 가능한 record -buffered의 수만큼 allocate하 고 시도하게 된다.

(V3.0) FORMS사용시 ORA-942 ERROR 처리

Bulletin no : 10272

==========
SQL*Forms30 BASE TABLE생성
==========

ACTION에서 NEW를 선택하였을때 ORA-942를 만나는 경우가 있다.
이것은 SQL*Forms3.0을 사용하기 위해 필요한 BASE TABLE들이 없기 때문에 발생하는 것이다. 다음은 SQL*Forms 3.0의 BASE TABLE을 생성하는 방법이다.

```
$ cd  $ORACLE_HOME/forms30/admin

$ sqlplus  system/manager

SQL> set  compatibility  v6

SQL> @iadtable

SQL> @iadindex

SQL> @iadview

SQL> @iadgrant
```

FORM에서의 2차원 TABLE집계 처리

Bulletin no : 10265

다음과 같은 화면구성을 갖고자 하는 경우

```
+-----------------------------------------------------------------+
¦        연   부서별 매출 현황                                    ¦
+--------+----+---+---+---+---+---+---+---+---+----+----+----+-----+
¦ 부서명 :1월¦2월¦3월¦4월¦5월¦6월¦7월¦8월¦9월¦10월¦11월¦12월¦총계¦
+--------+----+---+---+---+---+---+---+---+---+----+----+----+-----+
¦        ¦    ¦   ¦   ¦   ¦   ¦   ¦   ¦   ¦   ¦    ¦    ¦    ¦     ¦
+--------+----+---+---+---+---+---+---+---+---+----+----+----+-----+
¦        ¦    ¦   ¦   ¦   ¦   ¦   ¦   ¦   ¦   ¦    ¦    ¦    ¦     ¦
+--------+----+---+---+---+---+---+---+---+---+----+----+----+-----+
¦        ¦    ¦   ¦   ¦   ¦   ¦   ¦   ¦   ¦   ¦    ¦    ¦    ¦     ¦
+--------+----+---+---+---+---+---+---+---+---+----+----+----+-----+
```

BASE TABLE을 생성하여서는 위와 같은 표를 처리할 수 없다. 부서명 FIELD만 BASE TABLE FIELD로 생성하여 QUERY한 후 POST-QUERY TRIGGER에서 'SELECT SUM(..)... FROM 부서_TABLE WHERE DEPT = :DEPT' 의 방식으로 처리하면 PERFORMANCE가 나쁘다.

해결방법 예.

1> 우선 위의 표와 같은 형식을 갖는 빈 DUMMY TABLE을 생성한다.

```
CREATE TABLE  SUM_DUMMY  (  DEPT   CHAR(5),
                            M1     NUMBER(5),
                            M2     NUMBER(5),
                            M3     NUMBER(5),
                            .
                            .
                            M11    NUMBER(5),
                            M12    NUMBER(5),
                            MTOT   NUMBER(5));
```

2> FORM위에서 생성한 DUMMY TABLE을 BASE로 하는 BLOCK을 생성한다.

3> BLOCK DEFINITION 화면의 DEFAULT WHERE/ORDER BY절에 다음과 같이 기술한다.

```
WHERE   1 = 2
UNION
SELECT  DEPT, SUM(DECODE(SUBSTR(MDATE,3,2),*01*, AMT)),
              SUM(DECODE(SUBSTR(MDATE,3,2),*02*, AMT)),
              .
              .
```

```
                SUM(DECODE(SUBSTR(MDATE,3,2),*12*, AMT)),
                SUM( AMT),
                CHARTOROWID(1)
  FROM        매출
 WHERE      YMD LIKE *96%*
 GROUP BY    DEPT
```

4> POST-QUERY TRIGGER를 생성한다.

```
SELECT  DEPTNAME  INTO  :DEPTNAME
FROM   부서
WHERE  DEPT =  :DEPT
```

(V4.5) HOW TO IMPLEMENT MENU SECUTIRY IN FORMS4.5?

Bulletin no : 10255

ORACLE FORMS 4.5에서의 보안은 ORACLE7 SERVER의 ROLE의 개념에 기초를 둔다.
모든 USER들이 DATABASE에 저장된 모든 정보를 사용할 수 없기 때문에, ROLE은 USER들을 GROUPING하는데 유용하게 사용한다. (ROLE에 대한 자세한 개념은 MANUAL을 참조 바람.)
ORACLE SERVER에서 ROLE이 정의되면, FORMS개발자는 MENU MODULE과 MENU ITEM에 ROLE을 지정하여, MENU PROGRAM에 누가 그것을 사용할 수 있는지를 조정할 수 있게 된다.

* MENU APPLICATION을 사용하려면 EXP_FULL_DATABASE PRIVILEGE가 있어야 한다.

I. ROLE생성 방법

1. ORACLE FORMS4.5 DESIGNER에서 SYSTEM ACCOUNT로 CONNECT
2. FILE * ADMINISTRATION * DATABASE ROLES
   만약, ADMIN PRIVILEGE가 없는 USER로 CONNECT를 하게 되면
   이 OPTION이 DISABLE상태로 나타난다.
3. 새로운 ROLE을 생성하기 위해 NEW를 선택
   새로운 ROLE의 이름을 등록할 수 있는 WINDOW가 나타난다.
   새로운 ROLE의 이름을 입력하고, 그 ROLE을 갖게 될 USER명을 입 한다.
   MENU를 실행 시에 사용할 수 있는 OPTION (DEBUG MODE,O/S COMMAND,
   BACKGROUND MENU)을 원하는 데로 선택한다. MENU에서 위의 OPTION을
   사용하지 않으려면 BLANK로 남겨두면 된다.
4. OK BUTTON을 선택
   ROLE이 생성된다.

II. MENU MODULE에서의 SECURITY SETTING

1. MENU APPLICATION OPEN한 후 MENU APPLICATION의 PROPERTY화면을 띄운다.
2. SECURITY MENU SECTION에서
   1) MENU MODULE SECURITY * MORE BUTTON 선택 혹은 DOUBLE CLICK
      * 선택한 MENU APPLICATION의 사용을 허락할 ROLE명을 등록한다.
      * OK
      이 작업은 각각의 MENU ITEM을 사용할 수 있는 권한을 부여한 것은 아니다.
      단지, 각 ITEM마다 선택할 수 있는 ROLE을 지정한 것에 불과하다.
   2) USE SECURITY을 TRUE로 SETTING
      만약, TRUE로 SETTING하는 것을 잊어버린다면
      모든 USER들이 모든 MENU OPTION을 사용할 수 있게 된다.

III. MENU ITEM에 ROLE을 지정하는 방법

1. MENU ITEM의 PROPERTY화면을 띄운다.
2. SECURITY SECTION에서
   1) MENU ITEM ROLES에서 MORE BUTTON선택 혹은 DOUBLE CLICK
      * 앞의 과정에서 SETTING해 놓은 ROLE들이 나타난다.
      * 이 ITEM을 사용할 수 있게 하고자 하는 ROLE들을 선택한다.
        만약, 이 LIST에서 아무것도 선택하지 않으면,
        이 MENU는 모든 USER들이 사용할 수 없게 된다.
   2) DISPLAY W/O PRIVILEGE는 이 MENU ITEM을 사용할 수 없는 USER에게도 MENU ITEM을 보여 줄 것인가를 의미한다.
      만약, TRUE인 경우 이 ITEM을 사용할 수 없는 USER에게는 GRAYED OUT된 상태 즉 DISABLE된 상태로 나타난다.
      만약, FALSE인 경우 이 IETM을 사용할 수 없는 USER에게는 이 MENU ITEM이 나타나지 않게 된다.

IV. MENU 를 RUN시키는 방법

MENU을 RUN시키려면, 먼저 FORM에 MENU를 ATTACH하는 작업이 필요하다.

1. FORM MODULE을 OPEN한다.
2. FORM MODULE의 PROPERTY화면을 띄운다. (TOOLS* PROPERTIES)
3. MENU SECTION에서
   MENU MODULE에 생성해 놓은 MENU MODULE명을 등록한다.
   MENU STYLE을 선택한다.
   STARTING MENU를 입력한다.
   이것은 생성해 놓은 MENU MODULE내에 있는 MENU TREE구조 중 어느 POINT에서 부터 사용할 것인가를 지정하는 것이다.
4. FORM을 SAVE한 후 RUN시킨다.

V. 나타날 수 있는 ERROR들

FRM-10256 : "User Is Not Authorized To Run Oracle Forms Menu"
이 은 USER가 FORMS TABLE들을 ACCESS할 수 없다는 걸 의미한다.
FORMS에서의 SECURITY를 구현하기 위해서는, FORMS45에 관련된 TABLE과VIEW들이 생성되어 있어야 하고, 해당 USER는 이 TABLE과 VIEW를 사용할 수 있는 권한이 있어야 한다.
FORMS45에 관련된 TABLE과 VIEW를 생성하기 위해서는 관련된 SQL이SYSTEM ACCOUNT 하에서 실행되었어야 한다. (INSTALLATION GUIDE참조)

FRM-10249 : "No Authorization To Run Application"
이 ERROR는 MENU APPLICATION을 사용할 수 있도록 지정된 ROLE에 해당 USER가 ASSIGN되어 있지 않다는 것을 의미한다.

(V3.0) FORMS30 에서 LEFT,RIGHT KEY 를 사용하여 MENU DESIGN

Bulletin no : 10078

---

1. ?/forms30/admin/resource DIRECTORY 에서 RESOURCE FILE 을 COPY 한다.

```
$ cp vt220.r vt220a.r
```

2. ORATERM 을 이용 vt220a.r FILE 을 OPEN 하여 F0 및 F1 KEY 를 등록 한다.
   a. ORATERM 의 Product(RUNFORM) 에서 다음과 같이 등록한다.

Product Key Name	Function Code
Key 0	10022
Key 1	10023

b. Mapping 에서 Left,Right Key 의 Product Key Name 을 수정한다.

Product key name	Device Key Name
Key 0	Left
Key 1	Right

c. 수정된 내용을 Save 한다.

3. FORMS30 에서 Key-F0, Key-F1 TRIGGER 를 V2 TYPE 으로 정의한다.
Trigger Level : FORM
Trigger Style :V2 (V3 Trigger 화면에서 Trigger Style 을 V2 로 선택 하고 ZOOM IN 을 하면된다.)

Key-F0

```
-------
#exemacro case :system.trigger_field is
          when 'MENU.FLD1' then gofld MENU.LAST_FLD;
          when 'MENU.FLD2' then prvfld;
              .........
          when 'MENU.LAST_FLD' then prvfld;
          when  others then movleft
          end case;
```

```
Key-F1
-------
#exemacro case :system.trigger_field is
          when 'MENU.FLD1' then nxtfld;
              ........
          when 'MENU.LAST_FLD' then gofld MENU.FLD1;
          when  others then movright
          end case;
```

4. FORMS 기동한다.

```
$ runform30  menu -c vt220a:vt220  scott/tiger
```

(V4.5) MENU UPGRADE

Bulletin no : 10280

SQL*Menu 5.0에서 Oracle Forms 4.0로 혹은 From V4.0에서 V4.5로의 upgrade는 forms upgrade의 과정과는 다르게 구성되어 있다.

Upgrading From SQL*Menu 5.0
==========

Menu Upgrade를 위해서는 forms upgrade에서와 마찬 가지로 Oracle Forms Generator를 사용한다. Forms를 upgrade할 때는 File-To-File Conversion으로서 하나의 file (.INP)에서 forms 4.5 module인 .FMB를 생성한다. 그러나 SQL*Menu 5.0 Module DATABASE에 저장되어 있기때문에generate시에 source file을 명시하지 않는다. (DMM File은 menu module의 compile된 version으로 읽을 수 없는 file이다). 결국, menu를 upgrade하기 위해서 generator는 Base Table To Base Table Conversion이 이루어진다.

준비확인사항

Menu upgrade를 위해서는 우선적으로 SQL*Menu 5.0에 관련된 base table과 ORACLE forms 4.5에 관련된 base table이 생성되어 있어야 한다.

1. SQL*Plus에서 SYSTEM으로 DATABASE에 log on.한다.
2. DESC MENU_B_APPL
   만약 description이 나타나지 않 나, 어떤 column이 CHAR datatype으로 나타나면 SQL*Menu 5.0 base table을 install하여야 한다.
3. DESC FRM40__APP
   만약 ORA-942 error가 나타나면서 description이 나타나지 않으면 ORACLE forms base table을 다시 install하여야 한다.
4. forms V4.5 사용자는 위의 3번 사항대신에 다음을 확인하여야 한다.
   DESC FRM45__OBJECT
   만약 ORA-942 error이 나타나면 ORACLE forms V4.5 base table을 다시 install하여야 한다.

SQL*Menu 5.0 Application을 직접 V4.5로 upgrade할 때

$ORACLE_HOME/Forms45/Doc/Relnotes.Doc The Developer 2000 Release Notes참조.
SQL*Menu 5.0 Applications을 upgrade하려면 Forms 4.0 Base Table이 isnatll되어져 있어야 한다. 만약 Forms 4.0의 base table이 install되어 있지 않으면 다음의 작업들이 필요하다.
Forms 4.0 base table을 생성하는 SQL script들은 ORAWIN\FORMS45\SQL directory밑에 있으며 SYSTEM user상에서 수행되어져야 한다. SQL*Menu 5.0 Application을 migration하기 위해서는 SQL*Plus 에서 SYSTEM user로 다음의 script를 수행하여야 한다.

```
SQL> @Frm4bild.Sql
```

만약 SQL*Menu 5.0 Application의 base table을 생성하였었다면, 다음의 script를 이용하여 Forms 4.0 Base Tables을 drop시킬 수 있다.

SQL> @Frm4drop.Sql

위 script를 수행하기 전에, 사용자는 반드시 다른 사용자가 Forms 4.0 Base Tables을 생성해 놓았거나, 사용하고 있지는 않는지 확인하여야 한다.
반대로, menu upgrade를 수행하여 같은 이름의 application을 생성하면 다른 application을 없앨 수도 있으므로 VERSION=50 (command line으로 수행할 때) 혹은 *50* (generator 화면에서)을 반드시 지정하여야 한다.

Menu Module이 반드시 DATABASE에 저장되어 있어야 한다

---

1. SQL*Plus에서 SYSTEM으로 DATABASE에 log on.한다.
2. upgrade하고자 하는 menu module이 존재하는지 확인한다.
   SELECT * FROM MENU_B_APPL WHERE APPLICATION_NAME='MYAPPNAME';

How To Upgrade Menus From SQL*Menu 5.0

==========

menu의 upgrade작업은 Oracle Forms Generator를 통해서 이루어 진다. Generator을 호출하기 위해서 Icon을 double click하거나(windows에서), F40GENM (Or F45GENM For V4.5 Users)(UNIX에서)을 사용한다. 그러면 Generator Dialog Box가 나타난다.
기본적인 menu upgrade를 하려면 다음의 property를 지정한다.

File: DATABASE에 저장되어 있는 menu module . 대소문자 구별없음
Username, Etc.: Username, Password, Connect String
Logon To The Database: ON
Module Type: Menu
Module Access: Database
Upgrade 3.0 Form...: ON
Upgrade 5.0 Role...: OFF

나머 는 기본값으로 남겨둔다. OK button을 눌러 수행하면 주기적으로 수행중에 안내정보가 나타난다.

generator를 command line상에서 수행할 수도 있다.

```
F40GEN  MODULE=  USERID=  MODULE_TYPE=MENU
        MODULE_ACCESS=DATABASE  UPGRADE=YES
```

Upgrading From Oracle Forms 4.0 To Oracle Forms 4.5

==========

Oracle Forms 4.0에서 Oracle Forms 4.5 으로 upgrade하는 것은 상대적으로 쉽다.
각각의 V4.0 .MMB File을 단순하게 Oracle Forms 4.5에서 open하여 다시 Save하면 된다. 그러면 해당 application은 V4.5 file이 된다. 하지만 반대의 작업은 불가능하다. 즉, V4.5 file은 V4.0 file로 되돌릴 수 없다.
또 다른 방법은 다음과 같이 Oracle Forms 4.5 Generator을 사용하는 것이다.

File: .MMB File명. Wildcard 사용할 수 없다.

Username, Etc.: Username, Password, Connect String
Module Type: Menu
Module Access: File
Upgrade 3.0 Form...: OFF

Batch Mode에서는
F45gen Module=Mymmb Userid=Scott/Tiger Module_Type=Menu Batch=YES

Role Upgrading From SQL*Menu 5.0
==========

Menu V5.0 Role을 Oracle Forms로 upgrade하는 방법은 다음과 같다.
File: Blank
Username, Etc.: Username, Password, Connect String
Module Type: Menu
Module Access: Database
Upgrade 3.0 Form...: OFF
Upgrade 5.0 Roles...: ON
Batch Mode에서는
F40gen Userid=Scott/Tiger Module_Type=Menu Upgrade_Roles=YES

NOTE: Earlier Versions Of Oracle Forms Required A Menu Module To Be Upgraded. When Upgrading Roles, So In Versions Earlier That V4.0.13, You Need To Supply A Module Name To Upgrade And The Upgrade 3.0 Form Flag To ON.

V5.0 Role을 Forms V4.5에서 사용하는 ORACLE7 Roles로 upgrade하려면 다음과 같이 지정한다.
File: Blank
Username, Etc.: Your Username, Password, Connect String
Module Type: Menu
Module Access: Database
Upgrade 3.0 Form...: OFF
Upgrade 5.0 Roles...: ON
Version: 50

Batch Mode에서는 다음과 같이 사용한다.
F45gen Userid=Scott/Tiger Module_Type=Menu Upgrade_Roles=YES Version=50

Role Upgrading From Oracle Forms 4.0
==========

Menu security를 위한 Version7의 ROLE사용부분은 forms V4.0과V4.5의 기능면에서의 큰 차이점중의 하나이다. 과거에는 MENU에서의 role은 customer가 지정한 role에 의존하여 security를 관리하였고, 이것들은 base table내에 저장되어 관리되어 왔 . 그러나 V4.5에서는 ORACLE 7의 Role을 사용할

수 있다. V4.0에지정된 Role을 ORACLE 7 server의 Role로 upgrade하려면 SQL*Plus에서 SYSTEM user로 FRM45ROL.SQL script를 수행하여야 한다.
이script는window에서는 \ORAWIN\FORMS45\SQL에위치하고,
UNIX에서는$ORACLE_HOME/Forms45/Admin/SQL에 위치하며 Oracle:Scripts:Cde에 위치한다.
이 script는 PL/SQL script로써 Oracle Forms 4.0 Base Tables에서 자료를 읽어 ORACLE7 data dictionary table에 정보를 insert하는 script를 생성한다. 이 script가 또다른 scipt를 생성하고 실행하기 때문에 해당 user는 OS에 대해 writer permission을 가지고 있어야만 한다.

Batch Upgrading
==========

한개이상의 V5.0 Menu을 한번에 upgrade하기위해서는 Generator가 module명으로 wildcard를 받을 수 있어야 한다. 예를 들어 module명에 A%로 지정하면 owner에 관계없이 *A*로 시작하는 모든 menu module이 upgrade되어야 한다. 또한 menu module전부를 upgrade하려면 module명에 %만 입력하면 되도록 되어야 한다. 그러나 불행히도 File to file conversion은 wildcard처리가 되지 않기때문에 하나 이상의 V4.0 menu를 V4.5로 upgrade하려면 generator를 여러번 수행하거나 shell script를 이용하여 batch로 수행하여야 한다.

Troubleshooting
==========
Q. F40gen 을 수행하면 FRM-91500: Unable To Start/Complete The Generation 발생

A. F40genm을 사용한다면 DISPLAY Variable을 checking해보고, X Windows System이 정상적으로 run되는지 확인한다.
F40gen을 사용한다면 TERM과 ORACLE_TERM Environment 를 확인한다.

Q. SQL*Menu 5.0 Menu를 Oracle Forms Module로 upgrade하려면
MNU-10637: Error Performing Upgrade발생(FRM-10637, FRM-31637도 함께 발생)

A.
1. 이것은 SQL*Menu V5.0 application을 ORACLE forms V4.0으로 upgrade할 때 나타나는 일반적인 error이다. 이 error가 발생하였다면 이것은 대부분 SQL*Menu V5.0 base table과 ORACLE Forms V4.0 base table이 같은 database에 있지않거나 사용할 수 없을 때 발생한다.
2. 그외의 check할 점
  1) conversion하고자하는 application이 SQL*Menu base table에 저장되어 있는가?
     다음 sql을 이용하여 확인한다.
     SELECT * FROM MENU_B_APPL WHERE APPLICATION_NAME LIKE 'MYAPP';
  2) SQL*Menu V5.0 base table에 item이 CHAR type을 갖지 않는가?
     DESC MENU_B_APPL
  3) 충분한 tablespace영역과 rollback segment가 확보되어 있는가?
  4) 같은 이름의 forms application 이 이미 기존에 존재하지 않는가?
그렇다면 generator는 같은 이름의 menu application으로 convert할 수 없다.

Q. upgrade된 Menu Application을 수행하면서 다른 form program를 호출하는 menu를 선택하면 FRM-10242: Cannot Call Linked-In Oracle Forms From Oracle Forms 발생

A. 이것은 SQL*Menu V5.0에서 생성한 application을 upgrade한 경우에 발생한다. SQL*Menu V5.0 application는 대부분 사용자가 menu를 run하고,그 안에서 forms를 호출(type 4 command)하도록 되어 있다. 그러나 ORACLE forms에서는 form이 중심이 되고있다. 즉 사용자는 menu가 attach되어 있는 form을 사용하게 되어있다.
위의 문제는 type 4 commanad를 conversion한데서 발생한다. 이것은 link되어 있는 form을 실행하려 한다. 그러나 V4.5에서는 이미 form이기 때문에 error가 발생하는 것이다.
이 문제를 해결하려면 type 4인 command를 PL/SQL command로 바꾸고 command like에 지정되어 있는 runform30 ....의 문장을 CALL_FORM(*FORM_NAME*)으로 변경해야 한다.

Q. (MS Windows Only) Menu Item에서 SQL*Plus를 실행하려하면 화면이 번쩍하고나서 아무 변화가 없다.

A. SQL*Menu V5.0 Applications에서 upgrade한 경우에 많이 발생하는 현상이다.
SQL*Plus를 호출하는 command를 이전의 "Sqlplus Scott/Tiger" 에서 "Plus31 Scott/Tiger"로 변경하라. ORACLE Forms menu에서는 SQL*Plus 를 os command로 호출하게 된다.
PL/SQL menu item에서 SQL*plus를 HOST문을 이용하여 호출하거나, SQL*Plus menu item type을 이용하여도 마찬가지이다.
이것은 window에서의 window command로 인식하도록 되는 것이다. 그것이 실패하면 이것은DOS로 넘겨진다.
결국 ORACLE forms에서는 처음에 window program에서 SQLPLUS를 실행하려 한다. 그러나 windows program중에는 SQLPLUS라는 program이 없기 때문에(windows에서는 executable name이 PLUS31로 되어 있기 때문에) 실패하고 DOS로 넘겨져 그곳에서 실행하려 한다. 그러나 DOS에서도 실패하게 되는 것이다. 이것이 일반적으로 회면이 번쩍하게 되는 원인이다.

Q. SQL*Menu 5.0 Menu를 Oracle Forms 4.5 Menus 로 F45GEN을 사용하여 upgrade하려 할 때 FRM-10043 "Cannot Open File" 가 발생한다.

A. Upgrade opetion이 YES인 것과 Upgrade = 50으로 upgrade하려는 version을 정확히 주어있는지 확인한다.

(V4.5) MULTI FORM GENERATION
(UPGRADE)

Bulletin no : 10256

I. Introduction

Developer/2000의 'Forms Generate' Icon에서 실행하는 Oracle Forms Generator(F45gen.Exe)는 한 Module별로 처리하도록 되어있어서 User가 계속 지켜보다가 실행이 끝나면 다음 Module을 다시 입력하여야만 한다. 이 자료는 이러한 불편함을 해소할 수 있게 해 주며, Forms 2.0, Forms 2.3, Forms 3.0, Forms 4.0, And Forms 4.5 어느 Version에 대해서도 처리할 수 있게한다.
Forms 2.3까지는 File System에만 저장할 수 있었다. Forms 3.0부터는 Source를 File System과 Database중 하나를 선택할 수 있 되었다. 이 자료는 2가지의 큰 Category 즉 'Generating From The Filesystem' 와 'Generating From The Database*로 나누어 처리한다.

II. Generating From The Filesystem

1. Unix/VMS Files

Form의 모든 Version이 Source를 Filesystem에 저장할 수 있기때문에 Source File에 대한 Generate/Upgrade작업을 Batch로 처리 할 수 있다.

다음은 Forms 2.3에서 Upgrade하는 예이다. 이것을 하기위해서 Version=23을 Setting하여야 한다.

```
-------------------------------------------------------------------
    F45gen Module=Forma Userid=Username/Password Version=23 Upgrade=Yes
              .
              .
              .
    F45gen Module=Formz Userid=Username/Password Version=23 Upgrade=Yes
-------------------------------------------------------------------
```

SQL*Forms V3.0에서도 Version=30을 Setiing할 수 있으나, 꼭 해야 필요는 없다. Oracle Forms 4.0에 대해서는 Version이나 Upgrade Parameter를 특별히 지정 할 필요가 없다.

2. Windows Files

Microsoft Windows을 사용하는 System에서 Generation/Upgrade는 DOS Batch File로 Window Executable를 실행하는 것이 쉽지 않다. Form을 Generate/Upgrade하기 위해서는 Batch Processor의 역할을 하는 Form을 생성하여 처리하는 것이 좋다.

2.1 모든 Field를 PC에 가져다 놓는다.

2.2 위에서 언급한 바와 같이 F45gen은 Window Executable이어서 PC에서 Batch File을 생성하는 것이 어렵다. 대신 Generat/Convert할 File들을 Temporary File에 모아서, 이 File로 부터 이름을 읽어들여 Host Command를 이용하여 Generate하는 방법을 사용한다.

2.3 다음의 Item들을 갖는 Non-Base Block을 생성한다.

Item1: Forms Version
이 Field는 꼭 필요하다. 이것은 Forms 2.0,2.3과3.0에 대한 Version(20,23,30,40,45)을 넣기위한 것이다.
40 And 45에 대해서는 다음에서 언급한다.
2.0,2.3,3.0을 입력하지 않도록 한다.

Item2: Directory Name
이 Item은 File들이 위치하는 Directory를 지정하거나 Null로 비워져 있어야 한다. 만약 값이 주어지지 않으면 Icon과 연결된 현재의 Directory나 FORMS45_PATH에 지정된 Directory를 의미한다.

Item3: Username
이 Form을 가지는 User의 이름을 지정한다.

Item4: Password
위에 지정한 User에 대한 암호를 입력한다.

Item5: Connect String
Server Database에 Connect하기위한 String을 준다.

Button1: 작업을 시작할 Button을 생성한다. (Label=File)
다음의 Source Code를 갖는 When-Button-Pressed Trigger를 갖는 Button을 생성한다.

```
------------------------------------------------------------          DECLARE
    File_Pointer  TEXT_IO.FILE_TYPE;
    Record_Buf    VARCHAR2(200);
    Host_Buf      VARCHAR2(200);
    Usr_Name      VARCHAR2(40);
    Usr_Pwd       VARCHAR2(40);
    Usr_Cnct      VARCHAR2(40);
    File_Ext      VARCHAR2(4);
  BEGIN
    /* Checking The File Extension
    */
    IF :Item1 IN ('20', '23', '30') THEN
       File_Ext := 'INP';
    ELSE
       File_Ext := 'FMB';
    END IF;
    /*Get The Username And Password
    */
    IF :Item3 IS NULL THEN
       Usr_Name := GET_APPLICATION_PROPERTY(USERNAME);
       Usr_Pwd  := GET_APPLICATION_PROPERTY(PASSWORD);
       Usr_Cnct := GET_APPLICATION_PROPERTY(CONNECT_STRING);
    ELSE
```

```
   Usr_Name := :Item3;
   Usr_Pwd  := :Item4;
   Usr_Cnct := :Item5;
END IF;

IF Usr_Cnct IS NOT NULL THEN
   Usr_Pwd  := Usr_Pwd || '@' || Usr_Cnct;
END IF;

/* Build The Host String To Collect The .Inp/.Fmb Files
*/
IF :Item2 IS NOT NULL THEN
   Host_Buf := 'Dir ' || :Item2 ||  '\*.' || File_Ext || ' > Junk.Txt';
ELSE
   Host_Buf := 'Dir *.' || File_Ext ||' > Junk.Txt';
END IF;

/*  The Following Message Will Display The String To Be
   Executed In The HOST Command.  If Junk.Txt Does Not
   Contain The Correct File Name, Modify The Above
   Code To Fix The Problem.
*/
MESSAGE(Host_Buf);
SYNCHRONIZE;
HOST(Host_Buf, NO_SCREEN);
/* Opening Junk.Txt
*/
File_Pointer := TEXT_IO.FOPEN('Junk.Txt', 'R');
/*
   Looping To Read And Generate The File
*/
File_Ext := ' ' || File_Ext;
LOOP
   TEXT_IO.GET_LINE(File_Pointer, Record_Buf);
   IF INSTR(UPPER(Record_Buf), File_Ext)>0 THEN
      Host_Buf := REPLACE(SUBSTR(Record_Buf, 1, 8),* *,Null);
      Host_Buf := 'C:\ORAWIN\Bin\F45gen.Exe Module=' ||
                  Host_Buf || ' Userid=' || Usr_Name || '/' ||
                  Usr_Pwd|| ' Batch=Yes Upgrade=Yes Version=' || :Item1;
      HOST(Host_Buf);
   END IF;
END LOOP;

/*  Close The File When It Is Done
*/
EXCEPTION
```

```
        WHEN NO_DATA_FOUND THEN
           TEXT_IO.FCLOSE(File_Pointer);
    END;
```

III. Generating From The Database

Form이 Database에 저장되어 있는 경우를 살펴보자.

이 경우에 Generate가 Server에서 이루어지느냐, PC에서 이루어 지느냐에 따라 다르다. 어느 경우이든지 Forms 4.5에 관한 Base Table들은 반드시 Install이 되어져 있어야 한다.

Installatio Guide참조바람.

1. Server

Forms 2.0과 Forms 2.3에서는 Source가 File Syatem에만 저장되기 때문에 이 과정을 사용할 수 없다.

```
F45gen Module=Forma Userid=Username/Password Module_Access=Database Version=30 Upgrade=Yes
          .
          .
F45gen Module=Formz Userid=Username/Password Module_Access=Database Version=30 Upgrade=Yes
```

Forms4.0인 경우는 Version이나 Upgrade Parameter를 기술할 필요가 없다.

만약 Database에 저장된 Form의 이름을 모른다면, 다음에 설명하는 PC에서의 방법과 같이 사용하여도 된다.

2. PC

Database에 저장된 Form에 대해 Generate/Upgrade하는 경우는 PC의 Filesystem에서의 작업에 몇 가지의 과정이 더 필요하다.

Button2: Button To Start The Process (Label=Database)

When-Button-Pressed Trigger에 다음의 Source Code를 갖는 Button을 생성한다.

```
DECLARE
        Host_Buf      VARCHAR2(100);
        Usr_Name      VARCHAR2(40);
        Usr_Pwd       VARCHAR2(40);
        Usr_Cnct      VARCHAR2(40);

    /* Cursor Declaration For Forms 3.0
    */
    CURSOR C30 IS
       SELECT Appname
       FROM Form_App
```

```
   WHERE Appowner = Usr_Name;
/*  Cursor Declaration For Forms 4.0
*/
CURSOR C40 IS
   SELECT Appname
   FROM Frm40_App
   WHERE Owner = Usr_Name;
/* Cursor Declaration For Forms 4.5
*/
CURSOR C45 IS
   SELECT Name
   FROM Frm45_Object
   WHERE Owner = Usr_Name
   AND Objecttype = 1;

BEGIN
   /* Checking The File Extension
   */
   IF :Item1 NOT IN ('30', '40', '45') THEN
      MESSAGE('Only Valid Values Are 30, 40 And 45.');
      RAISE FORM_TRIGGER_FAILURE;
   END IF;

   /*  Get The Username And Password
   */
   IF :Item3 IS NULL THEN
      Usr_Name := GET_APPLICATION_PROPERTY(USERNAME);
      Usr_Pwd  := GET_APPLICATION_PROPERTY(PASSWORD);
      Usr_Cnct := GET_APPLICATION_PROPERTY(CONNECT_STRING);
   ELSE
      Usr_Name := :Item3;
      Usr_Pwd  := :Item4;
      Usr_Cnct := :Item5;
   END IF;

   IF Usr_Cnct IS NOT NULL THEN
      Usr_Pwd  := Usr_Pwd || '@' || Usr_Cnct;
   END IF;

   /*Opening The Cursor
   */
   IF :Item1 = '30' THEN
      OPEN C30;
   ELSIF :Item1 = '40' THEN
      OPEN C40;
   ELSIF :Item1 = '45' THEN
```

```
      OPEN C45;
   END IF;

   /*  Looping To Read And Generate The File
   */
   LOOP
      IF :Item1 = '30' THEN
         FETCH C30 INTO Host_Buf;
         EXIT WHEN C30%NOTFOUND;
         Host_Buf := 'C:\ORAWIN\Bin\F45gen.Exe Module=' ||
                     Host_Buf || ' Userid=' || Usr_Name || '/' ||Usr_Pwd
                     || '   Batch=Yes Module_Access=Database Version=30';
      ELSIF :Item1 = '40' THEN
         FETCH C40 INTO Host_Buf;
         EXIT WHEN C40%NOTFOUND;
         Host_Buf := 'C:\ORAWIN\Bin\F45gen.Exe Module=' ||
                     Host_Buf || ' Userid=' || Usr_Name || '/' ||
                     Usr_Pwd|| '   Batch=Yes Module_Access=Database';
      ELSIF :Item1 = '45' THEN
         FETCH C45 INTO Host_Buf;
         EXIT WHEN C45%NOTFOUND;
         Host_Buf := 'C:\ORAWIN\Bin\F45gen.Exe Module=' ||
                     Host_Buf || ' Userid=' || Usr_Name || '/' ||
                     Usr_Pwd|| '   Batch=Yes Module_Access=Database';
      END IF;

      HOST(Host_Buf);
   END LOOP;

END;
-----------------------------------------------------------------------
```

(V4.5) NEW FEATURES IN 4.5.7 (DEV/2000 R1.3)

Bulletin no : 10290

---

1. Native 32-Bit Windows95 And Windows NT Support

2. Improved Runtime Startup Performance
 Runtime Module의 Startup시간이 줄어들게 되었다.
 그것으로 인해 2개의 Runtime Executable을 가지게 되었다. 하나는 Pl/SQL Debugger를 사용하기위한 것이고 다른 하나는 Non-Debug Runtime을 위한 것이다. Forms Designer상태에서 Debug Option을 On 시킨 후 Form을 Run시키면 Debug Executable이 자동으로 호출되게 된다.
 만약 Debug=Yes로 Setting한 뒤 Non-Debug Version을 실행하면 다음과 같은 Error Message를 받게 된다.
 Frm-41838 : Use The Debugger-Enabled Executable If Specifying Debug=Yes

 새로운 Executable의 이름은 다음과 같다.

	Win16	Win32
Non-Debug	F45run	F45run32
Debug	F45rund	F45dbg32

3. List of Recently Used Files
 MS Word나 MS Powerpoiint에서와 같이 File Menu에 가장 최근에 사용했던 File들에 대한 List가 나타난다. 이 기능은 모든 Platform에서 가능 다.

4.1 Get_Application_Property
 Get_Application_Property Built-in에 Windows95에 해당하는 값이 추가되었다.
 Windows94에서 다음과 같은 문장을 실행하면
 ui := Get_Application_Property(User_Interface)
 os := Get_Application_Property(Operating_System)
 ui에는 "MSWINDOWS32"라는 값이 os에는 "WIN32COMMON"의 값이 들어온다.

5.1 Developer/2000 Fast Loader
 MS Windows의 strartup group에 D2KINIT.EXE icon을 설치해 두면, Developer/2000 DLL 들이 Memory 내에 미리 상주하여, 다음에 있을 Developer/2000제품이 빠르게 Start Up하게 해준다.
 단, 이 기능은 MS Windows에서만 가능하고 Windows95에서는 사용할 수 없다.

6. New Get_Group_Record_Number Built-In
 Get_Group_Record_Number Built-in이 새로 추가되었다.
 이 Built-in은 Record Group에서 보다 쉽게 Match되는 Record를 찾게 해 다.
 정의는 다음과 같다.

Get_Group_Record_Number(groupcolumn_id,cell_value) returns Number
Get_Group_Record_Number(groupcolumn_name,cell_value) returns Number

Group내에서 지정한 cell_value와 Match되는 첫번째 Record의 번호를 Return한다.
만약 Match되는 Record가 없으면 0(Zero)를 Return한다.
Cell_Value가 Column의 Datatype과 Match되지 않는다면, Error가 발생한다.
값은 정확히 Match될때만 Retrun된다. 대소문자를 구별하여 처리하여야 한다.

7. Japanese Shojhint.Dll For Ms Windows
Iconic Button에 Bubble Help를 표시하는데 사용하는 SHOWHINT.DLL의 일본어 version을 포함하고 있다. 만약 이것을 사용하고 있다면, 단순희 SHOWJHINT.DLL을 SHOWHINT.DLL로 같은 driectory내에서 Re-Name시켜주면 된다.

Windows95에서는 DLL들이 SHWHNT32.DLL과 SHJHNT32.DLL로 되어있다.
또한 Windows95에서는 32-bit SHWHNT32.DLL 을 사용하기위해 새로운 HINT.PLL이 제공된다.

8. New Generator Checks Relations
새로운 Generator는 불완전한 Relation에 대해서는 다음과 같은 주의Message를 준다. 주의 Message가 생겨도.FMX File은 생성된다. 그러나 Developer/2000 R2.0에서는 Warning이 아닌 Error로 대치되어 .FMX File이 생성되지 않게 될 것이므로 R 2.0으로 Upgrade하기 전에 사용자는 문제를 Fix해야만 할 것이다.

FRM-30370: Warning! Relation*s detail block is control block.
FRM-30371: Warning! Relation*s master block is control block.
FRM-30372: Warning! Relation*s detail block is the same as its master block.

다음의 새로운 Message는 Error로써, 이 Error가 발생하면 .FMX file이 생성되지 않게 된다.
FRM-30368: Warning! Relation has no detail block.
FRM-30369: Warning! Relation*s detail block does not exist.

9. New When-Form-Navigate Event For Closing Form
Open_Form Procedure를 사용하여 Open되었던 Form이 Close되면서 호출한 Form으로 되돌아올때 Fire되는 When-Form-Navigate Trigger가 추가되었다.

예를들어, Form1과 Form2가 있고 Form1에 When-Form-Navigate Trigger가 있다고 가정하자. 이전 version에서는 사용자는 Form2에 위치하고 있으면서 Exit_Form를 수행하였다면 Form1의 When-Form-Navigate Tigger는 Fire되지 않았었다.
현재의 Version에서는 이 Trigger가 Fire되어,Open_Form을 사용하여 Multi-Form을 사용할 때 부가적인 Event를 잡아낼 수 있게 되었다.

10. New Tk23_Icon Variable For Icon Search Path
Developer/2000 R1.3 (Forms4.5.7)에 는 이전 Version에서 ICON에 관한 PATH를 지정하던 TK21_ICON Variable대신에 TK23_ICON을 사용한다.
ORACLE Installer에서는 기존에 ORACLE.INI에 지정되어 있던 TK21_ICON애 대한 정보를 TK23_ICON으로 UPDATE 한다. 만약 사용자가 자신의 Application에서 TK21_ICON 수를 사용하였다면 수정해 주어야 할 것이다.

11. More Meaningful Errors If Unable To Write To Disk
Disk에 Buffer Data를 Write할 때 발생한 Error에 대해 좀더 자세한 Error를 제공한다.

FRM-41835 : Unable to open temporary record buffer file
FRM-41836 : Disk I/O error on temporary record buffer file

은 File명이거나 File의 위치를 나타낸다.
만약 이 Message를 만나게 되면, 사용자는 반드시 해당 Drive나 File System이 Write할 수 있는지 확인해 보아야 할 것이다.

12. Changed Behaviour Of Set_Record_Property
Non-Base Table Block에 대한 Set_Record_Property Built-in은 더 이상 작용하지 않는다. 이 Built-in은 Record Status만 바꾸도록 하기때문에 Non-Base Table Block에 대해서는 영향을 주지 않게 된다.

만약 Non-Base Table에 대해서 이 Built-in을 사용한다면 Consistency가 불일치하는 형상이 발생 할 수도 있으며, 다음과 같은 Message를 만나게 될 것이다.
FRM-40750 : Record .: Can*t set status to QUERY ot CHANGED in a control block

13. New Error Message For Character Sets
만약 다음과 같 Message를 만난다면
FRM-91123: internal Error : Unable to load NLS character set.
Forms가 Character Set에 관련된 *.D File을 찾지못하여 발생하는 것이다. 이문제를 해결하려면 환경변수중 ORA_NLS가 있는지 확인하고, *.D File이 위치한 곳을 지정하고 있는지, 실제 그 위치에 해당 File들이 있는지를 확인한다.

14. Improved Consistency Of Rendered And Non-Rendered Fields
이전 version에서는 Rendered, Non-Rendered Filed들이 Field가 Disable되었을 때 Color의 변화가 일정치 않았었으나 이제는 Consistency를 유지하게 되었다.

15. Re-Linking User Exits
이전 Version에서 제출되었던 문제를 해력하기 위해 F45XTB.DLL File이 Update되었다. Update된 이 File을 사용하기 위해서는 가지고 있는 User Exit들을 Re-link해주어야 한다.

16. New Windows95 Features
32-Bit Windows환경에 사용할 때Performance를 위해,Developer/2000에서는 Windows95의 다음 Feature들을 Support한다.

- Long Filename
- 32-Bit Executables
- MAPI Support
- OLE2 Drag-and-Drop
- Use Registry (ORACLE.INI대신)
- Explorer View
- UNC (Universal Naming Convention)
- Multiple Source Files Launched into Designers
- Recent List of Files Used

1) Ocx Support

Developer/2000 R1.3에서는 OCX Support의 제약사항이 있다. 이러한 점들은 Developer/2000에서 Full Support될 것이다.

Forms 4.5.7에서의 가능은 다음과 같다.

. OLE contatiner내에 OCX를 Insert한다.

. 오른쪽 Mouse를 눌러 Property화면을 불러 OCX의 Property를 Setting한다.

. 실행중에 OLE-style Automation Interface를 호출한다.

R2.0전까지는 다음 기능은 제공하지 않는다.

. Property palette에서 OCS Property를 확인하고 지정한다.

. 실행중에 Property를 Programmatically받거나 지정한다.

. 실행중에 Event를 받는다.

. OCX-style method를 호출한다.

2) Adoption Of Windows95 Look-And-Feel Standards

Windows95의 Look-And-Feel 형식을 따른다.

그중 특별 것중 하나는 Disable된 Field는Text와 Background가 전부 Greyed된다.

3) New Ballon Help For 32-Bit Applications

Developer/2000 32-Bit version에서는 Balloon Help를 사용하기 위해 새롭게 32-Bit용 SHWHNT32.DLL을 제공한다.

이것을 사용하기 위해서는 단순히 ORAWIN95\FORMS45\PLSQLLIB Directory내의 HINT.PLL을 사용하면 된다. 이것은 자동으로 새로운 32-Bit DLL을 찾아온다.

1 New Get_Applications_Property Results For Win95

Get_Application_Property Built-in에 Windows95에 해당하는 값이 추가되었다.

Windows94에서 다음과 같은 문장을 실행하면

```
ui := Get_Application_Property(User_Interface)
os := Get_Application_Property(Operating_System)
```

ui에는 "MSWINDOWS32"라는 값이 os에는 "WIN32COMMON"의 값이 들어온다.

4) Host()- Synchronous Vs Asynchronous

HOST( ) command는 일반적으로 synchronous하게 실행된다. 그러나 Windows95에서는 변칙적으로 16Bit Version의 Form에서 32-Bit Command를 Asynchronous하게 Hosting한다.

Win95에서의 HOST( ) 의 작용에 대한 Matrix는 다음과 같다.

Forms	16Bit Command	32Bit Command
f45run	synch	asynch
f45run32	synch	synch

5) Regenerating Files For 32bit (And Vice Versa)

16 Bit Version에서 생성한 File을 32Bit Tool에서 실행하려면 다시 Generate하면 된다.

같은 방식으로 32Bit에서 16Bit로 옮기려면 역시 다시 Generate하면 된다

6) Changes In Office 95 OLE2 Automation

MS WIndows3.1하에서는 AppActive Method만을 사용하여야 했으나 Windows95에서는 AppShow Method도 사용할 수 있게 되었다.

7) Win95 And Winnt Installers

Developer/2000 CD에는 2개의 Installer가 들어 있다. 하나는 Windows95용이고 하나는 WinNT용이다. 환경에 맞게 정확한 Installer를 실행하여야 한다.

만약 두가지 방식의 Boot를 하는 기계라면, 각각의 OS에 맞게 Install하여야 한다.

8) Double Click Events On Poplists

Windows95에서는 Poplist에서 Double Click이 실행되지 않는다. 그것은 처음 Click은 List를 Open하

는 것으로 인식하고 두번째 Click은 Close하는 것으로 인식하기 때문이다.

만약 Application의 Poplist에서 Double Click이 제공되게 하려면 Windows95에서Poplist가 다시 Coding되어야 한다.

9) Windows95 Memory Requirements

Windows95 에서 Developer/2000 Forms를 사용할때 Design시에는 16M, Runtime시에는 12M를 Recommand한다. 만약 Reports와 Graphics 역시 사용해야 한다면 각각 2M씩 더 필요하다고 본 .

(V3.0) ORATERM 을 이용한 FORMS30 FUCTION KEY 수정

Bulletin no : 10067

먼저 EMULATOR 상에서 FORMS30 을기동하였을경우 GRAPHIC(BOX 및 LINE)을 확인하여 정상적으로 DISPLAY 되는 RESOURCE FILE을 선택한다. EMULATOR 를 이용하는경우에는 VT220 TERMAINAL FILE 를 사용하면 FUNCTION KEY 를 쉽게 수정 할수 있다.

1. EMULATOR 를 사 하여 HOST 에 login 상태에서 각 FUNCTION KEY 의 ESCAPE SEQUENCE 값을 확인한다. 예를들어 F1 KEY 를 입력했을 경우 \eOP, F2 는 \eOQ 등이 나오는지 확인 하고 값이 나오지않으면 그 FUNCTION KEY 를 사용 할수 없으므로 EMULATOR 에서 값을 정의한다.

2. RESOURCE FILE 이 있는 DIRECTORY 로 변경하여 FILE 을 COPY 한다.

```
$ cd  $ORACLE_HOME/forms30/admin/resource
$ cp  vt220.r  vt220a.r
```

3. oratem 을 기동한다.

```
$ oraterm -c vt220
```

4. REOURCE FILE 을 OPEN 한다. (Accept key : ^O)

```
File                                        Help
-------------------- Oracle*Terminal --------------------------
New
Open <            Open Resource File
Save      +----------------------------------+
...       ¦      Filename : vt220a           ¦
Exit      +----------------------------------+
```

5. 수정하고자 하는 Device 를 선택하여 1. 에서 확인한 FUNCTION KEY 값을 등록 한다.

```
File    Product     Mapping          Device   Help
-------------------- Oracle*Terminal -------------------------
                                     Modify    <- a
                                     ------
                                     Keys      <- b
                                     Attribute
                                     Display

                          a
                          ¦
```

```
File    Product     Mapping         Device   Help
---------------------------------------------------------
------------------------+----------------------------------
 Device Name            ¦  Description
------------------------+----------------------------------
 VT220                  ¦  DEC VT220 terminal
                        ¦


                       b
                        ¦

File   Product      Mapping          Device   Help
------------------- Oracle*Terminal -----------------------
 ------------------------+----------------------------------
 Device Key Name         ¦  Physical Termianl Sequence
 ------------------------+----------------------------------
 Tab                     ¦  \011
 Return                  ¦  ^M
 ...                     ¦  ...
c  PF1   --> F1          ¦  \eOP
   PF2   --> F2          ¦  \eOQ
   PF3   --> F3          ¦  \eOR
   PF4   --> F4          ¦  \eOS
   ...                   ¦  ...
d  End                   ¦  \e[1~
```

사용하고자하는 PC 의 모든 KEY NAME 과 Terminal Sequence 값을등록하고(d), KEY NAME 이 틀린경우(c) 수정한다. FORMS30 에서는 위에서 등록된 KEY 만사용 할 수 있고 각 KEY NAME 에 해당되는 TERMINAL SEQUENCE 값이 일치 되어야만 동작이 된다.

6. 수정 하고자 하는 PRODUCT 를 선택 한다.(RUNFORM30 의경우)

```
File     Product     Mapping         Device      Help
------------------------ Oracle*Terminal -------------------
         Modify    <-
         ------
         Keys
         Attribute


                          ¦

File     Product     Mapping          Device     Help
------------------------ Oracle*Terminal --------------------
---------------------------+----------------------------------
        Product name       ¦  Description
---------------------------+----------------------------------
e  RUNFORM-RUNMENU  <-      ¦ SQL*Forms( Run Form) and SQL*Menu
```

```
f  SQLFORMS              ¦ SQL*Forms( Design ) key mapping
   SQLMENU               ¦ SQL*Menu( Design ) key mapping
g  ORATERM               ¦ Oracle*Terminal mapping
```

FORMS30 에서는 RUNFORM30 (e 선택) 과 SQLFORMS30 (f 선택) 별도로 KEY 를 정의한다. 특히 Oracle*Terminal(oraterm) 을 수정할때 KEY 가 틀린경우에는 g 를 선택하여 oraterm 의 KEY 를 정확하게 Setting 한다음 작업한다. 선택하는 방법은 CURSOR 를 해당 PRODUCT 에 놓고 ACCEPT 한면된다.

7.등록된 DEVICE KEY NAME 을 사용 할 수 있도록 해당 PRODUCT KEY와MAPPING 시킨다.(RUNFORM)

```
File     Product     Mapping         Device      Help
----------------------- Oracle*Terminal --------------------
                     Keys      <-
                     Attribute

                        ¦¦
File     Product     Mapping         Device      Help
-------------------------------------------------------------
----------------------+-------------------------------------
Product Key Name      ¦  Device Key Name
----------------------+-------------------------------------
Accept/Commit         ¦  Do         --> F3
                      ¦  ^O
Exit                  ¦  PF4        --> F4
Block Menu            ¦  PF1 F14    --> F1 F4
Copy                  ¦  PF1 Find   --> F1 End
....                  ¦  ....
```

8. SQLFORMS30 의 KEY 도 6.,7. 를 반복하여 수정한다.

9. RESOURCE FILE 을 저장하고 Exit 한다.
   4. 에서 Save 와 Exit 을 순서대로 선택한다.

10.수정한 Terminal File 로서 FORMS를 기동한다.

```
$ sqlforms30 -c vt220a:vt220

vt220a : Resource File name (생략하면  oraterm.r 이지정됨)
vt220  : Device Name
```

11. Attribute 를 수정하는 경우 Device 와 Mapping 에서 Attribute를 지정하여 수정한다.

(V3.0) OS의 환경변수를 받아들이는 방법

Bulletin no : 10112

UNIX SYSTEM을 사용하면서 ORACLE_SID와 같은 환경변수를 GLOBAL VARIABLE 이나 FORMS내의 FIELD로 받아들이고자 할 때가 있다. 그러나, 이러한 기능을 제공하는 FUNCTION이 없다.

- 해결방법

(1) USER EXIT을 사용한다.

예를들어 PRO*C PROGRAM에서 O/S 환경변수를 읽어 FORMS로 넘겨주도록 한다.

(2) 간단한 SQLPLUS SCRIPT를 만들어 사용한다.

< EXAMPLE >

. SQLPLUS에서

```
SQL> DROP TABLE ENV;
SQL> CREATE TABLE ENV ( VAR VARCHAR2(20));
```

. UNIX에서

```
$vi envvar.sql
   SET ECHO OFF;
   SET VERIFY OFF;
   DELETE FROM ENV;
   INSERT INTO ENV VALUES ( '&1');
   EXIT;
```

. FORMS 3.0에서

필요에 따라 :GLOBAL.SID 혹은 ENVVAR FIELD를 생성한다.

PROCEDURE생성

```
PROCEDURE GET_ENVVAR   IS
BEGIN
  HOST('sqlplus -s un/pw @envvar'||'$'||:envvar);
  SELECT  VAR  INTO  :GLOBAL.SID  FROM  ENV;
END;
```

(V3.0) SQL*FORMS V3에서 SQL*REPORTWRITER로 PARAMETER전송하는 방법

Bulletin no : 10337

---

1. ReportWriter에서

```
SELECT * FROM EMP
  WHERE b=:rb and c=:rc and d=:rd
```

2. SQL*Forms에서

```
HOST('runrep longs scott/tiger paramform=no rb='
            || to_char(:b) || ' rc='|| to_char(:c) ||' rd='|| to_char(:d));
```

(V3.0) FORMS30 및 STORED PROCEDURE 에서 ROLLBACK SEGMENT 사용

Bulletin no : 10063

---

1. FORMS30 에서 'SET TRANSACTION USE ROLLBACK SEGMENT' 사용

```
NAME = key-nxtblk
TRIGGER_TYPE = V3
TEXT = <<<
     BEGIN
        DECLARE
          rb  varchar2(10);
        BEGIN
           rb := 'R02';
           COMMIT;
           dbms_transaction.use_rollback_segment(rb);
           INSERT INTO emp SELECT * FROM emp;
        END;
     END;
```

2. STORED PROCEDURE 에서 'SET TRANSACTION USE ROLLBACK SEGMENT' 사용

```
SQL> CREATE PROCEDURE rbs_proc IS
  2  BEGIN
  3   DECLARE
  4    rb varchar2(10);
  5   BEGIN
  6    rb :='R02';
  7    COMMIT;
  8    dbms_transaction.use_rollback_segment(rb);
  9    INSERT INTO emp SELECT * FROM emp;
 10  END;
 11  END;
 12  /
```

RUNNING SQL_TRACE FROM ORACLE TOOLS

Bulletin no : 10245

---

1)SQL*Forms Version 3 에서
  Oracle Korea Thecnial Bulletins (RNO 101) 참조
  방법 1 : Designer에서 -> Option 서 statistics 선택 -> Run Form
  방법 2 : Runform30의 command line에 -s option 사용

2)Oracle Forms Version 4.5 에서
  the Oracle Forms Reference Manual, Volume 1참조
  방법 1 : Designer에서 -> Edit/Preferences 에서 statistics선택
          -> Run Form
  방법 2 : F45RUN command line에 "statistics=YES" 사용

3)SQL*Reportwriter Version 1.1 에서
  Designer에서 새로운 field생성

  Name   sql_trace
  Source &SQL alter session set sql_trace true
  Group   Report
  Width   1
  Skip   X

4)ORACLE Reports Version 2.5 에서
  Oracle Reports Reference Manual 참조
  방법 1 : Report Level의 Formula Column생성
          Formula: SRW.DO_SQL('ALTER SESSION SET SQL_TRACE TRUE');
  방법 2 : Before Form Trigger 을 생성
          SRW.DO_SQL('ALTER SESSION SET SQL_TRACE TRUE');

5)SQL*Plus 에서
  Oracle Korea Thecnial Bulletins (RNO 101) 참조

STORED PROCEDURE & DATABASE TRIGGER

Bulletin no : 10123

---

* STORED PROCEDURE

1. Oracle Forms 4.5에서는 PL/SQL Code를 Server-Side에 저장하거나 Forms내 Trigger, Procedure로 갖을 수도 있다.

다음과 같은 경우 Procedure를 Database Server에 저장하는 방법을 선택하는 것이 좋다.

- Procedure가 Standard한 기능을 제공하여 다른 Tool이 공유 할 수 있을 때.
- Procedure가 중요한 DML기능을 가질 경우.
- Oracle Forms4.5에서는 PL/SQL의 Version을 1.1까지 지원하므로, PL/SQL Version 2 의 기능이 필요한 경우.

2. Stored Procedure의 제한사항

(1) Server가 7.0 이상이어야 한다

(2) Pass되는 Variable은 2000 Byte이하이어야 한다.

- 2000이 넘으면 Truncate가 되고, PL/SQL에서 Value_Error Exception이 발생한다.

(3) Menu PL/SQL 에서는 Stored Procedure와 Function을 부를 수 없다.

(4) Stored Subprogram의 정의가 바뀐 경우, Form을 다시 Server와 Connect 하기전에는 영향을 주지 않는다.

3. Oracle 7 Server 에서 제공하는 Standard Package들

- dbms_alert
- dbms_ddl
- dbms_describe
- dbms_lock
- dbms_mail
- dbms_output
- dbms_pipe
- dbms_session
- dbms_snapshot
- dbms_standard
- dbms_transaction
- dbms_utility

< Oracle7 Developers Guide 참조>

4. Calling Stored Procedures

(1) Database Procedure은 Server-Side PL/SQL Engine에 의해 실행되도록 Design PL/SQL Block이다.

(2) Stored Procedure를 Call하기 위해서는 EXECUTE Privilege가 필 하다.

(3) Server-Side Procedure와 Forms에서의 PL/SQL의 중요한 차이점은 Server-Side 에서는 Forms의 Bind Variable (예 : :Block_Name.Item_Name등)을 인식하지 못한다는 것이다.

(4) Parameter/Return이 가능한 Data Type

- VARCHAR2 : Maximum of Varchar2(2000)
- NUMBER
- DATE
- BOOLEAN

제한사항

- TABLE%ROWTYPE,TABLE.COLUMN%ROWTYPE으로 정의된 것은 Reference가 안된다.
- Unsupported Parameter나 Return Value를 사용하면 다음의 Error가 발생한다.

* PL/SQL error 313 at line xxx,column yyy 'PROCNAME' not declared in this scope
* PL/SQL error 201 at line xxx, column yyy identifier 'FUNCTNAME' must be declared

(5) Formal Parameter의 Default Value

- Defulat Value는 제공하지 않는다.
- User가 임의로 줄 수 있다.
- Procedure Private_Test(a in VARCHAR2 := 'Hello' b in
  VARCHAR2 := 'There') IS

```
BEGIN
     Dbms_Output.Put_Line(a);
     Dbms_Output.Put_Line(b);
END;
```

(6) 다른 User의 Stored Subprogram이나 Remote Database를 Call하려면

- User Name이나 Database Link명을 숨기기 위하여 Synonym을 생성하여야 한다.
- CREATE SYNONYM lib_hr_syn FOR libowner.lib_hr;
- Program에서 Call할때는 ss_num := lib_hr_syn.get_ssn(:EMP.EMPNO);

5. PL/SQL Compiler가 해당 Procedure를 찾는 순서

(1) Current PL/SQL Block내에 정의되어 있는가?
(2) Standard PL/SQL Command인가?
(3) Oracle Forms내의 Built-in Procedure혹은 Function인가?
(4) User_Named Procedure혹은 Function인가?
(5) Server-side에 정의된 DBMS_STANDARD Package인가?
(6) Current user가 Server-Side의 어떤 Procedure나 Function을 Access하는가?

* 이상의 질문에 대하여 전부 'NO'라는 대답이 나오면 Compiler는 다음과 같은 Error Message를 준다.

- PL/SQL error 313 at line xxx,column yyy 'PROCNAME' not declared in this scope
- PL/SQL error 201 at line xxx, column yyy identifier 'FUNCTNAME' must be declared

* DATABASE TRIGGER

1. Database Trigger는 Forms에서의 Trigger와 개념적으로 거의 동일하다.
차이점은 Trigger가 Fire되는 원인을 제공하는 사건과, 후속코드가 실행되는 장소가 틀린 것이다.

2. Database Trigger는 Table과 연관된 PL/SQL Block들이다. Table에 대해 UPDATE,INSERT, DELETE등의 행위에 의해 Fire된다.

3. Error발생시에는 RAISE_APPLICATION_ERROR Built-In Procedure를 사용하여 Error를 표시한다. RAISE_APPLICATION_ERROR Procedure에서는 Error Number가 20000 - 20999의 Range를 가진다.

4. Creating & Editing Database Triggers

아래와같은 적절한 권한이 주어진다면 Forms Designer Mode에서 Database Trigger를 직접 Create, Edit, Compile, Browse를 할 수 있다.

- Execute Privilege, Create Privilege, Compile Privilege, Drop Privilege

* Create하려면

- Navigator에서 Database Objects와 Table Nodes를 Expand한다.
- 원하는 Table을 선택하고 Expand한다.
- Triggers Nodes를 선택하고 Menu Bar의 Navigator*Create를 선택한다.
- Database Trigger Editor에서 작성, compile한다.

* Edit 하려면

- Navigator에서 Database Objects와 Table Nodes를 Expand한다.
- 원하는 Table을 선택하고 Expand한다.
- 원하는 Trigger에서 Double-Click하면 Editor가 나타난다.
- Database Trigger Editor에서 변경, compile한다.

(V4.5) USING DATABASE ROLES WITH ORCLE FORMS 4.5

Bulletin no : 10253

Database Role에 관하여

==========

Oracle Databae의 Role은 권한들의 집합이라고 할 수 있다. 먼저 Role은 Create Role권한을 가진 User에 의하여 생성된다. Role이 생성되고나면 특정 권한이 해당 Role에 Grant된다. 일반적으로 Role에 주어지는 권한은 Table에 대한 Select,Insert,Update등 이다. Role은 다시 특정User나 Role에 Grant되고, 해당Role이 가지고 있는 권한이 계승된다.

Example : 다음의 예는 사원데이타를 입력,수정,조회하는 Role을 생성하는것이다.

```
--------------------------------------------------------------------------
--Create A Role Called Hr
Create Role Hr;

--Grant Specific Privileges To The Hr Role
Grant Select, Insert, Update On Emp To Hr;

--Create A Password Protected Role Called Payroll
Create Role Payroll Identified By Paycheck;

--Grant Specific Privileges To The Payroll Role
Grant Select, Insert, Update On Compensation To Payroll;

--Create A Role Called Auditor
Create Role Auditor;

--Grant Specific Privileges To The Auditor Role
Grant Hr, Payroll To Auditor;
--------------------------------------------------------------------------
```

이상은 Role이 다른 Role에 Grant되는 것을 보여준다.

다음은 위에서 생성한 Hr,Payroll,Auditor를 User에게 Grant하는 것이다.

```
--------------------------------------------------------------------------
Grant Hr To Johndoe;
Grant Hr To Janesmith;
Grant Payroll To Janesmith;
Grant Auditor To Suebrown;
--------------------------------------------------------------------------
```

만약, User의 Role이 Enable되어 있지 않으면 Set Role Command를 사용하여 특정한 User에게 Session중에 Role을 지정할 수 있다.

General Overview Of The DBMS_SESSION.SET_ROLE
==========

Oracle에서 제공하는 DBMS_SESSION Package는 Database Create시 Catproc.SQL을 실행하여 생성된다. 이 Package는 Oracle Forms에서 직접적으로 실행할 수 없는 SQL Command를 처리 할 수 있게 해준다.

DBMS_SESSION Package내의 SET_ROLE Procedure를 이용하여 SQL Set Role Command와 같은 결과를 처리 할 수 있다. 이것은 어느 Form의 Procedure나,Pl/SQL Block,Forms Trigger에서뿐만 아니라 Database Trigger에서도 사용 할 수 있다.
현재 Session을 맺어 실행중인 Forms Program에서 DBMS_SESSION.SET_ROLE Procedure를 이용하여 현재의 Session에서 필요한 특정한 Role을 Setting할 수 있다.

DBMS_SESSION.SET_ROLE의 Syntax는 다음과 같다.

```
DBMS_SESSION.SET_ROLE(Role_Cmd  Varchar2)
```

Example:

```
--------------------------------------------------------------------------
--Setting The Hr Role From Forms
DBMS_SESSION.SET_ROLE('Hr');

--Setting The Payroll Role From Forms
DBMS_SESSION.SET_ROLE('Payroll Identified By Paycheck');
--------------------------------------------------------------------------
```

Oracle7 Server Application Developer*s Guide참조.

Building A Generic Procedure To Set A Role Before Calling A Form
==========

개발자들은 종종 Form에 Database Role을 연관하여 처리하고 싶어한다.
다음은 그 예이다.

```
--------------------------------------------------------------------------
Procedure Check_For_Qualified_User(Role_Cmd  In Varchar2,
                                   Form_Name In Varchar2) Is
--The Procedure Has Two In Arguments:  The Role To Be Set(Role_Cmd)
--And The Form To Be Called(Form_Name)

--The Following Declared Exception And Pragma Are Used To "Trap" A User Who
--Does Not Have The Required Role.
Uncertified_User      Exception;
```

```
Pragma Exception_Init(Uncertified_User, -1924);
Begin
--Set The Role To The Role Identified By The Role_Cmd In Argument
  DBMS_SESSION.SET_ROLE(Role_Cmd);
--Call The Form Identified By The Form_Name In Argument
  Call_Form(Form_Name);
Exception
  When Uncertified_User Then
    Message('You Have Not Been Granted Privileges To Use The '
             || Upper(Form_Name)  || ' Form',Acknowledge);
    Message('Check With The Dba If You Think You Should Have Access.');
End;
```

General Restrictions

Call Form하여 Called된 Form에서 DBMS_SESSION.SET_ROLE을 실행 한 경우
Calling Form에서도 Called Form에서 적용된 것이 유효하다.
하지만 Open_Form으로 새로운 Form이 Run된 경우는 Calling Form에는 Role이 적용되지 않는다. 이 제한사항은 When-Window-Activated Trigger에서 Active된 Role을 Setting하여 주어야 한다.

Common Errors

Ora-1919 : DBMS_SESSION.SET_ROLE에서 지정한 Role이 존재하지 않는 경우
Ora-1979 : Role_Command에 Password가 필요한 경우

(V3.0) VT100에서 KEY-F0 ~ KEY-F9사용법

Bulletin no : 10261

---

1. oraterm을 invoke한다.
   (oraterm사용방법은 Oracle Korea Customer Support Technical Bulletin 101참조)

2. resource : vt100 open

3. [product]의 [key] option을 선택

4. product key name에 KEY 9 function code에 10031 을 입력

5. [mapping]의 [key] option을 선택

6. product key name에 KEY 9 device key name에 F9 을 입력

7. [device]의 [key] oprion을 선택

8. device key name에 F9 을 입력

9. [file]의 [save]를 선택

10. sqlforms30에서 KEY-F9 트리거에서 execute_query를 지정

11. runform30시에 [F9]키를 누르면 query가 수행된다.

(V4.5) WINDOWS PERFORMANCE TUNING FOR FORMS4

Bulletin no : 10187

Subject: Window Tuning For F4

PERFORMANCE TIPS FOR WINDOWS

==========

The following suggestions should assist you in maximizing the performance of your Windows 386 Enhanced mode installation (Windows is capable of using Virtual memory in 386 Enhanced mode). Many of these suggestions apply to standard and real modes as well.

Of course, running Windows in Standard mode is much faster than 386 Enhanced mode, but a lot of applications require Windows in 386 Enhanced mode, including our very own SQL*NET for Windows!

1. USE SMARTDRIVE. The Microsoft SMARTDrive disk caching driver can produce the largest single Windows 3.1 performance improvement.
   Use SMARTDrive whenever possible. You can enable SMARTDrive by loading the SMARTDRV.SYS driver in your CONFIG.SYS or by loading SMARTDRV.EXE in your AUTOEXEC.BAT. There are a variety of options available on these drivers.
   I recommend turning OFF the Write caching if you have a LOCAL Oracle RDBMS on your PC. For basic information on SMARTDrive installation and operation, refer to the "Microsoft Windows Users Guide" version 3.1 manual or even SMARTDRV /? at the DOS prompt.
   Caution: If you run into a a lot of GPFs in your Windows applications you will most likely have cross linked files on your hard disk due to SMARTDrive not being able to flush its write buffers before the crash. Run CHKDSK or SCANDISK at least once a week to guard against losing these files. Even better, run something like the Norton Disk Doctor.

2. KEEP YOUR HARD DISK OPTIMIZED. A fragmented hard disk greatly impacts Windows's performance, especially when a temporary swap file and/or SMARTDrive is installed. Use a hard disk optimizer program on a weekly basis to keep your disk contiguous.

3. CREATE A PERMANENT SWAP FILE. Using a permanent swap file improves performance over using a temporary one. Under Windows 3.1, if supported by your hardware, also select the Use 32-Bit Disk Access check box in Control Panel by choosing the 386 Enhanced icon, choosing the

Virtual Memory button, then choosing the Change button.

4. TURN OFF GRAPHICS PORT TRAPPING. The speed of MS-DOS applications running under 386 enhanced mode can be noticeably improved by not selecting any of the Monitor Ports options in the Advanced section of the PIF Editor. The High Graphics option provides the widest range of MS-DOS application compatibility but is not required for most applications.

5. TURN OFF THE FILESYSCHANGE=SETTING. Windows 3.1 386 enhanced mode can monitor disk access by MS-DOS applications and send directory update messages to File Manager. This allows File Manager to be automatically updated by changes MS-DOS applications have made to files or directories. However, this option is not a necessity, and leaving it off (the default) speeds file access by MS-DOS applications. To disable this feature, set FileSysChange=no in the [386enh] section of the SYSTEM.INI file.

6. TURN OFF THE RESERVEPAGEFRAME=SETTING. Turn this setting off if you do not require expanded memory support for MS-DOS applications. Turning this option off ensures that you're getting the most possible memory in virtual MS-DOS machines. To disable this feaure, set ReservePageFrame=no in the [386enh] section of the SYSTEM.INI file.

7. USE THE RIGHT NUMBER OF MS-DOS BUFFERS. If you are using SMARTDrive, set the number of MS-DOS disk access buffers in your CONFIG.SYS file to 15 (that is, BUFFERS=15). Using a greater number of buffers with SMARTDrive will actually decrease efficiency. If you are not using SMARTDrive, use BUFFERS=30.

8. USE THE LOWEST COMMON DISPLAY DRIVER. Using a display driver with a high resolution or large number of colors results in slower display performance. If you do not require the extra features of the display driver, use a driver with less capability. Usually this suggestion applies to display systems that are VGA compatible but offer an extended mode driver, such as the Video Seven or 8514. Using the standard VGA driver instead offers faster display performance but less resolution and/or color support.

(V4.5) ORACLE FORMS 4.5 에서 WINDOWS APPLICATION을 호출하는 방법

Bulletin no : 10276

General Overview

==========

Oracle Forms에서 Ora_Ffi를 이용하여 Windows Application을 호출하면 Forms Program과 Window Application이 Asynchronous하게 실행이 가능하다. 호출된 Window Application은 사용자가 임의오 Close하기 전까지 남아있게 된다. 현재 사용하고 있는 Form을 Close하거나, Forms Runtime에서 빠져나가도 호출한 Window Application을 Close시키지 않는다.
물론 사용자는 사용중인 Form과 호출한 Application간을 자유자재로 움직이면서 사용할 수 있게 된다. 이것은 이전 Version에서 사용하던 Host Command와는 다른 형태로 실행된다. Host Command를 이용하여 Window Application을 호출하면 이것은 Synchronous하게 실행된다. 즉, Host로 호출한 Window Application이 끝나기 전에는 사용중이던 Forms로 Focus가 돌아 오지 않게 된다.

Forms에서 Window Application을 호출하기 위해서, Ora_Ffi Command에서 Krn1386.Exe라는 Windows Library를 사용한다.
구체적인 예를 펴본다.

Creating Winexec Package Spec & Package Body

==========

Forms에서 다음과 같은 Winexec Package Spec을 생성한다.

```
Package Winexec Is
    Function Winexec(Cmd In Out Varchar2) Return Boolean;
End;
```

Forms에서 다음과 같은 Winexec Package Body를 생성한다.

```
Package Body Winexec Is
    Dll_Handle      Ora_Ffi.Libhandletype;
    Winexec_Handle  Ora_Ffi.Funchandletype;

Function Winexecicd(Handle In Ora_Ffi.Funchandletype,
    Cmd In Out Varchar2,
    Disp_Mode In Pls_Integer)
Return Pls_Integer;
Pragma Interface(C, Winexecicd, 11265);

Function Winexec(Cmd In Out Varchar2)
    Return Boolean Is
```

```
        Tmp Pls_Integer;
  Begin
  Tmp := Winexecicd(Winexec_Handle, Cmd, 1);
  If Tmp = 2 Then /* Cannot Find Program Or One Of Its Components */
    Message('Cannot Find File ' || Cmd ||' (Or One Of Its Components)');
    Return False;
  Else Return True;
  End If;
End Winexec;


Begin
  Dll_Handle := Ora_Ffi.Register_Library(Null, 'Krnl386.Exe');
  Winexec_Handle :=
      Ora_Ffi.Register_Function(Dll_Handle, 'Winexec', Ora_Ffi.Pascal_Std);
  Ora_Ffi.Register_Return(Winexec_Handle, Ora_Ffi.C_Int);
  Ora_Ffi.Register_Parameter(Winexec_Handle, Ora_Ffi.C_Char_Ptr);
  Ora_Ffi.Register_Parameter(Winexec_Handle, Ora_Ffi.C_Int);
End Winexec;
```

Winexecicd의 Disp_Mode Argument 의 설정된 값은 다음과 같다.

1 = Normal
2 = Minimized
3 = Maximized
4 = Starts Normal Size But Without Focus
7 = Starts Minimized Without Focus

Creating A Trigger To Call Windows Application

==========

```
When-Button-Pressed Trigger
Declare
   Tmp Boolean;
   Command Varchar2(30) := 'Calc.Exe';
Begin
   Tmp := Winexec.Winexec(Command);
End;
```

위의 Sample은 Windows의 Calculator를 호출하는 예이다.
다은 Windows Application을 호출하려면 Winexec.Winexec의 Parameter인 Command에 원하는 Executable을 지정하면 된다.
단, 만약 호출하고자 하는 Executable이 Dos Path내에 없는 것이라면, Command Parameter내에 Full Path를 지정해 주어야 한다.
예 : Command Varchar2(30) := 'C:\Winword\Winword.Exe'

PRO*C 2.0의 새로운 특징

Bulletin no : 10298

---

1. C Preprocessor 지원

C Preprocessor를 포함한다. 즉, #define 명령을 사용하여 constants와 macros를 정의하고 Varchar와 같은 Pro*c Data Type을 매개변수를 이용하여 선언할 수 있다. 또한 sqlca.h, oraca.h, sqlda.h같은 Pro*C header 파일들을 #include 명 을 사용하여 include 가능하다.

```
#include
#define ENAME_LEN 10
 .....
VARCHAR      ename[ENAME_LEN]
```

2. Host변수에 Structure 사용가능

Embedded SQL 문장에 Host Variables를 포함하는 일반적인 C Structure를 사용할 수 있다. 예를 들어, SELECT empno, ename, sal FROM emptable를 아래와 같이 정의된 C Host Structure에 넣을 수 있다.

```
struct
{
    int emp_number;
    char ename[10];
    float salary;
} emp_rec;
```

그리고 아래와 같은 Embedded SELECT 문장에서 이 Structure를 사용한다.

```
EXEC SQL SELECT empno, ename, sal
    INTO :emp_rec
    FROM emp
    WHERE empno = 7499;
```

Host Structures는 또한 Host Arrays를 포함할 수 있다.

3. Declare Sections가 필요치 않다

MODE=ORACLE 로 Precompile하 , Declare Section에서 Host변수, Structures 를 선언할 필요가 없다. 이것은 ANSI C-Style Function Prototypes를 사용하는 프로그래머에게는 커다란 장점이다.
앞의 버전에서 Function으로 Host 변수를 보내려면, Old-Style C Function Definition을 사용해야 했다. Name, Salary가 Host변수인 Function은 아래와 같이 사용해야 했다.

```
int my_func(name, salary)
EXEC SQL BEGIN DECLARE SECTION;
    char *name;
    float salary;
EXEC SQL END DECLARE SECTION;
{
/* function body using name and salary as
    host variables */
}
```

Pro*C Release 2.0에선 이 function을 다음과 같이 정의할 수 있다

```
int my_func(char *name, float salary)
{
/* function body using name and salary as
   host variables */
}
```

4. EXEC SQL WHENEVER

Whenever . . . Do사용시 Parameters를 받아 값을 돌려주는 Function Call을 지원한다. 예를 들어 아래와 같이 하면

```
EXEC SQL WHENEVER SQLERROR DO rv = error_func("Message");
```

message를 프린트하고 값을 return하는 에러처리 function을 call할 수 있다.

5. Optimizer Hints

SELECT, DELETE, UPDATE command에 Oracle7 optimizer hint syntax를 지원한다.

6. DBMS = V6_CHAR Option

이 옵션으로 모든 Standard Oracle7 기능을 가질 수 있다(The Semantics Of Character Strings를 제외한). 이 Option을 사용하면 Character Strings는 Fixed Length가 아닌 Oracle Version 6와 같이 Variable Length로서 사용된다.

AUTOMATIC STARTUP AND SHUTDOWN OF SQL*NET LISTENERS

Bulletin no : 10284

Introduction

This bulletin addresses the topic of automatic startup and shutdown of SQL*Net listeners when a UNIX system is brought up or down respectively.

Please refer to bulletin 105870.898 (Automatic Startup and Shutdown of Oracle Database Instances) for background information.

For more detailed technical explanations and alternatives, readers are advised to refer to the System Administrator's Guide for the appropriate UNIX port.

Files of interest in SQL*Net v1 and v2

SQL*Net v1.0:	$ORACLE_HOME/bin/XXXsrv	- SQL*Net Listener for protocol XXX. For example, orasrv(TCP/IP), spxsrv (SPX/IPX) etc.
	$ORACLE_HOME/bin/XXXctl	- Control program to start, stop or debug the corresponding listener process. For example, tcpctl(TCP/IP), spxctl (SPX/IPX) etc.
SQL*Net v2.0:	$ORACLE_HOME/bin/tnslsnr	- SQL*Net Listener for various incoming connection requests on different protocols.
	$ORACLE_HOME/bin/lsnrctl	- Control program to start, stop or debug the working of tnslsnr.
	/etc/tnsnames.ora or $TNS_ADMIN/tnsnames.ora	- lists aliases for connect strings for clients to connect to server
	/etc/listener.ora or $TNS_ADMIN/listener.ora	- lists details for listener specifying machine, port, protocol etc. that listener should be listening for.

Files common: /etc/oratab or - specifies Oracle instance information
to v1.0 & v2.0 /var/opt/oracle/oratab

/etc/services - specifies listener ports on the
UNIX system

For more details on individual SQL*Net files above, please refer to the appropriate SQL*Net documentation.

Multi Threaded Server (MTS)

MTS is a facility by which an incoming client connects with a shared dispatcher and server instead of a dedicated server, unless specifically requested. In SQL*Net v1.0, an oracle shadow process is spawned to serve each individual incoming connection. This leads of more memory and process usage.

MTS, on the other hand pre-spawns dispatcher and server processes that serve the client connection. The dispatcher process looks at the incoming connection,and depending on the protocol of the incoming request and the server load distribution, "introduces" the client to a server process, that might alreadybe serving other clients. Thus, a new dedicated process is not spawned for each incoming connection.

Thus the dispatcher and server processes are shared among various incoming connections. Processes named as d000 and s000 are shared dispatcher and server processes respectively.

Ownership and Permissions

The listener binaries, XXXsrv and tnslsnr, should be owned by the superuser(root) and the dba group. The permissions on the binary should be 4751. The initial '4' in the 4751 sets the SETUID bit for the bianry.

To change ownership and group ownership to root/dba, do:

```
chown root
chgrp dba
```

as superuser.

To change permissions on the listener binary, do:

```
chmod 4751
```

as superuser.

Sample startup and shutdown of SQL*Net Listeners

To start/stop/check status for a SQL*Net v1 listener:

```
XXXctl start          # starts XXXsrv
XXXctl stop           # stops  XXXsrv
XXXxtl stat           # gives  status for XXXsrv
```

To start/stop/check status for the SQL*Net v2 listener:

```
lsnrctl start          # starts tnslsnr
lsnrctl stop           # stops  tnslsnr
lsnrxtl stat           # gives  status for tnslsnr
```

Platform Specific Details

To automate the listener startup and shutdown process, the appropriate command should be listed in the correct directory/file so that the UNIX system startup process includes this command in its list of things to do.

Following are some details to implement the automatic startup and shutdown of SQL*Net listeners on six major flavors of UNIX.

NOTE: If you are using MTS, please make sure you start the SQL*Net listener(s) BEFORE the Oracle instance is started.

NOTE: In the following sections, refers to the owner of the Oracle installation, usually 'oracle' <$ORACLE_HOME> refers to the expanded form of $ORACLE_HOME since it might not be set for the superuser.

NOTE: In the following sections, 'XXXctl' refers to the appropriate control program for the corresponding listener. For example 'tcpctl' for 'orasrv', which listens for incoming TCP/IP connections.

(1) SunOS 4.1.3

Important Files

/etc/rc.local - The startup script for the system

Details

For startup, make the following entry in /etc/rc.local:

```
  For SQL*Net v1:   su  -c '<$ORACLE_HOME>/bin/XXXctl start'
  For SQL*Net v2:   su  -c '<$ORACLE_HOME>/bin/lsnrctl start'
```

Since only /etc/shutdown is invoked at system shutdown time, a script such as the following can be used to bring down Oracle before the system facilities are shutdown:

```
   #!/bin/csh

   /bin/su  -c '<$ORACLE_HOME>/bin/XXXctl stop'  # For SQL*Net V1
   /bin/su  -c '<$ORACLE_HOME>/bin/lsnrctl stop' # For SQL*Net V2
   /bin/su  -c <$ORACLE_HOME>/bin/dbshut

   /etc/shutdown $*
```

The script should be owned and executable only by the superuser.

-.-.-.

```
CAUTION: If there are separate scripts to shutdown the listeners and the
         Oracle instance, the order of invokation should be given some
         thought. Thus, you might want to call the startup script for
         SQL*Net listeners as 'S99oracle' and that for the instance
         startup as 'S98oracle'.

         At startup, the listeners should be started after the instance
         has been brought (unless you are using MTS, in which case, the
         order MUST be reversed).
         At shutdown, the listeners should be shutdown before the instance is.

The following commands should be included in the appropriate script:

For startup:
  /bin/su  -c '<$ORACLE_HOME>/bin/XXXctl start'   # For SQL*Net v1
  /bin/su  -c '<$ORACLE_HOME>/bin/lsnrctl start'  # For SQL*Net v2
For shutdown:
  /bin/su  -c '<$ORACLE_HOME>/bin/XXXctl stop'    # For SQL*Net v1
  /bin/su  -c '<$ORACLE_HOME>/bin/lsnrctl stop'   # For SQL*Net v2
NOTE:  The links to /sbin/init.d/oracle might not be correctly established.
       There may be more links than these 2, forcing the startup or shutdown
       to happen more than once.

-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-
```

(3) Solaris 2.3

==========

Important Files

```
    /etc/inittab          -- Controls the initialization process
    /etc/rc2              -- Script to start/stop processes at run level 2
    /etc/rc0              -- Script to start/stop processes at run level 0

    /etc/rc2.d/S99oracle  -- link to the script /etc/init.d/lsnrstart
    /etc/rc0.d/K01oracle  -- link to the script /etc/init.d/lsnrshut

    /var/opt/oracle/oratab-- Oracle oratab file
```

Details

For startup, put the following commands in the script /etc/init.d/lsnrstart:

```
  /bin/su  -c '<$ORACLE_HOME>/bin/XXXctl start'   # For SQL*Net v1
  /bin/su  -c '<$ORACLE_HOME>/bin/lsnrctl start'  # For SQL*Net v2
For shutdown, put the following commands in the script /etc/init.d/lsnrshut:

  /bin/su  -c '<$ORACLE_HOME>/bin/XXXctl stop'    # For SQL*Net v1
  /bin/su  -c '<$ORACLE_HOME>/bin/lsnrctl stop'   # For SQL*Net v2
Again, the order of startup/shutdown should be decided keeping MTS in mind.

-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-.-
```

(4) AIX

```
    ===
```

Important Files

```
---------------

        /etc/inittab   --  System initialization file
        /etc/rc        --  System initialization script
        /etc/mkitab    --  Utility to make an entry in /etc/inittab
        /etc/shutdown  --  System shutdown script
```

Details

```
-------

The user can choose one of the following methods for startup:

1. Use /etc/mkitab to make an entry in /etc/inittab :

   The command(s):

   /etc/mkitab ":2:wait:/bin/su   -c
     '<$ORACLE_HOME>/bin/XXXctl start' > /dev/console 2>&1"

   /etc/mkitab ":2:wait:/bin/su   -c
     '<$ORACLE_HOME>/bin/lsnrctl start' > /dev/console 2>&1"

   can be used to introduce XXXctl or lsnrctl respectively in the /etc/inittab
   file. Thus, on system startup, at run level 2, the appropriate listener
   control program is invoked by the system as the Oracle owner.

2. This method is essentially the same as (1) above, but with one level
   of indirection. Use /etc/mkitab as follows:

   /etc/mkitab "rcoracle:2:wait:/etc/rc.oracle > /dev/console 2>&1"
```

Also create a script /etc/rc.oracle, owned and executable only by root.
The contents of this script should be:

```
#
# Startup the oracle database
#
/bin/su  </bin/XXXctl start      # For SQL*Net V1
<$ORACLE_HOME>/bin/lsnrctl start     # For SQL*Net V2
EOF
```

3. Make the following entry at the end of the startup script /etc/rc:

```
/bin/su  -c '<$ORACLE_HOME>/bin/XXXctl start'  # For SQL*Net V1
/bin/su  -c '<$ORACLE_HOME>/bin/lsnrctl start' # For SQL*Net V2
```

For shutdown, write a script similar to the following. The script should be owned and executable by the superuser only:

```
#!/bin/sh

/bin/su  -c '<$ORACLE_HOME>/bin/XXXctl stop'  # For SQL*Net V1
/bin/su  -c '<$ORACLE_HOME>/bin/lsnrctl stop' # For SQL*Net V2
/etc/shutdown $*
```

-.-.-.

executed. To bring the listeners down, write a simple script as follows
and place it in /etc/shutdown.d:

```
#!/bin/csh

/bin/su  -c '<$ORACLE_HOME>/bin/XXXctl stop'  # For SQL*Net V1
/bin/su  -c '<$ORACLE_HOME>/bin/lsnrctl stop' # For SQL*Net V2
/bin/su  -c <$ORACLE_HOME>/bin/dbshut

/etc/shutdown $*
```

-.-.-.

CLINET/SERVER 환경에서의 PERFORMANCE TUNING

Bulletin no : 10186

PERFORMANCE TUNING에는 3가지면에서의 APPROACH가 가능하다.

. SYSTEM LEVEL : HARDWARE AND SOFTWARE
. DATABASE CONFIGURATION LEVEL : INIT.ORA AND STRUCTURE
. APPLICATION LEVEL : APPLICATION SPECIFIC DEVELOPMENT

다음에서는 위의 3가지 접근방법을 혼합하여 PERFORMANCE TUNING에 관하여 살펴본다.

INIT.ORA

INTERNAL PARAMETER중에서 ORACLE PERFORMANCE에 중요한 영향을 미치는 3가지의 PARAMETER를 살펴본다.

. DB_BLOCK_BUFFERS
. SHARED_POOL_SIZE
. SORT_AREA_SIZE

< DB_BLOCK_BUFFERS >

DB_BLOCK_BUFFERS는 DATA CACHING을 위해 SGA로 할당되는 MEMORY를 지정한다.
이 숫자가 증가 될수록 PERFORMANCE는 향상된다. SQL 문장이 실행될때, 만약 CACHE 내에 RECORD가 없다면 실DATA를 읽어들여 CACHE에 올려놓는다.
CACHE내에 올려진 DATA들은 LRU 알고리즘에 따라 삭제될때까지 SGA내에 위치하게 된다. 필요한 자료를 DISK로부터 읽어오지 않고 MEMORY내에서 찾아오는 비율이 높을수록 PERFORMANCE가 높다. DB_BLOCK_BUFFERS는 실제의 할당되는 MEMORY를 지정하는 것이 아니라, BLOCK수를 지정한다. 실제로 지정되는 MEMORY의 양은 DB_BLOCK_BUFFERS * DB_BLOCK_SIZE의 값이된다. DB_BLOCK_BUFFERS의 DEFAULT 값은 HARDWARE에 따라 다르게 지정된다.
일반적으로 실제 MAIN MEMORY의 25%이하로 잡도록 한다.

< SHARED_POOL_SIZE>(Data Dictionary Cache, Shared Sql Area)

SHARED_POOL_SIZE는 SGA내에 DATA DICTIONARY CACHING과 STORED SQL문장을 위한 MEMORY의 할당량을 지정한다. DATA DICTIONARY CACHE는 DATA DICTIONARY 요소의 BUFFERING과 관련되어 우 중요하다.

하나의 SQL문장이 실행될때 ORACLE이 DATA DICTIONARY자료를 여러번 참조하기 때문에 DATABASE와 APPLICATION SCHEMA와 STRUCTURE가 MEMORY에 많이 올라와 있을수록, DISK로 부터 읽어들여오는 자료

가 적어지게된다.

DATA DICTIONARY도 DB_BLOCK_BUFFERS의 경우와 마찬가지의 방법으로 CACHING 된다. 가장 이상적인 것은 모든 DATA DICTIONARY정보가 MEMORY에 올라와 있는 것이다. SHAED SQL AREA는 ORACLE INTERNAL이 실행하는 것이나 USER의 특별한 실행 문장을 갖고있다. SQL문장이 실행될때, SHARED SQL STATEMENT AREA에 같은 문장이 있는지를 확인하여 보아 있으면 그 문장이 실행된다. 이것은 PARSING TIME을 줄여 전체적인 PERFORMANCE를 향상시킨다. 이때 문장이 같다는 것은 문자열과 SPACE까지도 같아야 하는것을 의미한다. 만약, 문장이 다르다면 새로운 문장이 PARSING되어 실행되고 SHARED SQL AREA에 다시 저장된다.

<예>

```
SQL> select name,customer
 2   from customer_information;
SQL> select name,customer
 2   FROM customer_information;
```

위의 두 문장은 결과를 RETURN하지만, 'FROM'절의 대소문자 때문에 ORACLE 에서는 다른 문장으로 인식하여 SHARED SQL AREA내의 있는 CURSOR를 재사용하지 않게된다. 이와같이 SHARED SQL AREA내의 자료에 대한 재 이용율을 높이려면, STORED PROCEDURE를 가능한 많이 사용하는 것이 좋다. SHARED_POOL_SIZE PATAMETER의 값은 BYTE 단위로 지정된다. DEFAULT값은 SYSTEM에 따라 다양하다.

< SORT_AREA_SIZE >

SORT_AREA_SIZE는 USER당 SORTING DATA에 필요한 MEMORY의 양을 지정한다. 이것은 SGA 외부에 존재하며, 필요시에만 사용된다. SORT_AREA_SIZE PARAMETER의 값은 BYTE단위로 지정하며, USER당의 PROCESS에 해당하는 값이다.

< DATABASE STRUCTURE ANALYSIS >

ORACLE KERNAL의 기능과 저장 능력이 증가됨에 따라 각기 다른 TABLESPACE로 ORACLE STRUCTURE를 나누고, 다중 DISK와 CONTROLLER를 통하여 성능을 향상시키고, TABLESPACE의 FRAGMENTATION을 감소하는것이 중요하게 대두되고 있다.

ORACLE을 INSTALL하면 DEFAULT로 ROLLBACK SEGMENT TABLESPACE(ROLLBACK_DATA), TEMPORARY TABLESPACE(TEMPORARY_DATA),USER TABLESPACE(USER_DATA)가 생성된다. 가능하면 각기 다른 DISK와 CONTROLLER로 분산 시켜놓는 것이 좋다. 여기에 더하여 USER가 생성하는 APPLICATION이나 OBJECT들도 분산하여 TABLESPACE간의 DISK I/O를 분산시키는 것이 좋다.

< CLIENT-SERVER ANALYSIS >

. Database Communication Protocol

현재 ORACLE과 접속하기 위해서는 ODBC를 통하거나 ORACLE의 SQL*NET PRODUCT를 통하여 가능하다. PERFORMANCE 뿐아니라 ERROR TRAPPING, MESSAGE HANDLING을 위해서 ORACLE의 SQL*NET을 쓰는것을 권장한다.

. Client Versus Server Processing

CLIENT-SERVER환경을 생각할때 PC나 MACINTOSH WORKSTATION을 SERVER에 CONNECT 하는 것 뿐아니라 UNIX SYSTEM을 SQL*NET을 통하여 다른 UNIX와 CONNECT 하는 것까지도 포함하여 생각하여야 한다. EXECUTION에 대하여 CLINET 대 SERVER PERFORMANCE의 차이점을 TEST하려면, QUERY문을 CLIENT SQL*PLUS에서 (SET TIMING ON 상태에서) 실행하여 보고, 그 문장을 SERVER에 전송하여 SERVER SQL *PLUS에서 (SET TIMING ON 상태에서) 실행하여 본다. SERVER SIZE에서 수행하는 것의 주요한 잇점은 실행시간이 짧아지는 것이고, 결국 그것은 SYSTEM의 PERFORMANCE를 향상시키는 결과를 갖게된다.

. Client System Configuration

CLIENT 에서 DML 속도는 매우 중요하다. 더 빠른 PROCESSOR가 부가적인 차이를 가져오게된다. 그러나, 큰 차이점은 RAM의 증가에서 온다.

. Location Of Products And Source Code

대부분의 CLIENT-SERVER 구성은 NOVELL과 같은 NETWORK O/S를 FILE SERVER로 이용한다. 이 구성은 PRODUCT와 SOURCE CODE를 NETWORK DRIVE위에 놓고, CLIENT 에서는 참조만 하도록 한다. 이러한 구성은 제품의 UPGRADE시간의 절약이나 VERSION 관리에 용이하다. 그러나 SOURCE CODE를 포함하여 제품을 PC CLIENT에 INSTALL하는 것이 속 는 더 빠르다. NETWORK I/O가 감소되기 때문이다.

. Windows Swap File

WINDOWS SWAP FILE은 다른 것에의해 참조되는 FILE들이므로, CLIENT WORKSTATION 에서 가장 빠른 DRIVE에 위치하게 하는 것을 권장한다. 다른 방법은 별도의 MEMORY를 추가하여 (16M추 를 권장한다) 16M VIRTUAL DRIVE를 생성하여 WINDOWS SWAP FILE을 이 DRIVE에 생성하는 것도 좋다. 이 방법은 DISK I/O를 줄이고 MEMORY를 ACCESS하게 한다. 이러한 방법은 과거에 많이 사용하였으나, 몇몇 문제를 야기하기도 한다.

. Stored Procedures, Functions And Database Triggers vs. Pl/Sql In Source

STORED PROCEDURE나 DATABASE TRIGER는 이미 PARSING되어 있어 SQL문장이나 PL/SQL BLOCK을 직접 호출하는 것보다 실행에 필요한 OVERHEAD를 줄여준다.

. Eliminating Network I/O And Repetitiveness

CLIENT-SERVER 구성에서 PERFORMANCE를 향상시키는 또 다른 방법은 DATABASE를 호출하여 NETWORK OVERHEAD를 초래하는 반복적인 PROGRAM의 LOGIC을 재구성하는 방법이 있다. 이 방법은 각 RECORD를 조작하는 것이 아니라 한 문장에 의하여 DATA를 조작하는 방법을 간구하는 것이 . 이것은 DATABASE CALL의 숫자도 줄이는 결과를 가져온다.

다음의 방법에서 보면

Method A
1) Start;
2) Fetch Invoice Record;

3) Print Invoice Record;
4) Update Invoice Record To Printed;
5) Loop To Start;

Method B
1) Update all invoices not Printed to In Process;
2) Start;
3) Fetch Invoice record;
4) Print Invoice Record;
5) Loop To Start;
6) Update Invoice Record To Printed;

1000건의 해당 DATA가 있다면 , METHOD A에 의해서는 DATA가 1건씩 FETCH되어 1000번의 UPDATE가 발생하게 다. 반면 METHOD B의 경우, 한번의 UPDATE에 의해 1000건의 DATA가 UPDATE되게된다.

. RULE BASED vs. COST BASED OPTIMIZATION

ORACLE 7에 들어오면서, COSE BASED OPTIMIZER라는 새로운 ORACLE QUERY OPTIMIZER 개념이 도입되었다. ORACLE 7이전에는 실행 PLAN을 결정하는데 14가지 정도의 RULE에 따르는 RULE BASED OPTIMIZER가 사용되어져 왔다. 이 방법은 SQL OPTIMIZATION에 중요한 요소가 되는 DATA의 분산정도에 대해서는 전혀 인식하지 못하는 방법이었다. DATA의 분산정도는 QUERY 실행의 방법을 결정하는데 중요한 요소이 . 결국, ORACLE7 이전에 있어서의 PERFORMANCE TUNING은 SQL문장을 쓰는 개발자의 방법 설정에 의해 결정 되어져 왔다. COST BASED OPTIMIZER를 사용하는 ORACLE7 에서도 OPTIMIZER HINT를 추가함으로써 RULE BASED OPTIOMIZER에 의한 방법의 사용이 가능하다.

ORACLE RULE BASED OPTIMIZER는 INIT.ORA FILE의 OPTIMIZER_MODE PARAMETER를 'RULE'로 SETTING 함으로써도 사용가능하다. 이 PARAMERER의 DEFAULT 값은 'CHOOSE'이다. COST BASED OPTIMIZER는 DATA DICTIONARY에 저장되어 있는 DATA 분산 현황을 사용한다. 현황은 다음의 TABLE에서 확인할 수 있다.

USER_TABLES	ALL_TABLES	DBA_TABLES
USER_TAB_COLUMNS	ALL_TAB_COLUMNS	DBA_TAB_COLUMNS
USER_INDEXES	ALL_INDEXES	DBA_INDEXES
USER_CLUSTERS	DBA_CLUSTERS	

현황 정보는 'ANALYZE' COMMAND에 의하여 생성된다. 이 COMMAND는 각각의 TABLE 에 대하여 실행 되어야 한다. ANALYZE COMMAND에는 DATA DICTIONARY저장되는 현황의 종류를 결정 할 수 있는 몇개의 OPTION이 있다.

< ANALYZE COMMAND의 예 : OPTION포함 >

SQL*PLUS에서 실행한다.

1. TABLE 대한 ESTIMATED STATISTICS의 생성
: 1064 개의 RECORD를 RANDOM하게 SAMPLING한다.

SQL> ANALYZE TABLE TABLE_NAME ESTIMATE STATISTICS;

: DATA의 SAMPLING을 위한 2가지의 OPTION이 있다.

```
SQL>  ANALYZE TABLE TABLE_NAME ESTIMATE STATISTICS SAMPLE 5000 ROWS;
SQL>  ANALYZE TABLE TABLE_NAME ESTIMATE STATISTICS SAMPLE 25 PERCENT;
```

2. TABLE에 대한 COMPLETE STATISTICS의 생성
: TABLE의 모든 ROW에 대한 TABLE과 INDEX의 자료현황을 생성한다.

```
SQL>  ANALYZE TABLE TABLE_NAME COMPUTE STATISTICS;
```

3. TABLE에 대한 STATISTICS의 DELETE
: TABLE과 INDEX의 자료현황을 삭제한다.

```
SQL>  ANALYZE TABLE TABLE_NAME DELETE  STATISTICS;
```

4. CHAINED_ROWS TABLE에 CHAINED ROW를 등록
: UTLCHAIN.SQL SCRIPT(CHAINED_ROWS TABLE을 생성한다)를 아래의 COMMAND를 실행 하기전에 수행하여야 한다. CHAINED_ROWS TABLE에는 CHAINED ROW의 ROWID를 포함한 각각의 ROW들이 INSERT 된다. 이 ROWID를 이용하여 원래의 TABLE로 부터 CHAINED된 ROW들을 DELETE한 후, CHAINED_ROWS TABLE의 ROW를 이용하여 새로운 ROW를 생성하여 CHAIN을 없앨 수 있다.

```
SQL>  ANALYZE TABLE TABLE_NAME LIST CHAINED ROWS;
```

5. TABLE과 INDEX에 CORRUPTION이 있는지를 VALIDATE
: 만약 CORRUPTION이 존재한다면, 다음 COMMAND를 실행 하였을때 'DROP AND RECREATE' MESSAGE가 표시된다.

```
SQL>  ANALYZE TABLE TABLE_NAME VALID STRUCTURE CASCADE;
```

ANALYZE COMMAND가 실행되면, SHARED SQL AREA에 존재하면서 참조되던 OBJECT 들은 다음에 새로운 현황으로 참조되기 위해 SHARED SQL AREA에서 삭제된다.

ANALYZE COMMAND는 위에서와 같이 한 TABLE씩에 대하여 실행될 수도 있으나, ORACLE에서 공하는 PACKAGE PROCEDURE인 DBMS_UTILITY.ANALYZE_SCHEMA를 호출하여 한 USER의 전체 OBJECT에 대해서 전체적으로 실행 할 수도 있다.

```
SQL>  BEGIN
  2    DBMS_UTILITY.ANALYZE_SCHEMA('USER_NAME','COMPUTE');
  3   END;
  4   /
```

ANALYZE되니 않은 DATA DICTIONAY TABLE에 대해서는 RULE BASED OPTIMIZER가 사용된다. COST BASED OPTIMIZER를 사용하게 하기 위해서는, 위의 PACKAGED PROCEDURE를 SYS의 SCHEMA에 대해 실행 되어져야 한다.

. THE ORACLE MONITOR AND ORACLE V$TABLES
ORACLE MONITOR는 SQLDBA UTILITY상 서 가능하다. MONITOR UTILITY는 ORACLE의 V$VIEW이나 여러

개의 V$VIEW를 JOIN하여 정보를 가져온다.

. INDEXING STRATEGIES AND REVIEW

TABLE에 대한 INDEXING은 SYSTEM PERFORMANCE에 중요한 KEY가 되며, INDEXING의 생성에는 주의를 요한다. 한 TABLE에 대하여 INDEX의 갯수를 3개까지로 제한하는 것이 좋으며, 가능한한 5개 이상은 만들지 않는 것이 좋다.

3가지 TYPE의 TABLE에 대하여 각기 일반적인 INDEX기법에 대하여 다음에서 살펴본다.

1. 만약 TABLE이 QUERY만을 위한 것이라면, 위에서의 경우보다 많은 수의 INDEX 를 생성하여도 좋다.
2. 만약 TABLE에 대한 QUERY는 적고, INSERT나 UPDATE가 주로 이루어 진다면 INDEX를 적게 가져가도록 권장한다. (DATA의 운용은 TABLE의 DATA뿐만아니라 TABLE에 대한 INDEX까지 UPDATE가 이루어져야 하기때문이다)
3. 만약 TABLE이 INSERT,UPDATE,QUERY가 모두 필요하다면, INDEXING에 매우 주의를 요하나, INDEX은 꼭 필요하다.

. EXPLAIN PLAN

EXPLAIN PLAN COMMAND는 ORACLE OPTIMIZER가 SQL문의 실행에 사용할 EXECUTION PLAN을 개발자에게 제공해준다. 이 COMMAND는 SQL문장이 실제로 실행 되지는 않고, 실행 PLAN이나 OUTLINE만이 ORACLE TABLE에 저장되므로 SQL문장의 PERFORMANCE를 높이는데 매우 도움이된다. EXPLAIN PLAN COMMAND를 실행하기 전에 COMMAND를 수행할 ORACLE USER에 의해 UTLPLAN.SQL이 먼저 수행되어져야 한다. 이 SCRIPT는 PLAN_TABLE 생성한다.

```
SQL> EXPLAIN PLAN
  2  SET STATEMENT_ID = 'QUERY1' FOR
  3  SELECT CUST_ID, NAME FROM CUST_INFO WHERE CUST_ID = '199';
```

이 TABLE의 자료를 조회하기 위해서는 SQL문을 사용하여야 한다.

```
SQL> SELECT Operation, Options, Object_Name, Id, Parent_Id, Position
  2  FROM PLAN_TABLE
  3  WHERE STATEMENT_ID = 'QUERY1'
  4  ORDER BY Id;
```

< OUTPUT >

```
OPERATION        OPT     OBJECT_NAME     ID   P_ID  POS
--------------   ----    --------------  ---  ----  ---
SELECT STATEMENT                                    0
TABLE ACCESS     FULL    CUST_INFO            1     0
```

```
SQL> SELECT Lpad(' ',2*(Level-1))||
  2       Operation||' '||
  3       Options||' '||
  4       Object_Name||' '||
  5       Decode(Id,0,'Cost = '||Position)    'Query Plan'
```

```
 6   FROM    PLAN_TABLE
 7   START  WITH  ID = 0
 8   AND STATEMENT_ID  = 'QUERY1'
 9   CONNECT  BY  PRIOR_ID  = PARENT_ID
10   AND  STATEMENT_ID  =  'QUERY1';

       < OUTPUT >
QUERY PLAN
------------------------------------------
SELECT  STATEMENT    Cost =
  TABLE  ACCESS      FULL    CUST_INFO
```

SAMPLE: DB에 IMAGE FILE을 저장

Bulletin no : 10111

---

* 이미지 화일의 Type에는 여러가지 형태가 있다.
  예를 들면 BMP, PCX, TIF, ...
  이 중에서도 압축 효율이 높은 File의 형태를 선택하는것이 좋다. 가령, BMP Type보다는 PCX Format이 거의 1/4정도로 압축을 해주므로 보다 효율적이다.
* 일단 PC의 Windows의 PainterBrush 또는 Image 처리하는 Tool로써 이미지 화일을 적절한 형태로 처리하여 File로 만들어서 Unix Server로 올린다.
* Long Data를 처리하기 위해서는 File Size를 알아야하며, 추후 File의 Size로써 처리해야 하는 일이 있다면, Table에 File의 Size도 저장하는 것이 효율적이다.
* 아래의 프로그램은 Image File을 읽어서 Table에 저장하는 내용이다.

```
#include

typedef char bitmap[56262];            /* file size */
EXEC SQL BEGIN DECLARE SECTION;
    VARCHAR   username[20];
    VARCHAR   password[20];
    EXEC SQL  TYPE bitmap IS LONG RAW(56262) REFERENCE;
    bitmap    buffer;
EXEC SQL END DECLARE SECTION;
EXEC SQL INCLUDE sqlca;

void sqlerror();              /* handles unrecoverable errors    */
FILE *fp;
main()
{
        strcpy(username.arr, "SCOTT");
        username.len = strlen(username.arr);
        strcpy(password.arr, "TIGER");
        password.len = strlen(password.arr);

        EXEC SQL WHENEVER SQLERROR DO sqlerror();
        EXEC SQL CONNECT :username IDENTIFIED BY :password;
        EXEC SQL WHENEVER SQLERROR DO sqlerror();

        fp=fopen("xxx.bmp","r");
        fread(&buffer, 1, 56262, fp);

        EXEC SQL INSERT INTO IMAGE VALUES (20, :buffer);
        EXEC SQL COMMIT WORK RELEASE;
```

```
}
void sqlerror()
{
    EXEC SQL WHENEVER SQLERROR CONTINUE;
    printf("\nORACLE error detected:\n");
    printf("\n% .70s \n", sqlca.sqlerrm.sqlerrmc);
    EXEC SQL ROLLBACK WORK RELEASE;
    exit(1);
}
```

DIGITAL UNIX 4.0/4.0A 사용시 PRO*C PRECOMPILE 주의 사항

Bulletin no : 10389

최근 Digital unix 4.0/4.0a 를 사용하는 곳이 증가하고 있으며,이러한 장비에서 Pro*C로 precompile 하는 경우에는 몇가지 주의사항이 있습니다.

- 몇가지 precompile option을 정의하여야만 합니다.

```
code=ansi_c
define=bit64
define=_XOPEN_SOURCE
define=std1
  ¦
number
```

* 위의 사항들을 '$ORACLE_HOME/precomp/admin/pcscfg.h'에 정의해 놓으면 모든 사용자들에게 자동적으로 적용됩니다.
(pcscfg.h 는 normal text file로 vi등으로 편집하시면 됩니다.)

- 한가지 더 주의 하실 점은 C++ 를 사용하는 경우에는
parse=no (or partial)로 하여야만 합니다.

상기 사항들에 대한 자세한 설명은 'Programmer's Guide to the Oracle Pro*C/C++ Precompiler Release 2.2' 의 7장을 참조하시기 바랍니다.

DOS, WINDOWS에서의 PRO*C와 C COMPILER와의 관계

Bulletin no : 10110

---

1. Oracle 7의 Application(Pro*C)은 Oracle V6에서 사용할 수 없다.

2. Pro*C를 Compile하면 오로지 C code로 생성되며, C++ Code로는 생성할 수 없다.

3. Pro*C 1.5.9 이후의 Version은 MS C7.0은 지원하지 않는다.
   (MS C7.0은 Pro*C 1.4.8.0.0에서 지원한다.)

4. Forms 4.0, Report 2.0, Graphics 2.0 또는 상위 Version의 User Exit Program은 Pro*C 1.5에서만 지원한다. 이러한 USER EXIT은 Visual C++ 1.0 또는 1.0 Version과 Link되거나 Compile되며, Borland Compiler는 지원하지 않는다.

5. Pro*C 1.5.9의 DOS Applications은 SQLPME를 사용하여 DOS Protected Mode에서 만든다. Pro*C 1.5.10의 Applications은 MS Windows 환경에서 만든다.

6. Pro*C 1.5.10은 DOS와 Windows 2개의 Product로 구분된다. Windows Version은 MS-Windows하에서 GUI를 Interface하며, DOS Version은 이전 Version처럼 DOS Command Line을 Interface 한다.

DYNAMIC SQL METHOD1(CREATE TABLE)

Bulletin no : 10341

```
/*************************************************************************
Sample Program : Dynamic SQL Method 1( Create Table)
*************************************************************************/

#include
typedef char asciz[80];
EXEC SQL BEGIN DECLARE SECTION;

    EXEC SQL TYPE asciz IS STRING(80) REFERENCE;

    VARCHAR    username[20];
    VARCHAR    password[20];
    asciz      sqlstmt;
    asciz      tabname;
EXEC SQL END DECLARE SECTION;
EXEC SQL INCLUDE sqlca;

void sqlerror();    /* handles unrecoverable errors */
void connect();
main() {

    strcpy(username.arr, "SCOTT");        /* copy the username */
    username.len = strlen(username.arr);
    strcpy(password.arr, "TIGER");        /* copy the password */
    password.len = strlen(password.arr);
    EXEC SQL WHENEVER SQLERROR DO sqlerror();
    EXEC SQL CONNECT :username IDENTIFIED BY :password;
    strcpy(tabname,"TEST_TAB");
    strcpy(sqlstmt,"create table ");
    strcat(sqlstmt,tabname);
    strcat(sqlstmt," (aa number)");
    EXEC SQL EXECUTE IMMEDIATE :sqlstmt;
    EXEC SQL COMMIT;

    EXEC SQL COMMIT WORK RELEASE;    /* log off ORACLE */
    exit(0);
}
```

```
void sqlerror()
{
    EXEC SQL WHENEVER SQLERROR CONTINUE;
    printf("\nORACLE error detected:\n");
    printf("\n% .70s \n", sqlca.sqlerrm.sqlerrmc);
    EXEC SQL ROLLBACK RELEASE;
    exit(1);
}
```

SAMPLE: DYNAMIC SQL METHOD2 SAMPLE(INSERT)

Bulletin no : 10182

```
/************************************************************************
 Dynamic SQL Method 2  : INSERT

This program uses dynamic SQL Method 2 to insert a row into
the EMP table.
************************************************************************/
#define USERNAME "SCOTT"
#define PASSWORD "TIGER"
#include
#include

EXEC SQL INCLUDE sqlca;

EXEC SQL BEGIN DECLARE SECTION;
    char    *username = USERNAME;
    char    *password = PASSWORD;
    char     sqlstmt[2][80];
    int      empno   = 1234;
    int      deptno1 = 97;
    char     tmpi[160];
EXEC SQL END DECLARE SECTION;

main()
{
    EXEC SQL WHENEVER SQLERROR GOTO sqlerror;

    EXEC SQL CONNECT :username IDENTIFIED BY :password;
    puts((char *) "\nConnected to ORACLE.\n");

    sprintf(sqlstmt[1],
       "INSERT INTO EMP (%s, %s)","empno","deptno");
    sprintf(sqlstmt[2],"VALUES (:v%s, :v%s)","1","2");
    strcpy(tmpi,sqlstmt[1]);
    strcat(tmpi,sqlstmt[2]);
    /* Display the SQL statement and its current input host
       variables. */
```

```
    printf("   v1 = %d,  v2 = %d\n", empno, deptno1);

    EXEC SQL PREPARE S FROM :tmpi;

    EXEC SQL EXECUTE S USING :empno, :deptno1;

    EXEC SQL COMMIT RELEASE;
    puts((char *) "\nHave a good day!\n");
    exit(0);

sqlerror:
    printf("\n%.*s\n", sqlca.sqlerrm.sqlerrml,
                       sqlca.sqlerrm.sqlerrmc);
}
```

SAMPLE: DYNAMIC SQL METHOD2 SAMPLE(UPDATE)

Bulletin no : 10183

```
/************************************************************************
 Dynamic SQL Method 2  : UPDATE

This program uses dynamic SQL Method 2 to update a record on
EMP table
************************************************************************/
#define USERNAME "SCOTT"
#define PASSWORD "TIGER"
#include
#include
EXEC SQL INCLUDE sqlca;

EXEC SQL BEGIN DECLARE SECTION;
    char    *username = USERNAME;
    char    *password = PASSWORD;
    varchar  sqlstmt[80];
    int      empno   = 1234;
    int      deptno1 = 30;
EXEC SQL END DECLARE SECTION;
main()
{
    EXEC SQL WHENEVER SQLERROR GOTO sqlerror;

    EXEC SQL CONNECT :username IDENTIFIED BY :password;
    puts((char *) "\nConnected to ORACLE.\n");

    sprintf(sqlstmt.arr,
       "UPDATE emp SET %s = :v%s WHERE %s = :v%s",
       "empno","1","deptno","2");

    sqlstmt.len=strlen(sqlstmt.arr);

    EXEC SQL PREPARE S FROM :sqlstmt;

    EXEC SQL EXECUTE S USING :empno, :deptno1;

    EXEC SQL COMMIT RELEASE;
```

```
    puts((char *) "\nHave a good day!\n");
    exit(0);
sqlerror:

    printf("\n%.*s\n", sqlca.sqlerrm.sqlerrml,
                       sqlca.sqlerrm.sqlerrmc);
}
```

SAMPLE: DYNAMIC SQL METHOD3 SAMPLE(SELECT)

Bulletin no : 10205

```
/*************************************************************
Sample Program 8:  Dynamic SQL Method 3
*************************************************************/
#define USERNAME "SCOTT"
#define PASSWORD "TIGER"
#include
#include
EXEC SQL INCLUDE sqlca;
#ifdef TRUE
#undef TRUE
#endif
#define TRUE 1
EXEC SQL BEGIN DECLARE SECTION;
    char    *username = USERNAME;
    char    *password = PASSWORD;
    VARCHAR  sqlstmt[80];
    VARCHAR  ename[10];
    VARCHAR  tname[10];
    int      deptno = 10;
EXEC SQL END DECLARE SECTION;
main()
{
    EXEC SQL CONNECT :username IDENTIFIED BY :password;
    puts((char *) "\nConnected to ORACLE.\n");
    sprintf(sqlstmt.arr,
           "SELECT ENAME FROM %s WHERE DEPTNO = :v1","EMP");
    sqlstmt.len = strlen((char *) sqlstmt.arr);
    /* Display the SQL statement and its current input host
       variable. */
    puts((char *) sqlstmt.arr);
    printf("   v1 = %d\n", deptno);
    printf("\nEmployee\n");
    printf("--------\n");

    EXEC SQL PREPARE S FROM :sqlstmt;
    EXEC SQL DECLARE C CURSOR FOR S;
    EXEC SQL OPEN C USING :deptno;
```

```
    EXEC SQL WHENEVER NOT FOUND GOTO notfound;
    /* Loop until the NOT FOUND condition is detected. */
    while (TRUE)
    {
    EXEC SQL FETCH C INTO :ename;
    /* Null-terminate the array before output. */
    ename.arr[ename.len] = '\0';
    puts((char *) ename.arr);
    }
notfound:
    printf("\nQuery returned %d row%c.\n\n", sqlca.sqlerrd[2],
           (sqlca.sqlerrd[2] == 1) ? '\0' : 's');
    EXEC SQL CLOSE C;
    /* Commit any pending changes and disconnect from
       ORACLE. */
    EXEC SQL COMMIT RELEASE;
    puts((char *) "Have a good day!\n");
    exit(0);
sqlerror:
    printf("\n%.*s\n", sqlca.sqlerrm.sqlerrml,
                    sqlca.sqlerrm.sqlerrmc);
    EXEC SQL WHENEVER SQLERROR CONTINUE;
    /* Release resources associated with the cursor. */
    EXEC SQL CLOSE C;
    /* Roll back any pending changes and disconnect from
       ORACLE. */
    EXEC SQL ROLLBACK RELEASE;
    exit(1);
}
```

FUTURE TCP 와 SQL*NET V2 사용시 PERFORMANCE

Bulletin no : 10055

Subject: Re: Know How..[2. Future/tcp 사용시]

두번째 항목의 경우 future/tcp 에서 Named Server를 잘못설정하면 Host의 Named Server를 다 찾고서, 없으면 Client의 host file을 뒤져서 자기 host alias로 처리하는 경우가 있었습니다.(1-2초 걸리던 connection time이 12초까지 걸렸음)

경남은행에도 SLIP으로 테스트를 하는데 SQL*Net을 통하면 엄청 늦다고 계속 불평 해서 가서 보니까 SQL*Net의 문제가 아니라 future/tcp의 Client host file과 Named Server의 Host file의 불일치 때문으로 판명난 적이 있었습니다.

이런경우,

1. Client에서 future/tcp의 install program으로 Named Server를 없애든지
2. Host와 Client의 Host file을 일치 시키면 되었습니다.

---

sql*net v1 & v2 test 지원 중 발견된 사항입니다.

1.

아직도 많은 사용자가 sql*net v2 가 별 효과가 없다고 생각합니다. 이런 user는 주로 MTS 환경으로 setting은 하였으되 shared server와 dispatcher를 한개씩 뛰우고 사용하고 있었읍니다.

근본적으로 oracle의 mts는 FIFO 방식으로 SQL을 처리하기 때문에 server가 하나만 떠 가지고는 당연히 sql*net v1의 dedicated server 보다 느리게 처리됩니다.

따라서 mts 환경으로 setup 할 때에는 최소한 shared server와 dispatcher를 2개 이상씩 (3개이상 권유) 뛰워서 사용할 수 있도록 하 바랍니다. 일부에서는 이런이유로 oracle mts 기능이 별볼일 없더라 라고 오해하며 옆사람에게도 전달하고 있습니다.

2.

future tcp/ip 사용시에 tnsnames.ora에 host name 을 사용하면 sql*net v2 가 v1보다 3배 정도 더 느리게 되는데 이때에는 ip address number를 직접사용하면 됩니다.

삼성전관 pdss test시 connection time : sql*net v1 --> 6-8초
sql*net v2 host name --> 25초
sql*net v2 ip number --> 2-3초

3.

sql*less 의 사용 : 생각보다 참 괜찮은 tool 입니다.

sql parsing time, sql execution time, connection test & 경과시간 확인, long raw field의 image 실제 저장 size, db 내의 image data를 읽어내서 dos의 file로 write 하는 등의 기능을 제공하고 floppy 1장에 쏙 들어가는 size이므로 고객지원시나 sql*net v1 & v2 test시에 적극 활용하시기 바랍니다.

PRO*C 에서 HOLD_CURSOR,RELEASE CURSOR 의 의미

Bulletin no : 10164

==========
The HOLD CURSOR and RELEASE CURSOR Options
==========

What are the Oracle Precompiler options HOLD_CURSOR and RELEASE_CURSOR and how do they affect the performance of a PCC program?

First of all, these options apply to implicit and explicit cursors in all precompiled languages except Pro*Ada. They apply ONLY to implicit cursors associated with INSERT, UPDATE, DELETE, or a single row SELECT in Pro*Ada. An explicit cursor in a program is one explicitly created with EXEC SQL DECLARE C1 CURSOR FOR...

Following will be a discussion of what these options do internally and how changing them affects program performance.

What exactly do we mean by CURSOR? Unfortunately, we mean two differ-ent things:

1. The program cursor - A data structure associated with a SQL statement.

   A program cursor is declared for each SQL statement that the precompiler finds in your program. For the statements

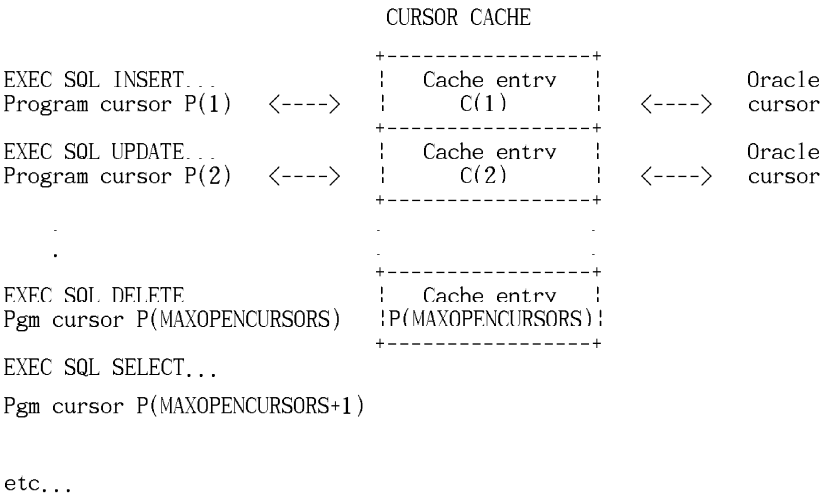
   EXEC SQL DECLARE SEL_EMP_CURS CURSOR FOR...
   EXEC SQL INSERT...

   PCC will declare two program cursors, say c1 and c2.

2. The Oracle cursor (also called the context area) - The work area created dynamically at run time: this area contains the parsed statement, the addresses of the host variables, and other information necessary to execute the SQL statement.

These two cursors are linked together via the cursor cache. The initial size of the cursor cache is determined by the MAXOPENCURSORS option.

The following diagram illustrates the relationship described above after an insert and an update have been executed in your program:



How are the HOLD_CURSOR and RELEASE_CURSOR options related to this view?

The HOLD_CURSOR option deals with the link between the program cursor and its cache entry.

The RELEASE_CURSOR option deals with the link between the Oracle cursor and the cache entry.

For SQL statements that are FREQUENTLY RE-EXECUTED, the bottom line is this: if you want to maximize performance, make sure these SQL statements stay "glued" to their respective Oracle cursor.

What does it mean when a SQL statement is "glued" to its Oracle cursor? It means that both links between the SQL statement and its Oracle cursor are made permanent.

Why would you want to keep a statement "glued" to its context area?
Because the context area contains the parsed statement and other information necessary to execute the statement, such as the addresses ofthe host variables. Maintaining access to this information makes subsequent execution of the statement much faster.

How do you "glue" a statement to a cache entry? By correct use of the HOLD_CURSOR and RELEASE_CURSOR options via the PCC command line or inline with EXEC ORACLE OPTION(...).

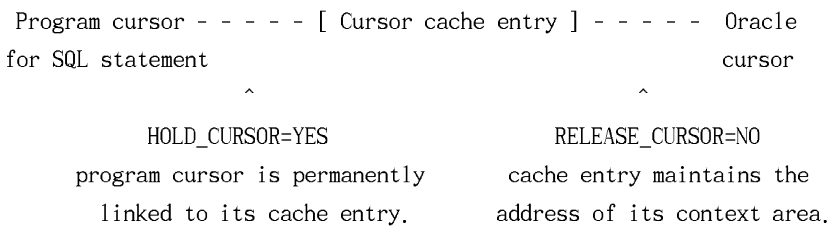
For instance, with HOLD_CURSOR=YES as the Oracle option, a cache entry cannot be flagged for reuse. This has important implications. If all cache entries have been used up and a new cache entry is needed for a new SQL statement such that the number of cache entries would now exceed the number specified in MAXOPENCURSORS, Oracle will use the first cache entry marked reuseable.

For example, in the above diagram, if the cache entry C(1) is marked reusable, and the program is about to execute the EXEC SQL SELECT...
(program cursor P(MAXOPENCURSORS+1), and the number of cache entries in use already equals

MAXOPENCURSORS, cache entry C(1) and its Oracle cursor will now be linked to the select statement. A subsequent execution of the insert statement would require pre-empting a cache entry and its Oracle cursor from another SQL statement and performing a reparse.

Correspondingly, with the default RELEASE_CURSOR=NO as the Oracle option, the link between the cache entry and the Oracle cursor (the context area) is maintained after the statement is executed so that the parsed statement and, more importantly, the allocated memory stay available.

The freeing up of this memory by RELEASE_CURSOR=YES means that the next statement that gets linked to this cache entry will require an expensive reallocation of memory in addition to a reparse. Ugh! Why would anybody want RELEASE_CURSOR=YES? We will see later on.



So the HOLD_CURSOR option is intimately tied to the MAXOPENCURSORS option. What exactly is the MAXOPENCURSORS option? First of all, MAX OPENCURSORS is a misnomer. It should more appropriately be called INITIAL_CURSOR_CACHE_SIZE. (Okay, so it's a mouthful.) Anyway, if all cursor cache entries are currently marked "not reusable" either because of the HOLD_CURSOR option or because the associated statement is currently being executed (an explicitly opened cursor is still being fetched on and hasn't been closed yet), then a request for a new cursor will actually result in the extension of the cursor cache at runtime (i.e. if MAXOPENCURSORS=10, and all 10 entries are active, then an 11th will be created). Just letting the precompiler reuse the oldest cache entry won't always work, as the following example illustrates: Imagine the case where the user has ten explicitly declared cursors opened, and wants to execute an eleventh. If the program actually reuses the oldest program cursor, the user would lose his current position in the first cursor and would not be able to fetch from it anymore.

By the way, if an eleventh cache entry is created, when that cursor is closed the eleventh entry is not removed. Setting MAXOPENCURSORS low saves memory, but causes potentially expensive dynamic allocations of new cache entries if they're needed. Setting it high assures quick execution, but may use more memory than necessary.

What if a statement is not executed repeatedly in a program? Then you could go with the other options HOLD_CURSOR=NO and RELEASE_CURSOR=YES.

With the HOLD_CURSOR=NO option, the link between a program cursor and its cache entry is not permanent. The cache entry is automatically marked reusable in case it is needed. With the RELEASE_CURSOR=YES option, the Oracle cursor (the context area) is automatically freed and the parsed statement lost. A reason you might use this option is if you are limited by the number of Oracle cursors (MAXOPENCURSORS) at your site due to memory issues. You may want to incur

the cost of reallocating memory and reparsing in order to manage memory more effectively.

An advantage of setting RELEASE_CURSOR=YES is that until the link between the cache entry and the Oracle cursor (context area) is removed,ORACLE keeps parse locks on any tables referenced in the SQL statement. These parse locks prevent other users and you from ALTERing or DROPping the tables (does ORA-0057 sound familiar?). Also, in Version5, it will free up the read-consistent image of the referenced tables stored in ORACLE's Before Image file.

What do we mean when we say that RELEASE_CURSOR=YES takes precedence over HOLD_CURSOR=YES? With RELEASE_CURSOR=YES, the link between the Oracle cursor and the cache entry is cut and the Oracle cursor is freed (closed), so even if your program cursor is permanently linked to the cache entry because HOLD_CURSOR=YES, you will still have to reallocate memory and reparse the statement. So subsequent executions of a statement don't benefit from the HOLD_CURSOR=YES option because RELEASE_CURSOR=YES.

For programmers experienced with OCI, here's the OCI equivalent of what's happening:

```
#define MAXOPENCURSORS 5

char     *sql_stmts[10];
curs_def cursor[MAXOPENCURSORS];

oopen(cursor[0],...);
osql3(cursor[0],...,sql_stmts[0],...);
```

An example of a "cache entry" being linked to another SQL statement
later on in the program is as follows:

```
osql3(cursor[0],...,sql_stmts[5],...);
```

I am forced to reuse one of my "cache entries" to execute the sixth
SQL statement.

An example of a context area being freed is:

```
oclose(cursor[0]);
```

Reusing cursor[0] would require another oopen() and another osql3()--
another dynamic allocation of memory and another reparse.

Conclusion

As a programmer, you will get the most from these options by using them selectively inline rather than specifying them as options at precompile time.

SQL-2101(ORA-2101): INCONSISTENT CURSOR CACHE (UCE/CUC MISMATCH)

Bulletin no : 10174

Causes of SQL-2101

'SQL-2101: Inconsistent cursor cache (UCE/CUC mismatch)' can be a very confusing and frustrating error to receive while running a Pro*C program. To start with, since the error is generated at runtime, the most logical place to begin looking for an explanation is in the Oracle Error Message and Codes Manual. However, there is no listing for an ORA-2101. It looks like it's time to call Oracle Support;
there's another undocumented error.

But there is a reason that this error is not listed among the ORA errors in the Error Messages and Codes Manual. It is because Oracle does not generate this error. Rather, it originates from the SQLLIB runtime libraries used by precompiler programs. This error is documented as RTL-2101 in the Programmer's Guide to the Oracle Precompilers V1.3 and V1.4 in Appendix A, in the Programmer's Guide to the Oracle Precompilers V1.5 in Appendex D, and in the Oracle7 Server Messages and Codes Manual on page 4-29. In the Programmer's Guide to the Oracle Pro*C Precompiler Release 2.0, it is documented in Appendex C as SQL-2101.

However, the three word message 'Inconsistent cursor cache' is more useful than the full paragraph cause and action given in the manuals. The true meaning of this error is that the cursor cache entry for the statement being executed has been corrupted. To understand why this corruption can occur, it is necessary to review the explanation of the cursor cache in the Precompiler Manual. The important fact here is that the cursor cache data structure used to manipulate every SQL statement is located in memory allocated by SQLLIB in the program's data space. Given this, the most likely culprit for corruption of a program's data space is the program itself.

The following is a list of program errors which can possibly corrupt the cursor cache:

- o Using a pointer returned by malloc, calloc, or realloc for more space than was actually allocated. For example:

```
char *p;

if (( p = malloc(5) ) != NULL)
    strcpy (p, '12345678');
```

Here, p points to a block of 5 bytes but a string of 8 characters was copied into it.

The extra 3 bytes can cause corruption in the cursor cache. Note that SELECTing or FETCHing into space allocated by malloc, calloc, realloc can also cause corruption if the selected data is larger than the space allocated.

- o Be sure to check the return value from calls to malloc, calloc, realloc, and free. If a NULL value is returned, then it means that the call did not complete successfully and the pointer should not be used.

- o The pointer passed to free must be the same as one that was returned from a call to malloc, calloc, or realloc. If it has been modified, then the free call may corrupt program space.

- o Check the usage of array indices. Make sure that they do not exceed the value used when declaring the array.

A temporary solution to the symptom (getting the SQL-2101) is to use the precompiler option RELEASE_CURSOR=YES. Many times this will get rid of the error, but the fact remains that the corruption is still happening. The full effect of this option is described in document#: 82810.067. In this case, it frees the corrupted memory in the cursor as soon as the cursor (implicit or explicit) is closed, before SQLLIB will detect the corruption. Tracking down stray pointers or checking for over-written memory can be the most difficult C programming errors to debug. However, the SQL-2101 has repeatedly been proven to be caused by memory corruption due the above types of errors.

PRO*C ON MS-DOS AND MS-WINDOWS PLATFORMS

Bulletin no : 10004

---

1. Support of Oracle 6.0 indicates that your applications will work against an Oracle 6.0 RDBMS(or Oracle 7 Server functionality is used,i.e., only V6 type calls are made).
   However, the reverse is not true. Oracle 7 applications cannot be run against Oracle 6.0

2. Some of the C (or C++) compilers listed above are capable of generating C++ code as straight C.
   You must put all C++ code in one file and all PROC/C code in another, then link these two modules.
   PROC can only generate straight C code and not C++ code.
   Include of C++ code in PROC modules is not supported.

3. Pro*C 1.5.9 and later do not support MS C7.0

4. User Exit support for Oracle Forms 4.0, Oracle Reports 2.0 and Oracle Graphics 2.0 are provided through Pro*C 1.5 only.
   These User Exits may be compiled and linked with Visual C++ 1.0 or 1.5 only (you cannot use the Borland compiler)

5. Pro*C 1.5.9 applications are built entirely in DOS protected mode and hence require the use of SQLPME.
   Pro*C 1.5.10 applications are built in the MS-Windows environment.

6. Pro*C 1.5.10 onwards is now two separete products, one for Windows and one for DOS.
   The Windows version runs completely under MS-Windows and has a GUI(no command line) interface.
   The DOS version will use the same DOS command line interface as previous version .
   Pro*C 1.5.10 is not yet available on DOS.

7. If you have any questions or concerns about future release of Pro*C please contact your Oracle representative.

   Oracle Customer Support does not have information on future releases.

쩽ro*C Ver	Apps For	Create In	RDBMS Ver	Supported Compliers	User Exits Support	OCI supp	PL/SQL supp
1.3	DOS	DOS	8	MSC 8.0	F30 for DOS	Yes	1.x
1.5.9	DOS WIN	DOS DOS	7 7	MSC 8.0 VC++ 1.0/1.5 VC++ 1.0/1.5	None OF4.0.12 OR2.0.13 OG2.0.9	Yes	2.x
1.5.10	WIN WIN	WIN WIN	7 7	VC++1.0/1.5 Borland C4.0	OF4.0.13 OR2.0.14 OG2.0.10 None	Yes Yes	2.x 2.x

기타 자세한 내용은 Support News Volume 3 issue 3 1994을 참조 바랍니다.

ORA-1002 FETCH OUT OF SEQUENCE IN PRO*C

Bulletin no : 10267

ORA-1002 : FETCH OUT OF SEQUENCE

ORA-1002 error가 의미하는 것은 user가 더이상 유효하지 않은 cursor로부터 ftech를 하려고 하기 때문이다.

이러한 경우의 가장 흔한 경우를 다음 두가지로 설명하겠다.

1) ORA-1403 등과 같이 NO DATA FOUND를 return하는 fetch작업을 수행할때

: 즉 fetch는 항상 현재의 row를 return하며, 마지막 row를 return한 후에 더 retrieve할 data가 있는지 check하기 위해 한번 더 fetch를 수행하는데 이때 특정 exit구문으로 loop를 종료시키지 않으면 ORA-1002 error를 return 한다.

```
DECLARE
   EMP_NAME  EMP.ENAME%TYPE;
   CURSOR C1 IS SELECT ENAME FROM EMP;
BEGIN
   OPEN C1;
   LOOP
       FETCH C1 INTO EMP_NAME;
   END LOOP;
   CLOSE C1;
END;
```

2) SELECT FOR UPDATE를 가진 cursor 의 fetch작업내에 commit이 있는 경우

```
:
   DECLARE
      EMP_NAME  EMP.ENAME%TYPE;
      CURSOR C1 IS SELECT ENAME FROM EMP
      FOR UPDATE OF SAL;
   BEGIN
      OPEN C1;
      LOOP
          FETCH C1 INTO EMP_NAME;
          UPDATE EMP SET SAL = 50;
                   -- cannot do this commit in version 6
```

```
        COMMIT;
    END LOOP;
    CLOSE C1;
END;
```

database consistency를 유지하기 위해 user는 자신이 update/delete 할 row를 select시점에서 lock을 걸어두는 경우가 있는데 commit이나 roollback을 만나기 이전까지는 lock이 걸려있다. 위 예에서 select for update 로 cursor가 open되는 시점에 lock을 걸어 놓지만 fetch를 수행중 commit을 만나 lock이 release된다. 이 때 cursor는 더이상 유효하지 않으며 fetch out of sequence를 return한다.
해결책은 commit을 fetch loop의 바깥쪽으로 빼거나 select for update문을 사용하지 않아야 한다.

ORA-1405 처리 방법

Bulletin no : 10079

ORA-01405 는 ERROR 가 아니고 WARNING MESSAGE 이다. 따라서 프로그램내에서 ERROR CHECK 하는 방법에따라 PRE*COMPILER OPTION 을 상이하게 설정 ERROR 를 처리 할 수있다.

1. SQLCODE 로 ERROR CHECK 하는경우

```
/* commission 이 null 인경우 */
EXEC SQL SELECT ENAME, SAL, COMM
         INTO :emp_name, :salary, :commission
         FROM EMP
         WHERE EMPNO = :emp_number;
      if (sqlca.sqlcode != 0 ) exit(0);
```

위 경우 sqlca.sqlcode 가 -1405 이므로 exit 하게된다. 따라서 값을 정상적으로 FETCH 를 하였으나 ERROR 로 처리되게 된다. 위의 프로그램을 변경하지않고 처리 할 수 있는방법은 dbms=v6 를 PRECOMPILER OPTION 에 추가해준다. dbms=v6 로 SETTING 할경우는 HOST 변수에 NULL 이 RETURN 되더라도 sqlca.sqlcode 는 0 이 된다.

2. NOT FOUND 로 CHECK 하는경우

```
EXEC SQL WHENEVER NOT FOUND DO break;
EXEC SQL SELECT ENAME, SAL, COMM
         INTO :emp_name, :salary, :commission
         FROM EMP
         WHERE EMPNO = :emp_number;
```

FOUND 인경우 1403 이 RETURN 되므로 위의 SQL 문은 정상적으로 처리가 된다.

ORA-3113,3114 의 발생원인 및 진단방법

Bulletin no : 10120

ORA-3113은 가장 포괄적인 오류입니다. 원초적인 의미로는 예기치 않은 이유 때문에 통신이 두절된 상태를 의미합니다. 이 오류뒤에는 대개 ORA-3114 'not connected to ORACLE'이 동반됩니다.

원인을 살펴보면

1. 가장 많은 원인은 서버의 Oracle 쉐도 프로세스가 예기치 않게 종료된 경우 입니다. 따라서 수행중에 갑자기 ORA-3113과 3114가 발생했다면, 우선 서버의 alert.log를 점검하여 다른 Oracle 오류가 발생했는지 알아보십시요.

<< alert.log >>
서버가 UNIX 인경우 $ORACLE_HOME/rdbms/log/alert_.log 화일에 ORA-3113 에러가 발생했던 시점에서 다른 에러가 발생했는지 점검 합니다. 특히 ORA-600[],[]이 발생했으면 에러 내용을 Oracle Technical Support Center로 연락 하십시오.

2. ORA-3113의 원인 중 그 다음으로 많은 것은 SQL*NET 드라이버가 Unix의 ORACLE 실행 파일과 연결되지 않아 발생한 경우입니다. 연결을 공식적으로 수신하고 그것을 ORACLE 쉐도 프로세스에 전달한다 해도, 쉐도 프로세스는 처리방법을 모르기 때문에 어떤 방법으로도 응답하지 못할 수 있습니다.
그러므로 클라이언트는 연결순간에 ORA-3113을 보게 됩니다.

3. 세번째로 많은 원인은 서버쪽의 기계 손상이나 네트워크 고장입니다.

4. 자주 있는 것은 아니지만 같은 네트워크에서 두 서버가 같은 노드 이름을 가질 때에도 이 오류가 발생합니다.

5. ORA-3113은 토큰링 카드의 공유 RAM 크기가 16KB가 아니라 8KB로 설정 되었음을 나타내기도 합니다. 토큰 링을 사용중이라면 공유 버크 크기를 점검하고 키워 보십시요.

6. ORA-3113은 INIT.ORA 매개변수 CONTEXT_AREA와 CONTEXT_INCR이 4096이라는 값으로 설정된 경우에도 발생합니다. 그럴때는 값을 8192로 키우면 ORA-3113이 해소됩니다.

이상 말한 모든 원인은 결국 클라이언트가 서버로부터 어떤 정보를 읽으러 갔다가 거기서 더 이상 연결이 없음을 발견했다는 뜻입니다. ORA-3113은 좀 더 진단해야 추적 가능한 더 큰 문제가 있음을 알리는 신호탄에 불과합니다. 다행히도 앞서 말한 여섯가지 정보를 참고하면 해결책을 찾는 방향은 잡힐 것입니다. 우선 ORA-3113을 디버깅하려면, 루프백을 수행중에 같은 CONNECTING 을 여러번 시도해 보는 것이 좋습니다. 즉, 서버의 어떤 툴이든 데스크탑 클라이언트에서 지정하는 것과 같은 연결 스트링을 사용하여 연결할 수 있습니다. 루프백을 수행중에도 똑같은 문제가 발생하면 데스크탑 클라이언트 쪽이 아니라 서버쪽에 문제가 있다고 보아야 합니다. 루프백을 수행하려면 서버에서 SQLPLUS 또는 SQLDBA를 호출하고, 서버의 SQLPLUS 또는 SQLDBA 프롬프트에서 다음과 같이 입력하십시요.

```
CONNECT USERNAME/PASSWORD@t:/:
```

예를 들어, SQL*NET TCP/IP를 통해 Unix 서버에 연결돼 있고 SQL*Plus를 호출하고, 같은 "t::" 연결 스트링을 사용하여, 같은 SELECT 문을 내서 루프백을 해 보십시오.

ORA-6401 ERROR 발생시 처리 방법

Bulletin no : 10058

---

sql*net v1,sql*net v2 TEST 시에 아래와 같은 error message가 발생할 경우에 다음과 같이 해보세요.

ORA-6401:NETCMN-invalid driver designator

1) $ORACLE_HOME/bin
   $adapters oracle
    sql* net v2 protocol adapters linked with oracle:
    v2 pipes
    ipc (unix domain scoket)
    tcp/ip ----------> 주의하여 볼것.

2) $ORACLE_HOME/bin
   $adapters sqlplus
    sql*net v2 protocol adapters linked with sqlplus:
    v2 pipes
     ipc (unix domain socket)

위와 같으면 error 발생 ..

sqlplus 실행 file 에 tcp/ip protocol link가 빠져서.

==> 1.cd $ORACLE_HOME/sqlplus/lib
    2. make -f sqlplus.mk install

기타 다른 실행 file 도 확인하여 make 작업을 다시 하세요.

ORACLE의 TWO-TASK DRIVERS

Bulletin no : 10233

Unix 시스템에서 User Process와 ORACLE Shadow Process 사이에 Communication 하기 위해 다양한 Driver 들이 사용된다.

1.Pipe(Default)와 Fast Driver

Single - System 환경에서 사용됨.
1개의 Machine에 있는 Multiple DB에 연결하기 위해 사용
Oracle 의 Standard Part로서 지원
Fast Driver는 V6에 만 지원

2.Networking Drivers(Sql*Net TCP/IP 등)

Networking 환경하에서 사용
분리된 Product로 판

3.SQL*NET V2

(1)BEQ

SQl*Net V2의 pipe adapter
Unix pipe에 만 사용. MTS가 아님

(2)IPC(Inter-Process Communication)

SQL*NET V2의 Default Adapter(UDS)(7.0.13 이후)
UNIX Domain Sockets 의 Top에서 수행됨
Local Host에 Running 중인 Process 간에 Socket-Based Communication
MTS에 Locally Connection 하기 위해 사용.
Oracle IPC Adapter 는 IPC Communication Protocol 과 Integrate 하기
위하여 TNS와 그것의 Application 을 허락해야 한다.
TNS Connect Descriptor으로 IPC-Specific Parameter를 사용할 수 있다.
IPC Adapter 는 SQL*NET V2를 설치 할 경우 자동적으로 설치된다.

```
IPC ADDRESS
    ( ADDRESS =
          [  ( COMMUNITY = community_ name ) ]
             ( PROTOCOL=IPC )
             ( KEY=alphanumeric )
    )
```

```
mts_dispatchers="ipc,1"
mts_max_dispatchers=10
mts_service=ORA7
mts_severs=1
mts_max_servers=10
mts_listener_address="(address=(protocol=ipc)(key=400))"
```

PC CLIENT TOOL 과 ORACLE DB 와의 INTERFACE 방법

Bulletin no : 10012

PC의 다양한 툴중에서 일부 잘 알려진 제품을 중심으로 Oracle과의 interface 방법을 기술 하였습니다. 업무에 참조 하시기 바랍니다.

분 류	제 품 명	회 사 명	Oracle 과의 Interface
데이타 베이스	Access	MS	Glue,ODBC
	FoxPro	MS	Connectivity
	Clipper		Non
	dBaseIV		Non
	Paradox		SQLLink
Spread Sheet	Excel	MS	Glue, Q+E
	Lotus1-2-3	Lotus	Glue, Q+E(?)
	Quattro Pro		Glue
Word Processor	Word	MS	Glue
	Word Perfect	WordPerfect	Glue
	Ami Pro	Lotus	Glue
Window Program Tool	Visual Basic	MS	Glue, Q+E
	ToolBook	Asymmetrix	Glue, Q+E

(Remarks):

- MS: Microsoft
- Q+E: Pioneer Software
- Glue: Oracle
- Connectivity:Microsoft
- SQLLink:Pradox
- 기타 기술된 내용 중에 잘못된 부분이나 추가시킬 내용이 있으면 메일로 보내 주시면 감사하겠습니다..

PRECOMPILE에서 MODE OPTION으로 저장하는 방법

Bulletin no : 10301

---

1. Pro*C Version 1.5 에서 대별되는 Mode option은 ORACLE (default)와 ANSI로 구분되어진다. 두 Mode의 큰 차이는 Char문자의 입력(insert)과 비교(where)에서 나타난다.

2. Oracle Version 6에서의 Pro*C Version 1.3을 이용하여 Application Program작성시 문자변수에 대하여 [char]를 선언할 때와 [varchar]를 선언할 때 입력과 비교에 별다른 차이를 발견할 수 없다.
Oracle Version 7에서 Pro*C Version 1.5사용 시는 Mode Option에 따라 커다란 차이를 볼 수 있다.

3. Mode=Oracle, 또는 Mode를 선언하지 않은 경우
(1) 문자 변수에 대하여 작업 (insert, updte, where)을 하는 경우 필히[varchar]의 TYPE을 선언해야 한다. 만일 문자변수에 대하여 [char]로 선언된 경우 insert시 정상적으로 작업된 것과 같이 수행이 되나 insert된 자료를 SQL*Plus에서 [where]를 이용하여 Select시 검색할 수 없다. 그러나 Pro*C로 프로그램을 작성하여 검색시는 정상적으로 발견할 수가 있다. [char]변수로 선언하여 insert시 보이지는 않으나 문자의 뒷부분에 [\0]값이 모두 들어가 있다.

예)
```
exec sql begin declare section;
char  name[10];
exec sql end declare section;
strcpy(name, "abc");
exec sql insert into samp (name) values (:name);
```

위의 경우, 실질적으로 [name]변수내에는 [abc\0\0\0\0\0\0]의 형태로 값이 저장되어 insert작업을 수행한다. 이를 확인하는 방법은 SQL*Plus에서 다음의 문장을 수행한다:

```
SQL> select vsize(name), name from samp;
```

이때 보이는 문자는 [abc]이나 vsize값은 [10]으로 나타나게 된다.
그러므로 SQL> select name from samp where name='abc';를 수행하면 검색된 자료가 없다는 message가 출력된다.

(2) 문자변수를 [char]로 선언하고 사용 가능한 방법
[char]로 변수를 선언한 경우 문자값 뒤에 [\0]값이 저장되므로 그 값을 [ ] (space)로 대치하면 된다. 위의 예제에서

```
strcpy(name, 'abc');
strcat(name, '         ');
```

를 이용하면 SQL*Plus에서 정상적으로 검색되고 vsize의 값도 정상적으로 출력된다.

4. Mode = ANSI를 선언한 경우

(1) ANSI Mode를 이용하기 위해서는 우선

```
Long  SQLCODE;
```

를 선언해 주어야 한다. 이 SQLCODE에는 SQL문장수행시 Error값을 저장한다.

(2) ANSI Mode의 장점은 Oracle Version 6 사용시와 동일하게 문자 변수에 대하여 [char]와 [varchar]를 같게 작업을 한다.

예)
```
exec sql begin declare section;
char       name[10];
varchar  name2[10];
exec sql end declare section;
strcpy(name, "abc");
strcpy(name2.arr, "def");
name2.len = strlen(name2.arr);
exec sql insert into samp(name) values (:name);
exec sql insert into samp(name) values (:name2);
```

(위와 같이 작업을 수행한 경우 SQL*Plus에서 동일한 작업이 가능하다.)

Note: Oracle Version 6에서는 ANSI 드가 없으므로 사용할 수 없다.

PROC 2.X 에서 INCLUDE FILE 사용방법

Bulletin no : 10200

PROC 2.0 에서는 include file 에대한 디렉토리 PATH를 INCLUDE option 으로 지정하여 사용 할수 있다. 이경우 Precompiler 는 다음 순서로 디렉토리를 SCAN 하면서 include file 을 찾는다.

1. 현재 디렉토리
2. standard header file 에대한 built-in 디렉토리
3. INCLUDE option 에 지정된 디렉토리

* 2 의 경우 sqlca.h, sqlda.h 와 같은 standard header file 에 대한 디렉토리는 지정할 필요가 없고, UNIX에서는 precompiler 가 ORACLE_HOME /sqllib/public 디렉토리에서 이 화일들을 찾는다.
* Nonstandard header file은 현재 디렉토리에 없으면 다음과 같이 INCLUDE Precompile option 으로 path를 지정한다.

```
... INCLUDE=path1 INCLUDE=path2 ..
```

<사용예>

<< 프로그램 >>

```
/*  sample2.pc */
#include
#include
#include
#include
typedef char asciiz[PWD_LEN];

EXEC SQL TYPE asciiz IS STRING(40) REFERENCE;
```

<< INCLUDE FILE >>

```
/users2/oracle7/lib/sam2.h
----------------------------
#define UNAME_LEN 20
#define PWD_LEN   40
```

<< MAKE FILE >>

```
/users2/oracle7/bin/proc INCLUDE=/users2/oracle7/lib  iname=$1.pc
                         ****************************
cc -I. -O -Aa -D_HPUX_SOURCE +ESsfc +ESlit -I/users2/oracle7/sqllib/public
-I/users2/oracle7/lib -c $1.c
**********************
cc -W1,-aarchive -L/users2/oracle7/lib -o $1 $1.o -lsql  -lsqlnet -lora
-lsqlnet  -lora -lnlsrtl -lcv6 -lcore -lnlsrtl -lcv6  -lcore  `cat
 /users2/oracle7/rdbms/lib/sysliblist`
```

SAMPLE: PROC 에서 SESSION 을 KILL 하는 방법

Bulletin no : 10184

```
#include
EXEC SQL BEGIN DECLARE SECTION;
    VARCHAR    username[20];
    VARCHAR    password[20];
    varchar    ser[30];
    varchar    str[130];
EXEC SQL END DECLARE SECTION;

EXEC SQL INCLUDE sqlca;
void sqlerror();    /* handles unrecoverable errors */

main() {

    strcpy(username.arr, "SYS");        /* copy the username */
    username.len = strlen(username.arr);
    strcpy(password.arr, "MANAGER");        /* copy the password */
    password.len = strlen(password.arr);

    EXEC SQL WHENEVER SQLERROR DO sqlerror();
    EXEC SQL CONNECT :username IDENTIFIED BY :password;
         EXEC SQL SELECT ''''||to_char(SID)||','||
                             to_char(SERIAL#)||''''
              INTO :ser
              FROM v$session
              WHERE sid=8;
    strcpy(str.arr,"alter system kill session ");
    strcat(str.arr,ser.arr);
    str.len=strlen(str.arr);
    printf("%s %d",str.arr,str.len);
    EXEC SQL EXECUTE IMMEDIATE :str;
}
void sqlerror()
{
    EXEC SQL WHENEVER SQLERROR CONTINUE;
    printf("\nORACLE error detected:\n");
    printf("\n% .70s \n", sqlca.sqlerrm.sqlerrmc);
    exit(1);
}
```

PROC 에서 외부 환경변수를 받아 들이는 방법

Bulletin no : 10213

```
#include
typedef char asciz[20];
EXEC SQL BEGIN DECLARE SECTION;
    /* Define type for null-terminated strings */
    EXEC SQL TYPE asciz IS STRING(20) REFERENCE;
    asciz     username;
    asciz     password;
    asciz     emp_name;
    short     ind;
    float     salary;
    float     commission;
EXEC SQL END DECLARE SECTION;
EXEC SQL INCLUDE sqlca;

void sqlerror();  /* handles unrecoverable errors */

main()
{
    /* Log on to ORACLE */
    strcpy(username, "SCOTT");
    strcpy(password, "TIGER");

/**
 **  ORA_NLS 는 ORACLE 7.2 에서만 필요 함.
 **/
    putenv("ORACLE_HOME=/oracle/oracle");
    putenv("ORACLE_SID=RC722");
    putenv("NLS_LANG=American_America.ko16ksc5601");
    putenv("ORA_NLS=/oracle/oracle/ocommon/nls/admin/data");

    EXEC SQL WHENEVER SQLERROR DO sqlerror();
    EXEC SQL CONNECT :username IDENTIFIED BY :password;
    printf("\nConnected to ORACLE as user: %s\n", username);
    EXEC SQL DECLARE salespeople CURSOR FOR
    SELECT ENAME, SAL, COMM
        FROM EMP
        WHERE JOB LIKE 'SALES%';
    EXEC SQL OPEN salespeople;
    for ( ; ; )
```

```
    {
        EXEC SQL FETCH salespeople
                 INTO :emp_name, :salary, :commission:ind;
        if (sqlca.sqlcode == 1403) break;
        if (sqlca.sqlcode ==0) {
        printf("%-11s%9.2f%13.2f\n", emp_name, salary, commission);
        }
    }
    EXEC SQL CLOSE salespeople;
    printf("\nHave a good day.\n");
    EXEC SQL COMMIT WORK RELEASE;
    exit(0);
}

void sqlerror()
{
    EXEC SQL WHENEVER SQLERROR CONTINUE;
    printf("\nORACLE error detected:\n");
    printf("\n% .70s \n", sqlca.sqlerrm.sqlerrmc);
    EXEC SQL ROLLBACK WORK RELEASE;
    exit(1);
}
```

PROC의 MAKE FILE을 이용하여 SHELL프로그램을 만드는 방법

Bulletin no : 10154

A. ORACLE Server 버젼이 7.0.X 인경우

[ORACLE7 버젼확인 방법]

```
$ sqldba lmode=y
  SQL*DBA: Release 7.1.6.2.0 - Production on Thu Nov 16 10:08:42 1995
  Copyright (c) Oracle Corporation 1979, 1994.  All rights reserved.
  Oracle7 Server Release 7.1.6.2.0 - Production Release
                         *********
```

[단계1]

1. $ cd $ORACLE_HOME/proc/demo

2. $ make -nf proc.mk sample1 > mk
   -- mk 내용이 "sample1 is up to date" 이면 sample1.pc 만 남겨놓고
   -- sample1.c sample1.o sample1 을 삭제후 다시 실행한다.

3. $ vi mk (sample1 부분을 $1 으로 수정 한다)

```
/usr3/ora7016/bin/proc ireclen=132 oreclen=132 select_error=no
iname=$.pc
/usr3/ora7016/bin/echodo cc -I. -O -L/usr3/ora7016/lib -o $1 $1.c
/usr3/ora7016/lib/libsql.a /usr3/ora7016/lib/osntab.o -lsqlnet -lora
/usr3/ora7016/lib/libpls.a -lsqlnet -lnlsrtl -lcv6 -lcore -lnlsrtl
-lcv6  -lcore  -lld -lmalloc -lseq `cat /usr3/ora7016/rdbms/lib
 /sysliblist`
```

4. $ chmod 777 mk

[프로그램 수행]

```
$ ./mk sample2
```

B. ORACLE Server 버젼이 7.1.X, 7.2.X 인경우

< PROC Version 1.X 를 사용하는 경우 >

[단계2]

1. cd $ORACLE_HOME/proc16/demo

2. make -nf proc16.mk sample1 > mk

3. [단계1] 의 3. 4. 를 수행

< PROC Version 2.X 를 사용하는 경우 >

[단계3]

1. cd $ORACLE_HOME/proc/demo

2. make -nf proc.mk sample1 > mk

3. [단계1] 의 3. 4. 를 수행

SQL*NET SESSION의 DEAD CONNECTION 처리 방법

Bulletin no : 10223

Client Server 환경에서 PC를 Client 로 사용시 비정상적인 방법으로 Server 와 Disconnect 하면 Server 쪽에 있는 Dedicate Server 가 남아 있다.
이는 Sql*net V1 & V2 에서 발생하던 문제로 Sql*net V2.1 이후에는 아래와 같은 방법을 이용하면 Dead connection 이 정상적으로 Disconnect 된다.

* 환경 : Server - Sql*net tcp/ip V2.1.3
Client - Sql*net V2.0
* Setup 방법 : Server 와 Client 에 있는
$ORACLE_HOME/network/admin/sqlnet.ora 에

sqlnet.expire_time=n

으로 setting 한다.
여기서 n 은 분단위이며 실제 disconnect 는 n 분보다 더걸린다.

SQL*NET과 관련된 기본적인 점검사항

Bulletin no : 10231

SQL*NET을 사용하면서 문제가 발생한다면 기본적으로 다음과 같은 사항들을 점검해 보아야 합니다.

==========
Client PC에서 점검할 사항(Windows 3.1의 경우)
==========

1)TCP/IP 체에서 제공하는 ping test가 제대로 되는지를 확인합니다.

2)hosts 화일에 사용하는 alias가 제대로 등록이 되어 있는지 확인합니다.

3)services 화일에 다음과 같은 항목이 등록이 되어 있는지 확인합니다.

```
listener        1521/tcp
orasrv          1525/tcp
```

port 번호는 작은 것부터 큰 순서로 정렬하도록 합니다. 경우에 따라서는 listener와 orasrv 를 services 화일의 맨 위에 등록을 한 다음 테스트를 해 볼 필요도 있습니다.

4)다른 프로그램보다 우선 nettest나 sqlplus 이 하여 connect test를 해봅니다.

5)ODBC driver를 사용하는 경우라면 오라클에서 제공하는 odbc test 프로그램을 이용하여 test를 해 봅니다. 그리고 autoexec.bat 화일에 다음과 같은 부분을 추가합니다(ORA-3121 에러의 경우)

```
SET CONFIG=C:\WINDWOS\ORACLE.INI
```

6)WINSOCK을 사용하는 경우라면 oracle.ini 화일에

```
TCP_VENDOR=WINSOCK
```

으로 등록이 되어 있는지 확인합니다.

7)host를 지정할 때 alias를 쓰지 말고 직접 ip adress 를 사용해서 접속해 봅니다. V1을 쓸 경우에는 connect string에서 alias를 ip address로 대체해서 테스트 하면 되고 V2의 경우는 tnsnames.ora 화일에서 alias로 정의된 부분을 ip address로 고쳐서 테스트 하면 됩니다.

8)winsock.dll 화일이 여러 디렉토리에 걸쳐 있어서 충돌하는 경우 문제가 발생합니다. 사용하는 TCP/IP에서 제공하는 winsock.dll 을 제외한 나머지 winsock.dll 화일들을 다른 이름으로 rename 하도록 합니다.

==========
SERVER 서 점검할 사항(UNIX의 경우)
==========

1)orasrv(V1)와 listener(V2)가 START 되어 있는지 확인합니다.

```
$tcpctl status  (V1)
$lsnrctl status (V2)
```

만약 start가 안되어 있다면 start를 시킵니다.

```
$tcpctl start   (V1)
$lsnrctl start  (V2)
```

2) /etc/services 화일에 다음과 같은 항목이 등록이 되어 있는지 확인합니다.

```
listener          1521/tcp
orasrv            1525/tcp
```

port 번호는 작은 것부터 큰 순서로 정렬하도록 합니다. 경우에 따라서는 listener와 orasrv 를 services 화일의 맨 위에 등록을 한 다음 테스트를 해 볼 필요도 있습니다.

3)orasrv, tnslsnr 화일의 owner가 root, permission이 4751인지를 확인합니다.(tnslsnr 화일은 owner가 root가 아니고 oracle user이어도 상관없지만 orasrv는 반드시 root 소유이어야 함)

4)root유저로 들어가서 $ORACLE_HOME/orainst 디렉토리에 있는 root.sh 스크립트를 실행합니다. 이것을 실행하면 현재 환경상태를 확인하여 옳고 그른가를 물어보는데 틀린 부분이 없으면 YES를 선택합니다. 그 다음에 local bin directory를 물어보면 /etc 를 입력합니다. 이 스크립트를 통하여 오라클 관련 환경을 정확하게 세팅하게 됩니다.

5)/etc/oratab 또는 /var/opt/oracle/oratab 화일이 있는지를 확인하고 다음과 같은 내용이 들어있는지 점검합니다.

```
ORA7:/home/oracle:N
```

여기서 ORA7은 ORACLE SID, /home/oracle 은 ORACLE HOME DIRECTORY, N은 system booting시에 ORACLE을 자동으로 STARTUP시키지 않는다는 것을 의미합니다. 이와같은 라인이 꼭 들어 있어야 합니다.

6)SERVER 문제인 CLIENT 문 인지를 알기 위해서 SERVER 상에서 LOOPBACK TEST를 해봅니다.

```
$sqlplus scott/tiger@t:krhp2:ORA7 (V1)
$sqlplus scott/tiger@TORA         (V2)
```

여기서 TORA는 $ORACLE_HOME/network/admin/tnsnames.ora 화일에 등록이 되어 있 service name 입니다.

7)UNIX SYSTEM을 rebooting 해봅니다.

8)executable file을 relink 해 봅니다.(V2)

```
cd $ORACLE_HOME/network/lib
make -f network.mk tnslsnr
cd $ORACLE_HOME/bin
mv tnslsnr tnslsnr.org
mv $ORACLE_HOME/network/lib/tnslsnr tnslsnr
chmod 4751 tnslsnr
```

이렇게 하고 listener를 죽였다가 다시 띄워 봅니다.

9)oracle kernel에 SQL*NET V2가 제대로 LINK되었는지 확인합니다.(V2)

```
cd $ORACLE_HOME/bin
adapters oracle
```

여기서 아무것도 출력되지 않으면 link가 안된 것입니다. 이럴 경우에는 ORACLE을 완전히 SHUTDOWN 하고 다음과 같이 합니다.

```
cd $ORACLE_HOME/rdbms/lib
make -f oracle.mk oracle
cd $ORACLE_HOME/bin
mv oracle oracle.org
mv $ORACLE_HOME/rdbms/lib/oracle oracle
chmod 4751 oracle
```

SQL*NET V1 FOR APPC/LU6.2 TEST

Bulletin no : 10064

Olivetti system에서 SQL*Net V1 for APPC/LU6.2 test 결과입니다.
customer : 한국컴퓨터

Test System : Olivetti UNIX System 2대
O / S version : SVR4 2.4

1. Server Install
   1.1 Shared Memory Parameter

       SHMMAX8388608
       SHMSEG6
       SHMMIN1
       SHMMNI100
   1.2 Kernel Parameter
       SVMMLIM0x7FFFFFFF
       HVMMLIM0x7FFFFFFF
       SFSZLIM0x7FFFFFFF
       HFSZLIM0x7FFFFFFF
       파라메터가 이보다 적으면 oracle kernel을 만들면서 에러가 발생함.
   1.3 Oracle Server Install 시 relink option 에서 Yes 선택.
       LU6.2를 사용하기 위해서 반드시 해주어야 함.
   1.4 Version이 2.3이하인 경우 kernel 생성시 에러 발생.

2. SQL*Net Install
   2.1 Transaction Program 을 Generate.
   $ $ORACLE_HOME/bin/gentpn ORACLE_SID Max_RU_size Min_RU_size

       RU_size는 256 에서 1024 범위내에서 set 해야함.

       ex) $ gentpn ORA7 512 512

         generate 후 TPORA7 이라는 Transaction Program 이 generate 되었다는 message가 떨어짐
         TP name은 ORACLE_SID 앞에 TP가 붙어서 생성.
       generate후 $ORACLE_HOME/lu62/server directory 에 TPORA7이라는 binary file이 생성.
   2.2 TPORA7을 APPC Services에 등록.
       sysadm tool을 이용하여 등록.
       TP_Name : TPORA7
       TP_Filespec : /home/oracle7/lu62/server/TPORA7

3. Connection.

3.1 Connect String

- @L:remote_LU_name::local_LU_name:mode_name:TP_name

remote_LU_name, local_LU_name, mode_name 의 value는 APPC
service에 등록이 되어있슴.

ex) $ sqlplus scott/tiger@L:BBBBBBBB::AAAAAAAA:CCCCCCCC:TPORA7

4. 특이사항.

-. gentpn시 Max_RU_size와 Min_RU_size 256으로 set:

sqlplus에서 접속시 SQL> prompt가 떨어지지 않고 waiting 상태.
반면 상대방 remote system에 session 은 연결됨.

-. gentpn시 Max_RU_size와 Min_RU_size 512으로 set:

sqlplus에서 접속은 됨.
그러나 512를 넘어서는 데이타일경우에는 ORA-6412 에러가 떨어지면서
disconnect 가 되어버림. (ORA-6412:bad read length)
remote에서 select 된 데이타는 local system 까지는 가져옴.
data size 는 548.
sqlplus 에서 array size를 '1'로 하면 정상적으로 작동.

```
ex) 1). select empno, ename from emp;          ----> 정상
    2). select * from emp;                     ----> disconnect
    3). select empno, ename, hiredate, sal,
               mgr, comm, deptno
           from emp;                           ----> disconnect
    4). set array 1
        select * from emp;                     ----> 정상
```

-. gentpn시 Max_RU_size와 Min_RU_size 1024으로 set:

데이타 사이즈가 1024를 넘어설경우 동일한 현상발생.

SQL*NET V1, V2의 STATUS를 확인하는 방법

Bulletin no : 10117

---

* SQL*NET V 1의 적절한 사용 방법

ORASRV의 Permission, Owner 및 Group은 다음과 같이 설정이 되어 있어야 하며 파일의 Size는 Version에 따라 다를 수 있다.

```
-rwsr-xr-x   1 oracle   dba      7544832 Jul 21 20:35 oracle
-rwsr-xr-x   1 root     dba       303104 Jul  5 19:24 orasrv
```

만일 위와 같지 않다면 Unix 명령 프롬프트에서 다음을 수행하여 모드를 바꾸어 준다.

```
#chmod  4755  oracle
#chown  root  orasrv <= orasrv의 owner는 반드시 root로 되어야 합니다.
#chmod  4755  orasrv
```

1. SQL*NET V1을 용하면서 가장 일반적으로 orasrv를 시작 및 정지 하는 경우이다.

```
#tcpctl start ( stop )
```

2. orasrv의 현상태를 확인하는 방법

```
#tcpctl status
 tcputl: Status summary follows
 Server is running          :
   Started                  : 4-OCT-95 15:58:48
   Total connections        : 0
   Total rejections         : 0
   Active subprocesses      : 0
   ORACLE SIDs              : RC,ORA722
   Default SID              : (null)
 Logging mode is ENABLED.
 DBA logins are DISABLED.
 OPS$ logins are ENABLED.
 OPS$ROOT logins are DISABLED.
 Orasrv is detached from the terminal.
 Break mode = OUT OF BAND.
 Debug level = 1
 Timeout (on orasrv handshaking) = 5 seconds.
 Length of listen queue = 10
 Orasrv logfile = /users2/oracle7/tcp/log/orasrv.log
 Orasrv mapfile = /etc/oratab
```

* SQL*NET V2의 적절한 사용 방법

SQL*NET V2를 사용하는데 있어서 갖추야 할 파라미터 파일이 있다.
기본으로 listener.ora,tnsnames.ora,sqlnet.ora가 $ORACLE_HOME/network/admin directory내에 있어야 합니다.(상기 파일에 대한 configuration은 SQL*NET Administrator's Guide v 2.0을 참고하시기 바랍니다.)

1. SQL*NET V2을 사용하면서 가장 일반적으로 listener를 시작 및 정 하는 경우이다.

```
        #lsnrctl start (stop)
```

2. Oracle과 Listener의 Process를 확인하는 방법

```
 $ ps -ef ¦grep ORA7
   ora7  1343   1  7   Aug 22 ?   30:53 ora_dbwr_ORA7
   ora7  1342   1  7   Aug 22 ?    0:32 ora_pmon_ORA7
   ora7  1346   1  7   Aug 22 ?    0:00 ora_reco_ORA7
   ora7  1347   1  7   Aug 22 ?    0:00 ora_s000_ORA7
   ora7  1345   1  7   Aug 22 ?   12:25 ora_smon_ORA7
   ora7  1344   1  7   Aug 22 ?   58:57 ora_lgwr_ORA7
   ora7  1348   1  7   Aug 22 ?    0:00 ora_s001_ORA7
   ora7  8036   1  7   Aug 22 ?    0:00 /ora/ora7/bin/tnslsnr LISTENER -
       inherit
   ora7 24279   1342  7 09:40:16 ? 0:05 ora_d001_ORA7
   ora7   899   1342  7 14:54:36 ? 0:21 ora_d000_ORA7
```

3. Listener의 현상태를 확인하는 방법

```
       $ lsnrctl service
 LSNRCTL for 88open UNIX: Version 2.0.14.0.0 - Production on 24-AUG-94
 09:19:37
Copyright (c) Oracle Corporation 1993.  All rights reserved.
Connecting to (ADDRESS=(PROTOCOL=IPC)(KEY=ORA7))
Services Summary...
    ORA7   has 3 service handlers
    DEDICATED SERVER established:0 refused:0
    DISPATCHER established:0 refused:0 current:0 max:60 state:ready
      D000 (machine: AViiON, pid: 899)
      (ADDRESS=(PROTOCOL=tcp)(DEV=6)(HOST=152.68.1.11)(PORT=2753))
    DISPATCHER established:0 refused:0 current:0 max:60 state:ready
      D002 (machine: AViiON, pid: 1357)
      (ADDRESS=(PROTOCOL=ipc)(DEV=5)(KEY=#1357.1))
The command completed successfully
```

4. SQL*NET V2를 MTS와 DEDICATE방식으로 사용코자 할때.

```
    $cd  $ORACLE_HOME/network/admin/tnsnames.ora

MTS =
  (DESCRIPTION =
    (ADDRESS_LIST =
        (ADDRESS =
          (PROTOCOL = TCP)
          (HOST = Hostname)
          (PORT = 1521)
        )
    )
    (CONNECT_DATA =
      (SID = ORA7)
```

```
    )
  )

MTS_DED =
  (DESCRIPTION =
    (ADDRESS_LIST =
        (ADDRESS =
          (PROTOCOL = TCP)
          (HOST = Hostname)
          (PORT = 1521)
        )
    )
    (CONNECT_DATA =
      (SID = ORA7)
      (SRVR = DEDICATED)
    )
  )
```

5. SQL*NET V2 Pipe Adapter를 사용중이라면 ps command를 통해서 oracleSID (LOACL=YES)라는 Shadow Process로써 확인 할 수 있다.

SQL*NET V1, V2의 차이점

Bulletin no : 10116

---

* 다음의 내용은 SQL*NET에 대해 사용자들이 가장 빈번히 문의한 내용을 간추린 것입니다.

1. SQL*NET V2와 SQL*NET V1은 서로 호환성이 있습니까 ?

- 아니오. SQL*NET V2과 SQL*NET V1은 Client 또는 Server사이에 주고 받는 Data Packets에 있어 서로 호환성이 없 니다. 서로 다른 SQL*NET Version으로 네트워크를 통하여 SQL*NET V1 을 사용하는 Client 또는 Server는 SQL*NET V2를 이용하는 Client 또는 Server와 통신할 수 없습니다.

2. SQL*NET V2와 SQL*NET V1을 같이 사용할 수 있습니까 ?

- 사용자는 Client 또는 Server상에서 SQL*NET V2와 SQL*NET V1을 같이 사용할 수 있는 데 이는 이 들이 동시에 공존할 수 있기 때문이다. 즉, Server에는 SQL*NET V2와 SQL*NET V1을 동시에 지원하며 두 가지 Version으로 접속을 할 수 있습니다. 다시 말하자면 Client에서 SQL*NET V1을 이용할 경우 scott/tiger@t:hostname:SID로 SQL*NET V2일 경우에는 scott/tiger@payroll으로 접속하며 여기서 payroll은 V2 접속기술자 입니다.

3. 통신 가능한 SQL*NET Version과 ORACLE Server Version에 대한 소개

. RDBMS V6.0	- SQL*NET V1 only
. ORACLE 7 Server 7.0.15	- SQL*NET V1
	- SQL*NET V2.0.14
. ORACLE 7 Server 7.0.16	- SQL*NET V1
	- SQL*NET V2.0.15
. ORACLE 7 Server 7.1.3	- SQL*NET V1
	- SQL*NET V2.1.3
. ORACLE 7 Server 7.1.4	- SQL*NET V1
	- SQL*NET V2.1.4
. ORACLE 7 Server 7.2	- SQL*NET V1
	- SQL*NET V2.2

4. SQL*NET V1에 없는 SQL*NET V2.0 만의 새로운 기능은 ?

. Comprehensive traceing and logging facilities of clients, servers and listeners
. Better error reporting and more granular debugging capabilities.
. Supprt for Oracle MultiProtocol Interchange
. Alternate Path Routing and network fault tolerance
. Improved generic architecture and improved performance (varies per protocol)
. Full support of the Oracle7 Multi-Threaded Server
. Consistent addressing schemes
. Oracle Forms-based network configuration tool any many more features

5. SQL*NET V2.1의 새로운 기능

. Oracle Network Manager, the graphical enterprise configuration tool

. Oracle Names, the global naming and directionary service for Oracle

. Dead Connection Detection, for detection of defunct connections

. Pre-started Server Process, for improved connection startup time.

. Support for Oracle Secure Network Services, the datastream encryption and data integrity product (a separate product, licensable separately)

. Encrypted passwords for SQL*NET utility

. More protocol support in SQL*NET and the MultiProtocol Interchange

SQL*NET V1 사용시 ORA-6107 조치 방법

Bulletin no : 10118

---

* Windows용 SQL*NET TCP/IP Ver 1.X는 SQLTCP.DLL과 SQLTCP1.DLL 두 개의 DLL이 하나로 구성되어 있다. 이들 DLL은 ORACLE용 연결 스트링이 TCP/IP 프로토콜 스트링으로 확인되면 작업진행중에 OCI DLL에 의해 올려진다. ORACLE Interface DLL은 SQLTCP.DLL을 먼 올리려고 한다. 이것이 실패하면 DOS TSR 버전의 드라이버를 찾는다. 두 가지 모두 실패하면 ORA-3121 메세지가 나온다.

1. SQL*NET TCP/IP DLL이 올려지고 초기화가 되는 즉시 Logon 시도를 시작한다.

2. 이때 모든 것이 정상적이라면 logon시 제공된 연결 스 링내의 I.P Address대신 Hostname을 사용하여 해결해 본다. 이러한 과정이 정상적이라면 클라이언트측의 통신관련 디렉토리에 있는 Services 화일에 Oracle Service's Portnumber를 제대로 부여한 것이며 SQL*NET TCP/IP DLL이 ORACLE.INI (또는 WIN.INI의 [Oracle] session의 ORA_CONFIG=\ORACLE.INI에 기술된 내용) 내에 기술된 TCP_SERVICES_FILE = \services 을 사용한다는 것이다.
ORACLE Service를 받는 Part Number SERVICES files내에 "orasrv" 라고 기술된다. SERVICES File의 예를 보면 하단과 같다.

```
# This list has been derived from RFC1060
#
# TCP Ports
# Service Name Port/Protocol Aliases

echo          7/tcp
discard       9/tcp    sink null
systat       11/tcp    users
listener   1521/tcp    oracle                  SQL*NET V2 인 경우
orasrv     1525/tcp    oracle                  SQL*NET V1 인 경우
```

3. 만일 TCP_SERVICES_FILE에 SERVICES화일이 구성되어있지 않았다면 "orasrv"의 접속이 실패된다. 만일 SERVICES File Name이 제대로 구성되어 있다면 file내에 표현되어 있는 "orasrv" 항목을 점검한다. SERVICES File 내의 각 항목 끝에 이 없을 경우나 "orasrv" line 상에 이 있을 경우에도 이러한 메세지를 수반한다.

* ORA-6107을 진단하는 방법

1. ORA-6107의 경우 ORACLE.INI내의 TCP_SERVICES_FILE이 무엇인지 (또는 WIN.INI내의 [ORACLE] Section에 ORA_CONFIG가 무엇인지)를 점검한다.

- ORACLE.INI의 경우
  TCP_SERVICES_FILE=\SERVICES
- WIN.INI의 경우
  ORA_CONFIG=c:\windows\oracle.ini
- 만약 설정이 되어 있지 않았다면 Client상에서 사용하고 있는 특정 TCP/IP Vendor에서 사용하고 있는 SERVICES 화일을 찾고 그 화일내에 Parameter를 설정한다.

2. 만일 TCP_SERVICES_FILE에 설정되어 있는 SERVICES 화일의 위치가 맞는다면 화일내에 "orasrv" 항목을 점검하여 본다. 화일내에 를 확인해 본다. 화일내의 각 항목 끝에 이 있는지 확인한다.

3. 만일 이 모든 것이 정상적이라면 새로운 SERVICES 화일내에 "orasrv" 항목을 추가한다.

4. 만약 SERVICES 파일의 도움없이 개발 응용프로그램에서 직접 접속을 시도할 경우에는 명시적으로 다음과 같이 Connect String에서 Port Number을 사용하여 접속할 수 있다.
  "t:/<port number>:<instance name>"
[주의] instance name = ORACLE SID

5. tcp_services_files=%drive1%\path%\services

[주의 1] ORACLE TCP/IP Listener의 기본 시작포트 번호는 십 수 1525 이다. SERVICES 파일은 대부분의 장비에 /etc 디렉토리안에 있으며 또한, "tcpctl" Utility를 사용하여 UNIX 장비상의 ORACLE orasrv 를 Start 시키고 Stop할 수 있다. Orasrv 를 정지하려면 "tcpctl stop"을 사용하고 다시 시작하려면 "tcpctl start" 사용한다.

BUG 1 : 다음과 같은 경우에도 에러가 발생할 수가 있다.
  orasrv 1525/tcp<CR>
위와 같이 기술되었을 경우 다음과 같이 하게되면 ora-6107 error 없이 접속이 가능하다. 즉, orasrv항목에서 <CR>전에 공란이 있을 경우에 에러없이 접속할수 있다.

  orasrv 1525/tcp <CR>

* 이 버그는 v1.1.7.10.1 이후 버젼에서는 수정되었다.

BUG 2 : 동일한 응용 프로그램에서 동일한 Connect String으로 접속을 시도하였을 때 에러가 발생한다면 다음과 같이 Connect String에 Port Number를 사용한다.

  t:<host name>/<port number>:<oracle_SID>

* 이 버그는 v1.1.7.9 이후 버젼에서는 수정되었다.

SQL*NET V2 SETUP 방법

Bulletin no : 10074

UNIX에서 SQL*NET V2를 사용하려면 ORACLE 인스톨 후에 다음과 같이 SETUP을 한 다음 사용하면 된다.
여기서

ORACLE HOME DIRECTORY는 /usr/oracle7
ORACLE SID는 ORA7

이라고 가정한다.

1. Multi-threaded server 프로세스들을 먼저 기동을 하여야 한다.
$ORACLE_HOME/dbs/initSID.ora 화일에 다음과 같은 내용을 추가하고 DB 를 다시 startup 을 한다

```
########################################
# Filename :  /usr/oracle7/dbs/initSID.ora
# Protocol :  tcp
########################################

mts_dispatchers="tcp,3"
mts_max_dispatchers=10
mts_servers=2
mts_max_servers=10
mts_service=ORA7
mts_listener_address="(ADDRESS=(PROTOCOL=tcp)(port=1521)(host=hostname)"
```

2. /etc/services file 에 다음내용을 추가한다. services 화일은 Machine에 따 디렉토리가 다를수 있다.

```
...
listener      1521/tcp
```

3. tnsnames.ora 및 listener.ora 를 작성 한다. 이 화일들은 /usr/oracle7/network/admin 디렉토리에 만들어야 하며 유닉스 환경변수를

```
TNS_ADMIN=$ORACLE_HOME/network/admin
```

와같이 지정하고 사용하는 것이 좋다.

```
#####################################################
# File name : /usr/oracle7/network/admin/listener.ora
# This configuration supports tcp
#####################################################

LISTENER =
  (ADDRESS_LIST =
        (ADDRESS =
          (PROTOCOL = tcp)
          (HOST = hostname)
          (PORT = 1521)
        )
  )

STARTUP_WAIT_TIME_LISTENER = 0
CONNECT_TIMEOUT_LISTENER = 0

SID_LIST_LISTENER =
  (SID_LIST =
    (SID_DESC =
      (SID_NAME = ORA7)
      (ORACLE_HOME=/usr/oracle7)
    )
)

#####################################################
#  Filename : /usr/oracle7/network/admin/tnsnames.ora
#####################################################

TORA =
  (DESCRIPTION =
    (ADDRESS_LIST =
        (ADDRESS =
          (PROTOCOL = tcp)
          (HOST = hostname)
          (PORT=1521)
        )
    )
    (CONNECT_DATA =
      (SID = ORA7)
    )
  )
```

4. listener 프로세스를 start 한다.

```
$lsnrctl start
```

5. SQL*NET v2 를 사용하여 MTS 에 접속한다.(LOOPBACK TEST)

```
$ sqlplus scott/tiger@TORA
```

위의 5가지 과정이 성공적으로 수행되면 UNIX SERVER 쪽에서의 SQL*NET V2 SETUP은 완료된 상태로 볼 수 있다. 만약 CLIENT에서 접속을 하는 경우라면 SERVER의 /usr/oracle7/network/admin 디렉토리에 있는 화일 전체를 CLIENT의 $ORACLE_HOME/network/admin 디렉토리로 COPY하면 CLIENT 쪽에서 SQL*NET V2를 사용하여 접속할 수 있다. 접속방법은 위의 LOOPBACK TEST 하는 경우와 같이 CONNECT STRING 부분에 tnsnames.ora 에 등록된 SERVICE NAME(위에서는 TORA)를 사용하여 접속하면 된다.

*SERVER가 WINDOWS NT인 경우에는 다른 부분은 UNIX와 동일하게 셋업하면 되고 단지 listener를 띄우는 방법만 다르다. 제어판 -> SERVICES 에서 ORACLE 관련 SERVICE 들을 모두 START 시켜 주면 된다.

SQL*NET V2 사용시 ORA-12154 조치 방법

Bulletin no : 10129

---

* ORA-12154의 원인은 tnsnames.ora 파일의 Alias처럼 정의된 Connect String 으로 사용하는 db_alias를 찾지 못할 경우 발생한다.

다음의 사항을 점검한다.

* WINDOWS, OS/2 CLIENT PART

1. Login Box를 통해 접속을 할 경우에는 Connect String의 Field에서 @ 호로 시작하지 않는다. @ 기호는 패스워드뒤에 공란없이 바로 Connect String(예:username/password@db_alias)을 지정하는 경우에 사용한다. Connect String을 입력하는 Field에서 아래와 같이 기술한다.

```
tns:db_alias
```

또는

```
db_alias
```

2. TNSNAMES.ORA 파일은 반드시 Client PC의 ORACLE_HOME\network\admin 디렉토리에 있어야 하며, 사용자가 지정한 Connect string은 TNSNAMES.ORA파일에 정의된 db_alis명이 기술되어 있어야 한다.

3. TNSNAMES.ORA 파일의 괄호(괄호쌍)를 제대로 열고 닫았는지 확인한다.

4. 만일 사용자가 TCP/IP를 사용하고 있다면 TNSNAMES.ORA 파일에 기술되어 있는 HOST=HOSTNAME을 HOST=IP ADDRESS로 변경한다.

5. SQLNET.ORA 파일을 점검하여 'names.default_zone=' 파라메타를 확인한다. 만일 SQLNET.ORA 파일에 이 파라메타가 존재하면 '#'으로 막는다.

```
#names.default_zone=
```

6. 하나의 단일 Community에서 이용한다면 TNSANMES.ORA에 기술되어 있는 'COMMUNITY' 파라메타를 '#' 으로 막는다.

```
=
 (DESCRIPTION =
   (ADDRESS_LIST =
   (ADDRESS =
#         (COMMUNITY = SAMPLE_COMMUNITY)
          (PROTOCOL = TCP)
          (HOST = )
          (PORT = 1521)
        )
   )
   (CONNECT_DATA =
```

```
        (SID = <SID>)
      )
    )
```

[주의] 상기의 <...>에 기술된 내용의 db_alias는 사용자 정의 필드이고, Server는 현재 사용자가 사용중인 server의 host_alias name 또는 IP Address를, SID는 설치된 ORACLE Instance 명을 기술하여야 하며, db_alias 명은 SID와 동일해서는 안된다. 접속 기술자로 사용되는 db_alias의 경우 속시 대.소문자 구분을 한다.

7. SQLNET.ORA 파일을 점검하여 본다.
만일 파일내에 names.default_domain=<string> 파라메타가 있다면 TNSNAMES.ORA의 db_alias 이름의 확장자로서 .<string>을 가지고 있어야 한다. 그렇지 않으면 SQL*NET V2 접속을 할때 ora-12154 에러가 발생한다. 만일 TNSNAMES.ORA 파일에 db_alias명에 .<string>을 포함하고 있고 사용자가 확장자 없이 db_alias로 접속을 시도시에 ora-12154 에러가 발생하였다면 SQLNET.ORA 파일에 names.default_domain=<string> 을 추가해야만 한다.

< 예제 : SQLNET.ORA >

```
TRACE_LEVEL_CLIENT = OFF
sqlnet.expire_time = 1
names.default_domain = world
name.default_zone = world
```

< 예제 : TNSNAMES.ORA >

```
NT_SRV.world =
  (DESCRIPTION =
    (ADDRESS_LIST =
        (ADDRESS =
          (COMMUNITY = NTCommun.world)
          (PROTOCOL = TCP)
          (Host = 152.68.31.100)
          (Port = 1521)
        )
    )
    (CONNECT_DATA = (SID = ORCL)
    )
  )
```

< 예제 : LISTENER.ORA >

```
LISTENER =
  (ADDRESS_LIST =
        (ADDRESS=
          (PROTOCOL=IPC)
          (KEY= oracle.world)
```

```
        )
        (ADDRESS=
          (PROTOCOL=IPC)
          (KEY= ORCL)
        )
        (ADDRESS =
          (COMMUNITY = NTCommun.world)
          (PROTOCOL = TCP)
          (Host = 152.68.31.100)
          (Port = 1521)
        )
  )

STARTUP_WAIT_TIME_LISTENER = 0
CONNECT_TIMEOUT_LISTENER = 10
TRACE_LEVEL_LISTENER = ADMIN
SID_LIST_LISTENER =
  (SID_LIST =
    (SID_DESC =
      (SID_NAME = ORCL)
    )
  )
```

* UNIX CLIENT PART

1. 상기의 WINDOWS, OS/2 Client의 1, 2 번 참조

2. 환경 변 로 TNSNAMES.ORA나 LISTENER.ORA 파일이 위치하고 있는 디렉토리가 TNS_ADMIN에서 제대로 지정하고 있나 확인한다.

Cshell을 사용한다면 : % env ¦ grep TNS_ADMIN

정확한 디레토리를 지정하고 있지 않으면 다음과 같이 TNS_ADMIN 환경변수에 정확한 디렉토리 설정한다.

% setenv TNS_ADMIN <directory path>

3. 사용자의 Configuration 파일들에 혹시 CTRL-M의 문자가 라인의 끝에 삽입되어 있나 확인한다.

4. 사용자의 Connect String이 정확하다면 TNSNAMES.ORA의 db_alias 이름을 SID명과 동 하게 주었는지 확인한다. SID와 db_alias는 다르게 기술해야 하며, 혹 Host명과 동일하게 주는 경우도 있는데, 이렇게 기술하면 Error를 발생시킬 수도 있다.

Sample V2 Connect String :
sqlplus <user>/<passwd>@<tnsnames.ora에 있는 db_alias>

Example : sqlplus scott/tiger@V715tcp

TNSNAMES.ORA 파일의 예제 :

```
V715tcp =
   (DESCRIPTION =
     (ADDRESS_LIST =
         (ADDRESS =
           (protocol = TCP)
           (host = korea)
           (port = 2000)
         )
     )
     (CONNECT_DATA =
       (SID = V715)
     )
   )
```

5. TNSNAMES.ORA File이 여러곳에 있는지 확인한다.
   TNSNAMES.ORA 파일은 3곳중 한곳에 있을 수 있다.

* SQL*NET V2의 Search Path Priority.

여기서는 SQL*NET V2 구성파일(LISTENER.ORA와 TNSNAMES.ORA)을 찾는 순위에 대해 기술한다. ORACLE에서는 구성파일을 다음과 같은 순서로 찾는다.

1. 만일 TNS_ADMIN 환경변수가 설정되어 있고 또한 디렉토리를 지정하고 있으면 확인한다.
2. /etc 디렉토리를 확인한다.
3. $ORACLE_HOME/network/admin을 확인한다.

SQL*NET V2 에서 BUFFER SIZE 지정 방법

Bulletin no : 10176

SQL*NET V2와 ODBC Driver를 사용하는 경우 Power Builder, Visual Basic,SQL Windows, Object View, Excel, Access 등과 같은 프로그램을 사용하면서 대량의 데이타를 처리하는 경우 세션이 끊어지거나 하는 문제가 발생을 하면 SQL*NET V2의 Buffer Size를 줄여 줌으로써 문제를 해결할 수 있습니다.

PC의 tnsnames.ora 화일(보통 c:\orawin\network\admin 디렉토리에 있음) 에 SDU 를 적용하여 다음과 같이 수정하면 됩니다. 만약 SDU를 적용하여도 문제가 해결이 안된다면 DEDICATED 방식으로 접속을 하여서 사용하기 바랍니다.

```
=======================
SDU (Session Data Unit)
=======================
      Syntax :  SDU=n (Byte)
      Range for 'n': 512 <= n <= 2048
      Default value = 2048 Bytes.

=========
바꾸기 전
=========
     TORA =
          (DESCRIPTION=
              (ADDRESS=
                 (PROTOCOL=TCP)
                 (PORT=1521)
                 (HOST=krhp2)
              )
              (CONNECT_DATA=(SID=RC))
          )

=======
바꾼 후
=======
     TORA =
          (DESCRIPTION=
              (ADDRESS=
                 (PROTOCOL=TCP)
                 (PORT=1521)
                 (HOST=krhp2)
              )
              (CONNECT_DATA=(SID=RC))
              (SDU=1024)
          )
```

SQL*NET V2에서 DEDICATED로 접속하는 방법

Bulletin no : 10230

SQL*NET V2를 사용하면서 MTS(Multi Threaded Server) 방식이 아닌 DEDICATED 방식으로 서버에 접속하려면 Client PC의 tnsnames.ora 화일을 다음과 같이 수정하면 됩니다 (tnsnames.ora 화일은 보통 c:\orawin\network\admin 디렉토리에 있음). 여기서 ORA_M을 사용하여 접속하면 MTS로 접속되고 ORA_D로 접속하면 DEDICATED로 접속됩니다.

Client가 PC가 아닌 경우에도 동일한 방법으로 수정하면 됩니다.

```
====================
MTS로 접속하는 경우
====================

ORA_M =
  (DESCRIPTION =
        (ADDRESS =
          (PROTOCOL = tcp)
          (HOST = krhp2)
          (PORT=1521))
    (CONNECT_DATA =
      (SID = ORA7)))

=========================
DEDICATED로 접속하는 경우
=========================

ORA_D =
  (DESCRIPTION =
        (ADDRESS =
          (PROTOCOL = tcp)
          (HOST = krhp2)
          (PORT=1521))
    (CONNECT_DATA =
      (SID = ORA7)(SRVR=DEDICATED)))
```

WINDOWS PERFORMANCE TUNING

Bulletin no : 10119

WINDOWS 386 확장모드에서 설치후 PERFORMANCE 향상을 최대화하기 위한 방안을 다음과 같이 제시한다. (WINDOWS는 386확장모드에서 가상메모리를사용할 수 있다.)
이러한 제시 사항들은 표준 모드와 실제모드에 적용할 수 있으며 잘 작동한다.
물론 표준모드에서 WINDOWS의 수행은 386 확장모드 보다는 빠르나 대부분의 많은 APPLICATION은 386 확장모드를 요구하고 있으며 SQL*NET 또한 이에 포함되어 있다.

1. USE SMARTDRIVE

* MICROSOFT SMARTDrive 디스크 캐쉬 드라이버는 WINDOWS 3.1 PERFORMANCE 향상을 얻을 수 있는 가장 큰 요소이다. SMARTDrive는 언제든지 사용이 가능하며 사용자는 CONFIG.SYS에 SMARTDRV.SYS를 로드하거나 AUTOEXEC.BAT에 SMARTDRV.EXE를 로드하여 SMARTDrive를 사용할 수 있다. 이 drivers를 사용시 적용할 수 있는 Option에는 여러가지가 있다. 사용자의 PC에 LOCAL Oracle RDBMS를 사용중이라면 Write caching을 OFF로 한다. (SMARTDrive의 설치및 운영에 대한 기본적인 정보는 "MICROSOFT WINDOWS USER'S GUIDE" VERSION 3.1 메뉴얼이나 DOS prompt에서 SMARTDRV /?를 이용하여 참조.)

2. KEEP YOUR HARD DISK OPTIMIZED.

* WINDOWS 수행에 있어 가장 크게 영향을 주는 것은 하드디스크의 단편화로서 특히 임시 SWAP 화일과 또는 SMARTDrive가 설치 된후 파일이 여러부분으로 나뉘고 각기 다른 위치에 기록될 때 발생한다. 하드 디스크 최적화 프로그램을 주단위로 수행하여 디스크의 단 화를 줄이는 것이 좋다.

3. CREATE A PERMANENT SWAP FILE

* WINDOWS 3.1에서 사용자의 하드웨어가 사용중인 영구 SWAP 화일을 임시 SWAP 화일로 사용하여 PERFORMANCE를 향상시키려면 기본프로그램 그룹의 제어판 내에 있는 386확장 아이콘을 선택하고, 가상메 리 버튼을 누르고 변경 버튼을 누른 후 32 비트 디스크 엑세스 체크박스를 선택한다.

4. TURN OFF GRAPHICS PORT TRAPPING

* 386 확장모드에서 수행하는 MS-DOS Application의 속도는 PIF 에디터의 고급옵션 버튼을 누르고 디스플레이 옵션의 모니터 포트에서 어떠한 것도 선택하지 않는 다면 눈에 띄게 PERFORMANCE가 향상될 수 있다. 고해상도 그래픽 옵션은 광범위하게 MS-DOS 응용프로그램에 대해 호환성을 제공하지만 대부분의 응용프로그램에서는 이를 요구하지 않는다

5. TURN OFF THE FILESYSCHANGE=SETTING

* WINDOWS 3.1 386 확장모드에서는 수정된 디렉토리의 메세지를 파일 관리자에게 보낼수 있고 MS-DOS Applications에 의해 접근하는 디스크를 감시할 수 있다. 이는 MS-Dos Applications에서 생성한 화일이나 디렉토리의 변경이 자동적으로 수정될 수 있도록 허용한다는 것을 의미 다. 이 기

능을 사용하지 않으려면 SYSTEM.INI 화일의 [386enh]부분에 FileSysChange =no로 설정한다.

6. TURN OFF THE RESERVEPAGEFRAME=SETTING

* 사용자가 MS-DOS 응용프로그램에서 확장메모리 지원을 요구하지 않는 다면 이것을 Off로 설정한다. 이 설정을 Off로 경하면 가상 MS-DOS 기계상에서 사용할 수 있는대부분의 메모리를 활용할 수 있다. 이 기능을 사용하지 않으려면 SYSTEM.INI 화일의 [386enh]부분에 ReservePage Frame=no로 설정한다.

7. USE THE RIGHT NUMBER OF MS-DOS BUFFERS.

* SMARTDrive를 사용중이라면 CONFIG.SYS 에 있는 MS-DOS disk에 접근할 수 있는 BEFFERS의 수를 15로 설정한다. (즉, BUFFERS=15). SMARTDrive와 함께 BUFFERS를 너무 크게 설정하여 사용하면 실질적으로 효율이 감소할 수도 있다. 만약 SMARTDrive를 사용하지 않으면 BUFFERS=30을 사용한다.

[주의] Oralce Client Tool들을 사용할 때 권고하는 config.sys 환경변수는 아래와 같다.

```
Buffers = 16
  Files = 60
```

WINDOWS용 SQL*NET V1 사용할 때 ORA-6136 조치방법

Bulletin no : 10127

Client and Server 환경에서 간혹 SQL*NET으로 Server에 접속하려고 할때, ORA-6136 Error가 발생하면서, Unix Server에서 $tcpctl stop 으로 orasrv의 Process를 정지시키려고 해도 아무런 반응없이 Holding되는 경우가 발생하면서 다음과 같은 메서지가 나타납니다.

```
06136, 00000, "NETTCP: error during connection handshake"
// *Cause:  Network I/O failure occurred while communicating with the
//          host's SQL*Net TCP/IP server.
//      See the SQL*Net TCP/IP server log file for more details.
// *Action: Contact your customer support representative.
```

상기의 에러가 나타날 경우에는 아래와 같은 방법으로 orasrv를 기동시키십시요.

1. TCPCTL Utility를 이용하여 다음의 Option을 부여하여 Start하는 방법.

```
   $tcpctl start listen=30 timeout=30 forkon
```

  listen=이며, 청취자 열의 크기를 지정.
  timeout=이며, 지정된 시간에 orasrv와의 응답 확인 시간을 나타냄.
  forkon SQL*net이 orasrv process에 접근하는 방법을 나타냄.
  System에 따라서 forkon option이 적용 않되는 경우도 있음.

2. Client에서 접속을 하여 사용 다가 비정상 종료(Session이 맺어진 상태에서 Client의 Power Off)를 하여 Server에 Processor가 남아 있고, orasrv를 통해 접속할 수 있는 Session의 수가 점점 줄어들 경우가 있는 데 이러한 경우에는 orasrv를(tcpctl stop or UNIX kill command를 이용)강제로 종료 시고 다음과 같이 Start 시켜 주십시오.

```
#nohup tcpctl start ( Client의 비정상 종료가 Orasrv에는 영향을 미치지 않음)
  또는
#orasrv (ORACLE_HOME\bin directory에서 직접 orasrv processor를 띄운다)
```

WINDOWS용 SQL*NET V1,V2 사용시 ORA-9241, ORA-9301 조치방법

Bulletin no : 10128

ORA-9241, 9301 에러는 개발툴이 해당 툴 또는 SQL*NET에 필요한 메세지 화일을 찾을 수 없을 때 발생한다.

* SQL*NET V 1

1. DOS 환경에서 SET을 수행하여 CLIENT상에 CONFIG라는 환경변수가 설정되어 있는지 확인한다.

2. 설정되어 있지 않으면 ROOT directory에서 CONFIG.ORA 또는 ORACLE.INI라는 화일을 찾는다. (Directory Search)
(상기의 화일은 보통 \ ORACLE_HOME 또는 \ WINDOWS Directory 내에 있다.)
상기의 화일이 존재하면 DOS환경에서 다음의 CONFIG 환경변수를 추가한다.

```
C:\> SET CONFIG = \ORACLE.INI
```

이로써 문제가 해결될 수 있으며, DOS 환경에서 SET과 PATH 명령을 수행하여 ORACLE_HOME 디렉토리가 PATH에 설정되어 있는지, 그리고 CONFIG가 DOS환경에 설정되어 있는지 확인하고 WINDOWS를 실행해야 한다.

3. 환경변수 CONFIG가 이미 설정되어 있고 ORACLE.INI 또는 CONFIG.ORA를 지시하고 있는데도 ORA-9241 또는 ORA-9301 에러가 계속 발생한다면 WINDOWS를 완전히 빠져나와서 ORACLE.INI를 CONFIG.ORA 라는 또 다른 화일로 복사를 하고 DOS 환경에서 환경변수 CONFIG를 추가한다.

```
C:\> SET CONFIG = \CONFIG.INI
```

4. 만일 ORACLE.INI 를 autoexec.bat 파일에 SET CONFIG = \ORACLE.INI 와 같이 추가하여도 계속 에러가 발생한다면 ORACLE.INI 파일에 LOCAL = t:hostname:sid 을 추가한다.

5. 단계2 또는 단계3 에서 문제가 해결된다면 이를 AUTOEXEC.BAT 화일에 SET CONFIG문을 추가한다.

* SQL*NET V 2

1. ORACLE.INI 파일에 LOCAL = 을 추가한다. 만일 ORACLE.INI 파일에 LOCAL 파라미터를 추가한 후에도 계속 ora-9301 에러가 계속 발생한다면 autoexec.bat 파일에 SET CONFIG = \ORACLE.INI를 추가한다.

[주의 1] CONFIG가 ORACLE.INI를 지시하도록 설정하면 나중에 다시 설치할 문제가 발생할 수 있다. 그럴 때는 AUTOEXEC.BAT 에서 SET CONFIG 행을 삭제하고 다시 Booting 한후 설치를 시작한다.
[주의 2] MS ACCESS를 이용하여 ORACLE의 데이타를 질의할 경우는 환경변수를 다음과 같이 설정한다.

```
SET CONFIG_FILES = \ORACLE.INI
```

[주의 3] SET 다음의 CONFIG 나 CONFIG_FILES 은 반드시 대문자 이어야 한다.

WINDOWS용 SQL*NET 사용시 ORA-6106

Bulletin no : 10133

다음은 먼저 WINDOWS용 SQL*NET TCP/IP와 ORA-6106을 간단하게 소개한 다음 ORA-6106을 진단하는 방법을 설명하겠습니다.

* WINDOWS용 SQL*NET TCP/IP

WIN V1.X용 SQL*NET TCP/IP는 현재로서는 SQLTCP.DLL과 SQLTCP1.DLL 등 두개의 DLL이 한 세트로 되어 있습니다. 이들 DLL은 ORACLE용 연결 스트링이 TCP/IP 프로토콜 스트링으로 변환되면 OCI DLL에 의해 작업 진행중에 올려집니다. ORACLE INTERFACE DLL은 SQLTCP.DLL을 먼저 올리려고 합니다. 이것이 실패하면 DOS TSR 버전의 드라이버를 찾습니다. 두 가지 모두 실패하면 ORA-3121 메세지가 나옵니다.

1. SQL*NET DLL은 올려지자마자 MSOCKLIB.DLL(ORACLE에서 제공됨)이 올려집니다.
이 DLL은 ORACLE.INI로부터 TCP_VENDOR 스트링으로 전달받습니다 ㅏ. 그 후, 이 MSOCKLIB.DLL은 TCP_VENDOR 스트링을 사용하여 항상 WINDOWS 디렉토리에 있는 VSL.INI라는 파일을 엽니다. VSL 파일은 JSB 호출을 각 업체 고유의 API에 대응시킵니다. VSL.INI가 없으면 MSOCKLIB.DLL이 초기화에 실패하여 ORA-6120 "NETTCP : network driver not loaded" 메세지가 나오게 됩니다. 이것은 고객이 다른 기계에서 SQL*NET DLL을 복사하거나 XCOPY 명령으로 ORAWIN 디렉토리 전체를 복사하기만 하고 이 VSL.INI 파일을 복사하지 않을 때 자주 발생합니다.

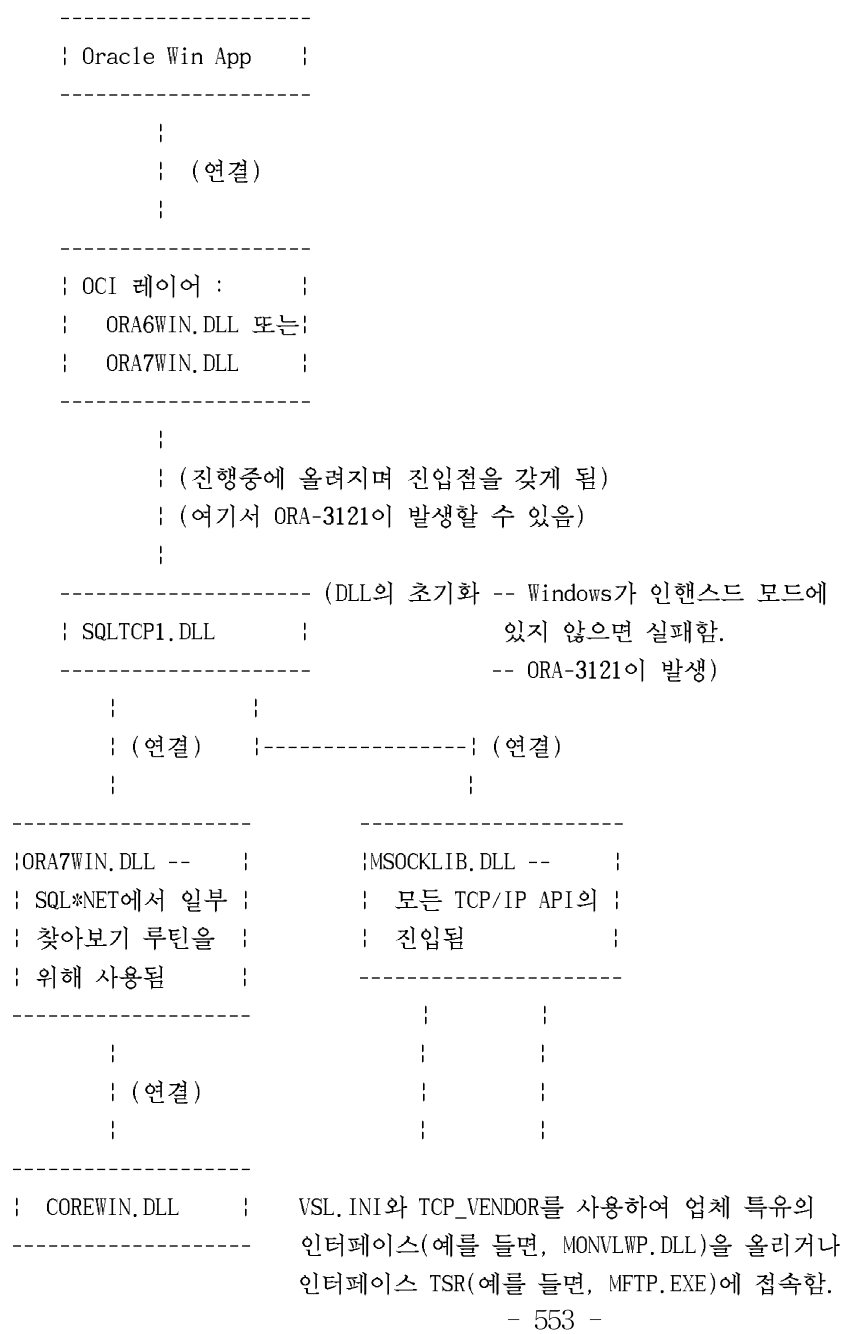
2. VSL.INI가 있다 하더라도 TCP_VENDOR 매개 변수에 ORA-6120 또는 ORA-6106 "NETTCP : socket creation failure"를 유발할 수 있는 오류가 적지 않습니다.

3. 이 문제의 다른 원인은 MSOCKLIB.DLL이 대응을 제대로 결정할 수 있지만 MSOCKLIB.DLL API를 해당 업체의 TRANSPORT 인터페이스 또는 API에 결합하도록 글루(glue)에 지시할 수 없는 것입니다. 이 글루는 SQL*NET TCP/IP를 설치할 때 선택하는 TCP/IP 업체에 따라 DLL 또는 TSR일 수 있습니다. 예를 들면, WINDOWS 로 들어가기 전에 글루 DLL이 실패할 수도 있고 (이를테면 MNOVLWP.DLL) 벤더 고유의 TSR(예를 들면 MFTP.EXE)이 올려지지 않았을 수도 있습니다. 이 글루를 구성하는 DLL과 TSR의 목록은 다음과 같습니다.

- m3open.dll : 3Com 3+Open v2.0
- mhparpa.dll : HP ARPA v2.0 & v2.1 그리고 Micorsoft TCP V1.0(LAN Manager v2.2의 번들)
- mnovlwp.dll : Novell LAN WorkPlace v4.0, v4.01, & v4.1
- mpcnfs4.dll : Sun PC-NFS v4.0a
- mpathsay.dll : Wollongong Runtime TCP v1.1 & v1.2
- mwinsock.dll : VSL 일반형 windows sockets DLL 버전 1.1
  - Frontier Technology Super - TCP v3.0
  - NetManage Chameleon TCP v3.1과 v3.22
  - FTP PCTCP v2.2 - Winsock Interface

(1993년 5월 4일자 WINSOCK.DLL)
. SUN PC/NFS 5.0 - Winsock Interface
(1993년 3월 2일자 WINSOCK.DLL)
. DOS용 IBM TCP v2.1.0.2
(1993년 3월 30일자 WINSOCK.DLL)

. m3open.exe : 3Com 3+Open v1.1 & v1.2 및 DOS용 Digital Pathworks(TCP/IP) v4.1
. mb2.exe : Beame 및 Whiteside TCP/IP v3.0
. mftp.exe : FTP PC/TCP v2.1 부터 v2.2까지
. mpcnfs.exe : Sun PC-NFS v3.0
. mpcnfs2.exe : Sun PC-NFS v3.5



여기서 ORA-3120 또는 ORA-6106이 발생할 수 있음.

4. 지원되는 TCP/IP 패키지 또는 버전을 설치하지 않은 것도 ORA-6106의 원인이 될 수 있습니다. 예를 들어서 TCP_VENDOR=PCNFS4를 가지고 있는 데 Sun PC-NFS v4.0를 실행하면, TCP_VENDOR=PCNFS4 를 실 할 때 반드시 사용해야 하는 버전인 Sun PC_NFS v4.0a를 올리자 않았기 때문에 ora-6106이 발생할 수 있습니다. 우리는 SUN PC-NFS v4.0을 지원하지 않습니다. 지원되는 버전과 지원되는 TCP/IP 제품을 사용중인지 확인하십시요. (3단계 참조)

5. 지원되는 TCP/IP패키지와 버전을 설치 했더라도 TCP/IP 패키지에 필요한 실행 파일을 모두 설치하지 않았으면 ORA-6106이 발생할 수 있습니다. 예를 들어, RTM과 RNMFILE(또는 NIS를 사용할 경우 RNMNIS)를 올리지 않았다면 PC-NFS를 사용할 때 ORA-6106이 발생할 수 있습니다. Microsoft TCP를 사용할 경우에는 반드시 SOCKETS.EXE를 올려야 합니다.

6. 또한 CONFIG.SYS의 FILES와 BUFFERS를 충분히 높게 설정하지 않으면 ORA-6106이 발생할 수 있습니다. 다음은 이들 매개변수의 최소 요구사항입니다.

```
BUFFERS = 16
FILES = 60
```

7. ORA-6106의 또 다른 인은 실제 메모리의부족입니다.

* ORA-6106을 진단하는 단계

1. TCP/IP 업체의 PING과 TELNET이 잘 작동하는 지 점검하십시요. 이들 유틸리티의 GUI 버전이 도움이 될수 있습니다. 이들 유틸리티가 작동하지 않으면 TCP/IP 설치 책자를 참조하거나 TCP/IP 업체에 연락 십시요.

2. 이들 유틸리티가 작동하지 않으면 WINDOWS 디렉토리에 VSL.INI 파일이 있는 지 점검 하십시요. 만약, 없으면 WINDOWS용 SQL*NET TCP/IP를 다시 설치하거나 다른 곳에서 복사해 오십시요.

3. WINDOWS 디렉토리에 VSL.INI가 있으면 TCP/IP 업체를 바르게 선택 하면서 TCP/IP를 설치 했는지 점검 하십시요.

4. 제대로 설치했으면 ORACLE.INI의 TCP_VENDOR 매개 변수를 읽어 보십시요. TCP_VENDOR의 값에 정확한 TCP/IP 패키지와 버전을 사용중인지 확인하십시요. 그런다음 VSL.INI로 가서 그 TCP_VENDOR의 프로파일을 살펴 보십시요. 프로파일의 "netmodule"은 무엇입니까? "netinterface"가 DLL이라면. 그 DLL을
찾을 수 있습니까? 필요할 때 WINDOWS가 DOS PATH를 통해 그 DLL을 올릴 수 있습니까? "netinterface"가 TSR이라면 WINDOWS로 들어가기 전에 반드시 그 TSR을 실제로 올리십시요.

5. CONFIG.SYS의 FILES와 BUFFERS를 충분히 높게 설정했는 지 확인 하십시요.
다음은 이들 두 매개변수의 최소 요구사항입니다.

```
BUFFERS = 16
FILES = 60
```

6. 앞서 말한 단계를 다 수행했는데도 여전히 ORA-6106이 발생하면 기본 메모리의 일부를 해제해 (불필요한 TSR을 내리거나 일부 TSR을 상위 메모리로 올려서) 보십시요.

7. Micorsoft TCP/IP를 사용 한다면 TCPUTILS.INI의 SCOKETS값을 점검하십시요. 값이 너무 낮으면 올리십시요. 또한 SOCKETS.EXE를 호출했는 지도 확인하십시요.

8. WINDOWS를 통해 SUN PC-NFS V5를 사용할 때 문제점이 생길 때가 있습니다. 그러한 경우에는 SQL*NET TCP/IP를 다시 설치하고, SUN PC-NFS V4.0a(V5가 아니라)를 업체로 선택해 보십시요. 이 때 RTM과 RNMFILE(또는 NIS를 사용할 경우 RNMNIS)을 반드시 올려야 합니다.

WINDOWS용 SQL*NET 사용시 ORA-6108

Bulletin no : 10135

다음은 먼저 Windows용 SQL*Net TCP/IP와 ORA-6108을 간단하게 소개하고 끝부분에서 ORA-6108을 진단하는 과정을 요약합니다.

* WINDOWS용 SQL*NET TCP/IP 개요

1. 연결을 시도하는 중에 클라이언트 응용프로그램이 소켓을 열고 Connect()를 사용하여 원하는 서버를 출합니다. Destination은 orasrv listener가 listerning하는 machine의 port와 destination address를 지정합니다. 이것의 디폴트는 십진 1525입니다.

```
----------------------
¦ 클라이언트         ¦   서버의 IP 주소와 목적지의 포트번호, 즉 서비스
¦ 응용 프로그램이    ¦   화일의 orasrv를 가지고 Connect()을 합니다.
¦ 소켓 #z를 엽니다.  ¦
----------------------
```

2. 서버 응용 프로그램은 대개 호스트(여기서는 BSD 소켓이라고 가정)에서 소켓을 연 다음, Bind() 호출을 통해 호스트 machine의 internet 주소와 수신 포트 번호를 소켓에 결합합니다. 이어 Accept()를 내서 수신 Connect() 요구를 받겠다는 의사를 표시합니다. 그 후, Listen() Operation을 수행하는 타이트 루프(tight loop)로 들어갑니다. 이 Listen()은 TCP 세션 진행을 위한 클라이언트의 Connect()와 일치합니다.

```
--------------------------  (a) 서버의 IP 주소 (또는 기계에서 이용할 수
¦ 서버 응용 프로그램,   ¦     있는 모든 IP주소) 와 수신포트번호, 즉
¦ 즉 orasrv listener가  ¦     기계의 /etc/services에서 가져온 orasrv를
¦ 소켓 #x를 엽니다.     ¦     가지고 Bind()를 냅니다.
--------------------------  (b) MAX_PENDING_CONNECTS_QUEUE_SIZE를 매개
                              변수로 하여 Accept()를 냅니다. 이 매개
                              변수는 tcpctl 프로세스에서 구성할 수 있는
                              경우도 있습니다. 서버에 대한 Connect()
                              요구가 쇄도할 때는 일부 Connect()요구를
                              큐에 넣어서 TCP/IP 레이어를 멈춥니다.
                            (c) 루프  들어감 : Listen()을 냅니다.
                              TCP/IP는 미결 Connect() 요구가 없으면
                              orasrv 프로세스를 휴면 상태로 만듭니다.
                              orasrv는 Connect()가 그 Listen()과 쌍을
                              이룰 때 작동을 재개합니다.
```

3. 서버는 Listen() 호출에서 작동을 재개할 때 복귀 코드로서 하나의 추가 소켓을 더 가져옵니다. 이어 두번째 소켓을 계승하는 ORACLE Shadow process를 만들며, 이때 Listener 자체는 원래 소켓상에

서 Listening 루프로 되돌아갑니다.

```
--------------------- (a) orasrv가 소켓 #x에서 Listen()을 내서 Listening
¦  ORASRV          ¦   합니다. 클라이언트 응용 프로그램의 Connect()
¦  Listener가 소켓  ¦ (b) orasrv는 Listen()으로부터 작동을 재개합니다.
¦  #x를 엽니다.     ¦   새 소켓 번호 #y를 가져오고 fork() 만듭니다.
---------------------   페어런트는 소켓 #y를 닫고 루프를 계속합니다.
                        차일드는 이제 소켓 #x를 닫고 나서 소켓 #y로부터
                        send()와 recv()를 실행하며, 그 후로는 페어런트
                        프로세스를 계승하게 됩니다. SID 참조가 일어나고
                        만약 성공하면 이 차일드 orasrv는 SID에 대한
                        ORACLE 이미지와 겹쳐집니다. 이렇게 해서
                        클라이언트의 연결을 위한 ORACLE 쉐도 프로세스가
                        이루어집니다.

--------------------  ORASRV 차일드 프로세스 -- 클라이언트에 대한
¦   ORACLE 쉐도     ¦     ORACLE 쉐도 프로세스가 됩니다.
¦   프로세스가      ¦    <- SID에 대한 ORACLE 이미지와 겹치기 전에
¦   만들어집니다.   ¦    차일드가 이 소켓을 닫습니다.
--------------------    <- 쉐도 프로세스는 이 소켓을 계승합니다.
                           recv()를 사용하여 클라이언트로 부터 받음.
                           send()를 사용하여 클라이언트로 보냄.
```

4. ORA-6108은 부적절한 machine, 또는 machine는 맞지만 틀린 포트를 지정할 때 발생합니다. 포트와 주소가 맞더라도 TCP Listener는 그냥 종료될 수 있습니다. 사용량이 많은 시스템에서는 Listener가 정상적으로 실행할 때에도 ORA-6108 에러가 발생할 수 있습니다. 이것은 TCP Listener가 Accept() 호출중에 해당 소켓에 대해 MAX_PENDING_CONNECT_REQS를 설정했을 수 있기 때문입니다. 결국 TCP/IP 레이어는 모든 연결 요구를 Listener의 소켓 큐에 넣을 수 없으므로 ORA-6108이 발생할 수 있습니다.

5. SQL*NET V1은 SQL*NET V1로만 송화한다는 점도 알 두어야 합니다. SQL*NET V2로는 송화할 수 없습니다.

6. 네트워크가 아주 혼잡하고 호스트에 도달하려는 중에 시간이 종료할 경우에도 ORA-6108이 나올 수 있습니다. SQL*NET가 기하기까지 연결을 재시도할 횟수를 알려주는 conn_retries라는 SQL*NET TCP/IP 연결 스트링에서 conn_retries를 지정하는 구문은 다음과 같습니다.

```
t:host[/service]:SID[,buffer_size][:conn_retires]
```

* ORA-6108을 진단하는 단계

1. 클라이언트에서 호스트 Machine에 대해 ping을 실행하십시요. 대부분의 PC TCP/IP업체는 "ping" 유틸리티를 제공합니다. 클라이언트 Machine에서 다음을 입력하십시요

```
ping
```

이 방법으로 잘 되지 않으면 아마도 호스트 machine이 down된 것입니다. IP 주소를 사용하여 호스트

에 대해 ping을 성공적으로 실행할 수 없으면, 서버의 호스트 이름을 사용하여 ping을 실행해 보십시요.

```
ping
```

호스트 이름을 사용하여 ping을 실할 수 없으면 TCP/IP 구성을 점검하십시요. 호스트 이름을 가지고 ping을 실행할 수 없으면, 연결 스트링에 호스트 이름을 사용하여 SQL*NET와 연결할 수 없습니다.

2, SQL*NET TCP/IP Listener가 해당 서버에서 실행중인지 점검하십시요. 서버의 UNIX프롬프트에서 다음을 입력하면 됩니다.

```
ps -al¦grep "orasrv"
```

이 때 최소한 한 행이 표시되어야 합니다. 그렇지 않으면 UNIX 프롬프트에서 "orasrv" 또는 "tcpctl start"를 입력하여 수화자를 띄우십시요. SYSADMIN 특권을 가지고 해당 기계에 로그인해야 합니다.

3. 서버쪽에서 루프백을 할 수 있는 지, 다시 말해서 PC 클라이언트에서 지정한 것과 같은 연결 스트링을 사용하여 서버의 툴을 연결할 수 있는 지 점검하십시요. 예를 들면, 서버의 SQLPLUS 또는 SQLDBA를 호출하고 서버의 SQLPLUS 또는 SQLDBA 프롬프트에서 다음을 입력하십시요.

```
CONNECT USERNAME/PASSWORD@t:/:
```

4. 루프백 성사되면 호스트 서버의 ORASRV 포트 번호를 확인하십시요.
(대부분의 기계에서 SERVICE 파일은 /etc 디렉토리에 있습니다.) 또한 "tcpctl" 유틸리티를 사용하면 대부분의 UNIX 기계에서 ORACLE Listener를 시작하거나 멈출 수 있습니다. "tcpctl stop"로 Listener를 종료하십시요. "tcpctl start" 으로 다시 시작하십시요. 이때 시작 포트에 관한 정보가 표시됩니다.

5. 이것이 성공하면 포트를 지정하지 말고 포트를 연결해 보십시요.

```
t::
```

연결되지 않으면 클라이언트에서 SERVICE 파일을 정확하게 설정하 않았기 때문입니다.

a) WINDOWS\WIN.INI를 점검하여 [Oracle] 부분의 ORA_CONFIG 매개 변수가 어떤 구성 파일을 지시하고 는지 알아보십시요. 이폴트는 다음과 같습니다.

```
[Oracle]
ORA_CONFIG=C:\WINDOWS\ORACLE.INI
```

b) ORACLE.INI 파일을 보고 TCP_SERVICES_FILE 매개변수가 설정되었고 SERVICES 파일을 지시하고 있는 지 확인하십시요.

c) SERVICES 파일을 보고 다음 항목이 있는 지 확인하십시요.

```
orasrv 1525/tcp oracle
```

7. 또한 서버가 SQL*NET V2가 아니라 SQL*NET V1을 실행중인지 확인하십시요.

8. 결 스트링의 재시도 매개변수를 증가시켜 보십시요. 재시도 횟수를 지정하는 구문은 다음과 같습니다.

t:host[/service]:SID[,buffer-size][:conn-retries]

conn-retries의 디폴트는 1입니다.

9. VAX에 연결할 경우에는 VAX config.ora 파일에 다음행이 있는지 확인하십시요.

SQLNET USERNAMEMAP*=*

이것은 VAX account가 없는 PC가 디폴트 사용자 account을 사용하여 연결 할 수 있게 해 줍니다.

WINDOWS용 SQL*NET 사용시 ORA-6110

Bulletin no : 10136

ORA-6110과 버퍼 크기의 개요

ORA-6110 "NETTCP: message send failure"는 대개 Windows 클라이언트의 TCP/IP사이에 버퍼 조정문제가 있을 때 발생합니다.

버퍼 크기를 이해하기 위해 SQL*NET을 통해 응용 프로그램과 원격 데이타베이스 사이의 데이타 흐름을 살펴보기로 합니다. 클라이언트쪽에서 응용 프로그램은 SQL*NET에 데이타를 전달하고, SQL*NET는 그 데이타를 기반 TCP/IP 계층에 전달하며, TCP/IP 소프트웨어는 그 데이타를 네트워크에 전달합니다. 서버쪽에서 TCP/IP 소프트웨어는 네트워크에서 데이타를 선정하여 SQL*NET에 전달하고, SQL*NET는 그 데이타를 데이타베이스에 전달합니다. 데이타베이스가 데이타를 다시 응용 프로그램에 보낼 때의 과정 이와 반대입니다.

양쪽에서 SQL*NET TCP/IP는 데이타를 다음 계층에 실제로 전달하기 전에 메모리에 저장해 놓습니다. 이와 같이 SQL*NET 버퍼 크기까지 데이타를 저장했다가 전달합니다.

클라이언트가 서버에 연결할 때 한 쪽의 SQL*NET 계층은 먼저 버퍼 크기를 조정해야 합니다. 양쪽의 기본 SQL*NET 버퍼 크기는 서로 다를 수 있으며, 어느 한 쪽이 다른 쪽에서 다룰 수 있는 SQL*NET보다 많은 데이타를 보내지 않도록 하기 위해 둘 중의 더 작은 크기를 사용합니다.

SQL*NET TCP/IP 가 버퍼를 사용하듯이 기반 TCP/IP 소프트웨어도 버퍼를 사용합니다. 이 버퍼 조정이 어떤 이유에서든 잘못 이루어지면 ORA-6110이 발생할 수 있습니다.

[해결책]
ORA-6110의 결책은 버퍼 크기를 연결 스트링에 포함시켜 일정한 크기로 고정하는 것입니다.

```
t::,
```

연결 스트링에 버퍼 크기를 포함시킨 후에도 여전히 ORA-6110이 발생하면 더 작은 값을 사용해 보십시요. WINDOWS용 SQL*NET TCP/IP의 기본 퍼 크기는 4096입니다. 이것을 1024로 사용하면 대개 ORA-6110에러가 없어집니다.

[서버쪽]

1. 서버에서 루프백을 수행할 수 있습니까? 다시 말해서 PC 클라이언트에서 지정한 것과 같은 연결 스트링을 사용하여 서버의 툴을 연결할 수 있습니까? 예를 들어 서버에서 SQLPLUS 또는 SQLDBA를 호출하고 에러가 없어집니다.

```
CONNECT USERNAME/PASSWORD@t::
```

루프백을 실행하면 실제 문제를 더 잘 보여주는 다른 에러가 나타나는 수도 있습니다.

2. ORA-6110은 Oracle실행 파일에 사용 권한이 정확하게 설 되어 있지 않으면 Unix 서버에 연결할 때도 발생할 수 있습니다. Oracle과 orasrv의 사용권한은 다음과 같이 설정되어야 합니다.

Oracle v6의 경우:

Unix 프롬프트에서 다음 두 행을 입력하십시요.

```
chmod 4755 oracle
chmod 4755 orasrv
```

이 때, Permission는 다음과 같이 설정됩니다.

```
-rwsr-xr-x    1    oracle    dba      ...size ...date   oracle
-rwsr-xr-x    1    root      dba      ...size ...date   oracle
```

Oracle v7의 경우

Unix 프롬프트에서 다음 두 행을 입력하십시요.

```
chmod 6755 oracle
chmod 6755 orasrv
```

이 때, Permission는 다음과 같이 설정됩니다.

```
-rwsr-xr-x    1    oracle    dba      ...size ...date   oracle
-rwsr-xr-x    1    root      dba      ...size ...date   oracle
```

ORA-6110과 틀린 네트워 주소

TCP/IP 네트워크에서 중복 IP주소가 살아 있으면 ORA6110이 발생할 수 있습니다. 네트워크의 모든 IP주소가 고유의 것인지 확인하십시요.

WINDOWS용 SQL*NET 사용시 ORA-6120

Bulletin no : 10137

Windows용 SQL*NET TCP/IP와 ORA-6120에 대해서 간단하게 소개하고 ORA-6120을 진단하는 방법을 요약합니다.

* WINDOWS용 SQL*NET TCP/IP

WIN V1.X용 SQL*NET TCP/IP는 현재로서는 SQLTCP.DLL과 SQLTCP1.DLL등 두 개의 DLL이 한 세트로 되어 있습니다. 이들 DLL은 ORACLE용 연결 스트링이 TCP/IP 프로토콜 스트링으로 확인되면 작업 진행중에 OCI DLL에 의해 올려집니다. ORACLE INTERFACE DLL은 SQLTCP.DLL을 먼저 올리려고 합니다. 이것이 실패하면 DOS TSR 버전의 드라이버를 찾습니다. 두 가지 모두 실패하면 ORA-3121 메세지가 나옵니다.

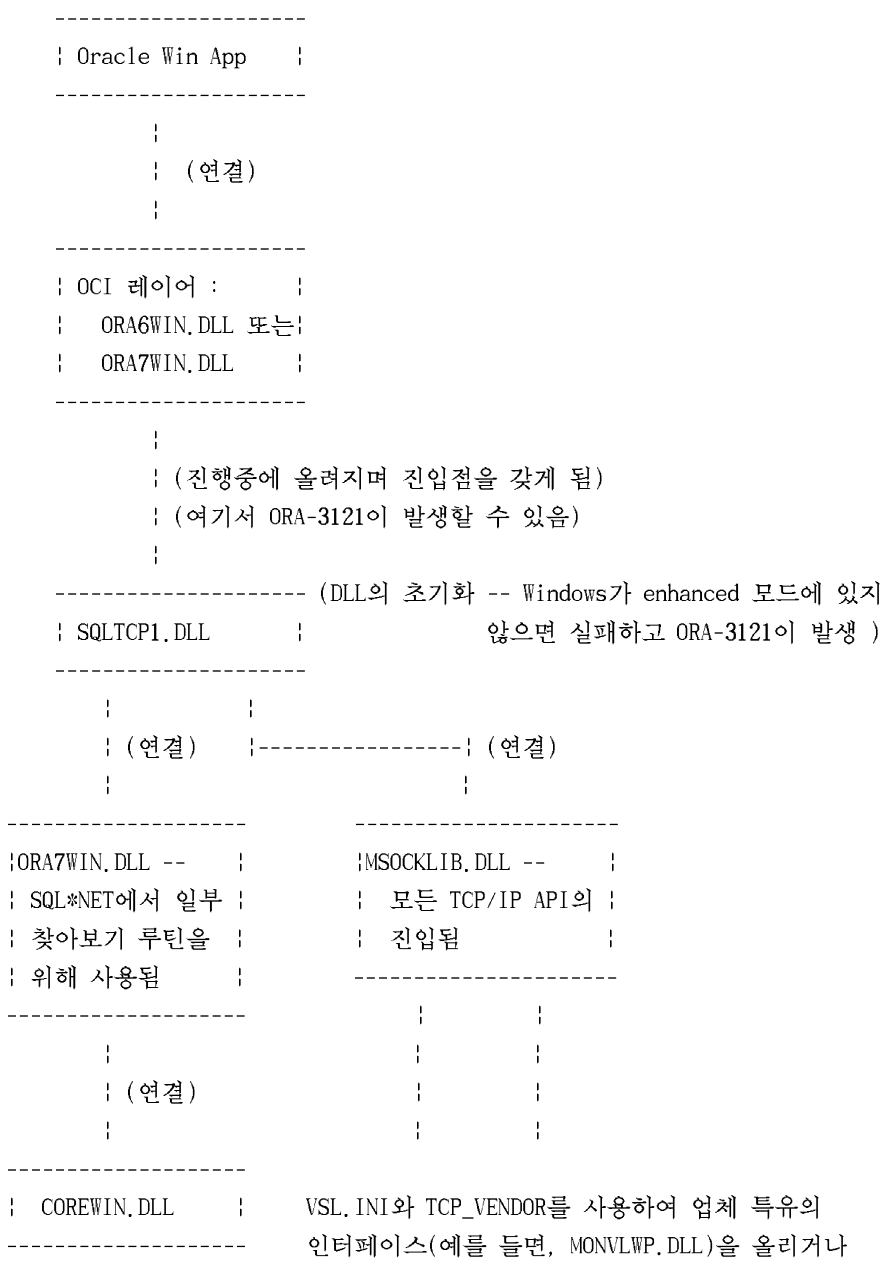
1. SQL*NET DLL이 올려지자 마자 MSOCKLIB.DLL(ORACLE에서 제공됨)이 올려집니다. 이 DLL은 ORACLE.INI로부터 TCP_VENDOR 스트링을 전달받습니다. 그 후, 이 MSOCKLIB.DLL은 TCP_VENDOR 스트링을 사용하여 WINDOWS 디렉토리에 있는 VSL.INI라는 파일을 엽니다. VSL 파일은 JSB 호출을 각 업체 고유의 API에 대응시킵니다. VSL.INI가 없으면 MSOCKLIB.DLL을 복사하거나 XCOPY 명령으로 ORAWIN 디렉토리 전체를 복사하기만 하고 이 VSL.INI 파일을 복사하지 않을 때 자주 발생합니다.

2. VSL.INI가 있더라도 TCP_VENDOR 매개변수에 오류가 생길 수 있습니다. 예를 들어 WINDOWS용 SQL*NET TCP/IP V1.1.7.5에서 VENDER 스트링의 오른쪽에 쓸데 없는 스페이스가 있으면 VSL.INI에서 Mapping을 분석할 때 MSOCKLIB.DLL이 실패하게 됩니다.

3. 이 문제의 다른 원인은 MSOCKLIB.DLL이 Mapping을 제대로 결 할 수 있지만 MSOCKLIB.DLL API를 해당 업체의 TRANSPORT INTERFACE 또는 API에 결합하도록 글루(GLUE)에 지시할 수 없을 때 발생합니다. 이 글루는 SQL*NET TCP/IP를 설치할 때 선택하는 TCP/IP 업체에 따라 DLL(이를테면 MNOVLWP.DLL)이 실패할 수도 있고 벤더 고유의 TSR(예를 들면 MFTP.EXE)이 올려지지 않았을 수도 있습니다. 이 글루를 구성하는 DLL과 TSR의 목록은 다음과 같습니다.

. m3open.dll : 3Com 3+Open v2.0
. mhparpa.dll : HP ARPA v2.0 & v2.1 그리고 Micorsoft TCP V1.0(LAN Manager v2.2의 번들)
. mnovlwp.dll : Novell LAN WorkPlace v4.0, v4.01, & v4.1
. mpcnfs4.dll : Sun PC-NFS v4.0a
. mpathsay.dll : Wollongong Runtime TCP v1.1 & v1.2
. mwinsock.dll : VSL 일반형 windows sockets DLL 버전 1.1
  . Frontier Technology Super - TCP v3.0
  . NetManage Chameleon TCP v3.1과 v3.22
  . FTP PCTCP v2.2 - Winsock Interface
  (1993년 5월 4일자 WINSOCK.DLL)
  . SUN PC/NFS 5.0 - Winsock Interface
  (1993년 3월 2일자 WINSOCK.DLL)

. DOS용 IBM TCP v2.1.0.2
(1993년 3월 30일자 WINSOCK.DLL)
. m3open.exe : 3Com 3+Open v1.1 & v1.2 및 DOS용 Digital
Pathworks(TCP/IP) v4.1
. mb2.exe : Beame 및 Whiteside TCP/IP v3.0
. mftp.exe : FTP PC/TCP v2.1 부터 v2.2까지
. mpcnfs.exe : Sun PC-NFS v3.0
. mpcnfs2.exe : Sun PC-NFS v3.5
. winsock.dll : Microsoft Windows



VSL.INI와 TCP_VENDOR를 사용하여 업체 특유의
인터페이스(예를 들면, MONVLWP.DLL)을 올리거나
인터페이스 TSR(예를 들면, MFTP.EXE)에 접속함.
여기서 ORA-3120 또는 ORA-6106이 발생할 수 있음.

4. ORA-6120 에러를 없애려면, 먼저 TELNET, PING 등 TCP/IP 유틸리티가 잘 작동하는 지 점검하십시요. 이들 유틸리티가 작동하지 않으면 TCP/IP 업체에 연락하거나 TCP/IP의 설치 책자를 읽어보아야 합니다.

5. 그렇지 않으면 WINDOWS 디렉토리에 VSL.INI가 있는지 점검하고, ORACLE.INI의 TCP_VENDOR가 무엇인지 알아 보십시요. 그런다음 VSL.INI에서 VENDOR의 프로파일을 찾아보십시요. 다음은 VSL.INI의 일부분입니다.

```
[ vsl-transports ]
3open="3Com 3+Open TCP"
3open2="3Com 3+Open TCP version 2.0"
pcnfs4="Sun PC-NFS v4.0"
.....................

[ 3open-transport ]
protocol=rlogin,telnet,rsh,rexec
dllmodule=MSOCKLIB.DLL
netmodule=M3OPEN
netinterface=TSR
;private=

[ pcnfs4-transport ]
protocol=rlogin,telnet,rsh,rexec
dllmodule=MSOCKLIB.DLL
netmodule=MPCNFS4.DLL
netinterface=DLL
;private=
```

글루 모듈이 DLL이라면, WINDOWS에서 그것을 올릴 수 있는 지 확인하시고 글루 모듈이 TSR이라면 그것은 WINDOWS를 시작하기 전에 올려져 있는지 확인하십시요.

6. CONFIG.SYS의 FILES와 BUFFERS를 충분히 높게 설정하지 않아도 ORA-6120이 발생할 수 있습니다. 다음은 이들 두 환경변수에 대한 최소 요구사항입니다.

```
BUFFERS=16
FILES=60
```

7. 일반형 WINDOWS 소켓 인터페이스(MWINSOCK.DLL)를 사용한다면, 업체 특유의 WINSOCK.DLL 파일이 손상되거나 버전이 틀릴 경우 ORA-6120이 발생할 수 있습니다.

ORA-6120을 진단하는 단계

1. TCP/IP 업체의 PING이나 TELNET이 잘 작동하는 지 점검하십시요. 이들 유틸리티의 GUI 버전이 도움이 될 수 있습니다. 이들 유틸리티가 작동하지 않으면 TCP/IP 설치 책자를 조하십시요.

2. 이들 유틸리티가 잘 작동하면 WINDOWS 디렉토리에 VSL.INI 파일이 있는지 점검하십시요. 만약 없으면 WINDOWS용 SQL*NET TCP/IP를 다시 설치하거나 다른 곳으로 복사해 오십시요.

3. WINDOWS 디렉토리에 VSL.INI가 있으면 TCP/IP 업체에 대한 바른 선택 하면서 SQL*NET TCP/IP를 설치했는지 점검하십시오

4. 제대로 설치 되었으면, ORACLE.INI의 TCP_VENDOR 매개변수를 읽어 보십시요. 그런다음 VSL.INI로 가서 그 TCP_VENDOR의 프로파일을 살펴보십시요. 프로파일의 "netmodule"은 무엇입니까? "netinterface"가 DLL이라면, 그 DLL을 찾을 수 있습니까? 필요할 때 WINDOWS는 DOS PATH를 통해 그 DLL을 올릴 수 있어야 합니다. "netinterface"가 TSR이라면 WINDOWS로 들어가기 전에 반드시 그 TSR을 실제로 올리십시요.

5. CONFIG.SYS의 FILES와 BUFFERS를 충분히 높게 설정했는 지 확인하십시요.
다음은 이들 두 매개변수의 최소 요구사항입니다.

```
BUFFERS = 16
FILES = 60
```

6. TCP/IP 업체가 winsock(Windows 소켓)을 사용한다면 정확한 또는 최신버전의 winsock.dll을 가지고 있지 않을 경우 GPF 또는 ORA-6120이 발생할 수 있습니다. 다음 표는 일 winsock TCP/IP 업체가 공인하는 몇 가지 WINSOCK.DLL입니다. 여기에 나열하지 않았지만 우리가 지원하는 winsock TCP/IP 업체라면, 그 업체로부터 최신의 winsock.dll을 구해다가 쓰는 것이 최선입니다. 그러나 TCP/IP 제품의 winsock.dll 복사본을 구하려면 해당 TCP/IP 업체에 직접 연락하십시요.

제품이름	WINSOCK.DLL 버전 또는 일자
FTP PC/TCP V2.2	1993년 5월 4일자 V1.06
IBM TCP/IP V2.1	1993년 3월 30일자
Sun PC-NFS V5.0	1993년 3월 2일자
3COM TCP/IP V2.1	1993년 5월 13일자

예:FTP의 PC/TCP V2.2를 사용하고 1993년 3월 10일자 winsock.dll을 사용하면, 실행중인 프로그램이 PCTCPAPI.DLL에서 GPF를 야기한다는 에러가 발생합니다. winsock.dll을 1993년 5월 4일자로 개정하 그 문제가 해결됩니다.

주: 또한 다른 디렉토리에 winsock.dll의 이전 복사본이 없는 것을 확인하십시요. 그 드라이브의 루트 디렉토리에서 "DIR WINSOCK.DLL/S" 명령을 내면 특정 드라이브의 모든 디렉토리를 점검할 수 있습니다. 경로에 있는 모든 드라이브에 해 이것을 수행해야 합니다.

WINSOCK.DLL의 다른 모든 복사본은 WINDOWS가 유효한 WINSOCK.DLL 대신 그것을 올릴 우려가 있으므로 이름을 바꾸어 주어야 합니다.

7. FTP PC/TCP V2.2이상을 사용할 경우에는 반드시 system.ini에 vpctcp386 드라이버가 있어야 합니다.

```
...
[386Enh]
device = (path)\vpctcp.386
...
```

8. VSL 일반형 Windows 소켓 인터페이스를 사용할 경우에는 WINSOCK.DLL이 손상 되지 않은 것을 확인해야 합니다.

WINDOWS용 SQL*NET 사용시 ORA-6122

Bulletin no : 10138

ORA-6122 "NETTCP: setup failure"는 SQL*NET 구성이 적절하게 설정되지 않은 상태에서 WINDOWS용 SQL*NET TCP/IP를 가지고 연결하려 할 때 발생합니다. 구성을 점검하여 다음 사항을 제대로 설정했는지 확인하십시요.

1. WINDOWS\WIN.INI를 조사해 보십시요. ORA_CONFIG 매개 변수를 정의하는 ORACLE 부분이 있어야 합니다

```
[Oracle]
ORA_CONFIG=C:\WINDOWS\ORACLE.INI
```

2. ORACLE.INI(또는 ORA_CONFIG 매개변수가 지시하는 파일)을 보십시요. ORACLE_HOME이 WINDOWS용 SQL*NET TCP/IP와 다른 Oracle Windows 응용 프로그램이 설 된 디렉토리를 지시하는 지 확인하십시요.
디폴트는 다음과 같습니다.

```
ORACLE_HOME=\ORAWIN
```

3. 또한 ORACLE.INI에 TCP_VENDOR를 정확하게 설정했는지도 확인하십시요.
유효한 업체 키 목록은 WINDOWS용 SQL*NET TCP/IP설정의 표 3-4 '업체키'를 참조하십시요.

4. 경로에 C:\ORAWIN\BIN(또한 ORACLE_HOME을 설정한 BIN 디렉토리)이 있는지 확인하십시요. DOS 프롬프트에서 SET을 입력하고 을 누르면 됩니다. 이 명령은 경로를 보여줍니다.

5. ORAWIN\BIN 디렉토리에 SQLTCP.DLL과 SQLTCP1.DLL이 모두 있는지 확인하십시요.

6. 경로의 다른 어떤 디렉토리에도 SQLTCP.DLL이 없는 것을 확인하십시요.

7. ORAWIN\BIN 디렉토리와 경로의 다른 디렉토리에 MSOCKLIB.DLL이 있는 지 확인하십시요. 또한 파일의 두 복사본을 가지고 있지 않도록 하십시요.
복사본이 둘일 경우, 이전 복사본의 이름을 바 면 문제가 줄어들 수 있습니다.

8. WINDOWS 디렉토리에 VSL.INI 파일이 있는지 확인하십시요. 만약 없으면 ORACLE INSTALLER를 통해 SQL*NET를 다시 입력하십시요.

WINDOWS용 SQL*NET 사용시 TNS-12203

Bulletin no : 10139

TNS-12203 "TNS:unable to connect to destination"은 여러가지 원인 때문에 발생할 수 있습니다. 여기서는 가장 일반적인 몇 가지 원인을 설명합니다.

1. WINDOWS용 TCP/IP 어댑터를 설치하지 않은 상태에서 연결하려 하면 TNS-12203이 발생할 수 있습니다. SQL*NET TCP/IP V2는 SQL*NET V2와 V2 TCP/IP 어댑터 등 두가지 제품으로 구성됩니다. 이들은 별도의 두 키트로 되어 있는데, 반드시 두 키트를 모두 설치해야 합니다.

2. TNS-12203 에러는 WINDOWS용 ORACLE SQL*NET소프트웨어가 ORACLE 홈 디렉토리를 찾을 수 없다는 의미일 수 있습니다. 이 문제를 해결하려면, 먼저 WIN.INI파일의 ORACLE 부분에 다음 항목이 있는지 확인하십시요.

```
[ORACLE]
ORA_CONFIG=C:\WINDOWS\ORACLE.INI
```

WINDOWS디렉토리가 C:\WINDOWS가 아니면, 위 행의 C:\WINDOWS 부분을 그 이름으로 바꾸십시요 그런 다음, ORACLE 소프트웨어를 다시 설치하십시요. ORACLE.INI 파일이 있으면 그 ORACLE.INI파일에 다음 행이 있는지 확인하십시요.

```
ORACLE_HOME=C:\ORAWIN
```

ORACLE 홈 디렉토리가 C:\ORAWIN이 아니면 위 행의 C:\ORAWIN 부분을 그 이름으로 바꾸십시요.

3. TNSNAMES.ORA가 ORACLE 홈 디렉토리 아래의 NETWORK\ADMIN에 있는지 확인하십시요. ORACLE 홈 디렉토리는 ORACLE.INI(또는 WIN.INI의 ORA_CONFIG매개변수가 지시하는 파일)의 ORACLE_HOME 에 의해 정의됩니다.

4. 서버쪽에서 실행중인 SQL*NET V2 TCP/IP Listener가 없어도 TNS-12203이 발생 수 있습니다. 서버쪽에서 실행중인 SQL*NET V2가 있는지 확인하십시요. 서버쪽에서 Listener Control 유틸리티(LSNRCTL)를 사용하면 서버의 V2 Listener가 실행중인지 확인할 수 있습니다. 서버에서 "LSNRCTL STATUS"명령을 실행하십시요. Listener Control이 유틸리티 대해서는 SQL*NET Administrator's Guide를 참조하십시요.

5. 연결할 SERVICES 이름에 대해 CONNECT DESCRIPTOR에 정확한 ADDRESS 매개변수를 지정했는 지 확인하십시요. 정확한 ADDRESS 매개변수를 사용중인지 확인하는 방법은, WINDOWS 클라이언트에서 사용 할 것과 같은 ADDRESS 매개 변수를 가진 TNSNAMES.ORA를 사용하여 서버에서 루프백을 해 보는 것입니다. 루프백을 수행한다는 것은 데이타베이스와 같은 기계에서 SQL*DBA 등을 호출하고 연결 스트링을 지정함으로써 SQL*NET V2를 통해 연결한다는 뜻입니다.

6. JSB VSL 소켓이 초기화되지 않으면 TNS-12203이 발생할 수 있습니다. 이 문제를 해결하려면 다음 사항을 점검하여 JSB VSL 레이어가 정확하게 초기화되었는 지 확인하십시요.

1) ORACLE.INI 파일(또는 WIN.INI의 ORA_CONFIG 매개변수가 지시하는 파일)의 업체키 매개변수 TCP_VENDOR가 정확하게 설정되었는 지 확인하십시오

2) ORACLE_HOME\BIN 디렉토리에 MSOCKLIB.DLL이 있는지 확인하십시요.

3) ORACLE_HOME\BIN 디렉토리에 선택된 JSB 업체 특유의 DLL이 있는지, 또는 그 JSB 업체 특유의 TSR 파일이 실행되는 지 확인하십시요.

4) WINDOWS 홈 디렉토리에 VSL.INI 파일이 있는 지 확인하십시요.

7. TNS-12203에 이어 실제 문제가 무엇인지 더 자세하게 나타내 주는 또 다른 에러가 발생할 수 있습니다. 화면에서 다른 에러가 보이지 않으면

ORACLE_HOME\NETWORK\LOG 디렉토리의 SQLNET.LOG 파일을 점검하십시요.

WINDOWS용 SQL*NET 사용시 TNS-12533

Bulletin no : 10140

TNS-12533에러는 대개 TNSNAMES.ORA에서 기술자의 ADDRESS 부분에 프로토콜 CONNECT DESCRIPTOR의 매개 변수가 틀릴 때 발생합니다. TCP/IP 어댑터의 경우, 프로토콜 특유의 매개 변수는 다음과 같습니다.

```
(ADDRESS =
  [(COMMUNITY=community_name)]
  (PROTOCOL=TCP)
  (HOST=host_name 또는 ip_address)
  (PORT=port_number)
)
```

TCP/IP 어댑터를 사용할 때 TNSNAMES.ORA 의 완전한 구문은 다음과 같습니다.

```
구문                                    해설
-----------------------------------    -----------------------------
service_name=
  (DESCRIPTION=
    (ADDRESS_LIST=                      ADDRESS_LIST는 생략할 수 있다.
      (ADDRESS=
        (COMMUNITY=community_name)      COMMUNITY는 생략  수 있다.
        (PROTOCOL=TCP)
        (HOST=host_name 또는 ip_address)
        (PORT=port_number)
      )
    )
    (CONNECT_DATA=
       (SID=sid)
    )
  )
```

TNSNAMES.ORA에 정의된 서버에 연결하려면, 연결중인 응용 프로그램의 Log On dialog box에서 호스트 스트링 필드에 연결 스트링 "TNS:service_name"을 지정하십시요. 또한 WINDOWS\WIN.INI의 ORA_CONFIG가 지시하는 파일에 LOCAL 매개 변수를 지정하여 기본 데이타베이스를 정의하는 것이 좋습니다. ORA_CONFIG는 대개 WINDOWS\ORACLE.INI를 지시합니다. ORACLE.INI에서 "LOCAL=TNS:service_name"행을 추가할 수 있습니다. 그 후, 연결을 할 때는 호스트 스트링 필드를 비워 둘 수 있습니다.

WINDOWS용 SQL*NET 사용시 TNS-12545

Bulletin no : 10141

에러 TNS-12545 "TNS:name lookup failure"는 TCP/IP 프로토콜 어댑터를 가진 SQL*NET V2를 통해 툴에서 연결하려는 데 TCP/IP 프로토콜 어댑터가 TNSNAMES.ORA 파일의 ADDRESS 부분에 제공된 호스트 이름을 분석할 수 없을 때 발생합니다.

TNS-12545는 SQL*NET TCP/IP V1에서 발생하는 ORA-6105 "NETTCP:remote host is unknown"과 비슷합니다.

* TNS-12545를 진단하는 단계

1. 호스트 이름을 이요하여 호스트에 대해 TELNET과 PING을 실행해 보십시요. 즉 PING 또는 TELNET <server>를 실행 해 보십시요. 호스트 이름을 용하여 PING이나 TELNET을 성공적으로 실행할 수 없으면, 먼저 TCP/IP 구성을 점검해야 합니다. PING나 TELNET의 WINDOWS버전을 사용하여 해 보십시요. 몇몇 DOS 버전이 잘 작동하더라도 WINDOWS 구성에 문제가 있으면 WINDOWS버전에서 에러가 발생할 수 있습니다.

2. 버에서 루프백을 수행하여 서버쪽이 정확하게 설정되었는 지 확인하십시요. 루프백을 수행하려면 데이타베이스가 있는 MACHINE에서 아무 툴이나 호출하고, TNSNAMES.ORA에 WINDOWS 클라이언트에 사용한 것과 같은 ADDRESS 매개 변수를 사용하여 SQL*NET V2를 통해 데이타베이스에 연결해 보십시요.

3. 다음에는 사용자의 TCP/IP 소프트웨어가 IP 주소의 이름 분석을 위해 PC에서 DOMAIN NAME SERVICE(DNS)를 사용하는 지 아니면 파일 찾아보기를 사용하는 지 점검하십시요.

4. DNS를 사용한다면, DNS에 해당하는 모든 드라이버를 올리도록 하십시요. 예를 들어, SUN PC/NFS는 RNMNIS.EXE를 올려야만 Naming Information Service를 사용할 수 있게 됩니다.

5. DNS를 사용하지 않는 다면, HOST 파일에 연결해야 할 호스트의 항목이 있는 지 확인하고, 만약 없으면 추가하십시요. WINDOWS용 TCP/IP 어댑터는 원 TCP/IP 업체가 예상한 디렉토리에 상주하는 HOST 파일을 사용합니다. HOST 파일의 각 항목 뒤에는 반드시 키리지 리턴을 넣어야 합니다. 또한 그러한 서비스에 필요한 드라이버를 모두 점검해야 합니다. 예를 들어, SUN PC/NFS는 그러한 서비스를 올리려면 RNMFILE.EXE 필요합니다.

6. WINDOWS 호출을 사용하지 않고 open() 호출을 사용했기 때문에 파일 핸들이 초과할 수 있습니다. SQL*NET V1은 여기에 해당하지 않습니다. 이 버그는 버전 2.1.5에 와서 해결되었습니다.

7. 연결을 시도하여 여전히 TNS-12545가 나오면, TNSNAMES.ORA 파일의 ADDRESS 부분 아래에 있는 HOST 매개 변수에 호스트 이름을 지정하지 말고 IP 주소를 지정하는 것도 한 가지 해결책이 될 수 있습니다. 다음 예를 참조하십시요.

```
HOST=139.141.61.64
```

WINDOW용 SQL*NET 를 사용시 ORA-3121

Bulletin no : 10121

다음은 ORA-3121을 해결하기 위해 단계별 조치방법을 설명하겠습니다.

* SQL*NET에서의 ORA-3121

SQL*NET V2를 통해 연결하려 할 때 연결 스트링에 'tns:'net접두어를 사용하지 않으면 ORA-3121이 나오게 됩니다. 구버전인 SQL*NET V1의 net 접두어 (SQL*Net TCP/IP에 대한 "t:"등)를 사용하지 않도록 주의하십시요.

* SQL*NET V1에서의 ORA-3121

1. Windows에서 나와서 DOS 프롬프트로 가십시요.

2. DOS 프롬프트에서 PATH를 입력하고 을 누르십시요. 현재 경로에 있는 모든 드라이브를 적어 두십시요. (C:, D:)

3. 2에서 적은 DOS 경로의 모든 드라이브에서 ORA6WIN.DLL 이라는 화일을 찾으십시요. (Norton에서 FILEFIND 유틸리티를 사용하거나 DOS 5.0 이상의 경우 루트 디렉토리에서 간단하게 DIR ORA6WIN.DLL/S를 실행하는 것이 좋은 방법입니다. Windows 파일 관리자의 파일 찾기 유틸리티를 사용할 수도 있습니다.)

ORA6WIN.DLL이 여러개이면 한 가지만 제외하고 모든 복사본의 이름을 바꾸십시요. (ORA6WIN.DLL같은 것으로). 한 가지 예외는 4/13/92(285573 byte) 또는 10/15/92(380639 bytes)라고 적힌 ORA6WIN.DLL은 복사본입니다. 이들 두 버전의 ORA6WIUN.DLL은 Oracle Corporation이 적접 배포한 것입니다.
10/15/92이후 날짜로 되어있고 380639 바이트보다 큰 파일을 가지고 있다면 그것을 사용해도 좋습니다.

(이들 날짜의 ORA6WIN.DLL 복사본이 없으면 제3자 응용 프로그램 업체에 연락해서 이 파일의 복사본을 구하십시요. 디스크 레이블이 "Windows Required Support Files Version 6.0xx.x.x"인 Oracle제품이라면, 틀림없이 ORA6WIN.DLL의 최신 복사본이 있을 것입니다.)

4. ORA6WIN.DLL이 있는 디렉톨기를 적어 두십시요.

5. 현재 경로에 앞서 말한 디렉토리가 있는 지 확인하십시요. DOS프롬프트에서 PATH를 입력하고 을 누르면 됩니다.

6. 현재 경로에 ORAWIN\BIN 디렉토리가 있는 지 확인하십시요. (또는 Windows용 SQL*NET이 설치된 곳, 잘 모르겠으면 ORACLE_HOME\BIN 디렉토리를 보십시요. ORACLE.INI 파일에 ORACLE_HOME이 정의도어 습니다.) 또한 ORACLE_HOME\BIN 디렉토리에 SQL*NET DLL이 있는지 확인하십시요. (예를 들어, Windows용

SQL*NET SPX의 경우는 SQLSPX.DLL과 SQLSPX1.DLL, WINDOWS용 SQL*NET TCP/IP의 경우는 SQLTCP.DLL과 SQLTCP1.DLL). ORACLE_HOME\BIN디렉토리에 ORA7WIN.DLL이 있는 지도 확인해야 합니다.

7. WINDOWS 디렉토리의 WIN.INI 파일을 편집하십시요. 다음과 같은 ORACLE이라는 구룹을 반드시 넣어야 합니다.

```
[Oracle]
ORA_CONFIG=C:\WINDOWS\ORACLE.INI
```

8. A) DOS 프롬프트에서 SET을 다시 입력하고 을 누르십시요.
CONFIG라 DOS환경 변수가 CONFIG.ORA 라는 파일에 인쇄하도록 설정되어 있는지 점검하십시요. CONFIG 환경 변수는 다음과 같아야 합니다.

```
CONFIG=C:\ORACLE6\CONFIG.ORA
```

B) 환경 변수에 CONFIG 변수가 설정되어 있지 않으면 AUTOEXEC.BAT 파일에 다음 명령문을 추가하고 나서 다시 부팅하십시요

<server : unix, tool : windows>

```
SET CONFIG=C:\WINDOWS\ORACLE.INI
(Windows 디렉토리에 해당 경로에 넣으십시요)
```

CONFIG가 설정되지 않아서 8B) 단계를 수행했으면 9단계는 무시하십시요.

9. CONFIG.ORA와 ORACLE.INI FILE의 다음 매개변수를 일치 시키십시요. (이들 파일의 위치에 대해서는 8단계를 참조하십시요.) 이들 매개변수 중에는 두 파일에 없는 것도 있습니다. 이들 매개 변수가 CONFIG.ORA나 ORACLE.INI에 있으면 정확히 일치하게 하십시요. 이들 개변수 중에 두 파일 중 어디에도 없는 것이 있다면, 그것을 추가하지 마십시요. 또한 이들 매개변수 중에 불필요한 것이 있을 수도 있지만, 그것을 CONFIG.ORA와 ORACLE.INI에 모두 추가해도 전혀 문제는 없습니다.

매개변수 목록 :

```
WIN_LOCAL_SESSIONS
WIN_REMOTE_SESSIONS
LOCAL
REMOTE
INTERRUPT
XMMITR
TCP_HOSTS_FILE
TCP_SERVICES_FILE
SQLNET DBNAME
```

10. 지금 WINDOWS를 시작한 다음, 프로그램 관리자의 도움말(HELP)을 클릭하고 ABOUT PROGRAM MANAGER를 클릭하여 WINDOWS가 인핸스드 모드인지 확인해 보십시요. WINDOWS용 SQL*NET은 표준 모드로 작동하지 않습니다.

11. 로그인 화면에 연결 스트링을 명시적으로 제공한 경우에는 연결 스트링 앞에 '@'를 지정하지 마십시요. 다음 예와 같이 그냥 연결 스트링만을 입력하십시요.

t:orasrv:orasid 혹은 x:orasrv

12. ORACLE.INI와 CONFIG.ORA에 LOCAL 매개 변수가 있으면, 로그인 화면에서 연결 스트링을 지정하지 않고 연결 스트링 필드를 비워 둘 수 있습니다.

LOCAL=t:orasrv:orasid

주: Oracle Corporation은 사용자의 \ORAWIN\BIN 디렉토리에 설치되는 NETTEST.EXE라는 WINDOWS 시험 프로그램을 배포합니다. 이 프로그램을 사용하면 원격 ORACLE 서버에 대한 연결을 시험할 수 있습니다. 이 프로그램은 연결되지만 제3자 툴이 연결되지 않으면 해당 툴 업체에 문의하십시요.

SAMPLE: XA PROGRAM

Bulletin no : 10363

```
===========
 myfuncb.sql
===========
create or replace function myfuncb ( aaa number )
        return number
        is
        bbb number;
begin
                for c1 in (
                select nvl(comm,0) as bbb
                from emp
                where empno=7499
                ) loop
        return c1.bbb;
        end loop;
        return 0;
end;
/

===========
 xa_prob.pc
===========
/*
*/

EXEC ORACLE OPTION (RELEASE_CURSOR=NO);
EXEC ORACLE OPTION (HOLD_CURSOR=YES);
/* EXEC ORACLE OPTION (SQLCHECK=SEMANTICS);
*/

/*#include */
#include

EXEC SQL INCLUDE SQLCA.H;
EXEC SQL INCLUDE ORACA.H;

#include

EXEC SQL BEGIN DECLARE SECTION;
```

```
        int emp_number;
        char emp_name[15];
        char *username = "SCOTT";
        char *password = "TIGER";
        VARCHAR  sqlstmt[80];
        int outval;
EXEC SQL END DECLARE SECTION;

int xarslt = 0;          /* xa result code */
int usexa = 1;           /* 1 -> use xa, otherwise normal sql */
int debug = 1;           /* print ALL result codes */
int rmid = 1;
XID xidx = { 0x1e0a0a1e, 12, 8, "KeesGTID01Branch0001" };
XID *xid = &xidx;
char xaoinfo[] =
      "oracle_xa+ACC=P/SCOTT/TIGER+SESTM=600";
int serialno = 1;        /* next transaction this pid */
#define LOOPMAX 1

void
xidgen(xid)
XID* xid;        /* output parameter */
{
  char seq [11];

  sprintf(seq, "%d", serialno);
  strncpy((&xid->data[6]), seq, 6);
  serialno++;
}

int dbopen()
{
  int anyerror=0;
/* open database */
  if ( usexa )
  {
    xarslt = xaoopen(xaoinfo, rmid, TMNOFLAGS);
    if ( xarslt || debug )
      printf("xaoopen = %d\n", xarslt);
    if ( xarslt )
      anyerror++;
  }
  else
  {

     /* enable oraca */
```

```
    oraca.oradbgf=1;
    oraca.oracchf=1;
    oraca.orastxtf=3;

    EXEC SQL CONNECT :username IDENTIFIED BY :password;
    if (sqlca.sqlcode || debug )
      printf("connect   sqlcode = %d\n", sqlca.sqlcode);
    if ( sqlca.sqlcode )
      anyerror++;
  }
  return(anyerror);
}

int starttr()
{
  int anyerror=0;
  int xarslt;

/* start transaction */
  if ( usexa )
  {
    xidgen(xid);
    xarslt = xaostart(xid, rmid, TMNOFLAGS);
    if ( xarslt || debug )
      printf("xaostart = %d\n", xarslt);
    if ( xarslt )
      anyerror++;
  }
  else
  {
    /* normal sql does by default */
  }
  return(anyerror);
}

int jointr()
{
  int anyerror=0;
  int xarslt;

/* start transaction */
  if ( usexa )
  {
    xidgen(xid);
    xarslt = xaostart(xid, rmid, TMJOIN);
    if ( xarslt || debug )
```

```
      printf("xaostart = %d\n", xarslt);
    if ( xarslt )
      anyerror++;
  }
  else
  {
    /* can't join a session w/o xa */
    printf("can't join if not xa\n");
    anyerror++;
  }
  return(anyerror);
}

int dosql()
{
  int anyerror=0;
  int xarslt;

/* do database operation */
  EXEC SQL EXECUTE
  begin :outval := myfuncb(1); end;
  END-EXEC;
  if (sqlca.sqlcode || debug )
    printf("execute sqlcode = %d, result=%d\n", sqlca.sqlcode,outval);
  if ( sqlca.sqlcode )
    anyerror++;

  return(anyerror);

}

int commit()
{
  int anyerror=0;
  int xarslt;

/* commit transaction */
  if ( usexa )
  {
    xarslt = xaocommit(xid, rmid, TMONEPHASE);
    if ( xarslt || debug )
      printf("xaocommit = %d\n", xarslt);
    if ( xarslt )
      anyerror++;
  }
  else
```

```
  {
    EXEC SQL commit work;
    if (sqlca.sqlcode || debug )
      printf("commit   sqlcode = %d\n", sqlca.sqlcode);
    if ( sqlca.sqlcode )
      anyerror++;
  }
  return(anyerror);
}

int rollback()
{
  int anyerror=0;
  int xarslt;

  if ( usexa )
  {
    xarslt = xaorollback(xid, rmid, TMNOFLAGS);
    if ( xarslt || debug )
      printf("xaocommit = %d\n", xarslt);
    if ( xarslt )
      anyerror++;
  }
  else
  {
    EXEC SQL rollback work;
    if (sqlca.sqlcode || debug )
      printf("commit   sqlcode = %d\n", sqlca.sqlcode);
    if ( sqlca.sqlcode )
      anyerror++;
  }
  return(anyerror);
}

int dbclose()
{
  int anyerror=0;
  int xarslt;

/* close database */
  if ( usexa )
  {
    xarslt = xaoclose(xaoinfo, rmid, TMNOFLAGS);
    if ( xarslt || debug )
      printf("xaoclose = %d\n", xarslt);
    if ( xarslt )
```

```
      anyerror++;
  }
  else
  {
    EXEC SQL rollback work release;
    if (sqlca.sqlcode || debug )
      printf("rollback   sqlcode = %d\n", sqlca.sqlcode);
  }
  return(anyerror);
}

int detach()
{
  int anyerror=0;
  int xarslt;

  if ( usexa )
  {
    xarslt = xaoend(xid, rmid, TMSUCCESS);
    if ( xarslt || debug )
      printf("xaoend = %d\n", xarslt);
    if ( xarslt )
      anyerror++;
  }
  else
  {
  }
  return(anyerror);
}

void main()
{
  int i;

  char temp[11];

  /* sprintf(temp,"%d", getpid()); */
  sprintf(temp,"%d", 100000);
  strncpy(xid->data, temp, 6);
  if (dbopen()) exit(1);
  for ( i=0; i < LOOPMAX; i++ )
  {
      if (starttr()) exit(1);
      if (dosql()) exit(1);
      if (detach()) exit(1);
      if (commit()) exit(1);
```

```
  }
  if (dbclose()) exit(1);

  exit(0);
}


========
 proc.mk
========
include $(ORACLE_HOME)/precomp/env_precomp.mk

.SUFFIXES: .pc .c .o

LDSTRING=
PRODUCT_LIBHOME=
MAKEFILE=proc.mk
PROCPLSFLAGS= sqlcheck=full userid=$(USERID) dbms=v6_char
PROCPPFLAGS= code=cpp include=/usr/include include=/opt/SUNWspro/SC3.0.1/include/CC
USERID=scott/tiger
INCLUDE=$(I_SYM). $(PRECOMPPUBLIC)

SAMPLES=sample1 sample4 sample6 sample7 sample8 \
        sample9 sample10 xa_prob sample12 oraca sqlvcp cv_demo
CPPSAMPLES=cppdemo1 cppdemo2 cppdemo3

# Rule to compile any program (specify EXE= and OBJS= on command line)

build: $(OBJS)
        $(CC) -o $(EXE) $(OBJS) -L$(LIBHOME) $(PROLDLIBS)

cppbuild:
        $(PROC) $(PROCPPFLAGS) iname=$(EXE)
        CC -c $(INCLUDE) $(EXE).c
        CC -o $(EXE) $(OBJS) -L$(LIBHOME) $(PROLDLIBS)

samples: $(SAMPLES)
cppsamples: $(CPPSAMPLES)

$(SAMPLES):
        $(MAKE) -f $(MAKEFILE) build OBJS=$@.o EXE=$@

$(CPPSAMPLES):
        $(MAKE) -f $(MAKEFILE) cppbuild OBJS=$@.o EXE=$@

sample5:
```

```
	@echo 'sample5 is a user-exit demo; use a forms makefile to build it.'

.pc.c:
	$(PROC) $(PROCFLAGS) iname=$*.pc

.pc.o:
	$(PROC) $(PROCFLAGS) iname=$*.pc
	$(CC) $(CFLAGS) -c $*.c

.c.o:
	$(CC) $(CFLAGS) -c $*.c

sample6.o: sample6.pc
	$(PROC) dbms=v6_char iname=$*.pc
	$(CC) $(CFLAGS) $(PRECOMPPUBLIC) -c $*.c

sample9.o: sample9.pc calldemo-sql
	$(PROC) $(PROCPLSFLAGS) iname=$*.pc
	$(CC) $(CFLAGS) $(PRECOMPPUBLIC) -c $*.c

cv_demo.o: cv_demo.pc cv_demo-sql
	$(PROC) $(PROCPLSFLAGS) iname=$*.pc
	$(CC) $(CFLAGS) $(PRECOMPPUBLIC) -c $*.c

xa_prob.o: xa_prob.pc
	$(PROC) $(PROCPLSFLAGS) iname=$*.pc dbms=v6
	$(CC) $(CFLAGS) $(PRECOMPPUBLIC) -c $*.c

calldemo-sql:
	sqlplus scott/tiger @../sql/calldemo
```

NUMBER DATA TYPE

Bulletin no : 10042

This article attempts to explain, with examples, the ORACLE Number datatype (type 2) as stored internally. Oracle Corporation reserves the right to change this internal representation at any time without reason and notice. Numeric data is stored internaly in variable length format,and consists of two parts

- SIGN/EXPONENT Byte (1st byte)
- DATA Byte(s) (2nd byte onwards)

SIGN/EXPONENT BYTE : The sign/exponent byte is divided into two parts again

- Sign bit (high-order bit)
- Exponent bits (lower order 7 bits)

The key to the calculation of a number is the sign bit which can take values 0 for negative and 1 for positive numbers.

The 7 lower order bits are the exponent, stored in excess 64 notation i.e. whatever the number you calculate from the 7 bits, you will have to deduct 64 from it to determine exponent value. The Sign/Exponent byte is evaluated as follows -

If the sign bit is set then the number is positive and the exponential is calculated in the following manner Determine the value of the exponent using the lower order 7 bits. As this value is in excess 64 notation, deduct 64 from this number that was calculated to give the exponent value.

E.g. Assume the SIGN/EXPONENT byte was 223 which in binary is 11011111.

From the binary representation of the example we determine that bit 8 is set, hence this is a positive number. To determine the exponent, clear bit 8 and then calculate the resultant which in this case is 95. As 95 is in excess 64 notation deduct 64 from it to give 31, which is the exponent.

If the sign bit is cleared i.e. 0 (number is negative), a little more work goes into calculating the number.

Perform a 1's complement of the sign/exponent byte (all 8 bits) Determine the value of the exponent using the lower order 7 bits. As this value is in excess 64 notation, deduct 64 from the value that was calculated to give the exponent value.

E.g. Assume the SIGN/EXPONENT byte was 100 which in binary is 01100100.

From the binary representation we can determine that the number is negative as bit 8 is clear. Now take the 1's complement of 01100100 to give 10011011. Mask of bit 8 (or just set it to 0) so that only bits 1 through 7 are used to calculate the exponent which in our example

will be 27. As 27 is in excess 64 notation, deduct 64 from it to give -37. Hence, exponent is -37.

DATA BYTE(s) : Data is stored in a series of bytes with the representation being in base 100. To avoid a binary zero a 1 is added to the number. Hence, each data byte represents digits in base 100 with values ranging from 1 to 100 representing values 0 to 99. Negative numbers have a terminator byte with a value of 102.

Let us take some examples and then try to understand the methodology of the calculation. These examples work off a table with the following description.

```
SQL> describe num_tab
 Name                           Null?    Type
 ------------------------------ -------- --------
 C1                                      NUMBER
```

CASE 1 : insert number 123456. Dump of column c1 gives the following

```
        DUMP(C1)
        ----------------------------------------
        Typ=2 Len=4: 195,13,35,57
```

1st byte = 195 decimal --> 11000011 binary Number is positive as bit 8 is set. The exponent value is 01000011 binary which is 67 decimal. As this number is in excess 64 notation deduct 64 from 67 giving 3. Exponent is 3 and positive.

2nd byte = 13 decimal

As the sign is positive deduct 1 from 13 (as it is stored with one added) to get 12

3rd byte = 35 decimal

As the sign is positive deduct 1 from 35 to get 34

4th byte = 57 decimal

As the sign is positive, deduct 1 from 57 to get 56

The exponent is 3, the data bytes are in base 100, so the number is

12 x (100 e 2) + 34 x (100 e 1) + 56 x (100 e 0) = 123456

CASE 2 : insert number -123456. Dump c1 gives the following

```
        DUMP(C1)
        ----------------------------------------
        Typ=2 Len=5: 60,89,67,45,102
```

1st byte = 60 decimal --> 00111100 binary

Number is negative as bit 8 is not set. Take the 1's complement of 00111100 which is 11000011. From this the exponent is 67 decimal (take the lower order 7 bits to calculate the

exponent). As this number is in excess 64 notation deduct 64 from 67 giving 3. Exponent is positive and is 3.

2nd byte = 89 decimal

Deduct 1 from this which gives 88. As the sign is negative, you need to take the 100's complement of this i.e. 100 - 88 to get 12.

3rd byte = 67 decimal
Deduct 1 from this which gives 66 and take the 100's complement to get 34.

4th byte = 45 decimal
Deduct 1 from this which gives 44 and take the 100's complement to get 56.

5th byte = 102 decimal
This is the last byte.

The calculation is
12 x (100 e 2) + 34 x (100 e 1) + 56 x (100 e 0) = 123456
and as the sign is negative, the answer is -123456.

CASE 3 : insert number 123456.789. Dump (c1) gives the following

```
DUMP(C1)
  ----------------------------------------
  Typ=2 Len=6: 195,13,35,57,79,91
```

1st byte = 195 --> 11000011 binary
Number is positive as bit 8 is set. The exponent value is 01000011 binary which is 67 decimal. As this number is in excess 64 notation deduct 64 from 67 giving 3 which is a positive number.

2nd byte = 13 decimal
As the sign is positive, deduct 1 from 13 to get 12.

3rd byte = 35 decimal
As the sign is positive, deduct 1 from 35 to get 34.

4th byte = 57 decimal
As the sign is positive, deduct 1 from 57 to get 56.

5th byte = 79 decimal
As the sign is positive, deduct 1 from 79 to get 78.

6th byte = 91 decimal
As the sign is positive, deduct 1 from 91 to get 90. The exponent is 3, the data bytes are

in base 100, so the number is
12 x (100 e 2) + 34 x (100 e 1) + 56 x (100 e 0) + 78 x (100 e -1) +
90 x (100 e -2) = 123456.789

CASE 4 : insert number -123456.789 Dump of c1 gives the following

```
DUMP(C1)
      ----------------------------------------
        Typ=2 Len=7: 60,89,67,45,23,11,102
```

1st byte = 60 decimal --> 00111100 binary

Number is negative as bit 8 is not set. Take the 1's complement of 00111100 which is 11000011. From this the exponent is 67 decimal (take the lower order 7 bits to calculate the exponent). As this number is in excess 64 notation deduct 64 from 67 giving 3. Exponent is positive and is 3.

2nd byte = 89 decimal
Deduct 1 from this which give 88. As the sign is negative, you need to take the 100's complement of this i.e. 100 - 88 to get 12.

3rd byte = 67 decimal
Deduct 1 from this which give 66. As the sign is negative, you need to take the 100's complement of this i.e. 100 - 66 to get 34.

4th byte = 45 decimal
Deduct 1 from this which give 44. As the sign is negative, you need to take the 100's complement of this i.e. 100 - 44 to get 56.

5th byte = 23 decimal
Deduct 1 from this which give 22. As the sign is negative, you need to take the 100's complement of this i.e. 100 - 22 to get 78.
6th byte = 11 decimal
Deduct 1 from this which give 11. As the sign is negative, you need to take the 100's complement of this i.e. 100 - 11 to get 89.
7th byte = 102 decimal
This is the last byte.
The calculation is
12 x (100 e 2) + 34 x (100 e 1) + 56 x (100 e 0) + 78 x (100 e -1) +
90 x (100 e -2) = 123456.789
and as the sign is negative, the answer is -123456.789

The above explanation is intended to further the knowledge of ORACLE's internal datatype representation only.

Note : Given below are examples of calculating the 1's and 100's complement. 1's complement of 10001100 = 01110011 (just 'not' the bits) 100's complement of 26 = 100 - 26 = 74

DATE TYPE 데이타를 효과적으로 조회하는 방법

Bulletin no : 10073

DATE TYPE 데이타는 오라클 TOOL을 사용하여 시간,분 등을 지정하지 않고 년월일만 입력하면 시간, 분,초 값이 '0'으로 입력된다. 이 데이타는 다음과 같이 조회하면 확인해 볼 수 있다.

```
SQL>  SELECT TO_CHAR(HIREDATE,'YYMMDD HH24:MI') HIREDATE FROM EMP;

       HIREDATE
       --------
       940506 00:00
       940407 00:00
       940408 10:11  (*)

SQL>  SELECT TO_CHAR(HIREDATE,'YYMMDD HH:MI') HIREDATE FROM EMP;

      HIREDATE
      --------

      940506 12:00
      940407 12:00
      940408 10:11  (*)
```

만일 시간,분이 입력되어 있는 데이타를 (*) 년월일로 조회하는 경우는 다음 SQL은 데이타를 조회할 수 없다.

```
SQL>  SELECT * FROM EMP WHERE HIREDATE = '08-APR-94';
      no rows selected
```

데이타를 조회 하기위해서는 다음과 같은 방법으로 SQL을 작성 한다.

```
A. SQL> SELECT * FROM EMP WHERE TO_CHAR(HIREDATE,'YYMMDD')
                = '940408'

B. SQL> SELECT * FROM EMP WHERE HIREDATE BETWEEN '08-APR-94' AND
                TO_DATE('08-APR-94') + .99999;
```

위 SQL들은 같은 결과의 데이타가 조회되지만 B. INDEX를 사용할 수 있으로 보다 더 효율적인 SQL이다.

ORACLE7 OPTIMIZER 에 대하여

Bulletin no : 10077

OPTIMIZER의 기능은 QUERY 를 실행하기위해 가장좋은 방법을 선택하는 것이다.
ORACLE V6와 비교하여 ORACLE7은 QUERY OPTIMIZER에 관한 2가지 주요기능 즉, COST-BASED 접근방법과 HINT-MECHANISM을 이용할 수 있는 것이다.

COST-BASED 접근방법은 OPTIMIZER가 이용 능한 접근 경로를 결정하고 그 접근 경로에 근간을 둔 여러개의 EXECUTION PLAN을 만든다. 이 EXECUTION PLAN들에 대해서 COST가 측정되고 OPTIMIZER는 가장 COST가 적은 EXECUTION PLAN을 선택한다.

OPTIMIZER는 EXECUTION PLAN의 COST를 측정하기 위하여 통계치를 사용한다. 통계치들은 ANALYZE명령에 의해 생성되고 다음과 같은 DATA DICTIONARY VIRES를 통해서 볼 수 있다.

DBA/ALL/USER TABLES, DBA/ALL/USER TAB COLUMNS,
DBA/USER CLUSTERS, DBA/ALL/USER INDEXES

syntax ANALYZE명령어 :

ANALYZE	INDEX	index	COMPUTE	STATISTICS
	TABLE	table	ESTIMATE	
	CLUSTER	cluster	DELETE	

[COMPUTE] : 정확한 정보를 얻을수있지만 데이타베이스에 상당한 LOAD 가 있으므로 RECORD 건수가 작은 TABLE 에 적용한다.(약 20000 RECORD 이하)
[ESTIMATE] : RECORD 수가 20000 건이상 TABLE 에 적용한다.

ORACLE7 OPTIMIZER는 통계치가 적어도 한개 TABLE에 존재하면 COST-BASED OPTIMIZER를 사용하고 어느 TABLE에도 통계치가 없으면 RULL-BASED OPTIMIZER를 사용한다.

또한 ORACLE7은 HINT MECHANISM을 이용하여 USER가 원하는 EXECUTION PLAN을 optimizer 에게 강제적으로 사용하도록 할 수 있다.

HINT에는 COST (cost-based approach), NOCOST (rule-based approach), FULL (full table scan), INDEX (use a certain index), ORDERED (from절에 나오는 순서대로 TABLE을 JOIN) 등이 있다.

다음 예들을 통하여 OPTIMIZER의 기능을 이해하기로 하자.

CASE1) ORACLE Version6 OPTIMIZER

Table TAB1 : 16,384 rows
Table TAB2 : 1 rows

a. TAB2를 DRIVING TABLE 로 하는 경우

select count(*) from TAB1, TAB2 0.96 seconds elapsed

	Count	Phys	cr	cw2	rows
Parse	1	0	0	0	
Execute	1	95	100	4	0
Fetch	1	0	0	0	1

b. TAB1을 DRIVING TABLE 로 하는 경우

select count(*) from TAB2, TAB1 26.09 seconds elapsed

	Count	Phys	cr	cw2	rows
Parse	1	0	0	0	
Execute	1	95	49247	32770	0
Fetch	1	0	0	0	1

위의 경우 TAB2 DRIVING인 경우가 더 효율적이므로 APPLICATION 은 a 와같이 사용하여야 한다.

CASE2) ORACLE Version7 Optimizer

각 TABLE에 ANALYZE COMMAND 를 시키지 않으면 결과는 VERSION 6의 경우와 유사하고 만일 두개의 TABLE 에 ANALYZE 를 실행시킬 경우는 다음과 같은 결과를 얻을수 있다.

a. select count(*) from TAB1, TAB2 1.20 seconds elapsed

	Count	Phys	cr	cw2	rows
Parse	1	0	0	0	
Execute	1	0	0	0	0
Fetch	1	123	124	5	1

b. select count(*) from TAB2, TAB1 1.32 seconds elapsed

	Count	Phys	cr	cw2	rows
Parse	1	0	0	0	
Execute	1	0	0	0	0
Fetch	1	123	124	5	1

위의 결과로서 ORACLE7에는 ANALYZE COMMAND 를 실행할 경우 TABLE 의 순서를 바꾸더라도 성능에 큰차이가 없이 향상된 결과를 얻을 수 있고 APPLICATION TUNING에 부담을 줄일수 있다.

한글이 ??? 로 DISPLAY 되는 경우

Bulletin no : 10097

ORACLE TOOLS(SQLPLUS,FORMS30,FORMS40,REPORT20 등)을 이용하여 한글DATA를 조회할 때 ???로 출력는데 해결 방법

==========

DATABASE는 SQL COMMAND 'CREATE DATABASE'를 포함하는 STATEMENT를 수행할 때 만들어지는데 우리가 그STATEMENT를 수행하기 앞서 고려해야 사항중의 하나가 DB CHARACTERSET 이다.

DB를 CREATE할때 DATABSE CHARACTERSET을 명시해야만 하는데 한번 선택되고 난후에는 CHARACTERSET을 변경하는 것은 쉽지가 않다.

DATA DICTIONARY에 있는 DATA를 포함해서 모든 DATA는 선택된 CHARACTERSET 에 의해 입출력 되기 때문에 USER가 다른 CHARATERSET으로 ACCESS한다면 한글 데이타가 ???로 출력된다.

또한 분산 DB 환경에서나 UPGRADE할 경우에는 DATABASE CHARACTERSET이 같아야 함으로 사용자들은 DATABASE의 CHARACTERSET을 알아아 두어야 한다.

< 현재 DATABASE 의 CHARACTERSET 확인 및 변경 >

1. 데이타베이스 CHARACTERSET 확인

```
$ sqldba lmode=y
SQLDBA> connect internal
SQLDBA> select * from v$nls_parameters;
PARAMETER                              VALUE
-------------------------------        --------------
    ...                                .....
NLS_CHARACTERSET                       KO16KSC5601 (or US7ASCII)
                                                   (A)
```

2. 환경 변수의 NLS_LANG 확인

```
$ env
...
NLS_LANG=American_America.US7ASCII
                          (B)
```

위의 (A)와 (B) 가 동일 해야만 한글 데이타 처리가 잘되고 서로 다른 상태에서 한글 데이타를 조회 할경우 ??? 로 출력 된다.

3. CHARACTERSET 을 일치 시키는 방법

* NLS_LANG 환경 변수를 변경하여 일치시키는 방법

Bourne shell, k-shell 을 사용 하는경우 . profile 을 수정

NLS_LANG = American_Americ a.KO16KSC5601; export NLS_LANG

c-chell 을 사용하는 경우 .cshrc 혹은 .login 수정
	setenv NLS_LANG American_America.KO16KSC5601
수정후 다시 $ env 를 실행하여 변경 되었는지 확인한다.

C:\WINDOWS\ORACLE.INI 수정
	NLS_LANG=American_America.KO16KSC5601
WINDOW 재기동

* 그리고 예를 들어서 SERVER의 character set이 US7ASCII 이고 PC의 NLS_LANG이 American_America.KO16KSC5601 과 같이 서로 다르게 설정되어 있는 경우 다음을 하나 더 oracle.ini에 추가하면 한글문제가 해결됩니다.

ORA_NLS_CHARACTERSET_CONVERSION=NO_CHARACTER_SET_CONVERSION

WINDOWS 95에서는 NLS_LANG이 ORACLE.INI에 들어 있지 않고 REGISTRY에 기록되므로 REGISTRY EDITOR를 이용하여 수정해야 한다.

MSDOS 창으로 나가서 REGEDIT.EXE 샐행
HKEY_LOCAL_MACHINE -> SOFTWARE -> ORACLE 선택
오른쪽 마우스 버튼을 이용하여 NLS_LANG을 수정

REGISTRY 변경 후에 PC를 REBOOTING 할 필요는 없습니다.

WINDOWS NT 에서도 WINDOWS 95의 경우와 마찬가지로 REGISTRY 에 기록된 정보를 변경해 주면 됩니다. 다음과 같이 합니다.

DOS 창에서 REGEDT32.EXE 실행
HKEY_LOCAL_MACHINE -> SOFTWARE -> ORACLE 선택
메뉴를 선택하여 NLS_LANG을 수정

SQLPLUS 기동시 WARNING MESSAGE 처리

Bulletin no : 10106

SQL*PLUS 를 기동할때 다음과 같은 경고 MESSAGE가 나타나는 경우가 있다.

```
$ sqlplus scott/tiger
SQL*Plus: Release 3.1.2.3.1 - Production on Tue Aug 30 17:07:36 1994
Copyright (c) Oracle Corporation 1979, 1992.  All rights reserved.

Warning:  Product user profile information not loaded!   <----
Error in disabling roles in product user profile.        <----

Connected to:
ORACLE7 Server Release 7.0.15.4.0 - Production
With the procedural and distributed options
PL/SQL Release 2.0.17.1.0 - Production
```

이 경고 MESSAGE는 사용자나 DBA 가 SQL*PLUS 를 사용하는데 방해가 될 수 있기 때문에 다음과 같은 방법으로 제거 한 후 사용한다.

Unix 의 경우

```
$ cd $ORACLE_HOME/sqlplus/admin
$ sqlplus system/manager
  SQL >  start pupbld
```

OpenVMS 의 경우

ORA_SQLPLUS_DEMO:V7PUP.SQL 를 SQL*PLUS SYSTEM 사용자로 LOGIN하여 실행.

Windows 의 경우

c:\orawin\plus31\pupbld.sql 를 SQL*PLUS SYSTEM 사용자로 LOGIN 하여 실행.

SQL,SQL*PLUS 를 사용하여 SQL SCRIPT를 동적으로 생성,수행

Bulletin no : 10107

이 Documents 는 SQL 과 SQL*PLUS를 동적으로 생성하고 수행하는 방법을 소개하고 있다. 이를 위해 자주 사용하는 3가지 사용 예를 소개한다.

이 Document는 다음과 같이 구성 되어져 있다.

CREATING A DYNAMIC DROP TABLE SCRIPT
CREATING A DYNAMIC GRANT PRIVILEGES SCRIPT
CREATING A DYNAMIC SCRIPT TO SET SQL PROMPT TO DATABASE NAME

1. CREATING A DYNAMIC DROP TABLE SCRIPT

개발 과정 중 우리는 Table을 Drop하고 Original Data를 Import하는 작업을 빈번하게 수행하는데, 이때 개개의 Drop 문을 사용하지 않고 SET_UP_DROP.SQL 이라 불리우는 다음의 Script를 사용한다.

```
set heading off
set feedback off
spool drop_all.sql
select 'drop table'||table_name||';'from user_tables;
spool off
set feedback on
start drop_all.sql
```

DROP_ALL.SQL script 는 다음과 같다.

```
drop table DEPT;
drop table BONUS;
drop table SALGRADE;
drop table DUMMY;
drop table CUSTOMER;
drop table ORD;
drop table ITEM;
drop table PRODUCT;
drop table PRICE;
```

2. CREATING A DYNAMIC GRANT PRIVILEGES SCRIPT

자신의 모든 Table 을 다른 사람에게 SELECT 권한을 부여하고자 할때 다음의 SET_UP_GRANT.SQL 을 이용하면 빠르고 손쉽게 해결할 수 있다.

```
set heading off
set feedback off
spool grant_all.sql
select 'grant select on'||table_name||'to SCOTT;'from user_tables;
spool off
set feedback on
start grant_all.sql
```

GRANT_ALL.SQL 의 script 는 다음과 같다.

```
grant select on DEPT to SCOTT;
grant select on BONUS to SCOTT;
grant select on SALGRADE to SCOTT;
grant select on DUMMY to SCOTT;
grant select on CUSTOMER to SCOTT;
grant select on ORD to SCOTT;
grant select on ITEM to SCOTT;
grant select on PRODUCT to SCOTT;
grant select on PRICE to SCOTT;
```

3. CREATING A DYNAMIC SCRIPT TO SET SQL PROMPT TO DATABASE NAME

위의 개념들을 보다 더 잘 사용하기 위해서는 Sql Prompt를 자신의 Database로 변환해주는 SET_UP_PROMPT.SQL Script를 login.sql File 에 추가시켜 자동적으로 SQL*PLUS 를불러 올수있게 한다. 이는 Multiple Databases에 유용하다.

```
set termout off
set heading off
set feedback off
spool set_prompt.sql
select 'set sqlprompt'||value||'>"from v$parameter
       where name='db_name';
spool off
start set_prompt.sql
```

SET_PROMPT.SQL 은 다음과 같다.

```
        set sqlprompt V715>
```

V$PARAMETER 는 Dynamic Performance Table 의 View 이다. Install후 SYS만이 Dynamic Performance Table을 수행할 수 있으므로 위의 Script를 사용하기 위해 View를 Create 하거나 User에게 Access 한을 부여하여야 한다.

SQL TIPS(1): SORT

Bulletin no : 10108

[Q] SQL*PLUS 에서 Salary가 많은 순서로 1위부터 5위까지 Fetch하여 Ename에 대해 Ordering하는 방법은?

[A] ROWNUM을 이용하면 안된다. Subquery를 이용한다.

잘못된 문장 :

```
select ename, salary
from emp
where rownum < 6
order by sal desc;
```

바른 문장 :

```
select ename, salary
from emp a
where 5 > (select count(*)
           from emp b
           where b.sal > a.sal)
order by sal desc;
```